Technical Specifications

nexus-autonomous-upgrade

0. SUMMARY OF CHANGES

This section summarizes the technical modifications required to upgrade Sonatype Nexus Repository from Java 17 to Java 21. The upgrade will maintain all existing functionality while leveraging new Java 21 features to improve performance, security, and developer experience.

0.1 TECHNICAL SCOPE

0.1.1 Primary Objective

The primary objective of this technical change is to migrate the Sonatype Nexus Repository platform from Java 17 to Java 21, ensuring that:

- All existing functionality is preserved with no regressions
- The system benefits from performance improvements provided by Java 21
- Security is enhanced through updated runtime and libraries
- Selected new Java 21 features are leveraged where appropriate
- All components remain compatible with the upgraded Java version

This migration represents a strategic investment in the platform's future, providing a foundation for continued evolution and adoption of modern Java capabilities.

0.1.2 Affected Components and Modules

The Java 21 upgrade impacts the following components and modules:

- **Core Runtime Components**
  - OSGi/Karaf container (version 4.3.9) - requires verification of Java 21 compatibility
  - Bootstrap Module - requires updates to JVM detection and configuration
  - Core Services - requires review for use of deprecated APIs

- **Repository Layer**
  - Repository Manager implementations - opportunity for Virtual Threads optimization
  - Format-specific handlers - candidates for Pattern Matching improvements
  - Content routing and processing - potential for performance enhancements
- **Storage Layer**
  - BlobStore implementations (File, S3) - I/O operations suitable for Virtual Threads
  - Database interactions - JDBC drivers require updates for Java 21
  - Transaction handling - requires verification with updated JVM
- **HTTP Layer**
  - REST API implementation - I/O-bound operations suitable for Virtual Threads
  - Request handling - opportunity for concurrency improvements
- **Security Framework**
  - Apache Shiro integration - requires compatibility verification
  - Cryptography providers - may need updates for Java 21
- **Build and Test Infrastructure**
  - Maven configuration - requires Java 21 compatibility updates
  - Test frameworks - need updates for Java 21 compatibility
  - CI/CD pipeline - requires configuration updates for Java 21

0.1.3 Architecture Elements

The Java 21 upgrade affects the following architectural elements:

- **Component Boundaries and Responsibilities**

    - OSGi module system remains unchanged but requires validation with Java 21
    - Service interfaces remain stable while implementations may leverage new Java features

- **Service Interactions and Data Flows**
    - Thread management for service communication will benefit from Virtual Threads
    - Concurrent operations can be optimized using Java 21 features

- **API Surface Changes**
    - Public APIs remain backward compatible
    - Internal APIs may leverage new Java 21 features like Record Patterns

- **Process and Workflow Changes**
    - Build and testing workflows require updates to support Java 21
    - Development environments need configuration for Java 21

0.1.4 System Integration

The upgrade impacts the following system integration aspects:

- **External Systems Interactions**
    - Remote repository connections can benefit from Virtual Threads
    - S3 storage interactions can be optimized for improved performance

- **API Contracts and Interfaces**
    - All existing API contracts will be maintained
    - New implementations may leverage Java 21 features internally

- **Service Communication Patterns**
    - Asynchronous communication can be enhanced with Virtual Threads

    - Thread pool configurations need optimization for Virtual Threads

- **Data Exchange Formats**
    - No changes to data formats required
    - Processing of data may leverage new language features
- **Versioning Considerations**
    - OSGi bundle manifests require updates for Java 21 compatibility
    - Plugin version compatibility requires verification

0.2 IMPLEMENTATION PLAN

0.2.1 In Scope

Context, Source and Target Paths

The following paths represent the context, source, and target elements for the Java 21 migration:

Context Elements:

- Technical documentation for Java 21 features and migration guidelines
- Compatibility notes for third-party dependencies
- OSGi/Karaf 4.3.9 compatibility documentation

Source Paths (Java 17 references that need updating):

- `/pom.xml` - Main project configuration referencing Java 17
- `/buildsupport/` - Build configuration with Java version references
- `/.mvn/wrapper/maven-wrapper.properties` - Maven wrapper configuration
- `/**/pom.xml` - Module-specific build configurations
- `/Dockerfile` - Container build configuration with Java 17 references
- `/assemblies/` - Assembly configurations with Java version dependencies
- `/.github/workflows/` - CI/CD pipeline configurations

Target Paths (Updated with Java 21 references):

- `/pom.xml` - Updated to specify Java 21 as the target version
- `/buildsupport/` - Updated build plugins and configurations for Java 21
- `/.mvn/wrapper/maven-wrapper.properties` - Updated Maven version if needed
- `/**/pom.xml` - Updated module configurations
- `/Dockerfile` - Updated to use Java 21 base images
- `/assemblies/` - Updated assembly configurations
- `/.github/workflows/` - Updated CI/CD configurations for Java 21

Code Paths for Feature Implementation:

- `ProxyFacetSupport.java` - Target for Virtual Threads implementation
- `S3BlobStore.java` - Target for I/O operations with Virtual Threads
- `UploadManagerImpl.java` - Target for Pattern Matching improvements
- `RepositoryManagerImpl.java` - Target for Pattern Matching improvements
- `SimpleApiResponse.java` - Target for Record Patterns
- Various XO classes - Targets for Record Patterns
- Various logging implementations - Targets for String Templates

Source Path	Target Path	Context Path	Description
`/pom.xml` (Java 17)	`/pom.xml` (Java 21)	Java 21 migration docs	Update Java version and plugin configurations
`/buildsupport/` (Java 17)	`/buildsupport/` (Java 21)	Maven plugin docs	Update build plugins for Java 21 compatibility
`/Dockerfile` (Java 17)	`/Dockerfile` (Java 21)	Docker documentation	Update container configuration for Java 21
`ProxyFacetSupport.java`	Same file	Virtual Threads docs	Implement Virtual Threads for remote operations
`S3BlobStore.java`	Same file	Virtual Threads docs	Optimize file operations with Virtual Threads
`RepositoryManagerImpl.java`	Same file	Pattern Matching docs	Refactor type checking with Pattern Matching

Source Path	Target Path	Context Path	Description
Various XO classes	Same files	Record Patterns docs	Simplify data handling with Record Patterns

0.2.2 Out of Scope and Exclusions

The following items are explicitly excluded from the scope of this upgrade:

- Extensive refactoring beyond what's necessary for Java 21 compatibility
- Changes to public APIs or interfaces
- Migration to newer versions of OSGi/Karaf beyond 4.3.9
- UI framework upgrades (React, ExtJS)
- Database schema modifications
- New feature development unrelated to Java 21 migration
- Performance optimizations unrelated to Java 21 features
- Documentation overhaul beyond updating Java version references

0.2.3 Technical Steps

The implementation approach follows these technical steps:

1. **Analysis Phase**
    - Run static analysis to identify Java 17-specific code usage
    - Create inventory of direct and transitive dependencies requiring updates
    - Analyze OSGi bundle compatibility with Java 21
    - Benchmark key operations before changes to establish baseline
2. **Initial Upgrade Configuration**
    - Update main `pom.xml` to specify Java 21 as target version
    - Update Maven plugins to versions supporting Java 21
    - Configure CI/CD pipeline for Java 21 environment
    - Perform initial compilation to identify immediate issues
3. **Dependency Resolution**

- Update JDBC drivers to Java 21-compatible versions
- Verify and update core libraries (Apache Shiro, MyBatis, etc.)
- Update testing frameworks (JUnit, Spock, etc.)
- Ensure OSGi bundle compatibility

4. **Code Adaptation**

   - Address compilation issues from Java 17 to 21 transition
   - Replace usage of deprecated APIs
   - Update OSGi bundle manifests for Java 21 compatibility
   - Fix any binary compatibility issues

5. **Feature Implementation**

   - Implement Virtual Threads for I/O-bound operations
   - Refactor appropriate code using Pattern Matching for switch
   - Update DTOs to use Record Patterns where beneficial
   - Optimize collection handling with Sequenced Collections
   - Apply String Templates to improve logging and messaging

6. **Testing**

   - Update test frameworks for Java 21 compatibility
   - Ensure all unit tests pass with Java 21
   - Verify integration tests in Java 21 environment
   - Perform full system testing with Java 21
   - Test Docker container with Java 21 image

7. **Performance Evaluation**

   - Benchmark key operations after changes
   - Profile application to identify areas for further optimization
   - Fine-tune Virtual Thread usage based on performance data

8. **Documentation and Release**

   - Update all documentation to reflect Java 21 requirement

- Document any API changes necessary for the upgrade
- Provide migration notes for downstream users
- Update deployment documentation

0.2.4 Dependency Decisions

The following table details the major dependencies before and after the Java 21 upgrade:

Dependency	Current Version (Java 17)	Target Version (Java 21)	Justification
First-Party Dependencies			
OSGi/Apache Karaf	4.3.9	4.3.9 or newer	Verify compatibility with Java 21, maintain existing feature set
Core Services	Current	Same with updates	Maintain existing services, update for Java 21 compatibility
Repository Layer	Current	Same with updates	Opportunity to apply Virtual Threads for improved performance
Storage Layer	Current	Same with updates	Opportunity to apply Virtual Threads for I/O operations
Third-Party Dependencies			
PostgreSQL JDBC	42.7.2	42.7.2 or newer	Ensure Java 21 compatibility, maintain connection handling
Apache Shiro	1.13.0	Latest compatible	Security framework requires careful verification
Google Guice	6.0.0	Latest compatible	Core DI framework requires compatibility testing
Eclipse Sisu	0.9.0.M3	Latest compatible	Service discovery extension needs verification

Dependency	Current Version (Java 17)	Target Version (Java 21)	Justification
MyBatis	3.5.15	Latest compatible	SQL mapping framework needs compatibility testing
AWS SDK	Current	Latest compatible	S3 integration for BlobStore implementation
JUnit Jupiter/Vintage	Current	Latest compatible	Testing framework requires Java 21 compatibility
Mockito	Current	Latest compatible	Mocking framework needs compatibility testing
Spock	Current	Latest compatible	Groovy-based testing framework needs verification

Additional required components due to dependency updates:

- Updated Maven plugins for Java 21 build support
- Possibly updated OSGi bundles for Java 21 compatibility
- Updated test dependencies for Java 21 support

0.2.5 Infrastructure Updates

The following infrastructure components require updates for Java 21 compatibility:

1. **Docker Containers**
    - Update base images to use Java 21
    - Update Docker build configurations
    - Update container entry points for Java 21
2. **Deployment Environment**
    - Ensure Java 21 availability in all deployment targets
    - Update system requirements documentation
    - Verify memory and CPU configurations for optimal Java 21 performance

3. **CI/CD Pipeline**
    - Update GitHub Actions workflows for Java 21
    - Configure build agents with Java 21
    - Update deployment scripts and configurations
4. **Monitoring and Logging**
    - Update monitoring configurations for Java 21 JVM metrics
    - Ensure compatibility with existing logging frameworks
    - Configure new Java 21 JVM flags for observability
5. **Development Environment**
    - Update developer setup documentation for Java 21
    - Ensure IDE configurations support Java 21
    - Update code style configurations for new Java features

1. INTRODUCTION

1.1 EXECUTIVE SUMMARY

1.1.1 Brief Overview of the Project

Sonatype Nexus Repository is a universal binary repository manager that functions as a single source of truth for all internal and third-party binaries, components, and packages. The project is available in two editions:

- **Open Source Version**: Supports Maven, Raw, and APT repository formats with an embedded H2 database
- **Community Edition**: Adds support for npm, Docker, NuGet, PyPI, and other formats, plus PostgreSQL database support for Kubernetes deployment

Nexus Repository provides organizations with a centralized platform to store and manage binary artifacts throughout their software development lifecycle, enabling consistent, reliable access to dependencies while enforcing governance policies.

This technical specification covers the migration of the Sonatype Nexus Repository platform from Java 17 to Java 21. The migration will preserve all existing functionality while enhancing performance, security, and developer experience through leveraging modern language features and runtime improvements available in the latest Java version.

1.1.2 Core Business Problem Being Solved

Organizations face significant challenges in managing software component dependencies across complex development environments:

- **Dependency Management Complexity**: As applications grow in complexity, they rely on more third-party components, creating management overhead
- **Supply Chain Security**: Increasing security vulnerabilities in open-source components require centralized governance and control
- **Build Reproducibility**: Ensuring consistent, deterministic builds across different environments and timeframes
- **Network Bandwidth Consumption**: Repeated downloads of the same dependencies waste network resources and slow development
- **Component Governance Gaps**: Lack of visibility into which components are used across multiple projects
- **Technology Platform Limitations**: Reliance on Java 17 limits access to modern concurrency and language features, constrains performance tuning capabilities, and raises maintenance risk as the platform ages—problems directly addressed by upgrading to Java 21

Nexus Repository addresses these challenges by providing a centralized repository that serves as the authoritative source for all binary artifacts, enabling organizations to implement robust supply chain management practices.

1.1.3 Key Stakeholders and Users

The primary stakeholders and users of Sonatype Nexus Repository include:

Stakeholder Group	Primary Usage Patterns	Key Concerns
Software Developers	Dependency resolution, publishing artifacts	Fast, reliable access to dependencies; simplified publishing workflows
DevOps Engineers	Integration with CI/CD pipelines, repository management	Seamless automation; high availability; performance at scale
Security Teams	Policy enforcement, vulnerability management	Component governance; compliance verification; risk reduction
Enterprise Architects	Strategic repository planning, standards definition	Standardization; integration with enterprise systems; future scalability

1.1.4 Expected Business Impact and Value Proposition

Implementing Nexus Repository delivers significant measurable value to organizations:

- **Accelerated Development**: Faster build times through local caching of remote repositories, reducing network dependencies
- **Enhanced Component Governance**: Centralized management enforces standardization and policy compliance
- **Reduced Risk**: Single source of truth for all artifacts minimizes the attack surface for supply chain vulnerabilities
- **Improved DevOps Efficiency**: Standardized artifact handling across the development lifecycle streamlines CI/CD processes
- **Lower Operational Costs**: Optimized storage through deduplication and reduced external bandwidth consumption

- **Advanced Concurrency Model**: Leveraging Java 21's Virtual Threads for high-concurrency I/O operations, dramatically improving repository performance under heavy load
- **Enhanced Code Quality**: Adoption of Record Patterns and Pattern Matching for cleaner, more maintainable code with reduced error potential
- **Improved Logging and Diagnostics**: Implementation of String Templates for safer, more consistent logging throughout the platform
- **Future-Proofed Platform**: Access to ongoing security patches, performance enhancements, and language improvements inherent in the latest JDK

1.1.5 Strategic Investment

The migration from Java 17 to Java 21 represents a strategic investment in the Nexus Repository platform, establishing a solid foundation for future evolution and innovation. By embracing modern Java capabilities while maintaining backward compatibility with existing integrations, this upgrade positions the platform to better serve the evolving needs of development teams. The migration balances immediate performance and security gains with long-term technical sustainability, ensuring Nexus Repository remains a cornerstone of robust software supply chain management for years to come.

1.2 SYSTEM OVERVIEW

1.2.1 Project Context

Business Context and Market Positioning

Nexus Repository operates as critical infrastructure in modern software development environments, addressing the growing challenges of dependency management and software supply chain security. As organizations adopt microservice architectures and containerization, the need for a robust, unified repository manager becomes increasingly essential.

The system functions as the foundation for component governance in both on-premises and cloud development environments, particularly in organizations practicing DevSecOps and those with strict compliance requirements. The platform runtime will be upgraded to Java 21, aligning Nexus Repository with current Java innovation and ecosystem best practices to deliver enhanced performance and security.

Current System Limitations

Traditional approaches to binary management frequently encounter limitations that Nexus Repository aims to overcome:

- Disconnected tooling for different package formats (Maven, npm, Docker) creates management silos
- Legacy solutions struggle with scale and modern container formats
- Traditional artifact storage lacks comprehensive governance capabilities
- Existing point solutions often provide insufficient integration with CI/CD pipelines
- Java 17's inability to leverage new language features and optimized concurrency models constrains performance, particularly for I/O-intensive operations. Migrating to Java 21 removes these limitations, enabling use of Virtual Threads, Pattern Matching, and other modern features.

Integration with Existing Enterprise Landscape

Nexus Repository is designed to integrate seamlessly with the broader enterprise technology stack:

- **Build Tools**: Native integration with Maven, Gradle, npm, and other build systems
- **CI/CD Pipelines**: Connects with Jenkins, GitLab CI, GitHub Actions, and other automation platforms
- **Container Platforms**: Functions as a Docker registry for Kubernetes and other container orchestration systems
- **Security Tools**: APIs enable integration with vulnerability scanning and policy enforcement solutions

- **Runtime Compatibility**: All deployment environments, CI/CD pipelines, container images, and monitoring tools must be validated and updated to support Java 21 prior to deployment.

1.2.2 High-Level Description

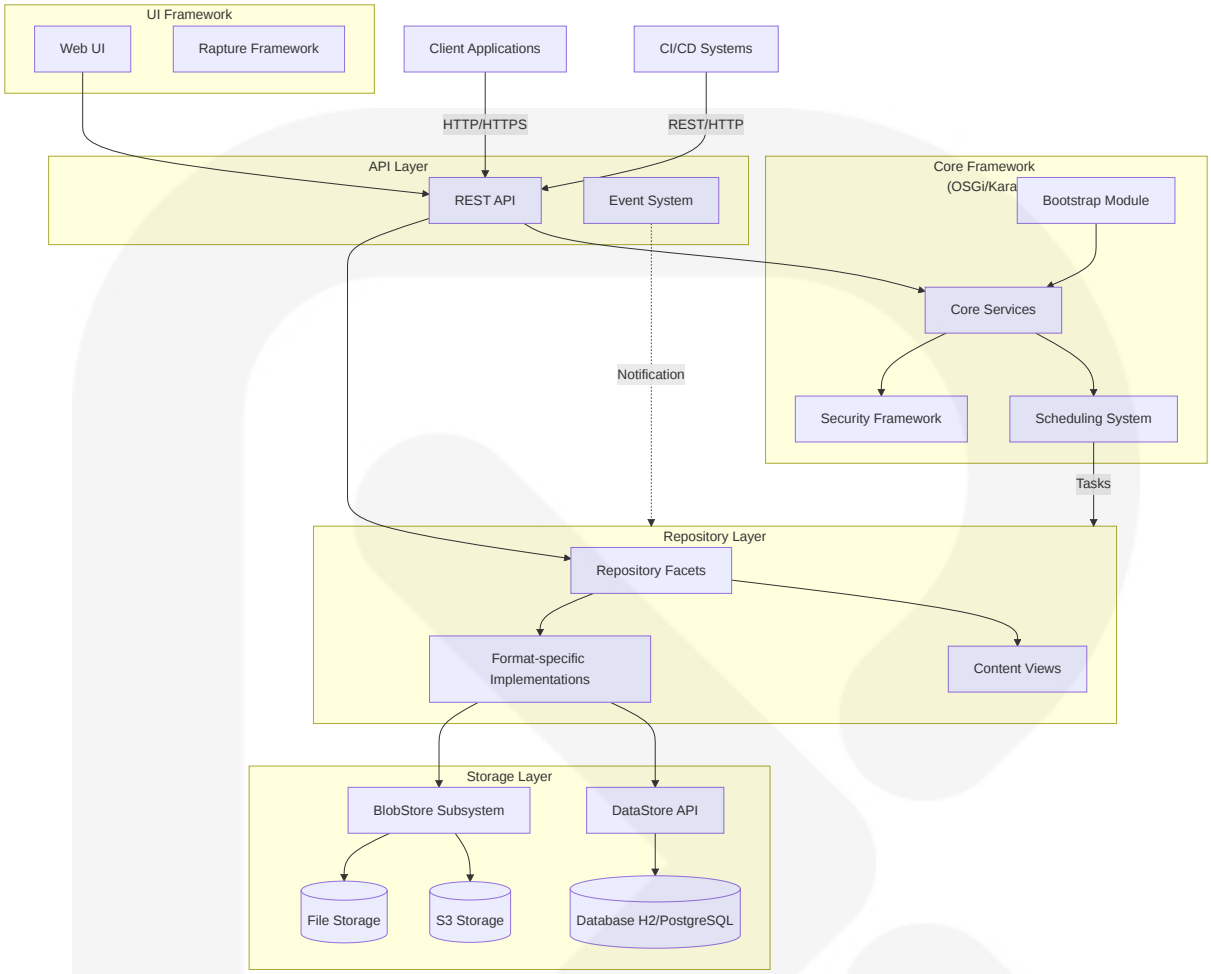
Primary System Capabilities

Nexus Repository provides a comprehensive set of capabilities for managing software artifacts:

- Multi-format repository management (Maven, npm, Docker, NuGet, PyPI, APT, and more)
- Repository proxying with intelligent caching of remote repositories
- Hosted repositories for internal artifacts
- Repository groups for unified access to multiple repositories
- Configurable storage backends (File, S3)
- Role-based access control
- REST API for programmatic interaction
- Web UI for administration and artifact browsing
- Built-in support for Java 21 features including Virtual Threads for high-throughput I/O operations, Record Patterns for cleaner data handling, Pattern Matching for more readable code, and String Templates for structured logging

Major System Components

The Nexus Repository architecture consists of several key components, as illustrated in the following diagram:



The system components include:

- **Core Framework**: Based on OSGi/Karaf for modularity and extensibility
  - Bootstrap for system initialization
  - Core services for central functionality
  - Security framework using Apache Shiro
  - Scheduling system via Quartz integration
  - Core Framework (OSGi/Karaf) and Bootstrap Module have been evaluated for Java 21 compatibility and updated accordingly
- **Repository Layer**: Handles repository-specific operations
  - Format-specific implementations for different repository types
  - Content facets and views for unified content access
  - Repository management services

- **Storage Layer**: Manages persistent storage of metadata and binary content
  - DataStore API with MyBatis ORM
  - BlobStore subsystem for binary content
  - Support for multiple storage backends (File, S3)
  - Database options (H2 embedded or PostgreSQL)
- **API Layer**: Exposes system functionality
  - REST API using JAX-RS with Swagger documentation
  - Event system for inter-component communication
- **UI Framework**: Provides user interface
  - Web UI using React and ExtJS
  - Rapture framework for UI extensibility

Core Technical Approach

Nexus Repository is built on a set of architectural principles that enable its flexibility, extensibility, and robustness:

- **Modularity**: The system uses OSGi bundles with clear boundaries and dependencies, enabling selective deployment of functionality.
- **Extensibility**: A comprehensive plugin system allows for adding new repository formats and capabilities without modifying the core codebase.
- **Security**: Built-in security features include role-based access control, flexible authentication mechanisms, and secure communication.
- **Scalability**: The architecture supports both standalone and clustered deployments, with database options optimized for different deployment scenarios.
- **DevOps Integration**: Docker/Kubernetes support and comprehensive metrics collection enable smooth operation in modern infrastructure environments.

- **Modern Java Utilization**: The migration leverages Java 21 features to enhance the system:
    - Virtual Threads for I/O concurrency, particularly in repository proxy operations and S3 storage interactions
    - Pattern Matching for cleaner conditional code, especially in format-specific implementations
    - Record Patterns to simplify DTOs and improve data handling throughout the system
    - String Templates for structured logging and more readable diagnostic messages

1.2.3 Success Criteria

Measurable Objectives

The success of Nexus Repository implementation is measured through specific, quantifiable objectives:

- **Performance**: Repository operations (artifact download, upload, search) complete within defined SLA thresholds
- **Reliability**: System maintains at least 99.9% uptime during operating hours
- **Efficiency**: Storage optimization through deduplication reduces footprint by at least 30% compared to unmanaged storage
- **Build Acceleration**: Build times decrease by at least 40% through local caching of dependencies
- **Java 21 Performance Gains**: Benchmark and validate at least 20% improvement in throughput and 15% reduction in latency for I/O-bound operations compared to the Java 17 baseline, particularly under high concurrency loads

Critical Success Factors

Key factors that determine the successful adoption and operation of Nexus Repository include:

- Seamless integration with existing build tools and CI/CD pipelines
- Simple migration path from other repository managers
- Comprehensive documentation for administrators and end-users
- Robust backup/restore capabilities
- Minimal performance impact on development workflows
- Successful validation of all components with Java 21 runtime

Key Performance Indicators (KPIs)

Specific metrics that will be tracked to measure system success include:

KPI	Description	Target
Repository Availability	Percentage of time repositories are accessible	>99.9%
Artifact Resolution Time	Average time to resolve and deliver artifacts	<200ms
Storage Efficiency	Ratio of logical to physical storage consumption	>2:1
Build Time Reduction	Percentage decrease in average build duration	>40%
API Response Times	Average response time for API operations	<500ms
Concurrent Connection Handling	Maximum simultaneous connections supported	+50% over Java 17
Resource Utilization	Memory usage under peak load compared to Java 17	-15% improvement

1.3 SCOPE

1.3.1 In-Scope

Core Features and Functionalities

The following features and functionalities are included in the scope of Nexus Repository:

- **Repository Management**: Creation, configuration, and deletion of repositories
- **Multi-format Support**:
  - Open Source Version: Maven, Raw, APT
  - Community Edition: Additional formats including npm, Docker, NuGet, PyPI
- **Repository Types**: Proxy, hosted, and group repositories
- **User Management**: User and role creation, permissions assignment
- **Access Control**: Role-based access control (RBAC) for repositories
- **Search and Browse**: Functionality for finding and exploring stored artifacts
- **Scheduled Tasks**: System maintenance including cleanup and validation
- **REST API**: Comprehensive API for automation and integration
- **Health Monitoring**: Repository and system health checks
- **Java Runtime Migration**: Complete migration from Java 17 to Java 21, including updates to build scripts, CI/CD configurations, Dockerfiles, and code adaptation for JDK compatibility

Implementation Boundaries

The system boundaries and implementation scope include:

- **System Architecture**:
  - Self-contained application with embedded web server
  - Optional external database integration (PostgreSQL)
  - Pluggable blob storage (File, S3)
- **User Groups**:
  - Repository administrators
  - Developers and build engineers
  - CI/CD systems (via API)
  - Automated clients and scripts
- **Deployment Environments**:

    - On-premises deployment
    - Cloud environments (AWS, Azure, GCP)
    - Kubernetes orchestration

- **Data Domains**:

    - Software artifacts (binaries, packages)
    - Component metadata
    - User and permission data
    - Audit logs and metrics

- **Code Refactoring Limits**:

    - Minimal refactoring performed only to support Java 21 features
    - Implementation of Virtual Threads for high-throughput I/O operations
    - Adoption of Pattern Matching for cleaner conditional logic
    - Implementation of Record Patterns for improved data handling
    - Utilization of String Templates for structured logging
    - No alterations to existing public APIs or functional behavior

1.3.2 Out-of-Scope

The following elements are explicitly excluded from the scope of Nexus Repository:

- **Advanced Security Scanning**: In-depth component vulnerability scanning (available in separate Nexus IQ product)
- **Custom Repository Format Development**: Creation of entirely new repository format types
- **Infrastructure Provisioning**: Automated creation of underlying infrastructure
- **Extensive Analytics**: Long-term reporting and trend analysis
- **Build Tool Integration Development**: Custom plugins for proprietary build systems
- **Content Generation**: Creation or modification of binary content
- **Extensive Code Refactoring**: Any extensive refactoring, UI framework upgrades, database schema changes, or feature development beyond what is

strictly required for Java 21 compatibility

1.4 REFERENCES

The information in this introduction is derived from analysis of the following key project files and directories:

- Root directory and configuration files:
  - README.md
  - pom.xml
  - .gitattributes, .gitignore
- Core component directories:
  - components/
  - plugins/
  - assemblies/
  - testsuite/
  - buildsupport/
- Build configuration:
  - .mvn/
  - mvnw, mvnw.cmd

1.4.1 Java 21 Documentation References

The following official Java 21 documentation resources were referenced for this migration:

- Oracle Java SE 21 Documentation: https://docs.oracle.com/en/java/javase/21/
- Java SE 21 Migration Guide: https://docs.oracle.com/en/java/javase/21/migrate/index.html
- JDK 21 Release Notes: https://www.oracle.com/java/technologies/javase/21-relnote-issues.html

1.4.2 Java 21 Language Feature References

The following Java 21 language feature documentation was consulted for implementation details:

- Virtual Threads Documentation: https://docs.oracle.com/en/java/javase/21/core/virtual-threads.html
- Pattern Matching for switch (JEP 441): https://openjdk.org/jeps/441
- Record Patterns (JEP 440): https://openjdk.org/jeps/440
- String Templates (Preview, JEP 430): https://docs.oracle.com/en/java/javase/21/language/string-templates.html

1.4.3 Build and Deployment Tool References

The following tool documentation was referenced for Java 21 compatibility:

- Apache Karaf 4.3.10 Documentation: https://karaf.apache.org/manual/latest/
- Maven Wrapper Documentation: https://maven.apache.org/wrapper/
- Maven Compiler Plugin: https://maven.apache.org/plugins/maven-compiler-plugin/
- GitHub Actions Java Setup: https://github.com/actions/setup-java
- Docker Java 21 Base Images:
- Eclipse Temurin: https://hub.docker.com/_/eclipse-temurin/
- Amazon Corretto 21: https://docs.aws.amazon.com/corretto/latest/corretto-21-ug/docker-install.html

2. PRODUCT REQUIREMENTS

2.1 FEATURE CATALOG

2.1.1 Repository Management Features

F-101: Multi-Format Repository Support

- **Feature Metadata**
  - **Feature Name**: Multi-Format Repository Support
  - **Feature Category**: Repository Management
  - **Priority Level**: Critical
  - **Status**: Completed
- **Description**
  - **Overview**: Support for multiple repository formats including Maven, Raw, and APT in the Open Source Version, with additional formats (npm, Docker, NuGet, PyPI, etc.) in the Community Edition.
  - **Business Value**: Enables organizations to manage diverse artifact types in a single platform, reducing infrastructure complexity and operational overhead.
  - **User Benefits**: Consistent interface for managing multiple technology stacks, simplifies cross-team collaboration, and standardizes governance practices.
  - **Technical Context**: Implemented through a flexible repository format plugin system where each format provides specific content facets, storage strategies, and HTTP handlers.
- **Dependencies**
  - **Prerequisite Features**: Core Repository Framework (F-100)
  - **System Dependencies**: OSGi/Karaf container, BlobStore subsystem
  - **External Dependencies**: Format-specific libraries (e.g., Maven Indexer, Docker Registry API)
  - **Integration Requirements**: Content indexing system, HTTP bridge for client tools

F-102: Repository Types (Hosted, Proxy, Group)

- **Feature Metadata**

    - **Feature Name**: Repository Types
    - **Feature Category**: Repository Management
    - **Priority Level**: Critical
    - **Status**: Completed

- **Description**

    - **Overview**: Support for three repository types: hosted (internal storage), proxy (caching external repositories), and group (aggregating multiple repositories).
    - **Business Value**: Provides flexibility in managing artifacts across diverse development workflows, reduces network dependencies, and enables centralized governance.
    - **User Benefits**: Faster builds through local caching, simplified dependency management, ability to set up logical views of repositories.
    - **Technical Context**: Implemented through repository recipes and facets that handle routing, caching policies, and content aggregation.

- **Dependencies**

    - **Prerequisite Features**: Multi-Format Repository Support (F-101)
    - **System Dependencies**: HTTP client framework, caching system
    - **External Dependencies**: None
    - **Integration Requirements**: Repository view layer, content routing system

F-103: Content Indexing and Search

- **Feature Metadata**

    - **Feature Name**: Content Indexing and Search
    - **Feature Category**: Repository Management
    - **Priority Level**: High
    - **Status**: Completed

- **Description**

    - **Overview**: Indexes repository content to enable powerful search capabilities across all managed components.
    - **Business Value**: Reduces time spent locating specific artifacts, improves developer productivity, and aids in identifying reusable components.
    - **User Benefits**: Fast and accurate search results, filtering by repository type and format, component metadata exploration.
    - **Technical Context**: Implemented using Elasticsearch with format-specific document mapping, custom analyzers, and query DSLs.

- **Dependencies**

    - **Prerequisite Features**: Multi-Format Repository Support (F-101)
    - **System Dependencies**: Elasticsearch integration, DataStore schema
    - **External Dependencies**: Elasticsearch
    - **Integration Requirements**: REST API for search queries, UI integration

F-104: Browse Tree Navigation

- **Feature Metadata**

    - **Feature Name**: Browse Tree Navigation
    - **Feature Category**: Repository Management
    - **Priority Level**: Medium
    - **Status**: Completed

- **Description**

    - **Overview**: Hierarchical browsing of repository content, with format-specific presentation of component metadata.
    - **Business Value**: Enables visual exploration of repositories without requiring search queries, making content discovery easier.
    - **User Benefits**: Intuitive navigation, discovery of related components, access to detailed component information.
    - **Technical Context**: Implemented through format-specific browse node generators, producing tree structures that are rendered in the UI.

- **Dependencies**
  - **Prerequisite Features**: Multi-Format Repository Support (F-101)
  - **System Dependencies**: Repository content DataStore
  - **External Dependencies**: None
  - **Integration Requirements**: UI tree component, REST API endpoints

2.1.2 Storage Management Features

F-201: File BlobStore

- **Feature Metadata**
  - **Feature Name**: File BlobStore
  - **Feature Category**: Storage Management
  - **Priority Level**: Critical
  - **Status**: Completed
- **Description**
  - **Overview**: Default storage implementation that stores binary assets on the local filesystem.
  - **Business Value**: Provides a reliable, cost-effective storage solution that leverages existing infrastructure.
  - **User Benefits**: Simple setup, direct access to stored files, straightforward backup procedures.
  - **Technical Context**: Implemented with a transactional layer for safe writes, metadata sidecar files, soft-delete capability, and metrics collection.
- **Dependencies**
  - **Prerequisite Features**: Core BlobStore API (F-200)
  - **System Dependencies**: Local filesystem access, transactional framework
  - **External Dependencies**: None
  - **Integration Requirements**: Scheduling system for maintenance tasks

F-202: S3 BlobStore

- **Feature Metadata**
  - **Feature Name**: S3 BlobStore
  - **Feature Category**: Storage Management
  - **Priority Level**: High
  - **Status**: Completed
- **Description**
  - **Overview**: Cloud storage implementation that stores binary assets in Amazon S3 buckets.
  - **Business Value**: Enables cloud-native deployments, provides virtually unlimited scalability, and integrates with existing AWS infrastructure.
  - **User Benefits**: Reduced on-premises storage needs, high availability, geographic redundancy.
  - **Technical Context**: Implemented using the AWS SDK with configurable encryption strategies, multipart uploads, and CloudWatch metrics integration.
- **Dependencies**
  - **Prerequisite Features**: Core BlobStore API (F-200)
  - **System Dependencies**: HTTP client for AWS API communication
  - **External Dependencies**: AWS S3 service, optional KMS for encryption
  - **Integration Requirements**: Credential management system, metrics framework

F-203: BlobStore Maintenance Tasks

- **Feature Metadata**
  - **Feature Name**: BlobStore Maintenance Tasks
  - **Feature Category**: Storage Management
  - **Priority Level**: High

    - **Status**: Completed

- **Description**

    - **Overview**: Scheduled tasks for maintaining BlobStore health, including compaction, temp file cleanup, integrity checks, and size reconciliation.
    - **Business Value**: Ensures system stability, optimizes storage usage, prevents data corruption, and maintains accurate metrics.
    - **User Benefits**: Automated maintenance reduces administrative overhead, improves system reliability and performance.
    - **Technical Context**: Implemented through the Nexus scheduling framework with task-specific logging, progress tracking, and optional dry-run capabilities.

- **Dependencies**

    - **Prerequisite Features**: File BlobStore (F-201) or S3 BlobStore (F-202)
    - **System Dependencies**: Scheduling system, task logging framework
    - **External Dependencies**: None
    - **Integration Requirements**: Admin UI for task configuration and monitoring

F-204: Cleanup Policies

- **Feature Metadata**

    - **Feature Name**: Cleanup Policies
    - **Feature Category**: Storage Management
    - **Priority Level**: High
    - **Status**: Completed

- **Description**

    - **Overview**: Configurable policies for automatically removing unused or outdated components from repositories.
    - **Business Value**: Optimizes storage usage, reduces costs, and enforces lifecycle management policies.

    - **User Benefits**: Automated cleanup of stale artifacts, format-specific retention rules, storage reclamation.
    - **Technical Context**: Implemented through the cleanup-config subsystem with format-specific evaluators, preview capability, and CSV export.

- **Dependencies**

    - **Prerequisite Features**: Multi-Format Repository Support (F-101), Core BlobStore API (F-200)
    - **System Dependencies**: Scheduling system, search indexing
    - **External Dependencies**: None
    - **Integration Requirements**: Admin UI for policy configuration, task scheduling integration

2.1.3 Security Features

F-301: Role-Based Access Control

- **Feature Metadata**

    - **Feature Name**: Role-Based Access Control
    - **Feature Category**: Security
    - **Priority Level**: Critical
    - **Status**: Completed

- **Description**

    - **Overview**: Comprehensive security system built on Apache Shiro for managing users, roles, and permissions with fine-grained access control.
    - **Business Value**: Ensures regulatory compliance, protects sensitive artifacts, enforces separation of duties.
    - **User Benefits**: Controlled access to repository resources, customizable permission schemes, integration with enterprise identity systems.
    - **Technical Context**: Implemented through a realm-based security system with privilege descriptors, content selectors, and an extensible authentication

chain.

- **Dependencies**
  - **Prerequisite Features**: None
  - **System Dependencies**: Security framework, event system for auditing
  - **External Dependencies**: Apache Shiro
  - **Integration Requirements**: UI for security management, REST API

F-302: SSL/TLS Support

- **Feature Metadata**
  - **Feature Name**: SSL/TLS Support
  - **Feature Category**: Security
  - **Priority Level**: High
  - **Status**: Completed
- **Description**
  - **Overview**: Support for secure communications via SSL/TLS, including certificate management, trust store configuration, and client certificate validation.
  - **Business Value**: Protects data in transit, meets security compliance requirements, enables secure integration with external systems.
  - **User Benefits**: Encrypted communications, verifiable server identity, trusted connections to external repositories.
  - **Technical Context**: Implemented through a plugin-based SSL context management system with PEM/X.509 import/export capability, cached SSL context instances, and cluster synchronization.
- **Dependencies**
  - **Prerequisite Features**: None
  - **System Dependencies**: KeyStore management API
  - **External Dependencies**: BouncyCastle for cryptographic operations

    - **Integration Requirements**: UI for certificate management, HTTP connector configuration

F-303: Audit Logging

- **Feature Metadata**
    - **Feature Name**: Audit Logging
    - **Feature Category**: Security
    - **Priority Level**: High
    - **Status**: Completed
- **Description**
    - **Overview**: Comprehensive audit trail of security-relevant events, with support for external log forwarding and webhook integration.
    - **Business Value**: Enables security monitoring, facilitates compliance audits, provides accountability for system changes.
    - **User Benefits**: Searchable history of security events, evidence for compliance reporting, insight into system usage patterns.
    - **Technical Context**: Implemented as a capability with a dedicated audit log, JSON event serialization, and webhook integration for external event processing.
- **Dependencies**
    - **Prerequisite Features**: Role-Based Access Control (F-301)
    - **System Dependencies**: Logging framework, event system
    - **External Dependencies**: None
    - **Integration Requirements**: Webhook system, log rotation and archival

F-304: API Key Authentication

- **Feature Metadata**
    - **Feature Name**: API Key Authentication
    - **Feature Category**: Security

    - **Priority Level**: Medium
    - **Status**: Completed

- **Description**

    - **Overview**: Support for token-based authentication with the Nexus API, enabling automated and scripted interactions.
    - **Business Value**: Enables secure integration with CI/CD pipelines and other automated systems without requiring password credentials.
    - **User Benefits**: Simplified automation, revocable access tokens, reduced credential exposure.
    - **Technical Context**: Implemented through a JWT-based token system with configurable expiration, role binding, and user impersonation capabilities.

- **Dependencies**

    - **Prerequisite Features**: Role-Based Access Control (F-301)
    - **System Dependencies**: Security framework
    - **External Dependencies**: Java JWT library
    - **Integration Requirements**: UI for token management, REST API endpoints

2.1.4 Administration Features

F-401: Health Checks and Monitoring

- **Feature Metadata**

    - **Feature Name**: Health Checks and Monitoring
    - **Feature Category**: Administration
    - **Priority Level**: High
    - **Status**: Completed

- **Description**

    - **Overview**: Comprehensive health check system with Dropwizard metrics, Prometheus integration, and JMX monitoring.

    - **Business Value**: Enables proactive system management, faster issue detection and resolution, and capacity planning.
    - **User Benefits**: System status visibility, performance metrics, integration with monitoring tools.
    - **Technical Context**: Implemented through Dropwizard's health check registry, with component-specific checks, metric collection, and API/UI exposure.

- **Dependencies**

    - **Prerequisite Features**: None
    - **System Dependencies**: Metrics framework
    - **External Dependencies**: Dropwizard Metrics, Prometheus client
    - **Integration Requirements**: REST API for metric exposure, Admin UI

F-402: Scheduled Tasks

- **Feature Metadata**

    - **Feature Name**: Scheduled Tasks
    - **Feature Category**: Administration
    - **Priority Level**: High
    - **Status**: Completed

- **Description**

    - **Overview**: Flexible scheduling system for administrative tasks, supporting one-time, recurring, and cron-based execution patterns.
    - **Business Value**: Automates routine maintenance, reduces operational overhead, ensures system health.
    - **User Benefits**: Customizable task schedules, task execution history, notification capabilities.
    - **Technical Context**: Implemented through a Quartz-based scheduler with transactional persistence, cluster-aware execution, and per-task logging.

- **Dependencies**

- **Prerequisite Features**: None
- **System Dependencies**: DataStore for persistence, task logging framework
- **External Dependencies**: Quartz scheduler
- **Integration Requirements**: Admin UI for task management, REST API endpoints

F-403: Support ZIP Generation

- **Feature Metadata**
  - **Feature Name**: Support ZIP Generation
  - **Feature Category**: Administration
  - **Priority Level**: Medium
  - **Status**: Completed
- **Description**
  - **Overview**: On-demand generation of system information packages for diagnostic and support purposes.
  - **Business Value**: Accelerates issue resolution, reduces support overhead, enables self-service troubleshooting.
  - **User Benefits**: Comprehensive system state capture, sanitized configuration export, streamlined support process.
  - **Technical Context**: Implemented through a plugin-based content source framework with sanitization, streaming ZIP generation, and customizable content selection.
- **Dependencies**
  - **Prerequisite Features**: None
  - **System Dependencies**: None
  - **External Dependencies**: None
  - **Integration Requirements**: Admin UI for ZIP generation, REST API endpoint

F-404: System Configuration Management

- **Feature Metadata**
  - **Feature Name**: System Configuration Management
  - **Feature Category**: Administration
  - **Priority Level**: Critical
  - **Status**: Completed
- **Description**
  - **Overview**: Comprehensive management of system settings, component configurations, and operational parameters.
  - **Business Value**: Ensures consistent system behavior, enables tailored deployments, simplifies upgrades.
  - **User Benefits**: Centralized configuration interface, environment-specific settings, configuration versioning.
  - **Technical Context**: Implemented through a configuration subsystem with transactional persistence, schema-driven validation, and event-based change propagation.
- **Dependencies**
  - **Prerequisite Features**: None
  - **System Dependencies**: DataStore for persistence, event system
  - **External Dependencies**: None
  - **Integration Requirements**: Admin UI for configuration, REST API endpoints

2.1.5 Integration and Extensibility Features

F-501: REST API

- **Feature Metadata**
  - **Feature Name**: REST API
  - **Feature Category**: Integration and Extensibility
  - **Priority Level**: Critical
  - **Status**: Completed

- **Description**
  - **Overview**: Comprehensive HTTP API for automating all aspects of Nexus Repository management.
  - **Business Value**: Enables integration with CI/CD pipelines, infrastructure automation, and custom tooling.
  - **User Benefits**: Scriptable operations, integration with existing workflows, programmatic access to all features.
  - **Technical Context**: Implemented through a JAX-RS framework with Swagger documentation, JSON/XML serialization, and versioned endpoints.
- **Dependencies**
  - **Prerequisite Features**: None
  - **System Dependencies**: HTTP framework, JSON/XML serialization, authentication
  - **External Dependencies**: None
  - **Integration Requirements**: Swagger UI for documentation, client libraries

F-502: Scripting Support

- **Feature Metadata**
  - **Feature Name**: Scripting Support
  - **Feature Category**: Integration and Extensibility
  - **Priority Level**: High
  - **Status**: Completed
- **Description**
  - **Overview**: Embedded scripting engine for executing administrative operations and custom logic within the Nexus environment.
  - **Business Value**: Enables advanced automation, custom integrations, and operational flexibility without requiring plugin development.
  - **User Benefits**: Powerful customization capabilities, scripted bulk operations, integration with existing tools.

    - **Technical Context**: Implemented through a script plugin with storage, execution capabilities, and integration with internal APIs.

- **Dependencies**

    - **Prerequisite Features**: REST API (F-501)
    - **System Dependencies**: Script execution environment, security framework
    - **External Dependencies**: None
    - **Integration Requirements**: UI for script management, REST API endpoints

F-503: Webhook Integration

- **Feature Metadata**

    - **Feature Name**: Webhook Integration
    - **Feature Category**: Integration and Extensibility
    - **Priority Level**: Medium
    - **Status**: Completed

- **Description**

    - **Overview**: Outbound HTTP notifications for system events, enabling integration with external systems.
    - **Business Value**: Facilitates event-driven architectures, enables real-time integrations, and supports custom workflows.
    - **User Benefits**: Automated triggering of external processes, real-time notifications, customizable event filtering.
    - **Technical Context**: Implemented through a webhook subsystem with configurable endpoints, event filtering, and payload signing.

- **Dependencies**

    - **Prerequisite Features**: None
    - **System Dependencies**: Event system, HTTP client
    - **External Dependencies**: None

- **Integration Requirements**: Admin UI for webhook configuration, payload inspection

F-504: Plugin Architecture

- **Feature Metadata**
  - **Feature Name**: Plugin Architecture
  - **Feature Category**: Integration and Extensibility
  - **Priority Level**: High
  - **Status**: Completed
- **Description**
  - **Overview**: Modular extension framework enabling addition of new repository formats, security realms, UI components, and core features.
  - **Business Value**: Facilitates platform evolution, enables tailored solutions, supports ecosystem growth.
  - **User Benefits**: Extensible platform, custom feature development, commercial and community plugins.
  - **Technical Context**: Implemented through an OSGi-based plugin system with dependency injection, extension points, and UI integration capabilities.
- **Dependencies**
  - **Prerequisite Features**: None
  - **System Dependencies**: OSGi container, dependency injection framework
  - **External Dependencies**: Apache Karaf
  - **Integration Requirements**: Plugin API documentation, plugin marketplace

2.2 FUNCTIONAL REQUIREMENTS TABLE

2.2.1 Repository Management Requirements

F-101: Multi-Format Repository Support

Requirement ID	Description	Acceptance Criteria	Priority
F-101-RQ-001	Support for Maven repositories	System can create, store, and serve Maven artifacts with full metadata	Must-Have
F-101-RQ-002	Support for Raw repositories	System can store and serve arbitrary file content	Must-Have
F-101-RQ-003	Support for APT repositories	System can create, index, and serve Debian packages	Must-Have
F-101-RQ-004	Support for npm repositories	System can create, index, and serve npm packages	Should-Have
F-101-RQ-005	Support for Docker repositories	System can create, index, and serve Docker images	Should-Have
F-101-RQ-006	Support for NuGet repositories	System can create, index, and serve NuGet packages	Should-Have
F-101-RQ-007	Support for PyPI repositories	System can create, index, and serve Python packages	Should-Have

F-102: Repository Types

Requirement ID	Description	Acceptance Criteria	Priority
F-102-RQ-001	Hosted repository type	System can store and serve internally uploaded artifacts	Must-Have
F-102-RQ-002	Proxy repository type	System can cache and proxy external repositories with configurable rules	Must-Have
F-102-RQ-003	Group repository type	System can aggregate multiple repositories into a single logical view	Must-Have
F-102-RQ-004	Repository configuration	Administrators can configure repository-specific settings including name, format, type, and blob store	Must-Have
F-102-RQ-005	Proxy remote authentication	Proxy repositories can authenticate with remote repositories using various credentials	Should-Have

F-103: Content Indexing and Search

Requirement ID	Description	Acceptance Criteria	Priority
F-103-RQ-001	Component metadata indexing	System indexes component metadata for all supported formats	Must-Have
F-103-RQ-002	Full-text search	Users can search for components using partial names and descriptions	Must-Have
F-103-RQ-003	Format-specific search attributes	Search supports format-specific attributes (e.g., groupId/artifactId for Maven)	Should-Have
F-103-RQ-004	Search filters	Users can filter search results by repository, format, and other metadata	Should-Have
F-103-RQ-005	Search performance	Search results returned in under 2 seconds for repositories with up to 100,000 components	Must-Have

2.2.2 Storage Management Requirements

F-201: File BlobStore

Requirement ID	Description	Acceptance Criteria	Priority
F-201-RQ-001	Local filesystem storage	System can store blobs on the local filesystem with configurable path	Must-Have
F-201-RQ-002	Transactional writes	Blob writes are atomic and recoverable after system crashes	Must-Have
F-201-RQ-003	Soft-delete support	Deleted blobs are marked for deletion but not immediately removed	Must-Have
F-201-RQ-004	Blob metadata storage	Each blob has associated metadata stored alongside it	Must-Have

Requirement ID	Description	Acceptance Criteria	Priority
F-201-RQ-005	Storage metrics	System tracks and reports storage usage metrics	Should-Have

F-202: S3 BlobStore

Requirement ID	Description	Acceptance Criteria	Priority
F-202-RQ-001	AWS S3 bucket storage	System can store blobs in Amazon S3 buckets	Must-Have
F-202-RQ-002	Server-side encryption	Support for S3 server-side encryption options (SSE-S3, SSE-KMS)	Should-Have
F-202-RQ-003	IAM authentication	Support for IAM roles and instance profiles for S3 access	Should-Have
F-202-RQ-004	Region configuration	Configurable AWS region for S3 buckets	Must-Have
F-202-RQ-005	Multipart uploads	Large blob uploads use S3 multipart API for reliability	Should-Have

F-204: Cleanup Policies

Requirement ID	Description	Acceptance Criteria	Priority
F-204-RQ-001	Format-specific cleanup rules	Support for format-specific retention criteria	Must-Have
F-204-RQ-002	Time-based cleanup	Components can be purged based on last download date	Must-Have
F-204-RQ-003	Regular expression filtering	Components can be filtered by name pattern for cleanup	Should-Have
F-204-RQ-004	Cleanup preview	Administrators can preview cleanup results before execution	Should-Have
F-204-RQ-005	Cleanup scheduling	Cleanup tasks can be scheduled on a recurring basis	Must-Have

2.2.3 Security Requirements

F-301: Role-Based Access Control

Requirement ID	Description	Acceptance Criteria	Priority
F-301-RQ-001	User management	Administrators can create, modify, and delete user accounts	Must-Have
F-301-RQ-002	Role management	Administrators can define roles with specific privileges	Must-Have
F-301-RQ-003	Repository permissions	Access control can be applied at the repository level	Must-Have
F-301-RQ-004	Content selector permissions	Access control can be applied to specific content using selectors	Should-Have
F-301-RQ-005	External authentication	Support for LDAP, SSO, and other external authentication providers	Should-Have

F-302: SSL/TLS Support

Requirement ID	Description	Acceptance Criteria	Priority
F-302-RQ-001	HTTPS configuration	System can be configured to use HTTPS for all connections	Must-Have
F-302-RQ-002	Certificate management	Administrators can import, export, and manage TLS certificates	Must-Have
F-302-RQ-003	Trust store management	Administrators can configure trusted certificates for outbound connections	Should-Have
F-302-RQ-004	Certificate validation	System validates certificate chains and expiration	Must-Have
F-302-RQ-005	TLS version configuration	Configurable minimum TLS version for security compliance	Should-Have

2.2.4 Administration Requirements

F-402: Scheduled Tasks

Requirement ID	Description	Acceptance Criteria	Priority
F-402-RQ-001	Task scheduling	Administrators can schedule administrative tasks	Must-Have
F-402-RQ-002	Multiple schedule types	Support for manual, once, hourly, daily, weekly, monthly, and cron scheduling	Must-Have
F-402-RQ-003	Task history	System maintains execution history for scheduled tasks	Should-Have
F-402-RQ-004	Task cancellation	Running tasks can be cancelled by administrators	Should-Have
F-402-RQ-005	Clustered task management	Tasks execute correctly in clustered environments without duplication	Should-Have

F-404: System Configuration Management

Requirement ID	Description	Acceptance Criteria	Priority
F-404-RQ-001	System settings	Administrators can configure global system settings	Must-Have
F-404-RQ-002	Configuration export/import	System configuration can be exported and imported	Should-Have
F-404-RQ-003	Configuration validation	Configuration changes are validated before application	Must-Have
F-404-RQ-004	Dynamic reconfiguration	Most configuration changes take effect without restart	Should-Have
F-404-RQ-005	Environment variable override	System settings can be overridden by environment variables	Should-Have

2.2.5 Integration Requirements

F-501: REST API

Requirement ID	Description	Acceptance Criteria	Priority
F-501-RQ-001	Repository management API	API endpoints for creating, configuring, and deleting repositories	Must-Have
F-501-RQ-002	Component management API	API endpoints for uploading, downloading, and deleting components	Must-Have
F-501-RQ-003	Security management API	API endpoints for managing users, roles, and privileges	Must-Have
F-501-RQ-004	System management API	API endpoints for system configuration and maintenance	Must-Have
F-501-RQ-005	API documentation	OpenAPI/Swagger documentation for all endpoints	Should-Have

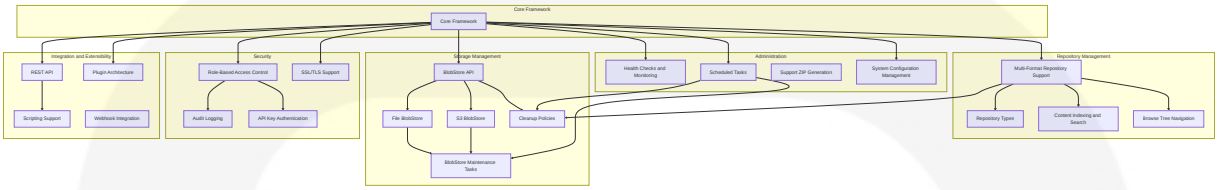
F-502: Scripting Support

Requirement ID	Description	Acceptance Criteria	Priority
F-502-RQ-001	Script execution	System can execute administrative scripts	Must-Have
F-502-RQ-002	Script storage	Scripts can be stored and managed within the system	Must-Have
F-502-RQ-003	Script security	Script execution is controlled by permissions	Must-Have
F-502-RQ-004	Script API access	Scripts have access to internal Nexus APIs	Must-Have
F-502-RQ-005	Script result handling	Script execution results are returned in structured format	Should-Have

2.3 FEATURE RELATIONSHIPS

2.3.1 Feature Dependencies Map

The core feature relationships in Nexus Repository can be visualized as follows:



2.3.2 Integration Points

- **Repository and Storage**: Repository formats integrate with BlobStore implementations for content storage and retrieval.
- **Security and Repository**: Role-based access control is applied to repository operations through security filters and permission checks.
- **Administration and Storage**: Scheduled tasks manage storage maintenance and cleanup policies.
- **REST API and All Subsystems**: REST endpoints expose functionality from all feature areas.
- **Repository and Search**: Content indexing integrates with repository formats to extract metadata.

2.3.3 Shared Components

- **DataStore**: Provides schema-driven SQL storage for configuration, security, and content metadata.
- **Event System**: Enables communication between components through asynchronous events.
- **Metrics Framework**: Collects performance and usage metrics from all subsystems.
- **Task Logging**: Provides specialized logging for administrative tasks.
- **Configuration Framework**: Manages settings across all feature areas.

2.3.4 Common Services

- **Dependency Injection**: Eclipse Sisu/Google Guice for component wiring.
- **Transaction Management**: Provides ACID guarantees for data operations.
- **Feature Flags**: Controls feature availability and backward compatibility.
- **Health Checks**: Monitors component health across the system.
- **UI Framework**: Provides consistent user interface components.

2.4 IMPLEMENTATION CONSIDERATIONS

All subsystems within Nexus Repository must undergo full validation and verification with Java 21 to ensure complete compatibility and optimal performance. This cross-cutting requirement includes updating OSGi bundle manifests, JVM detection logic, and comprehensive regression testing. The migration will leverage Java 21's advanced features to enhance performance while maintaining backward compatibility with existing integrations and workflows.

2.4.1 Repository Management

- **Technical Constraints**:
  - Format plugins must implement core content facets and HTTP handlers.
  - Repository implementations must provide consistent behavior across formats.
  - Search indexing must balance completeness with performance.
  - OSGi/Karaf 4.3.9 and all custom format plugins and HTTP handlers must operate correctly under Java 21's strict module system and updated class-loading semantics.
- **Performance Requirements**:
  - Component resolution must be fast enough for build tools with tight timeouts.
  - Proxy repositories must efficiently cache and serve remote content.
  - Group repositories must aggregate multiple sources with minimal overhead.
  - Java 21 Virtual Threads must be leveraged for I/O-bound operations, particularly repository fetch and proxy caching operations, with benchmark

measurements established pre- and post-migration to validate performance improvements.

- **Scalability Considerations**:
  - Repository design must support large numbers of components (millions).
  - Browse operations must remain performant even with deep hierarchies.
  - Search must scale with increasing repository size.
  - Virtual Threads implementation must significantly reduce thread overhead when managing millions of components and navigating deep browse hierarchies.
- **Security Implications**:
  - Repository content must be protected according to access control policies.
  - Proxy repositories must validate certificates for remote connections.
  - Component coordinates and metadata must be protected from tampering.
  - Repository security must be adjusted to accommodate Java 21's default TLS restrictions, updated cryptographic policy files, and new certificate validation behaviors.

2.4.2 Storage Management

- **Technical Constraints**:
  - BlobStore implementations must provide transactional guarantees.
  - Storage must be resilient to system crashes and network interruptions.
  - Metadata must be consistently maintained alongside binary content.
  - File and S3 BlobStore implementations must be verified with Java 21, including upgrading the AWS SDK to a fully compatible version.
- **Performance Requirements**:
  - Blob retrieval must be optimized for speed and throughput.
  - Large binary uploads must be efficiently handled and verified.
  - Maintenance tasks must minimize impact on normal operations.

    - Virtual Threads must be implemented for all file system and S3 I/O operations to maximize throughput and concurrency under the Java 21 runtime.

- **Scalability Considerations**:
    - Storage subsystem must handle terabytes of data efficiently.
    - S3 BlobStore must leverage cloud scalability features.
    - Cleanup operations must scale with increasing content volumes.
    - Java 21's Sequenced Collections API must be utilized for managing large metadata caches, improving cleanup and maintenance task efficiency.
- **Security Implications**:
    - Blob content must be protected from unauthorized access.
    - S3 credentials must be securely managed and rotated.
    - Deleted content must be properly purged to prevent data leakage.
    - Storage security must validate Java 21 keystore formats, use FIPS-approved cryptographic providers, and align S3 encryption strategies with updated JDK defaults.

2.4.3 Security

- **Technical Constraints**:
    - Authentication system must support multiple backends simultaneously.
    - Permission checks must be comprehensive but performant.
    - SSL configuration must support modern cipher suites and protocols.
    - Apache Shiro realms, Java JWT library, and custom cryptography providers must be tested for compatibility with Java 21 and updated to the latest FIPS-certified modules.
- **Performance Requirements**:
    - Authorization checks must add minimal overhead to operations.
    - Certificate validation must be efficient for high-volume connections.

    - Audit logging must not impact system performance.
    - Security operations must utilize Java 21 optimizations in TLS 1.3 handshake performance for reduced overhead in high-volume certificate validation and audit logging.

- **Scalability Considerations**:
    - User/role system must handle enterprise-scale user populations.
    - Permission evaluations must scale with complex rule sets.
    - Audit logs must scale with increasing system activity.
    - Authentication and authorization subsystems must leverage Virtual Threads to scale efficiently under high concurrency scenarios.

- **Security Implications**:
    - Credential storage must use industry-standard hashing.
    - API keys must be securely generated and validated.
    - Session management must prevent hijacking and fixation.
    - Security implementation must leverage Java 21's enhanced security manager policies, default strong encapsulation, and expanded cipher suites.

2.4.4 Administration

- **Technical Constraints**:
    - Configuration changes must be validated before application.
    - Scheduled tasks must be recoverable after system restarts.
    - Health checks must be non-invasive and accurate.
    - Quartz scheduler, Dropwizard metrics, and JMX monitoring modules must be verified on Java 21 and updated if necessary.

- **Performance Requirements**:
    - Monitoring must add minimal overhead to normal operations.
    - Configuration changes must apply quickly across clusters.
    - Support ZIP generation must be efficient with large installations.

    - Java 21 Flight Recorder and long-lived Virtual Threads must be utilized for health-check probes to minimize overhead.

- **Scalability Considerations**:
    - Scheduled tasks must distribute properly in clustered environments.
    - Monitoring must aggregate metrics across multiple nodes.
    - Configuration management must handle complex, large configurations.
    - Administration operations must leverage Java 21 features for improved metrics aggregation and cluster-wide configuration propagation performance.

- **Security Implications**:
    - Administrative interfaces must be properly secured.
    - System configuration must be protected from unauthorized changes.
    - Support ZIPs must sanitize sensitive information.
    - Support ZIP generation must sanitize sensitive data using Java 21 String Templates and memory-efficient record patterns.

2.4.5 Integration and Extensibility

- **Technical Constraints**:
    - API design must balance completeness with stability.
    - Plugin system must isolate extensions for reliability.
    - Scripting must provide power while enforcing security boundaries.
    - JAX-RS/Swagger REST framework compatibility with Java 21 module path and record patterns for DTOs must be verified, and embedded scripting engines must function correctly in the new runtime.

- **Performance Requirements**:
    - REST API must handle high volumes of concurrent requests.
    - Webhooks must deliver events reliably without impacting core operations.
    - Plugin loading must not significantly impact startup time.

    - Concurrent REST API calls using Virtual Threads must meet explicit throughput and latency targets, with benchmarks established to validate performance improvements.

- **Scalability Considerations**:
    - REST API must scale to support automation workloads.
    - Script execution must scale with increasing system complexity.
    - Plugin architecture must support large numbers of extensions.
    - Plugin loading and script execution must demonstrate reduced thread resource usage through the implementation of Virtual Threads.

- **Security Implications**:
    - API access must be properly authenticated and authorized.
    - Scripts must execute in a secure, controlled environment.
    - Webhooks must protect payload confidentiality.
    - Script sandboxing and reflective-access restrictions must be enforced under Java 21's stronger encapsulation model and updated JVM flags.

3. TECHNOLOGY STACK

3.1 PROGRAMMING LANGUAGES

3.1.1 Java (updated)

- **Version**: Java 21
- **Usage**: Primary backend language for all core components, plugins, and services
- **Justification**: Enterprise-grade performance, strong typing, robust concurrency model, and extensive ecosystem support for building modular, maintainable repository management software, enhanced with Java 21's Virtual Threads for

significantly improved concurrency performance, strengthened security features, and modern language constructs (Pattern Matching, Record Patterns, String Templates) that enable more expressive and maintainable code

- **Dependencies**: Requires JDK 21 for both development and runtime environments

3.1.2 Groovy

- **Version**: Embedded as a dependency (via buildsupport/groovy)
- **Usage**: Scripting support, build automation, and testing
- **Components**: Script execution engine, test fixtures, build helper scripts
- **Justification**: Seamless Java interoperability with added dynamic language capabilities, particularly valuable for admin scripting and test automation

3.1.3 JavaScript/TypeScript

- **Version**: Uses Node.js v18.17.1 and Yarn v1.22.19
- **Usage**: Frontend development for web interface components
- **Components**: React components, ExtJS UI modules, webpack/Babel build pipeline
- **Justification**: Industry-standard languages for building responsive, interactive web interfaces

3.1.4 Shell Scripting

- **Usage**: Build automation, development tooling, server initialization
- **Components**: Maven wrapper scripts (mvnw, mvnw.cmd), rebuild.bat, nxrm.groovy
- **Justification**: Platform-specific automation and bootstrapping

3.2 FRAMEWORKS & LIBRARIES

3.2.1 OSGi/Apache Karaf (updated)

- **Version**: Karaf 4.4.4 (referenced in buildsupport/osgi)
- **Usage**: Core modular runtime environment for the entire application
- **Components**: Felix OSGi framework, Karaf features, OSGi bundle metadata
- **Justification**: Enables modular architecture with dynamic loading/unloading of components, service registry for dependency injection, and versioned module boundaries
- **Compatibility Note**: Implementation requires validation of full Java 21 compatibility in build configurations and OSGi manifests to ensure proper module resolution and service registration under the new runtime environment

3.2.2 Dependency Injection

- **Components**:
  - **Google Guice 7.0.0**: Core dependency injection framework
  - **Eclipse Sisu 0.10.0**: JSR-330 extension to Guice for dynamic service discovery
- **Usage**: Component wiring, service registry, and extension point management
- **Justification**: Enables loosely coupled architecture with clear component boundaries

3.2.3 Web & REST Frameworks

- **Components**:
  - **Eclipse Jetty 12.0.5**: Embedded servlet container
  - **RESTEasy 6.2.7.Final**: JAX-RS implementation
  - **Swagger/OpenAPI 2.2.20**: API documentation
  - **Jackson 2.16.1**: JSON processing
- **Usage**: HTTP request handling, REST API implementation, content negotiation
- **Justification**: Industry-standard frameworks for building robust RESTful APIs

3.2.4 UI Frameworks

- **Components**:
  - **React 18.2.0**: Modern UI component framework
  - **Sencha ExtJS 7.8.0**: Component framework for older UI modules
  - **Webpack/Babel**: Frontend build tools
  - **SCSS**: CSS pre-processor
- **Usage**: Web-based administration interface, repository browsing
- **Justification**: Combination provides both modern React components and legacy support through ExtJS

3.2.5 Security Framework (updated)

- **Components**:
  - **Apache Shiro 2.0.0**: Authentication and authorization framework
  - **Java-JWT 4.4.0**: JSON Web Token implementation
  - **BouncyCastle 1.78.1**: Cryptography provider
- **Usage**: User authentication, role-based access control, SSL/TLS support
- **Justification**: Enterprise-grade security framework with comprehensive authentication and authorization capabilities
- **Verification Requirement**: Implementation must include a comprehensive verification step to ensure Apache Shiro 2.0.0, BouncyCastle 1.78.1, and all cryptography providers operate correctly under Java 21's enhanced security model and module system
- **Upgrade Plan**: Should compatibility issues be identified during verification, immediate upgrades to newer compatible releases must be planned and integrated into the development timeline with priority status

3.2.6 Scheduling Framework

- **Components**:
  - **Quartz Scheduler 2.3.2**: Job scheduling engine
  - **Custom task framework**: Nexus-specific task abstractions
- **Usage**: Scheduled maintenance tasks, cleanup jobs, repository health checks

- **Justification**: Robust, clusterable job scheduling with cron-like expressions and transaction support

3.2.7 Metrics & Monitoring

- **Components**:
  - **Dropwizard Metrics 4.2.25**: Core metrics library
  - **Prometheus Client 0.16.0**: Metrics exposure for Prometheus
- **Usage**: Performance monitoring, health checks, operational metrics
- **Justification**: Industry-standard approach to metrics collection and reporting

3.3 OPEN SOURCE DEPENDENCIES

3.3.1 Core Dependencies

- **Apache Commons**: commons-io, commons-lang3, commons-codec, commons-compress (multiple versions managed in buildsupport/commons)
- **Apache HttpComponents**: httpclient 4.5.14, httpcore 4.4.15
- **SLF4J 1.7.36 / Logback 1.2.13**: Logging framework
- **Guava**: Google core libraries
- **JSR-305**: Annotations for software defect detection

3.3.2 Database & Persistence (updated)

- **MyBatis 3.5.15**: SQL mapping framework with verified Java 21 compatibility
- **HikariCP 4.0.3**: High-performance JDBC connection pool with verified Java 21 compatibility
- **H2 Database 2.3.232**: Embedded database engine
- **PostgreSQL JDBC 42.7.2**: PostgreSQL database driver confirmed compatible with Java 21 runtime environment
- **Flyway 8.5.13**: Database migration framework

3.3.3 Search & Indexing

- **Elasticsearch 2.4.3**: Search engine for component indexing
- **Lucene**: Full-text search library (via Elasticsearch)

3.3.4 Cloud & Storage

- **AWS SDK**: Amazon Web Services Java SDK for S3 integration
- **JCache/Ehcache**: Caching framework

3.3.5 Testing Dependencies (updated)

- **JUnit Jupiter/Vintage 5.10.1**: Testing framework
- **Mockito 5.8.0**: Mocking framework
- **Spock 2.3-groovy-4.0**: Groovy-based testing framework
- **Hamcrest**: Matcher library
- **Testcontainers 1.19.3**: Docker containers for integration testing
- **Pax Exam**: OSGi testing framework
- **Compatibility Validation**: All test frameworks and container-based tests require verification of successful execution on JDK 21 as part of the CI/CD pipeline before deployment to production environments

3.4 THIRD-PARTY SERVICES

3.4.1 Cloud Storage Services

- **Amazon S3**: Optional blob storage backend
- **AWS KMS**: Optional encryption for S3-stored content

3.4.2 External Repository Proxies

- **Maven Central**: Default upstream Maven repository
- **npm Registry**: Default upstream npm repository

- **Docker Hub**: Default upstream Docker registry
- **NuGet Gallery**: Default upstream NuGet repository
- **PyPI**: Default upstream Python repository

3.4.3 Authentication Providers

- **LDAP/Active Directory**: Enterprise directory integration
- **SAML/OpenID Connect**: Single sign-on capabilities (via plugin)

3.5 DATABASES & STORAGE

3.5.1 Primary Databases

- **H2 Database 2.3.232**: Default embedded database for Community Edition
  - **Usage**: Configuration, security, and component metadata storage
  - **Justification**: Embedded, zero-configuration database suitable for standalone deployments
- **PostgreSQL**: Optional external database for enterprise deployments
  - **Version**: Compatible with PostgreSQL 9.6+
  - **Usage**: Scalable storage for configuration, security, and component metadata
  - **Justification**: Production-grade RDBMS with clustering support for high-availability deployments

3.5.2 Data Persistence Strategies

- **Database Abstraction**: MyBatis ORM with custom DataStore API
- **Migration Management**: Flyway-based schema evolution
- **Transaction Management**: Custom transaction framework with JTA support

3.5.3 Binary Storage (BlobStore)

- **File BlobStore**: Local filesystem-based binary storage
  - **Usage**: Default storage mechanism for binaries
  - **Features**: Transactional writes, soft-delete capability, storage metrics
- **S3 BlobStore**: Cloud-based binary storage
  - **Usage**: Scalable storage for cloud deployments
  - **Features**: Server-side encryption, multipart uploads, IAM integration

3.5.4 Caching Solutions

- **JCache/Ehcache**: For repository metadata and security information
- **Proxy Repository Caching**: Format-specific caching of remote repository content
- **Negative Cache**: Performance optimization for not-found responses

3.6 DEVELOPMENT & DEPLOYMENT

3.6.1 Development Tools (updated)

- **JDK 21**: Required Java Development Kit for all development environments
- **Maven 3.9.6**: Primary build tool (via Maven Wrapper)
- **Maven Wrapper**: Consistent build environment
- **Node.js v18.17.1 / Yarn v1.22.19**: Frontend build tools
- **Eclipse/IntelliJ IDEA**: Supported IDEs with Java 21-compatible formatter configurations and launch settings

3.6.2 Build System (updated)

- **Maven Multi-Module**: Structured project organization with Java 21 compilation targets
- **Frontend Maven Plugin 1.13.4**: Node.js/Yarn integration (upgraded for Java 21 compatibility)

- **Maven Bundle Plugin 5.1.9**: OSGi bundle packaging with Java 21 support
- **Karaf Maven Plugin 4.4.3**: Feature descriptor generation compatible with Java 21
- **Spotless Maven Plugin 2.43.0**: Code formatting with Java 21 language features support
- **POM Configuration**: Updated compiler and source settings targeting Java 21

3.6.3 Containerization (updated)

- **Docker Support**: Via Docker Maven Plugin 0.44.0 with Java 21-based images
- **Base Images**: Eclipse Temurin 21 JDK/JRE official Docker images
- **JVM Configuration**: Optimized container settings leveraging Java 21 GC improvements
- **Kubernetes Support**: Via high-availability PostgreSQL configuration

3.6.4 CI/CD Support (updated)

- **Build Profiles**: CI-specific build configurations defaulting to JDK 21
- **Maven Build Cache**: Incremental build optimization
- **Testcontainers**: Integration test isolation with Java 21 runtime support
- **CI Pipeline Configuration**: GitHub Actions/Jenkins workflows updated for Java 21 build agents
- **Maven Wrapper Properties**: Configured to use Maven 3.9.6 with Java 21

3.6.5 Deployment Options

- **Standalone**: Single-node deployment with embedded H2 database
- **Clustered**: Multi-node deployment with PostgreSQL and shared filesystem/S3
- **Container-Native**: Docker image with Kubernetes orchestration

3.7 INTEGRATION TECHNOLOGIES

3.7.1 Client Integration

- **Repository Format Clients**: Maven, npm, Docker, NuGet, PyPI, etc.
- **REST API**: JAX-RS with Swagger documentation
- **Webhook System**: Outbound event notifications

3.7.2 CI/CD Integration

- **Repository Connectors**: Jenkins, GitHub Actions, GitLab CI, etc.
- **API-based Automation**: Script-driven repository configuration
- **Token Authentication**: Secure API access

3.7.3 Extensibility

- **OSGi Plugin System**: Dynamic loading of extensions
- **Script Plugin**: Groovy-based administrative scripting
- **Format API**: Extensible repository format support

3.8 TECHNOLOGY CONSTRAINTS

3.8.1 Runtime Requirements (updated)

- **Java 21+**: Minimum JDK version for all deployment options
- **Memory**: Minimum 4GB for production use
- **Disk Space**: Varies based on repository size and format

3.8.2 Compatibility Requirements

- **Client Tools**: Compatible with standard build tools (Maven, npm, Docker, etc.)
- **Browser Support**: Modern web browsers (Chrome, Firefox, Safari, Edge)
- **Operating Systems**: Linux, Windows, macOS for standalone deployments

3.8.3 Security Constraints

- **TLS Requirements**: TLS 1.2+ for all communications
- **Authentication**: Strong password policies, optional multi-factor authentication
- **Authorization**: Role-based access control for all operations

The technology stack described above provides a robust, scalable foundation for Sonatype Nexus Repository, supporting its multi-format repository management capabilities while enabling extensibility and integration with modern development ecosystems.

4. PROCESS FLOWCHART

4.1 CORE BUSINESS PROCESSES

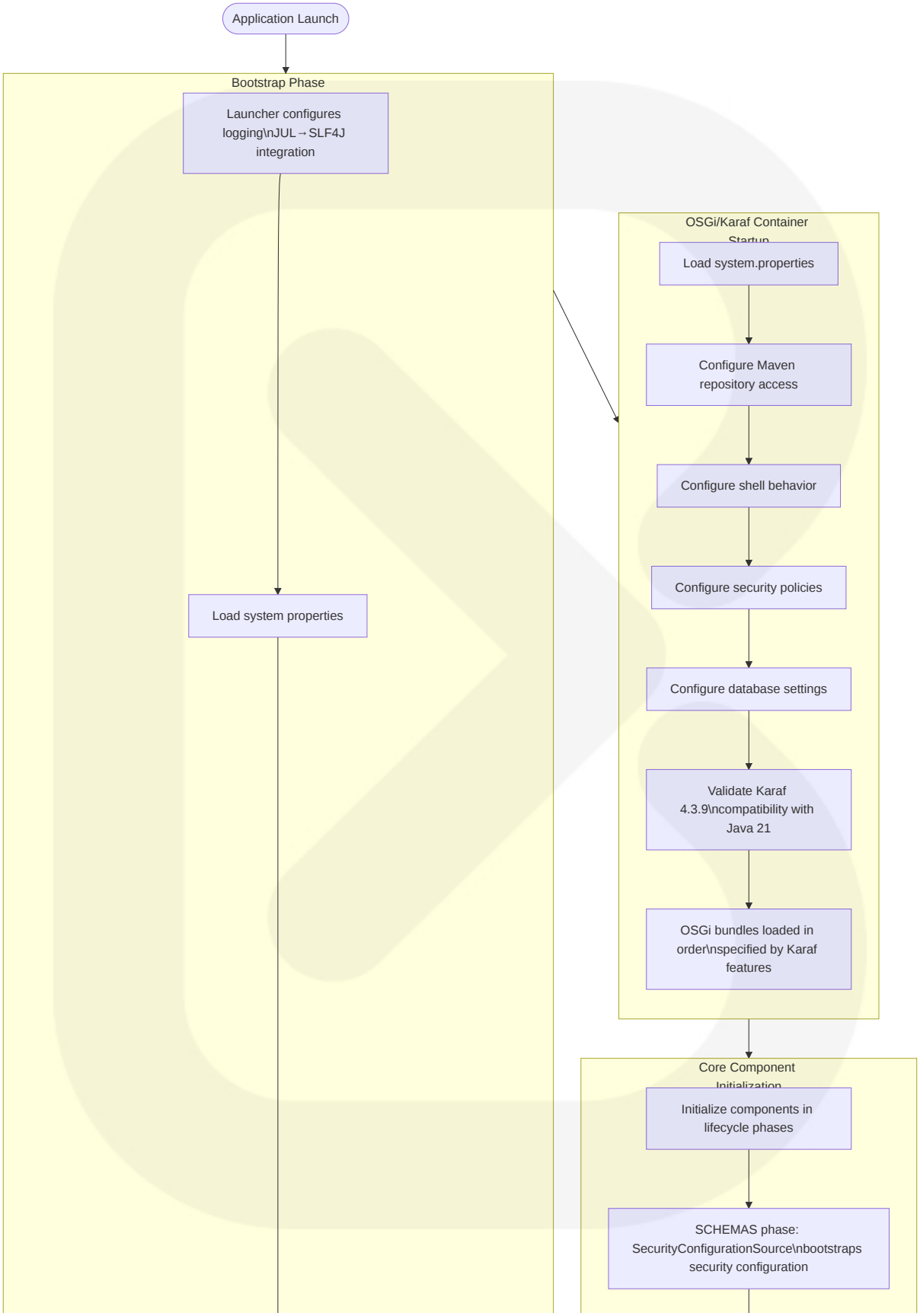
This section describes the fundamental operational workflows that form the backbone of the Nexus Repository system.

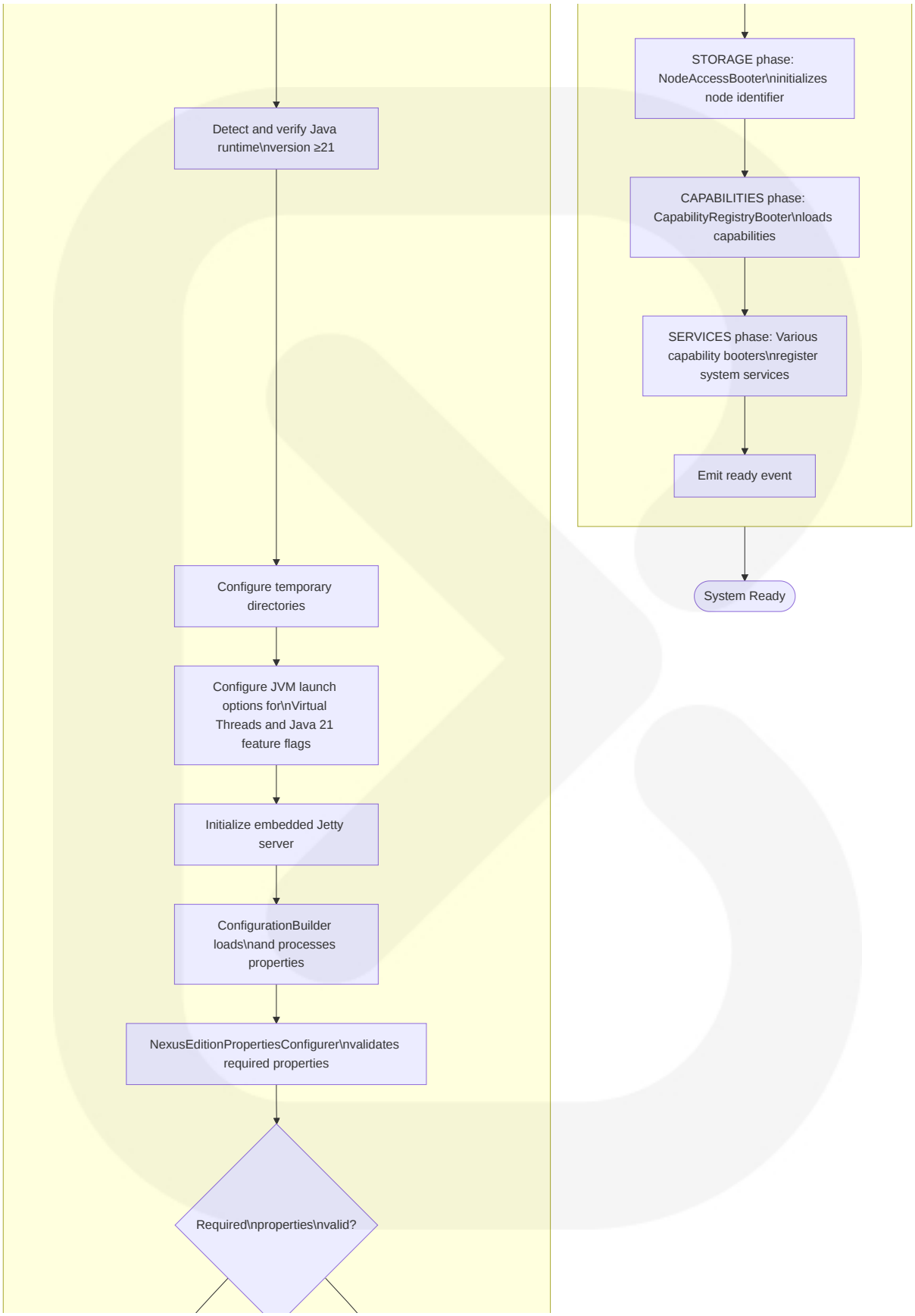
4.1.1 System Initialization and Startup (updated)

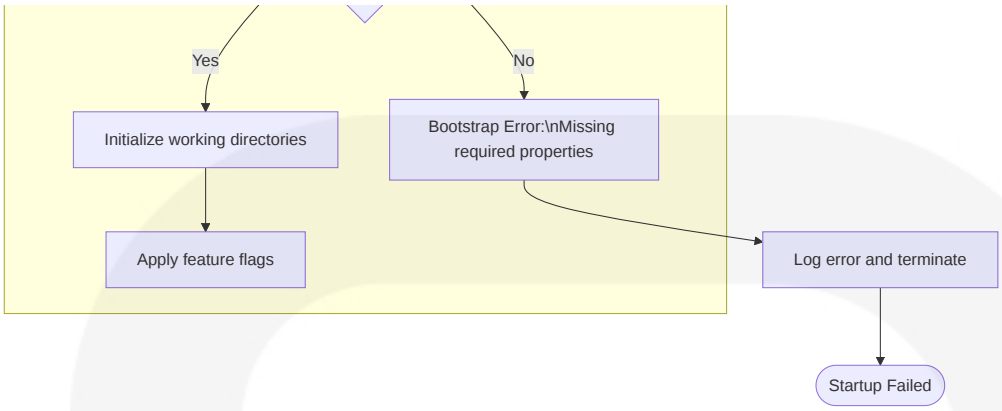
The Nexus Repository initialization and startup process follows a well-defined sequence of operations to bootstrap the application, initialize the OSGi container, and prepare all core components for operation.

The initialization flow consists of three major phases:

1. **Bootstrap Phase**: Initial system configuration and environment preparation
2. **OSGi/Karaf Container Startup**: Framework initialization and bundle loading
3. **Core Component Initialization**: Systematic initialization of application components in a controlled sequence


Application Launch
Bootstrap Phase
Launcher configures logging\nJUL→SLF4J integration
Load system properties
OSGi/Karaf Container Startup
Load system.properties
Configure Maven repository access
Configure shell behavior
Configure security policies
Configure database settings
Validate Karaf 4.3.9\ncompatibility with Java 21
OSGi bundles loaded in order\nspecified by Karaf features
Core Component Initialization
Initialize components in lifecycle phases
SCHEMAS phase: SecurityConfigurationSource\nbootstraps security configuration

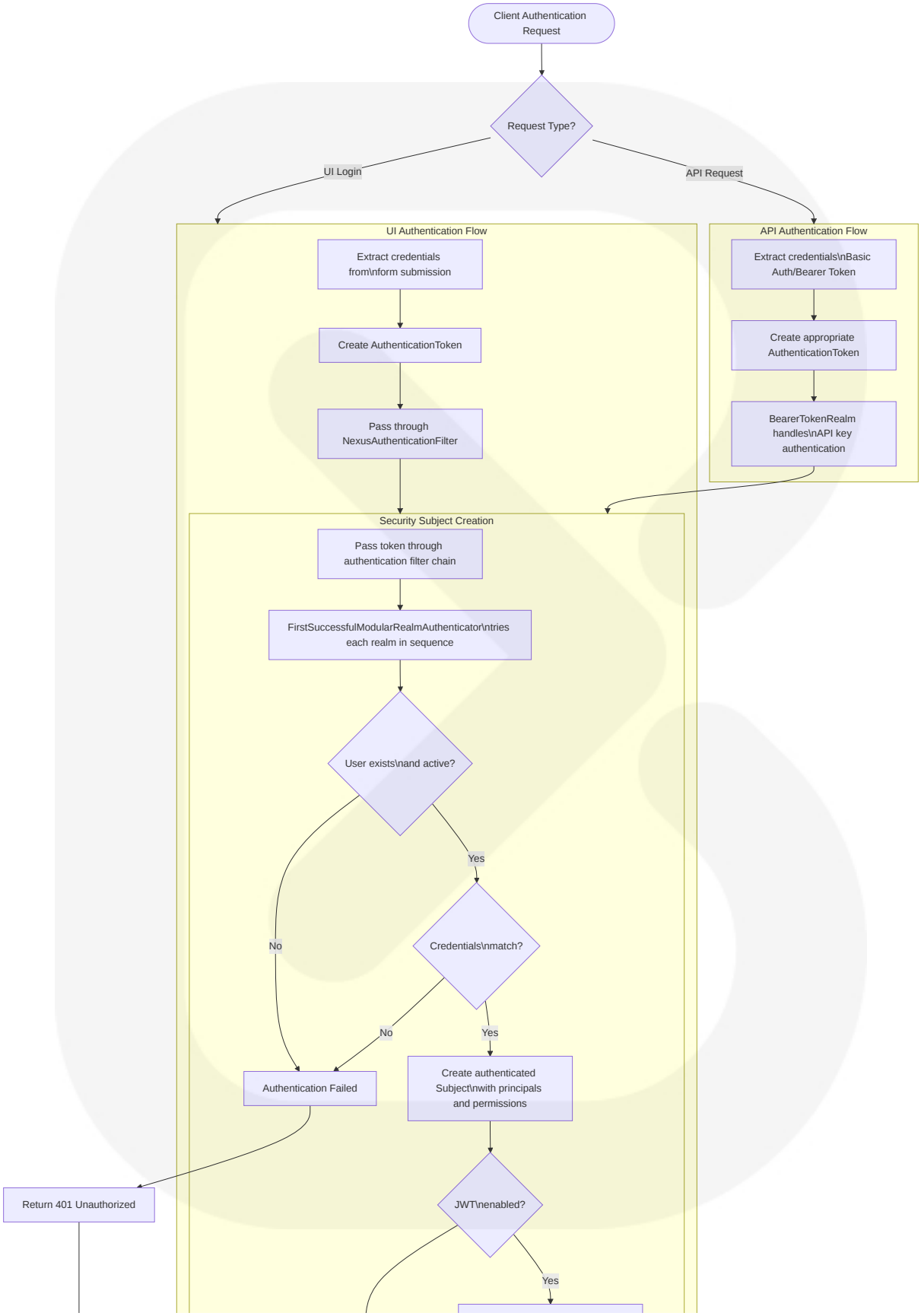

Detect and verify Java runtime\nversion ≥21
Configure temporary directories
Configure JVM launch options for\nVirtual Threads and Java 21 feature flags
Initialize embedded Jetty server
ConfigurationBuilder loads\nand processes properties
NexusEditionPropertiesConfigurer\nvalidates required properties
Required\nproperties\nvalid?
STORAGE phase: NodeAccessBooter\ninitializes node identifier
CAPABILITIES phase: CapabilityRegistryBooter\nloads capabilities
SERVICES phase: Various capability booters\nregister system services
Emit ready event
System Ready

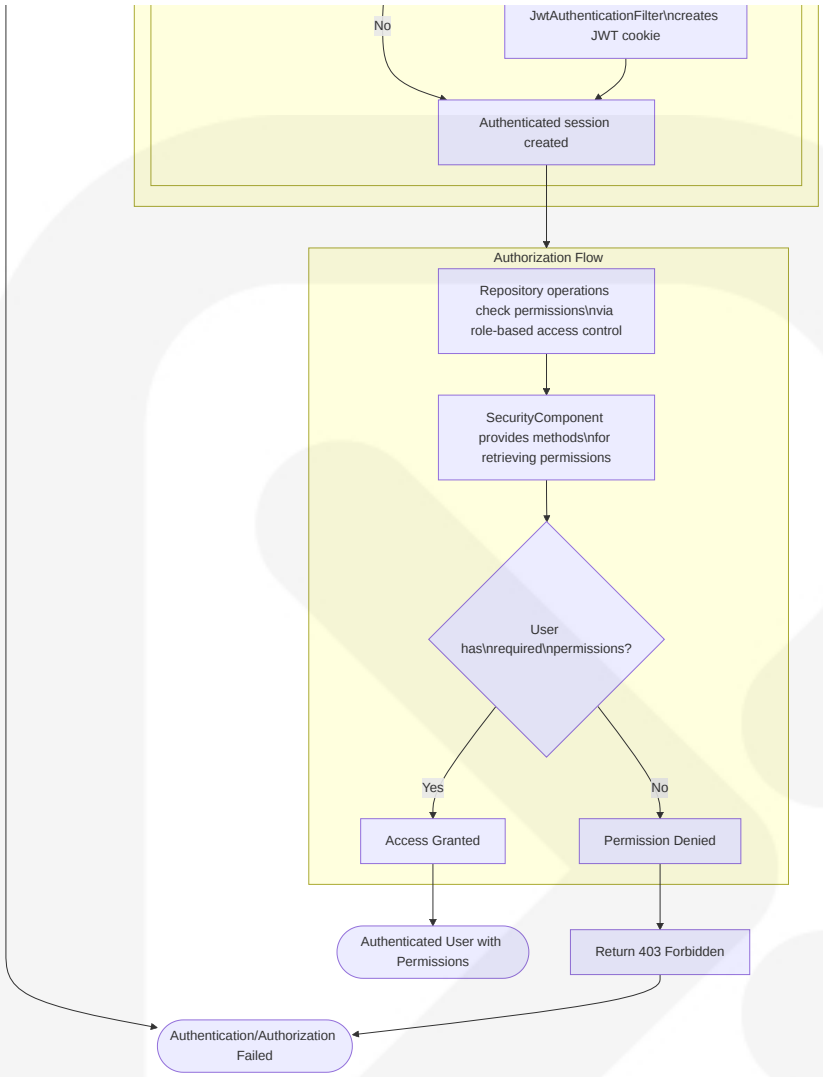


The initialization process ensures that all components are started in the correct order, with proper dependency resolution and configuration loading. The system now performs an early verification of the Java runtime version to ensure compatibility with Java 21 or higher, and configures the JVM to leverage the Virtual Threads capability available in modern Java versions. The use of distinct lifecycle phases (SCHEMAS, STORAGE, CAPABILITIES, SERVICES) enforces a deterministic startup sequence that respects component dependencies. Additionally, the system validates Karaf 4.3.9 compatibility with the Java 21 runtime before proceeding with OSGi bundle loading.

4.1.2 Authentication and Security

The authentication and security workflow manages user identity validation, credential verification, session management, and authorization enforcement throughout the Nexus Repository system.

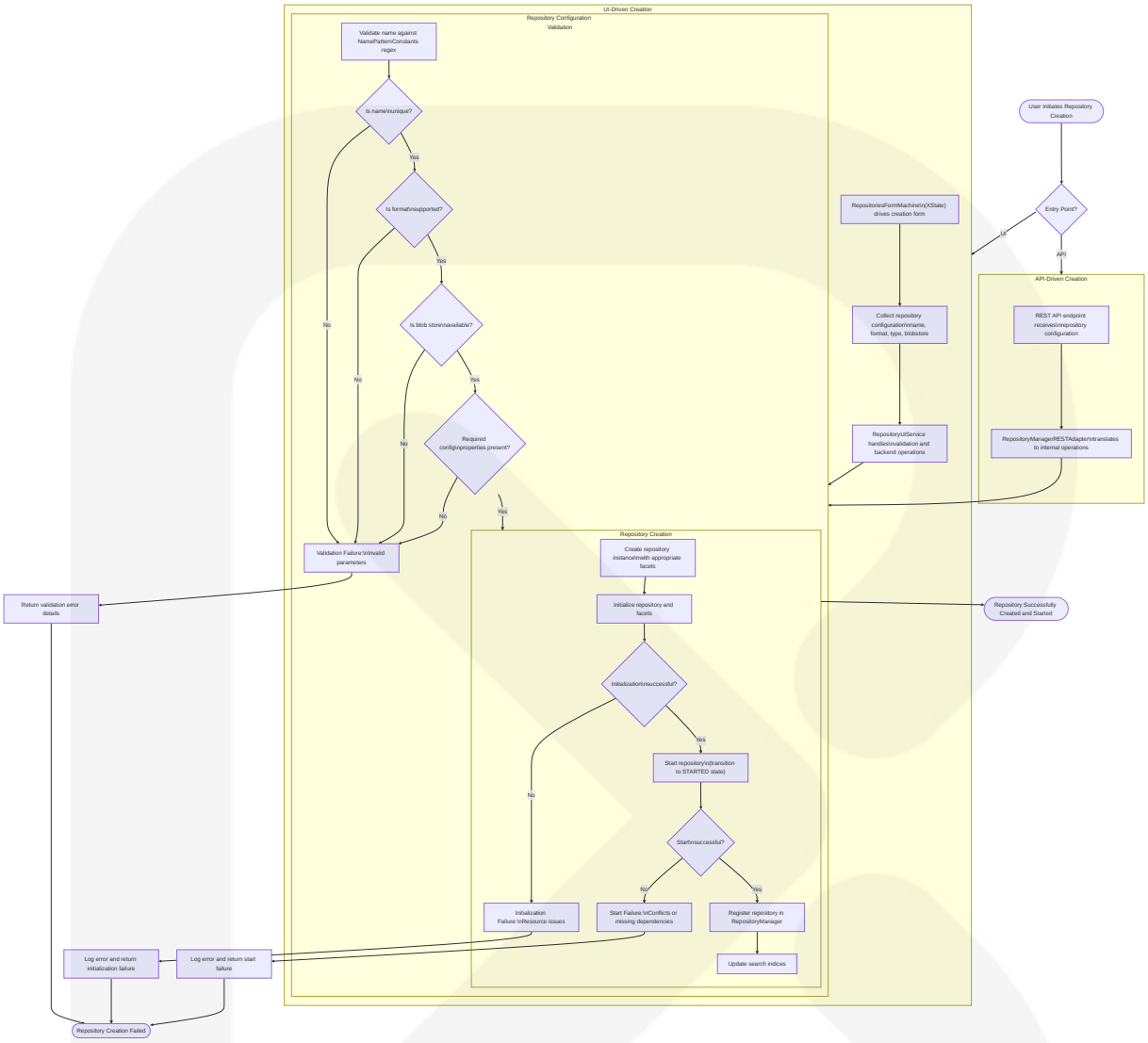

Client Authentication Request
Request Type?
UI Login
API Request
UI Authentication Flow
Extract credentials from\nform submission
Create AuthenticationToken
Pass through NexusAuthenticationFilter
API Authentication Flow
Extract credentials\nBasic Auth/Bearer Token
Create appropriate AuthenticationToken
BearerTokenRealm handles\nAPI key authentication
Security Subject Creation
Pass token through authentication filter chain
FirstSuccessfulModularRealmAuthenticator\ntries each realm in sequence
User exists\nand active?
Yes
No
Credentials\nmatch?
No
Yes
Authentication Failed
Create authenticated Subject\nwith principals and permissions
Return 401 Unauthorized
JWT\nenabled?
Yes



The authentication system uses a flexible, multi-realm approach that supports various authentication methods (basic auth, form-based, API keys, JWT tokens) and pluggable realms. The authorization system enforces role-based access control at the repository and function level.

4.1.3 Repository Management

The repository management workflow handles the creation, configuration, and lifecycle management of repositories within the Nexus system.

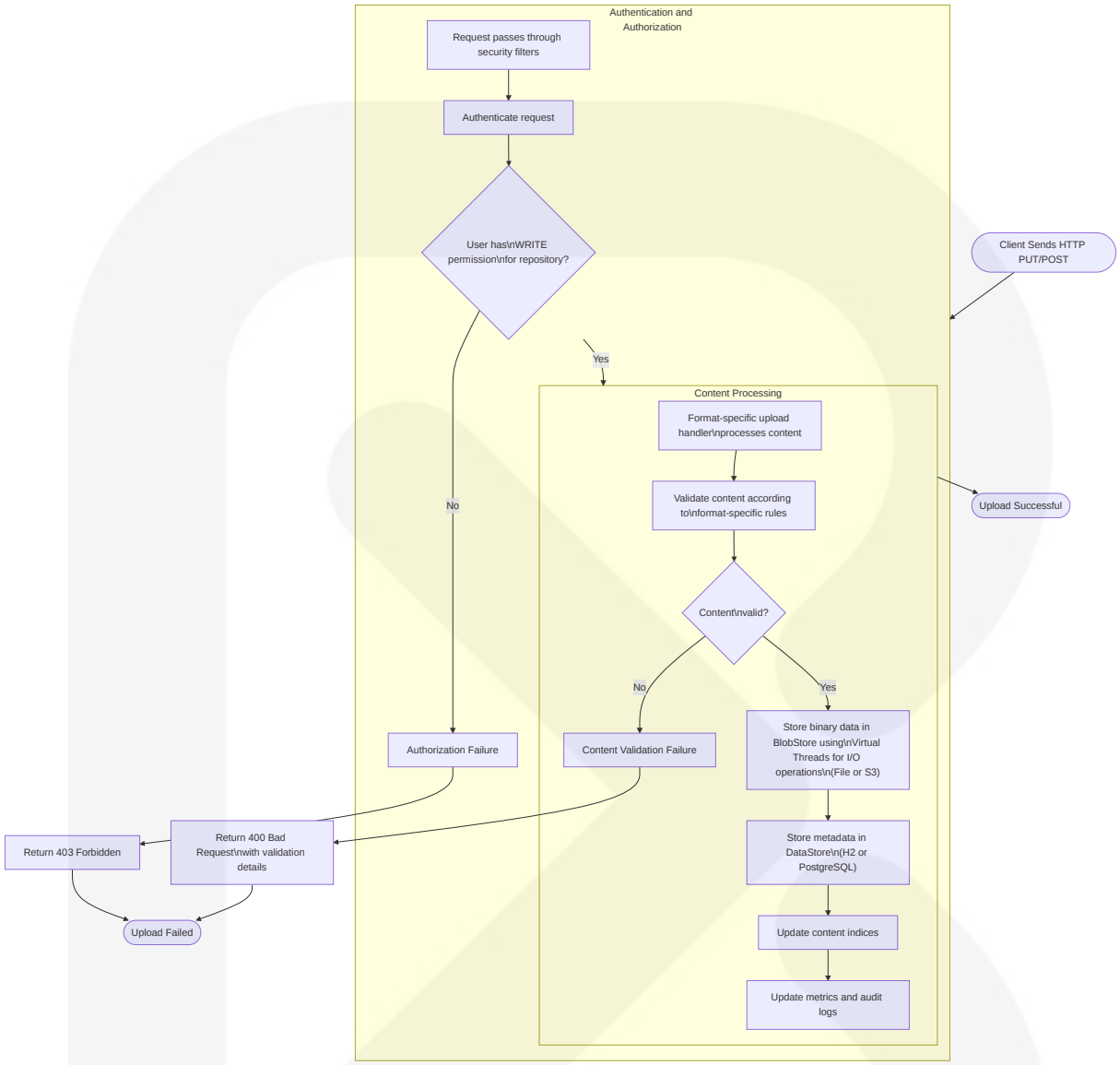


The repository management process includes validation of input parameters, creation of the repository instance with appropriate format-specific facets, initialization of the repository and its components, and state transitions that follow a well-defined lifecycle.

4.1.4 Artifact Upload/Download (updated)

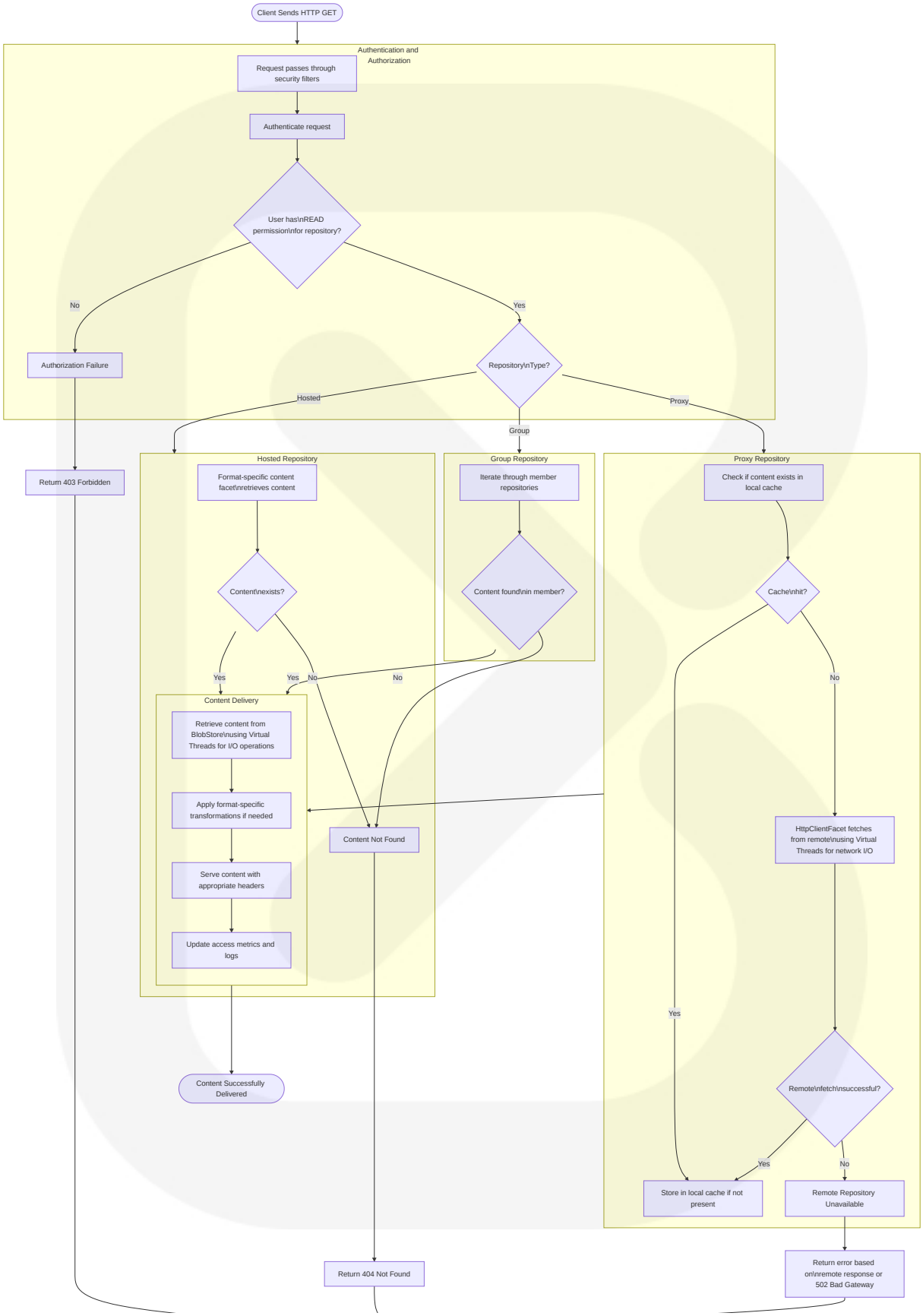
The artifact upload and download workflows manage the core content operations within Nexus Repository, handling security, storage, retrieval, and metadata management.

4.1.4.1 Artifact Upload Process (updated)



The artifact upload process now leverages Java 21 Virtual Threads to efficiently handle I/O-bound operations when storing binary data in the BlobStore. This improves scalability by allowing the system to handle more concurrent uploads without consuming excessive platform thread resources.

4.1.4.2 Artifact Download Process (updated)


Client Sends HTTP GET
Authentication and Authorization
Request passes through security filters
Authenticate request
User has\nREAD permission\nfor repository?
No
Yes
Authorization Failure
Repository\nType?
Hosted
Proxy
Group
Return 403 Forbidden
Hosted Repository
Format-specific content facet\nretrieves content
Content\nexists?
Yes
Yes
No
No
Content Delivery
Retrieve content from BlobStore\nusing Virtual Threads for I/O operations
Apply format-specific transformations if needed
Serve content with appropriate headers
Update access metrics and logs
Content Not Found
Content Successfully Delivered
Group Repository
Iterate through member repositories
Content found\nin member?
Proxy Repository
Check if content exists in local cache
Cache\nhit?
No
HttpClientFacet fetches from remote\nusing Virtual Threads for network I/O
Yes
Remote\nfetch\nsuccessful?
Yes
No
Store in local cache if not present
Remote Repository Unavailable
Return error based on\nremote response or 502 Bad Gateway
Return 404 Not Found

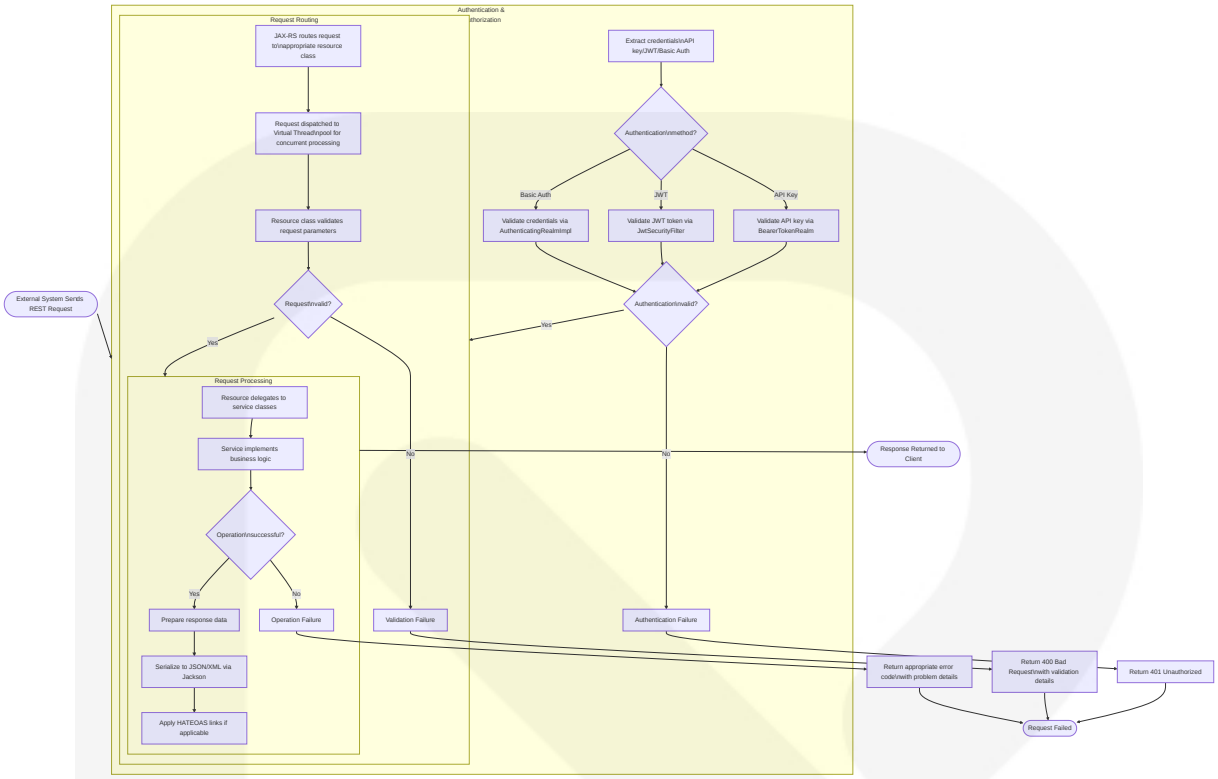
The artifact workflows demonstrate the multi-layered approach to content management in Nexus Repository, with security enforcement, format-specific handling, storage abstraction, and comprehensive logging and metrics collection. Both remote content fetching via HttpClientFacet and local content retrieval from the BlobStore now utilize Java 21 Virtual Threads to improve throughput and resource utilization. This enhancement allows the system to handle more concurrent downloads efficiently, especially for proxy repositories that may involve significant network I/O operations.

4.2 INTEGRATION WORKFLOWS

This section describes how Nexus Repository integrates with external systems and components through standardized interfaces and protocols.

4.2.1 REST API Integration (updated)

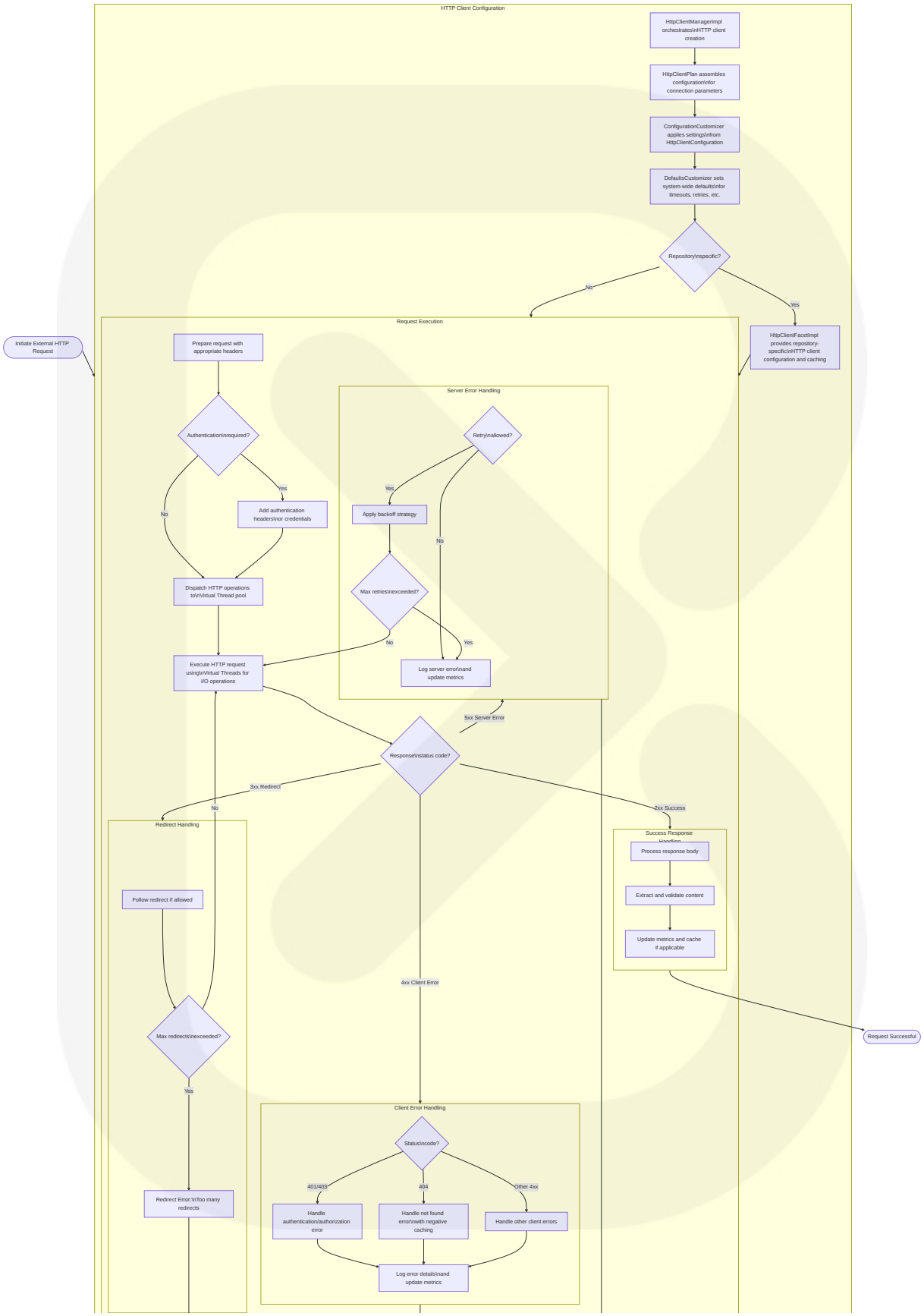
The REST API integration workflow illustrates how external systems interact with Nexus Repository through its JAX-RS-based API endpoints.

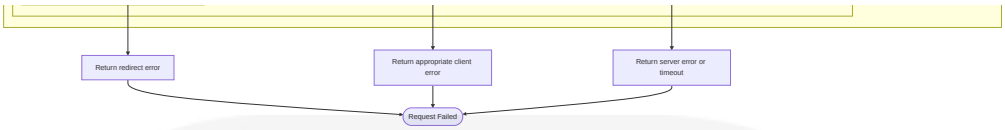


The REST API provides a standardized interface for external systems to interact with Nexus Repository, with robust authentication, validation, and error handling. Each API request is now dispatched to a Virtual Thread executor for processing, leveraging Java 21's lightweight thread implementation to improve throughput and scalability. The API follows RESTful principles and uses JSON/XML for data interchange. This Virtual Thread approach allows the system to handle thousands of concurrent API requests efficiently without exhausting platform thread resources, resulting in better performance under high load conditions.

4.2.2 HTTP Client Integration (updated)

The HTTP client integration workflow demonstrates how Nexus Repository communicates with external systems, particularly for proxy repositories and external services.


HTTP Client Configuration
HttpClientManagerImpl orchestrates\nHTTP client creation
HttpClientPlan assembles configuration\nfor connection parameters
ConfigurationCustomizer applies settings\nfrom HttpClientConfiguration
DefaultsCustomizer sets system-wide defaults\nfor timeouts, retries, etc.
Repository\nspecific?
No
Yes
Request Execution
Initiate External HTTP Request
Prepare request with appropriate headers
HttpClientFacetImpl provides repository-specific\nHTTP client configuration and caching
Server Error Handling
Authentication\nrequired?
Retry\nallowed?
Yes
Add authentication headers\nor credentials
Apply backoff strategy
No
Dispatch HTTP operations to\nVirtual Thread pool
Max retries\nexceeded?
Yes
No
Execute HTTP request using\nVirtual Threads for I/O operations
Log server error\nand update metrics
5xx Server Error
Response\nstatus code?
3xx Redirect
No
2xx Success
Redirect Handling
Success Response
Process response body
Follow redirect if allowed
Extract and validate content
Update metrics and cache if applicable
Max redirects\nexceeded?
4xx Client Error
Request Successful
Yes
Client Error Handling
Status\ncode?
401/403
404
Other 4xx
Redirect Error:\nToo many redirects
Handle authentication/authorization error
Handle not found error\nwith negative caching
Handle other client errors
Log error details\nand update metrics



The HTTP client integration provides robust capabilities for connecting to external systems, with configurable connection parameters, authentication methods, retry strategies, and error handling tailored to different response types. With the adoption of Java 21, the HTTP client now leverages Virtual Threads for all network I/O operations, significantly improving the system's ability to handle many concurrent external connections efficiently. This approach is particularly beneficial for proxy repositories that need to maintain numerous concurrent connections to remote package registries while minimizing resource consumption.

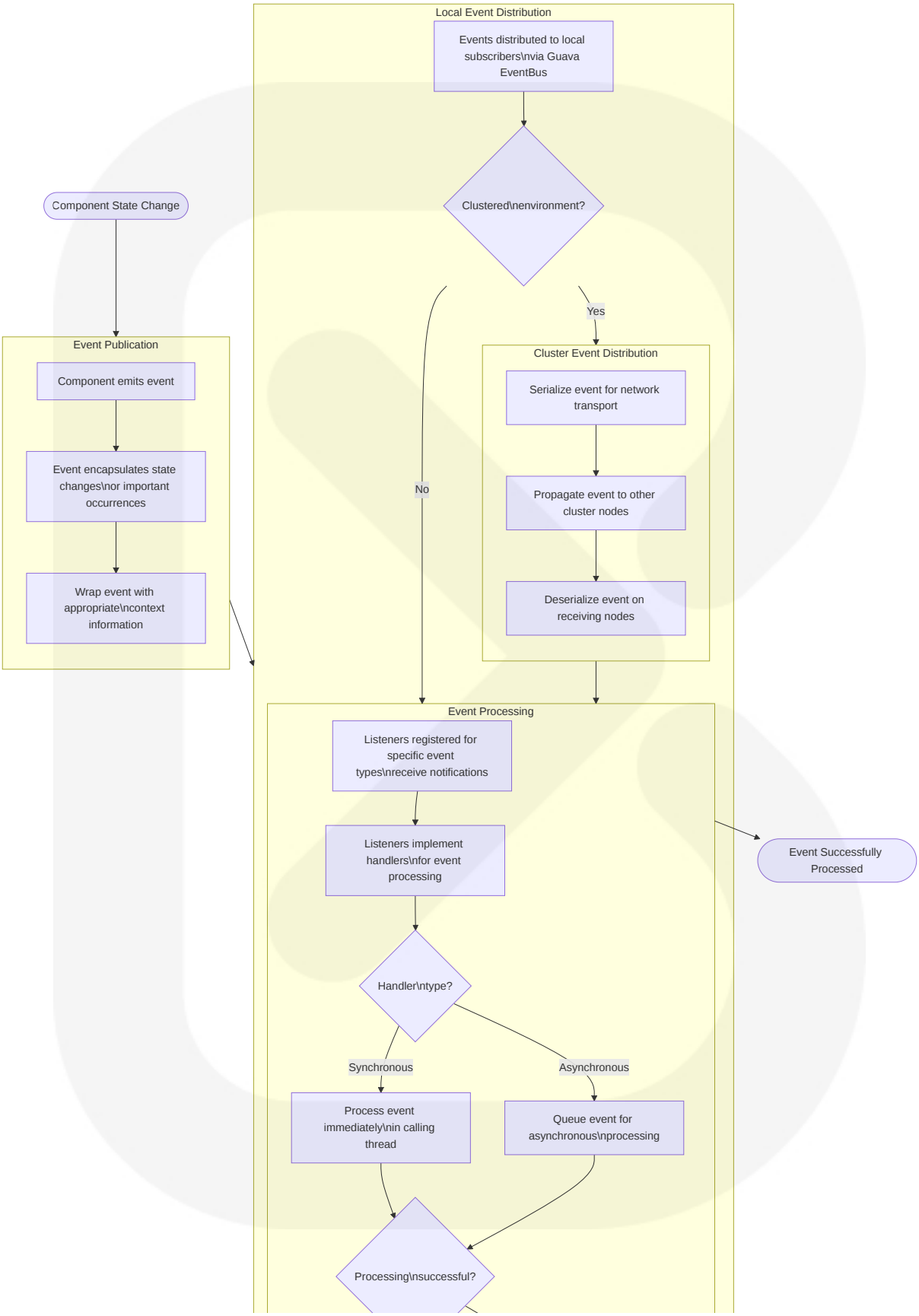
4.2.3 Scheduled Tasks

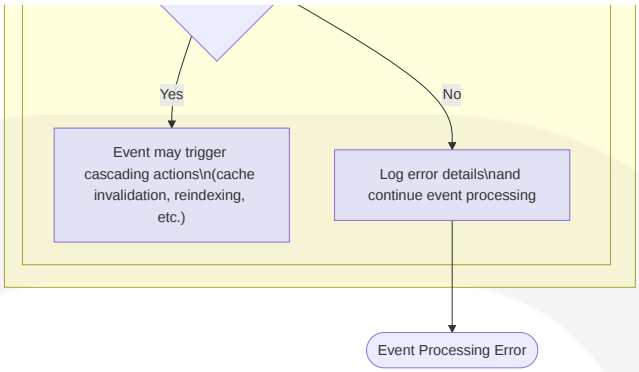
The scheduled tasks workflow illustrates how Nexus Repository manages and executes periodic maintenance and operational tasks.

The scheduled tasks system provides a flexible framework for periodic operations, with support for cron-like scheduling, manual execution, cluster-aware execution, dedicated task logging, and comprehensive execution history.

4.2.4 Event Processing

The event processing workflow demonstrates how events are published, distributed, and handled within the Nexus Repository system.


Component State Change
Event Publication
Component emits event
Event encapsulates state changes\nor important occurrences
Wrap event with appropriate\ncontext information
Local Event Distribution
Events distributed to local subscribers\nvia Guava EventBus
Clustered\nenvironment?
Yes
No
Cluster Event Distribution
Serialize event for network transport
Propagate event to other cluster nodes
Deserialize event on receiving nodes
Event Processing
Listeners registered for specific event types\nreceive notifications
Listeners implement handlers\nfor event processing
Handler\ntype?
Synchronous
Asynchronous
Process event immediately\nin calling thread
Queue event for asynchronous\nprocessing
Processing\nsuccessful?
Event Successfully Processed



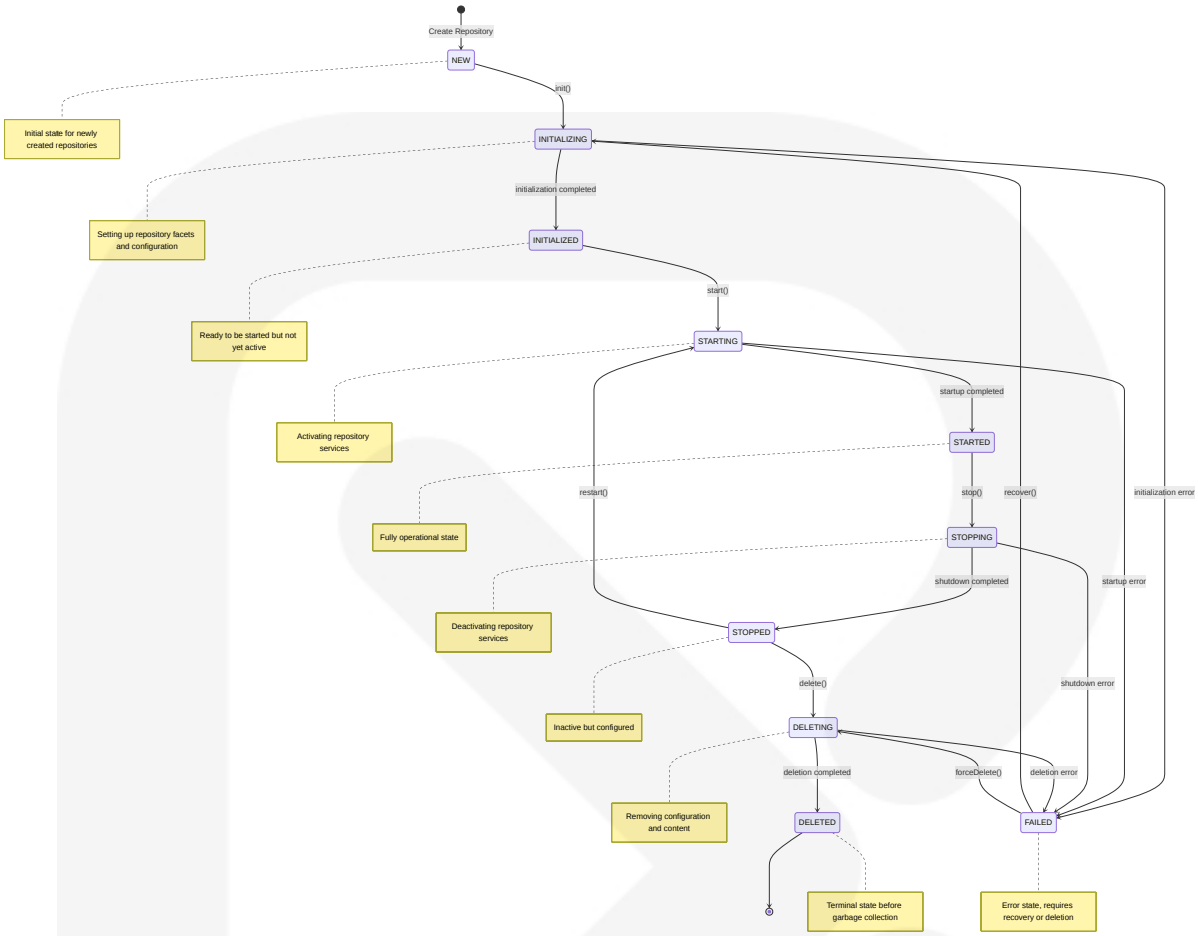
The event system provides a decoupled mechanism for components to communicate state changes and trigger responsive actions throughout the system, with support for both local and cluster-wide event distribution.

4.3 STATE MANAGEMENT

This section describes how Nexus Repository manages state transitions for various system components.

4.3.1 Repository Lifecycle States

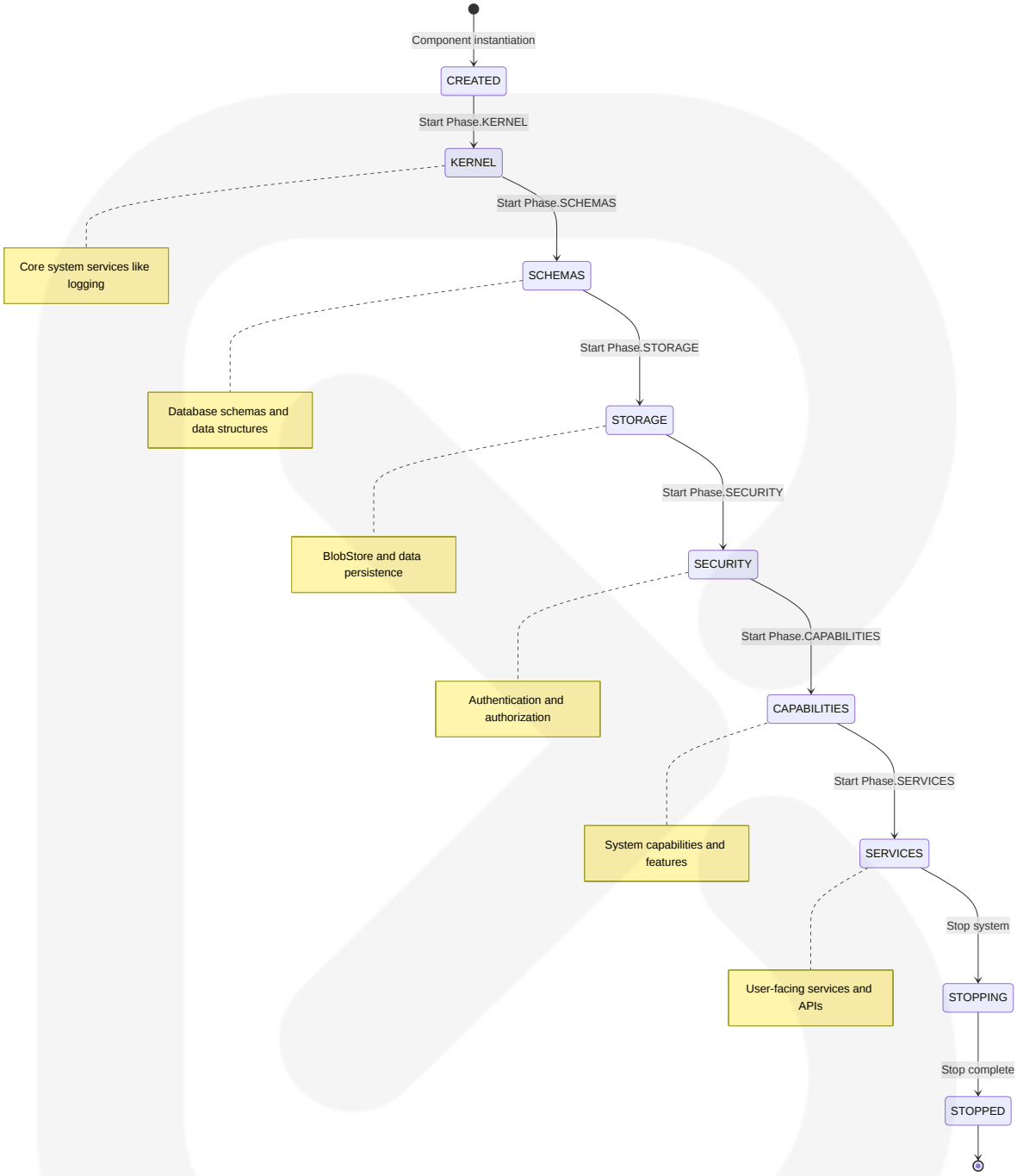
The repository lifecycle state diagram illustrates the valid states and transitions for repositories within the Nexus system.



Repository state transitions are enforced by the `StateGuard` mechanism in `RepositoryImpl`, with `@Guarded` and `@Transitions` annotations ensuring valid transitions between states. This strictly controlled lifecycle ensures that repositories maintain data integrity during configuration changes and system operations.

4.3.2 Component Lifecycle Management

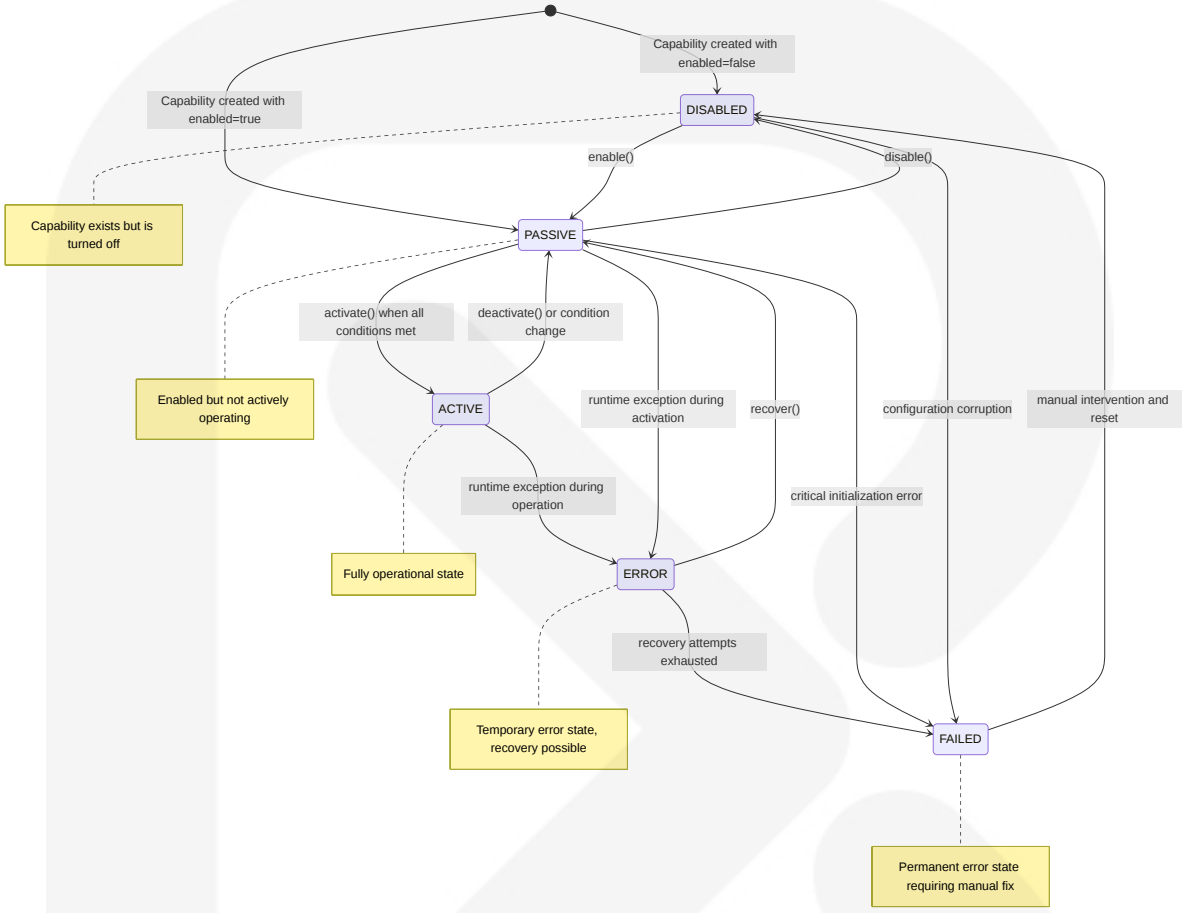
The component lifecycle management diagram illustrates how Nexus manages the startup order and lifecycle of core system components.



The component lifecycle is managed through the `ManagedLifecycle` annotation, which defines the phase in which a component should be started. Components are initialized in the order of their lifecycle phases, ensuring that dependencies are properly established before dependent components are started.

4.3.3 Capability State Management

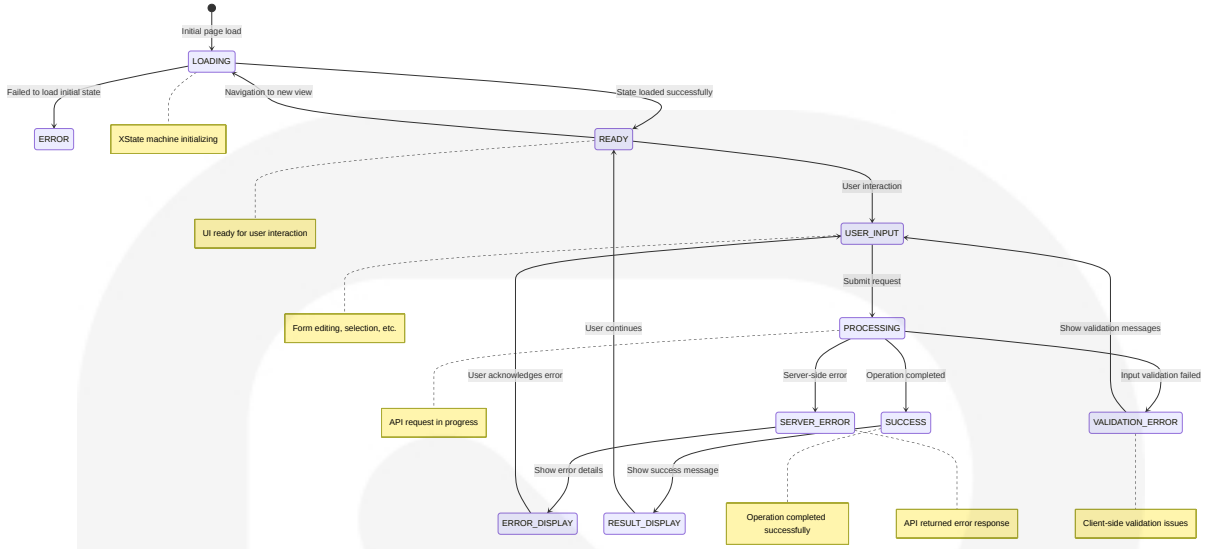
The capability state management diagram illustrates the states and transitions for system capabilities within Nexus Repository.



Capability state management is handled by the `CapabilityRegistryBooter`, which initializes capabilities during system startup. Capabilities transition between states based on configuration changes, system conditions, and operational status, allowing features to be dynamically enabled or disabled.

4.3.4 User Interface State Management

The user interface state management diagram illustrates how the Nexus web interface manages state transitions and user interactions.



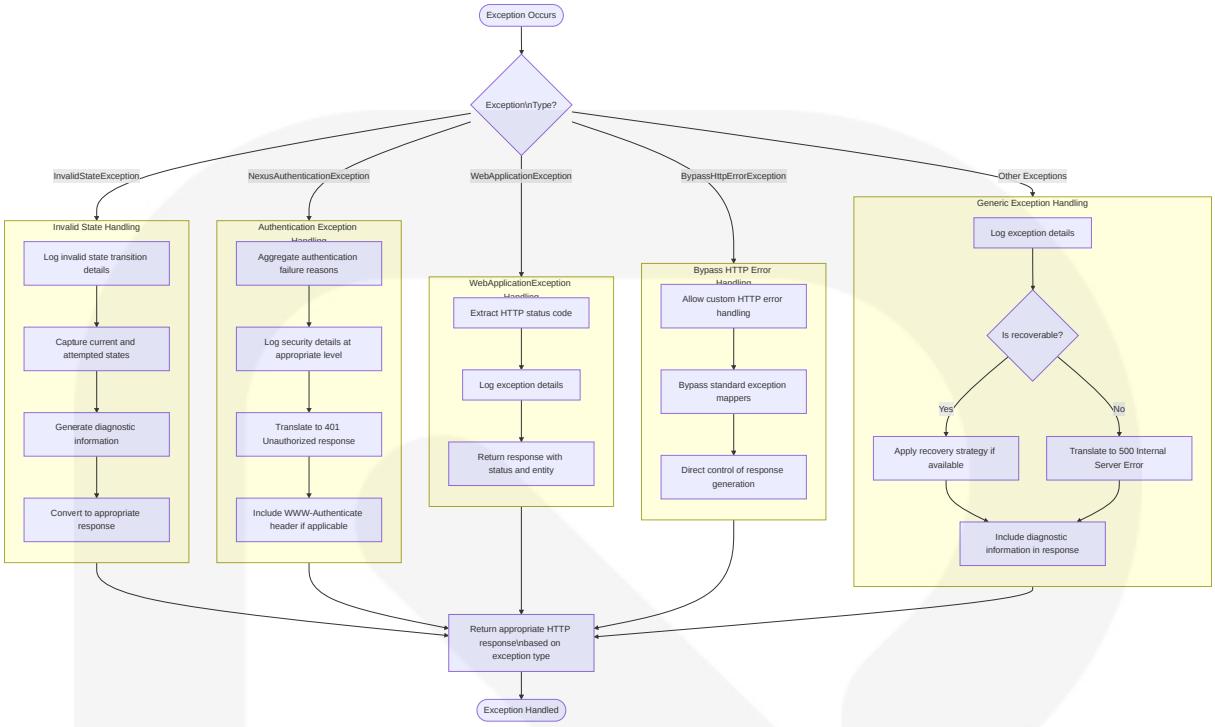
The UI state management is implemented using XState machines, such as `RepositoriesFormMachine`, which define the possible states and transitions for UI components. This state-based approach ensures consistent UI behavior and predictable user interactions across the application.

4.4 ERROR HANDLING WORKFLOWS

This section describes how Nexus Repository manages errors and exceptions throughout the system.

4.4.1 Exception Handling

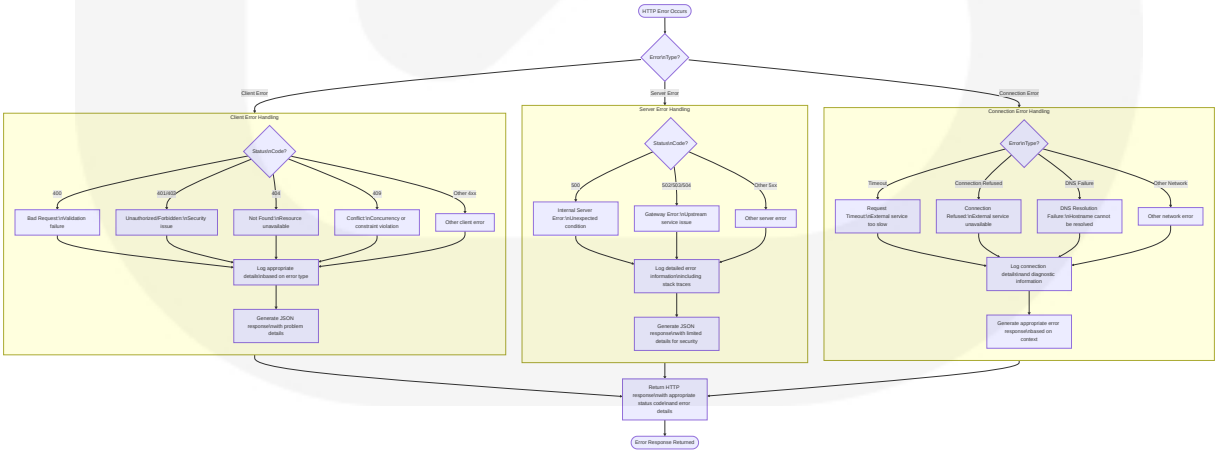
The exception handling diagram illustrates the hierarchy and flow of exceptions within the Nexus Repository system.



Exception handling in Nexus Repository follows a hierarchical approach, with specialized handlers for different types of exceptions. This ensures that errors are properly logged, diagnostic information is captured, and appropriate responses are generated for client applications.

4.4.2 HTTP Error Handling

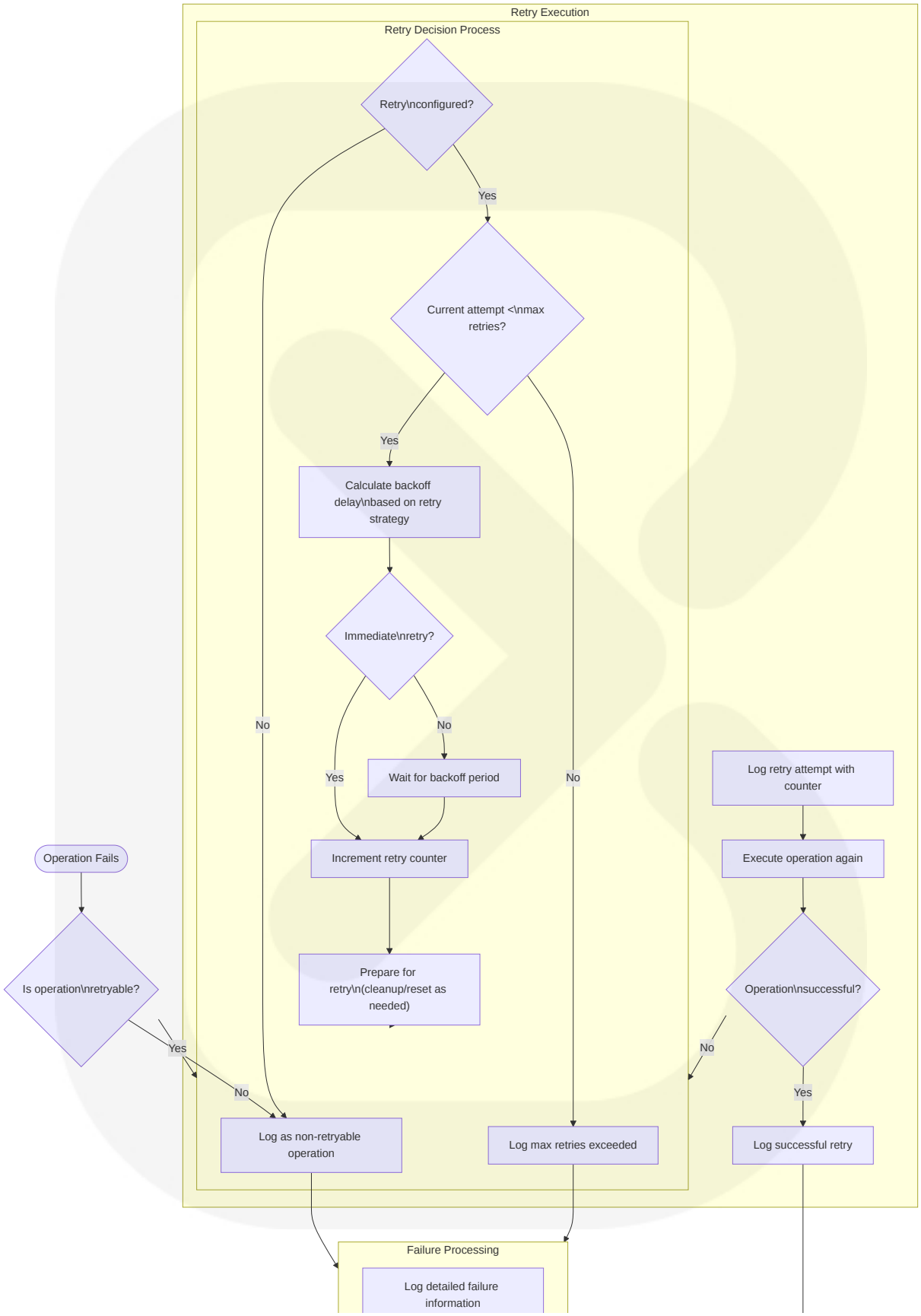
The HTTP error handling diagram illustrates how HTTP-related errors are processed and responded to.

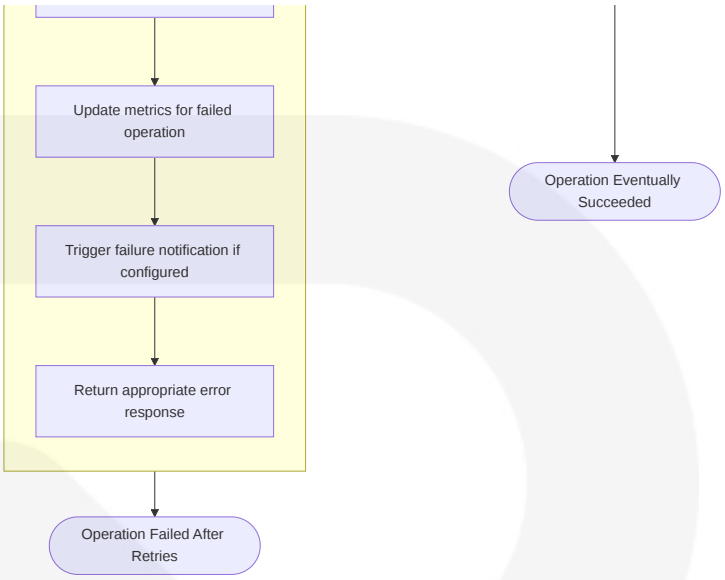


HTTP error handling in Nexus Repository provides comprehensive management of various error scenarios, with appropriate logging, diagnostic information capture, and client response generation tailored to the specific error type.

4.4.3 Retry Mechanisms

The retry mechanisms diagram illustrates how Nexus Repository handles retryable operations and failure recovery.

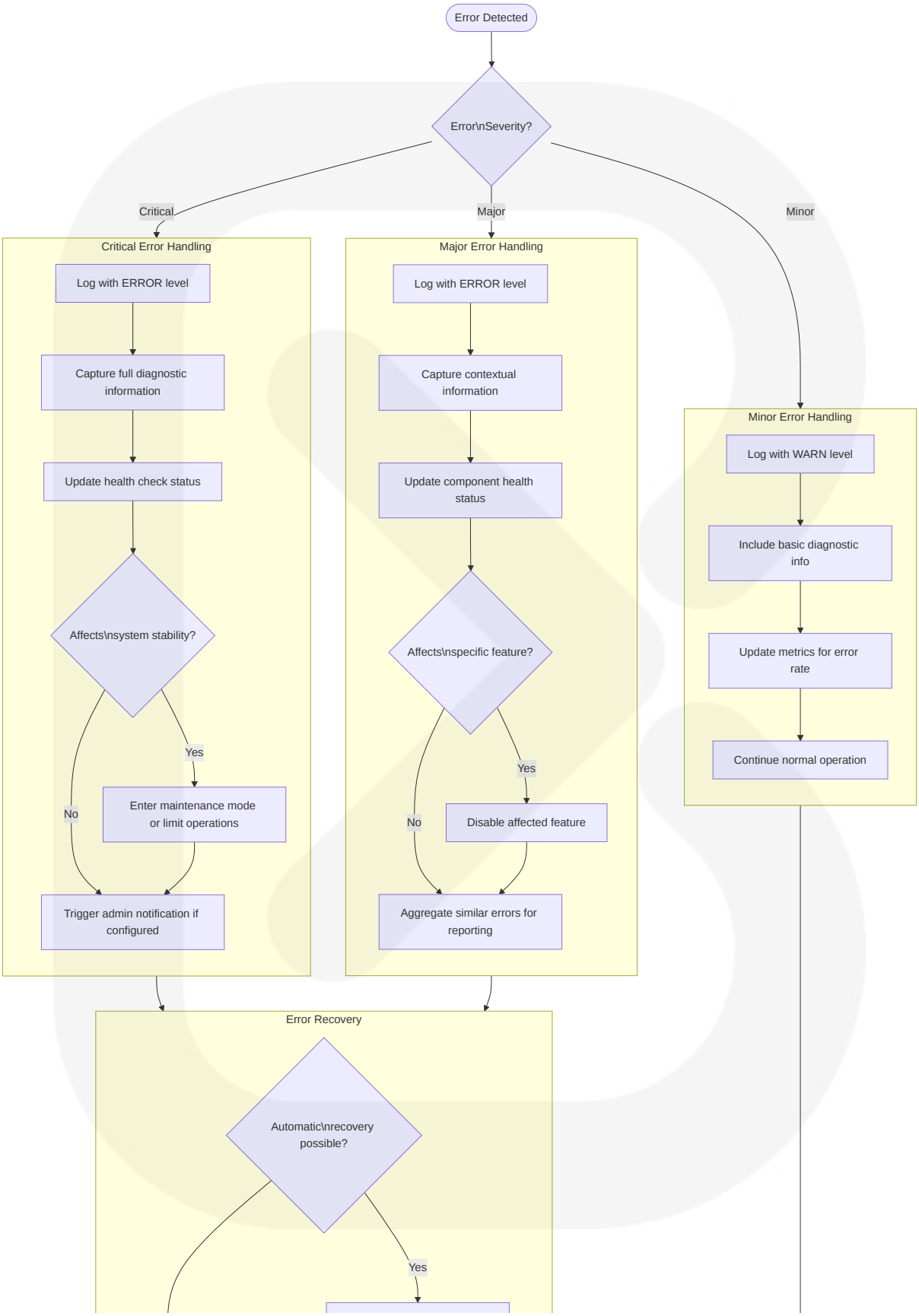

Retry Execution
Retry Decision Process
Retry\nconfigured?
Yes
No
Current attempt <\nmax retries?
Yes
No
Calculate backoff delay\nbased on retry strategy
Immediate\nretry?
Yes
No
Wait for backoff period
Increment retry counter
Prepare for retry\n(cleanup/reset as needed)
Operation Fails
Is operation\nretryable?
Yes
No
Log as non-retryable operation
Log max retries exceeded
Log retry attempt with counter
Execute operation again
Operation\nsuccessful?
No
Yes
Log successful retry
Failure Processing
Log detailed failure information

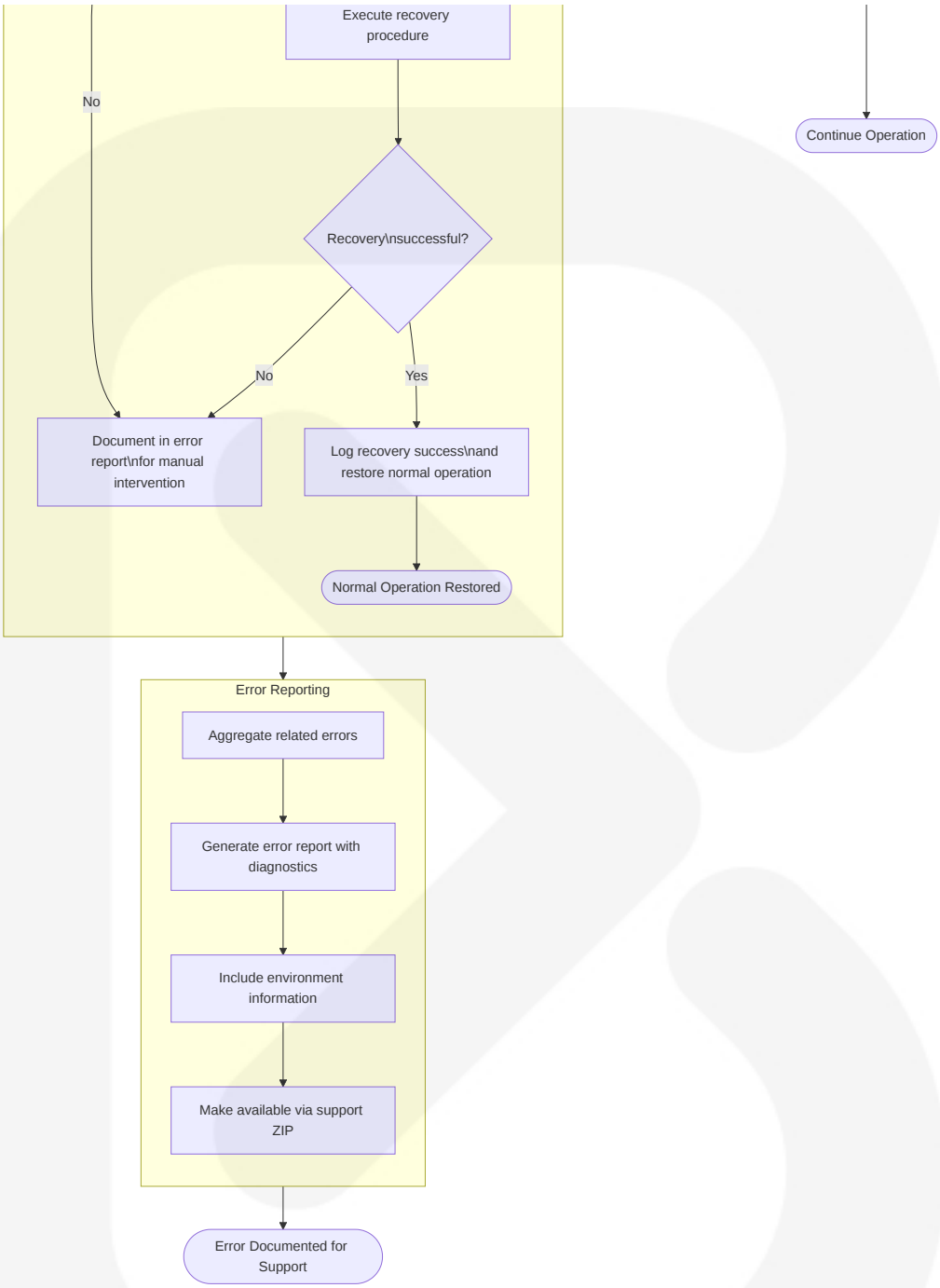


Retry mechanisms in Nexus Repository provide robust handling of transient failures, with configurable retry policies, backoff strategies, and comprehensive logging to aid in diagnosing intermittent issues.

4.4.4 Error Recovery and Reporting

The error recovery and reporting diagram illustrates how Nexus Repository handles error recovery, escalation, and reporting.


Error Detected
Error\nSeverity?
Critical
Major
Minor
Critical Error Handling
Log with ERROR level
Capture full diagnostic information
Update health check status
Affects\nsystem stability?
Yes
No
Enter maintenance mode or limit operations
Trigger admin notification if configured
Major Error Handling
Log with ERROR level
Capture contextual information
Update component health status
Affects\nspecific feature?
Yes
No
Disable affected feature
Aggregate similar errors for reporting
Minor Error Handling
Log with WARN level
Include basic diagnostic info
Update metrics for error rate
Continue normal operation
Error Recovery
Automatic\nrecovery possible?
Yes



Error recovery and reporting in Nexus Repository provides comprehensive handling of various error scenarios, with severity-based processing, automatic recovery where possible, and detailed diagnostic information collection for support and troubleshooting.

The process flowcharts presented in this section illustrate the key operational workflows within the Sonatype Nexus Repository system. These diagrams provide a

clear understanding of the system's runtime behavior, component interactions, decision points, and error handling mechanisms, serving as a valuable reference for both development and operational activities.

4.5 REFERENCES

The process flows documented in this section are based on analysis of the following source files:

- `components/nexus-bootstrap/src/main/java/org/sonatype/nexus/bootstrap/Launcher.java`
- `components/nexus-base/src/main/java/org/sonatype/nexus/internal/node/NodeAccessBooter.java`
- `components/nexus-core/src/main/java/org/sonatype/nexus/internal/capability/CapabilityRegistryBooter.java`
- `components/nexus-repository-config/src/main/java/org/sonatype/nexus/repository/Repository.java`
- `components/nexus-repository-services/src/main/java/org/sonatype/nexus/repository/manager/internal/RepositoryImpl.java`
- `plugins/nexus-coreui-plugin/src/frontend/src/components/pages/admin/Repositories/RepositoriesFormMachine.js`
- `components/nexus-security/src/main/java/org/sonatype/nexus/security/authc/NexusAuthenticationFilter.java`
- `components/nexus-security/src/main/java/org/sonatype/nexus/security/internal/AuthenticatingRealmImpl.java`
- `components/nexus-security/src/main/java/org/sonatype/nexus/security/token/BearerTokenRealm.java`
- `components/nexus-security/src/main/java/org/sonatype/nexus/security/JwtSecurityFilter.j`

`ava`

- `components/nexus-httpclient/src/main/java/org/sonatype/nexus/httpclient/HttpClientManager.java`
- `components/nexus-core/src/main/java/org/sonatype/nexus/internal/httpclient/HttpClientManagerImpl.java`
- `components/nexus-repository-view/src/main/java/org/sonatype/nexus/repository/httpclient/internal/HttpClientFacetImpl.java`
- `components/nexus-common/src/main/java/org/sonatype/nexus/common/stateguard/InvalidStateException.java`
- `components/nexus-repository-view/src/main/java/org/sonatype/nexus/repository/view/payloads/HttpEntityPayload.java`
- `components/nexus-repository-services/src/main/java/org/sonatype/nexus/repository/proxy/BypassHttpErrorException.java`
- `components/nexus-validation/src/main/java/org/sonatype/nexus/validation/ValidationModule.java`
- `components/nexus-rapture/src/main/resources/static/rapture/NX/util/Validator.js`
- `components/nexus-repository-view/src/main/java/org/sonatype/nexus/repository/proxy/ProxyFacetSupport.java` - Target implementation for Virtual Threads in remote repository operations to improve concurrent request handling
- `components/nexus-blobstore-s3/src/main/java/org/sonatype/nexus/blobstore/s3/internal/S3BlobStore.java` - Optimized I/O paths leveraging Virtual Threads for improved throughput in cloud storage operations
- `components/nexus-repository-services/src/main/java/org/sonatype/nexus/repository/upload/internal/UploadManagerImpl.java` - Implementation of Pattern Matching for enhanced content type detection and validation

- `components/nexus-repository-services/src/main/java/org/sonatype/nexus/repository/manager/internal/RepositoryManagerImpl.java` - Pattern Matching refactors for repository type handling and configuration validation
- `components/nexus-rest/src/main/java/org/sonatype/nexus/rest/SimpleApiResponse.java` - Record Pattern enhancements for improved API response handling
- `components/nexus-common/src/main/java/org/sonatype/nexus/common/log/LoggerFactory.java` - Implementation of String Templates for structured logging
- `components/nexus-logging/src/main/java/org/sonatype/nexus/logging/task/TaskLoggerHelper.java` - String Templates usage for task execution logging

5. SYSTEM ARCHITECTURE

5.1 HIGH-LEVEL ARCHITECTURE

5.1.1 System Overview

Sonatype Nexus Repository is built on a modular, service-oriented architecture leveraging the OSGi component model with Apache Karaf as the runtime container validated against Java 21 runtime. This architectural approach delivers several key advantages:

- **Modularity and Extensibility**: The core system is composed of loosely coupled, independently deployable components that communicate through well-defined interfaces. This enables a plugin architecture where new repository formats, capabilities, and integrations can be developed and deployed without modifying the core codebase.

- **Service Orientation**: Components expose and consume services through dependency injection (primarily Google Guice and Eclipse Sisu), allowing for flexible component wiring and runtime service discovery.
- **Dynamic Runtime**: The OSGi/Karaf container provides dynamic loading and unloading of components, hot service replacement, and versioned module boundaries.
- **Multi-format Repository Support**: The architecture abstracts repository format-specific operations behind common interfaces, allowing uniform handling of diverse artifact types (Maven, Docker, npm, etc.) through consistent APIs.
- **Layered Design**: The system follows a clear layering pattern from UI down to storage layers, with well-defined responsibilities at each level.

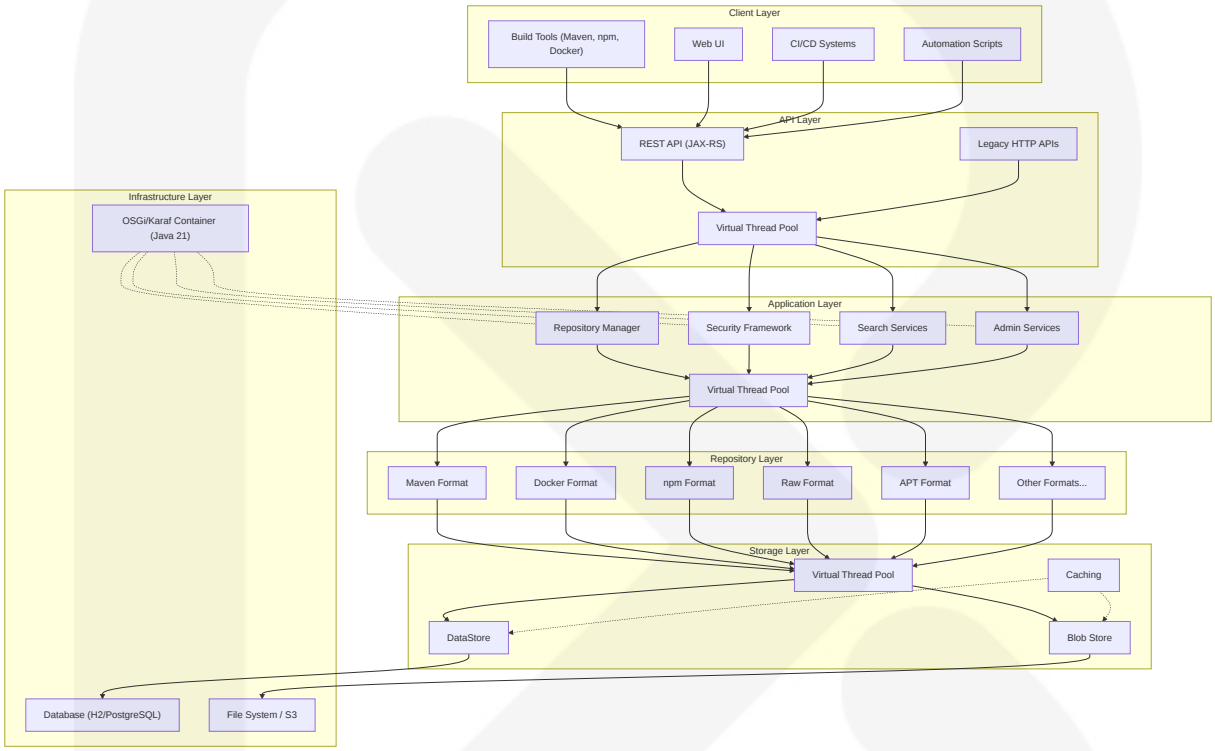
The architecture now targets Java 21, enabling features such as virtual threads, record patterns, pattern matching, string templates, and sequenced collections to improve concurrency and code maintainability.

Key architectural principles include:

1. **Separation of Concerns**: Each component has clearly defined boundaries and responsibilities, with interfaces for cross-component communication.
2. **Pattern-Based Design**: Extensive use of design patterns such as Facet, Repository, Factory, Builder, and Command for consistent implementation approaches.
3. **Aspect-Oriented Integration**: Cross-cutting concerns like security, transactions, and auditing are implemented using interceptors and event-driven patterns.
4. **Multi-Tenancy**: The system supports multiple independent repositories of various formats sharing common infrastructure.
5. **Stateless Services**: Core services are designed to be stateless where possible, with state managed in persistence layers.

6. **Enhanced Concurrency Model**: Leverage Java 21 virtual threads for I/O-bound operations across HTTP, storage, and external integration layers.

The overall architectural style can be described as a modular monolith with service-oriented internals, combining the deployment simplicity of a monolithic application with the component isolation benefits of a service-oriented architecture.



5.1.2 Core Components Table

Component Name	Primary Responsibility	Key Dependencies	Integration Points	Critical Considerations
OSGi/Karaf Framework	Runtime container, bundle lifecycle, service registry	Apache Felix, Apache Karaf, Java JDK 21	Bootstrap sequence, extension loading	Version compatibility, start-level ordering, Java 21 compatibility verification

Component Name	Primary Responsibility	Key Dependencies	Integration Points	Critical Considerations
Bootstrap Module	System initialization, launcher, configuration	Java JDK 21, SLF4J/Logback	OS environment, JVM arguments	File locking, proper shutdown hooks, Java 21 compatibility verification
Core Services	Central services registry, application lifecycle	Nexus Common, Guice/Sisu, Java 21-compatible JDBC drivers	All other components	Initialization ordering, lifecycle phases
Security Framework	Authentication, authorization, user management	Apache Shiro, JWT libraries, Java 21-compatible crypto libraries	HTTP layer, repository access	Secret management, realm coordination
Repository Layer	Repository management, type-specific facets	Repository Config, Content API	Storage layer, HTTP bridge	Format compatibility, migration paths
Storage Layer	Binary and metadata persistence	BlobStore API, DataStore API	Repository layer, backup systems	Transactional integrity, file locking
HTTP Layer	RESTful API and legacy HTTP interfaces	JAX-RS, Siesta, Jackson	Client applications, UI layer	Content negotiation, error handling
UI Framework	Web administration interface	React, ExtJS, Rapture	HTTP/REST API	Browser compatibility, asset bundling

5.1.3 Data Flow Description

Nexus Repository's architecture implements several primary data flows:

1. **Artifact Upload Flow**:

- Client applications submit artifacts via HTTP PUT/POST requests to the repository HTTP endpoint
- The Repository HTTP Bridge routes requests to the appropriate repository based on path
- Format-specific handlers validate and process content according to repository type rules
- Content is stored as a blob in the BlobStore (file system or S3) using virtual threads for improved throughput
- Metadata is extracted and stored in the DataStore (H2 or PostgreSQL) via virtual threads
- Search indices are updated to reflect new content
- Event notifications are dispatched for audit and webhook triggers

2. **Artifact Download Flow**:

- Client applications request artifacts via HTTP GET
- For hosted repositories, content is retrieved directly from the BlobStore
- For proxy repositories, content is checked in local cache first, with remote repository calls executed using virtual threads
- If not present, the remote repository is queried, content is cached, and then served
- For group repositories, member repositories are checked in sequence
- Content transformations may be applied based on format-specific rules
- Access is logged and metrics are updated

3. **Repository Configuration Flow**:

- Administrator actions via UI or REST API trigger configuration changes
- Changes are validated against schema and business rules
- Configuration is persisted to the DataStore
- Events are emitted to notify components of changes
- Repository components are initialized, started, or reconfigured
- In clustered environments, changes are propagated to other nodes

Thread management between HTTP Bridge, Repository Manager, and Storage layers utilizes the Java 21 concurrency model, with virtual threads handling I/O operations for improved scalability while maintaining code readability.

Key data stores and caching mechanisms include:

1. **BlobStore**: Stores binary content (artifacts) with configurable backend (file system, S3)
2. **DataStore**: Stores metadata, configuration, and relationships (H2 embedded or PostgreSQL)
3. **Search Index**: Elasticsearch-backed index for component/asset discovery
4. **Configuration Cache**: In-memory cache of frequently accessed configuration
5. **Component Metadata Cache**: Caches frequently accessed component metadata
6. **Proxy Cache**: Caches content from remote repositories to reduce network traffic

Data transformation primarily occurs at repository boundaries, such as:

- Format-specific content normalization during upload
- Metadata extraction from binary artifacts
- Search document generation for indexing
- Cache entry creation for proxy repositories
- Content filtering or transformation for client requests

5.1.4 External Integration Points

System Name	Integration Type	Data Exchange Pattern	Protocol/Format	SLA Requirements
Maven Central	Proxy Repository	Pull on demand with cache (virtual threads)	HTTP(S)/Maven2	High availability, <500ms response time
Docker Hub	Proxy Repository	Pull on demand with cache (virtual threads)	HTTP(S)/Docker Registry API	High availability, <1s response time
npm Registry	Proxy Repository	Pull on demand with cache (vi	HTTP(S)/npm Registry A	High availability, <500ms res

System Name	Integration Type	Data Exchange Pattern	Protocol/Format	SLA Requirements
		rtual threads)	PI	ponse time
CI/CD Systems	Repository Client	Push/Pull	HTTP(S)/REST	High throughput, <200ms response time
LDAP/Active Directory	Authentication Provider	Validate on login	LDAP/LDAPS	<200ms authentication time
AWS S3	Storage Backend	Streaming upload/download with virtual threads	S3 API/HTTP	<50ms operation time, 99.99% availability
PostgreSQL	Metadata Storage	SQL query/update	JDBC/SQL	<50ms query time, 99.99% availability
Monitoring Systems	Metrics Consumer	Push metrics, pull health status	HTTP(S)/Prometheus	<100ms response time
Webhook Receivers	Event Consumer	Push notifications	HTTP(S)/JSON	Best effort delivery, retry on failure

5.2 COMPONENT DETAILS

5.2.1 OSGi/Karaf Framework (updated)

- **Purpose and Responsibilities**:
  - Provides the runtime container for all Nexus components
  - Manages bundle lifecycle (install, start, stop, update)
  - Maintains service registry for dynamic service discovery
  - Handles classloading isolation and dependency resolution
  - Enforces versioned module boundaries
  - Support Java 21 runtime
- **Technologies and Frameworks**:

    - Apache Felix OSGi Framework
    - Apache Karaf container and features
    - Eclipse Equinox Region for class isolation
    - SLF4J/Logback for logging
    - Eclipse Sisu for service registration
    - Java JDK 21

- **Key Interfaces and APIs**:

    - OSGi BundleContext for service registration
    - Karaf Features Service for bundle provisioning
    - BundleActivator for lifecycle hooks
    - Blueprint XML for service wiring

- **Data Persistence Requirements**:

    - Bundle cache directory for installed bundles
    - Configuration files in etc/ directory
    - Feature repositories in system/ directory

- **Scaling Considerations**:

    - Memory allocation for bundle cache
    - Startup time increases with number of bundles
    - Class space separation for third-party dependencies
    - Verify OSGi/Karaf 4.3.9 Java 21 compatibility and module startup behavior under Java 21

5.2.2 Bootstrap Module (updated)

- **Purpose and Responsibilities**:

    - System initialization and environment preparation
    - JVM configuration and validation
    - Logging setup and redirection
    - Configuration loading and processing

    - Container startup and lifecycle management

- **Technologies and Frameworks**:
    - Java JDK 21
    - Karaf Main extension
    - JUL to SLF4J bridging
    - ConfigurationBuilder with property interpolation
    - NexusFileLock for process isolation

- **Key Interfaces and APIs**:
    - NexusMain extends Karaf Main
    - ConfigurationBuilder fluent API
    - StateGuard for lifecycle enforcement
    - File locking mechanism for process exclusivity
    - JVM configuration logic with Java 21 version string detection

- **Data Persistence Requirements**:
    - Lock file in data directory
    - Configuration files in etc/ directory
    - Temporary directory for staging files

- **Scaling Considerations**:
    - JVM memory allocation
    - File descriptor limits
    - Process isolation on shared servers
    - Default JVM flags including Java 21 recommended GC and string template options

5.2.3 Core Services Module (updated)

- **Purpose and Responsibilities**:
    - Centralized service registry and lifecycle

    - Event-driven component communication
    - Application state management
    - Configuration services
    - Health monitoring and metrics

- **Technologies and Frameworks**:
    - Guice/Sisu dependency injection
    - Guava EventBus
    - Dropwizard Metrics
    - JSR-330 annotations
    - StateGuard for lifecycle management
    - Java JDK 21

- **Key Interfaces and APIs**:
    - ManagedLifecycle annotation and phases
    - EventManager for component communication
    - ComponentSupport base class
    - ApplicationDirectories for filesystem access
    - MetricRegistry for operational metrics
    - Virtual threads for asynchronous event processing and scheduled tasks

- **Data Persistence Requirements**:
    - Application state in DataStore
    - Configuration in property files
    - Metrics gathered in memory with optional persistence

- **Scaling Considerations**:
    - Event bus performance under high load
    - Memory requirements for service registry
    - Component initialization ordering
    - Replacement of deprecated Java 17 APIs with Java 21 equivalents

5.2.4 Security Framework (updated)

- **Purpose and Responsibilities**:
  - Authentication and user management
  - Role-based access control
  - Permission evaluation and enforcement
  - Token-based security (JWT)
  - Credential encryption
- **Technologies and Frameworks**:
  - Apache Shiro
  - Java JWT libraries
  - BouncyCastle for cryptography
  - LDAP, SAML integrations
  - Nexus Crypto services
  - Updated Shiro and JWT library versions verified for Java 21
- **Key Interfaces and APIs**:
  - SecuritySystem for centralized security operations
  - Realms for authentication source pluggability
  - Permission system for access control
  - JwtService for token management
  - CryptoHelper for sensitive data handling
  - Updated BouncyCastle provider configuration for new JDK 21 defaults
- **Data Persistence Requirements**:
  - User, role, and privilege data in DataStore
  - Credential storage with encryption
  - Token storage and revocation lists
  - Audit logs for security events
- **Scaling Considerations**:

    - Authentication caching
    - Token validation performance
    - Realm synchronization in clusters
    - Security context propagation
    - Java 21 compatibility verification for all security components

5.2.5 Repository Layer (updated)

- **Purpose and Responsibilities**:
    - Repository lifecycle management
    - Format-specific content handling
    - Recipe-based repository creation
    - Content routing and resolution
    - Format validation and conversion
- **Technologies and Frameworks**:
    - Repository Facet pattern
    - Format recipes and templates
    - Content handlers and views
    - Repository types (hosted, proxy, group)
    - Format-specific implementations
    - Java 21 Virtual Threads
    - Pattern Matching enhancements
- **Key Interfaces and APIs**:
    - Repository interface for core operations
    - Facet interfaces for capability extension
    - Format interface for type identification
    - Recipe classes for repository configuration
    - ContentFacet for artifact management
- **Data Persistence Requirements**:

    - Repository configuration in DataStore
    - Repository content metadata in format-specific tables
    - Repository relationship mapping for groups
    - Repository statistics and metrics

- **Scaling Considerations**:

    - Number of concurrent repositories
    - Repository group depth and breadth
    - Format-specific scaling characteristics
    - Cache utilization for proxy repositories
    - Concurrent repository operations leveraging virtual threads

5.2.6 Storage Layer (updated)

- **Purpose and Responsibilities**:

    - Binary content storage and retrieval
    - Metadata persistence and querying
    - Transactional operations
    - Content deduplication
    - Backup and restore operations

- **Technologies and Frameworks**:

    - BlobStore API for binary content
    - DataStore API for metadata
    - MyBatis for SQL mapping
    - H2 or PostgreSQL databases
    - AWS S3 SDK for cloud storage
    - Updated JDBC drivers compatible with Java 21

- **Key Interfaces and APIs**:

    - BlobStore interface for content CRUD
    - DataSessionSupplier for database sessions

    - TransactionSupport for atomic operations
    - BlobId and BlobRef for content addressing
    - ContentAccess interfaces for data retrieval

- **Data Persistence Requirements**:

    - Binary content in file system or S3
    - Metadata in relational database
    - Transaction logs for recovery
    - Content references and relationships
    - Soft-deleted content tracking

- **Scaling Considerations**:

    - Storage capacity planning
    - Database connection pooling
    - Blob chunking for large artifacts
    - Transaction isolation levels
    - Database partitioning strategies
    - BlobStore implementations refactored to run I/O operations on virtual threads for higher throughput

5.2.7 HTTP Layer (updated)

- **Purpose and Responsibilities**:

    - RESTful API exposure
    - Legacy HTTP compatibility
    - Content upload and download
    - Request routing and dispatching
    - Content negotiation and format handling

- **Technologies and Frameworks**:

    - JAX-RS (RESTEasy)
    - Siesta framework for REST

    - Jackson for JSON processing
    - Jetty for HTTP serving
    - Apache Commons FileUpload
    - Virtual thread-per-request model enabled by Java 21

- **Key Interfaces and APIs**:

    - Resource classes with JAX-RS annotations
    - ViewServlet for HTTP request handling
    - HttpResponseSender for content delivery
    - Repository HTTP bridge
    - ExtDirect for UI communication

- **Data Persistence Requirements**:

    - REST configuration in DataStore
    - HTTP metrics and logs
    - Session state if enabled
    - Request caching information

- **Scaling Considerations**:

    - HTTP connection pooling
    - Thread pool sizing defaulting to virtual threads
    - Request timeout configuration
    - Payload size limitations
    - Response streaming for large artifacts

5.2.8 UI Framework

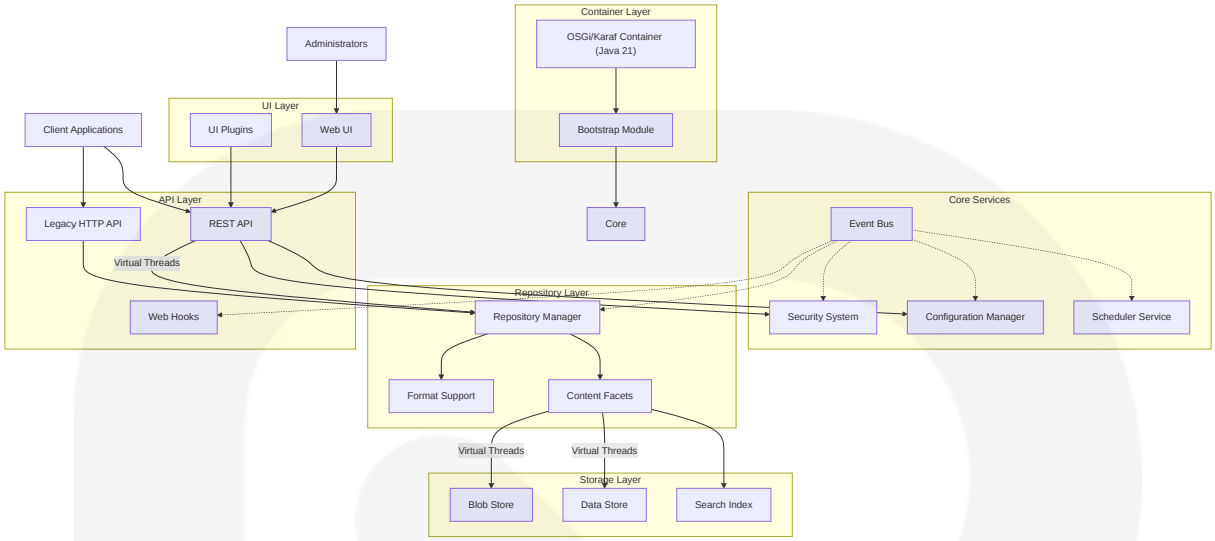
- **Purpose and Responsibilities**:

    - Web-based administration interface
    - Repository browsing and management
    - User, role, and privilege management
    - System configuration and monitoring

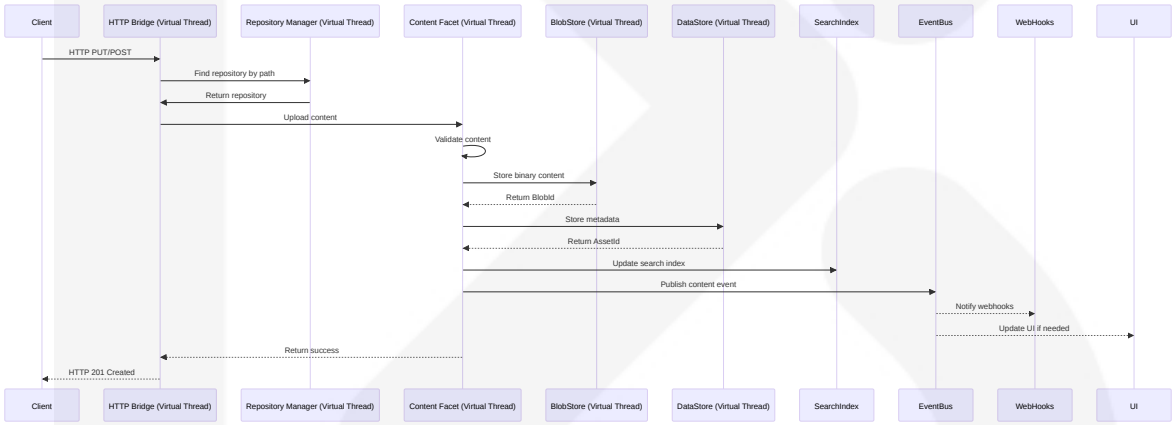
    - Plugin-specific UI extensions

- **Technologies and Frameworks**:
    - React for modern UI components
    - ExtJS for legacy UI components
    - Rapture framework for UI integration
    - XState for state management
    - Webpack for asset bundling
- **Key Interfaces and APIs**:
    - UiPluginDescriptor for plugin integration
    - REST clients for backend communication
    - StateContributor for UI state management
    - ExtDirect for RPC-style communication
    - Plugin extension points for UI customization
- **Data Persistence Requirements**:
    - User preferences in browser storage
    - UI state in memory
    - UI configuration via REST APIs
    - Static assets in file system
- **Scaling Considerations**:
    - Browser memory usage
    - Asset bundling and optimization
    - API request batching
    - UI responsiveness under load
    - Incremental rendering for large datasets

5.2.9 Component Interaction Diagram (updated)



5.2.10 Sequence Diagram for Artifact Upload (updated)



5.3 TECHNICAL DECISIONS

5.3.1 Architecture Style Decisions (updated)

Decision Area	Selected Approach	Alternatives Considered	Rationale
Overall Architecture	OSGi/Karaf Modular	Monolithic, Microservices	OSGi provides modular boundaries with runtime dynamism while maintaining

Decision Area	Selected Approach	Alternatives Considered	Rationale
			deployment simplicity, avoiding the operational complexity of microservices
Component Model	Facet Pattern	Inheritance Hierarchy, Pure Services	Facets allow dynamic extension of core entities like repositories without deep inheritance trees, enabling mix-in capabilities
Dependency Injection	Guice with Sisu	Spring, CDI, Manual Wiring	Guice provides lightweight DI with strong typing, while Sisu adds dynamic service discovery capabilities in OSGi environments
Database Access	MyBatis with Custom DAOs	JPA/Hibernate, JDBC Direct	MyBatis offers SQL control with type safety, avoiding ORM complexity while maintaining good performance characteristics
REST Implementation	JAX-RS with Siesta	Spring MVC, Jersey, Direct Servlets	JAX-RS provides standard annotations, while custom Siesta framework adds zero-wire DI for simpler resource registration
Language and Concurrency Features	**Java 21 virtual threads, record patterns, pattern matching, and string templates**	**Traditional thread pools, ExecutorService implementations, Java 17 without modern language features**	**Virtual threads enable high-throughput I/O operations with reduced overhead; record patterns and pattern matching provide cleaner, type-safe code; string templates increase readability and reduce string concatenation errors**

5.3.2 Communication Pattern Choices (updated)

Pattern	Implementation	Use Cases	Benefits
Event-Driven	Guava EventBus	Component notifications, Cache invalidation, Audit trails	Loose coupling, **Async processing utilizing Java 21 virtual threads for high concurrency**, Simplified cross-cutting concerns
REST	JAX-RS with Jackson	External API, UI Backend	Standard HTTP semantics, Content negotiation, Wide client support
Direct Method Calls	Service Injection	Performance-critical paths	Lower overhead, Type safety, Simpler debugging
Store-and-Forward	Scheduled Tasks	Cleanup operations, Index rebuilds	Resilience to failures, Background processing, Reduced peak loads

5.3.3 Data Storage Solution Rationale

Storage Type	Technology Choice	Primary Usage	Justification
Binary Content	File BlobStore / S3 BlobStore	Repository artifacts	File system provides simplicity and direct OS caching; S3 offers cloud scalability and durability
Metadata	H2 / PostgreSQL with MyBatis	Configuration, Component metadata, User data	H2 for simplicity in standalone deployments; PostgreSQL for clustering and enterprise scale
Search	Elasticsearch	Component/asset indexing	Full-text search capabilities, faceted navigation, high performance for complex queries
Configuration	File-based + Database	System settings, Repository configs	Files for bootstrap configuration; Database for runtime configuration that needs transactions

Storage Type	Technology Choice	Primary Usage	Justification
Caching	Ehcache / JCache	Proxy content, Frequently accessed data	In-memory performance with optional persistence, standard JCache API for pluggability

5.3.4 Caching Strategy Justification

Cache Type	Implementation	Invalidation Strategy	Rationale
Proxy Repository Cache	File BlobStore	TTL, Manual invalidation	Persistence needed across restarts, Large content unsuitable for memory
Security Context	In-memory, JCache	Event-driven, TTL	Performance critical, Relatively small data size, Needs quick invalidation
Configuration	In-memory	Event-driven	Small data size, Frequently accessed, Changes trigger events
Search Results	In-memory	Time-based, Size-based	Query results benefit from caching, Results may become stale with time
HTTP Resources	HTTP Caching Headers	Client-driven, ETag validation	Standard HTTP caching reduces server load, Clients manage own caches

5.3.5 Security Mechanism Selection

Security Area	Mechanism	Alternatives	Justification
Authentication	Apache Shiro with custom Realms	Spring Security, JAAS	Shiro provides flexible realm system, Good OSGi support, Simple API
Authorization	Role-based Access Control	ACLs, Attribute-based Access	RBAC offers balance of flexibility and simplicity, Matches repository permission model

Security Area	Mechanism	Alternatives	Justification
API Security	JWT + API Keys	OAuth, Basic Auth	JWT provides stateless security with rich claims, API keys offer simplicity for automation
Transport Security	TLS with configurable versions	Custom encryption	Industry standard, Wide client support, Good performance with proper configuration
Credential Storage	Encrypted with master key	Plain text, HSM integration	Balance of security and simplicity, Master key can be externalized

5.3.6 Architecture Decision Records (updated)

mermaid

flowchart TD

Start([Error Occurs]) --> Detect[Error Detection]

```
Detect --> Classify{Error Type}

Classify -->|Validation Error| Validation[Validation Error Handling]
Classify -->|System Error| System[System Error Handling]
Classify -->|External Error| External[External Dependency Error]

Validation --> Log1[Log Warning Level]
Validation --> Map1[Map to 400 Bad Request]
Validation --> Detail1[Include Validation Details]

System --> Log2[Log Error Level]
System --> Capture[Capture Stack Trace]
System --> FaultId[Generate Fault ID]
System --> Map2[Map to 500 Server Error]
```

```
External --> Retry{Retryable?}

Retry -->|Yes| BackOff[Apply Backoff Strategy]
Retry -->|No| Log3[Log Error with Context]

BackOff --> Attempt{Max Retries?}
Attempt -->|No| Retry2[Retry Operation]
Attempt -->|Yes| Failed[Mark as Failed]

Retry2 --> Detect

Failed --> Log3
Log3 --> Map3[Map to Appropriate Status]

Map1 --> Response[Return Error Response]
Map2 --> Response
Map3 --> Response

Detail1 --> Response
FaultId --> Response

Response --> Client[Return to Client]
Response --> Notify[Notify Administrators]

Client --> End([Error Handled])
```

```
## 5.5 DEPLOYMENT ARCHITECTURE

### 5.5.1 Deployment Models (updated)

Nexus Repository supports several deployment architectures:

1. **Standalone Deployment**:
   - Single server installation
   - Embedded H2 database
   - Local file system BlobStore
   - Suitable for development and small teams

2. **Enterprise Deployment**:
   - Multiple application servers
   - External PostgreSQL database
   - Shared file system or S3 BlobStore
```

```
   - Load balancer for traffic distribution
   - Suitable for large organizations

3. **Cloud Deployment**:
   - Containerized application (<span style="background-color: rgba(91, 5
   - <span style="background-color: rgba(91, 57, 243, 0.2)">Kubernetes or
   - Cloud database services
   - S3 or equivalent object storage
   - Auto-scaling capabilities
   - <span style="background-color: rgba(91, 57, 243, 0.2)">Optimized for

<span style="background-color: rgba(91, 57, 243, 0.2)">All deployment mod

### 5.5.2 Networking Architecture

<div class="mermaid-wrapper" id="mermaid-diagram-xysi4lsf0">
         <div class="mermaid">
graph TD
    subgraph Internet
        Clients[Client Applications]
        RemoteRepos[Remote Repositories]
    end

    subgraph LoadBalancer
        LB[Load Balancer/Proxy]
    end

    subgraph Application
        NexusA[Nexus Instance A]
        NexusB[Nexus Instance B]
    end

    subgraph Storage
        DB[(PostgreSQL)]
        SharedStore[Shared Storage&lt;br/&gt;NFS/S3]
    end

    Clients --&gt; LB
    LB --&gt; NexusA
    LB --&gt; NexusB

    NexusA --&gt; DB
    NexusB --&gt; DB
```

```
    NexusA --&gt; SharedStore
    NexusB --&gt; SharedStore

    NexusA --&gt; RemoteRepos
    NexusB --&gt; RemoteRepos
</div>
        </div>

### 5.5.3 Resource Requirements

| Component | CPU | Memory | Disk | Network |
| --------- | --- | ------ | ---- | ------- |
| Nexus Application | 4+ cores | 4GB+ | 10GB for application | 1Gbps |
| BlobStore | Varies | Cache as needed | Sized for artifacts | 1Gbps+ |
| Database (H2) | Shared | Shared | 1GB+ | N/A |
| Database (PostgreSQL) | 2+ cores | 2GB+ | 10GB+ | 1Gbps |

The actual resource requirements vary significantly based on:
- Number and size of artifacts
- Repository formats in use
- Concurrent user load
- Proxy caching patterns
- Search and analysis requirements

### 5.5.4 Scalability Considerations (updated)

Nexus Repository addresses scalability through several mechanisms:

1. **Vertical Scaling**:
   - Increased memory for caching
   - Additional CPU cores for concurrent operations
   - Faster storage for improved I/O performance

2. **Horizontal Scaling**:
   - Multiple application instances
   - Database clustering
   - Load balancing

3. **Resource Optimization**:
   - Connection pooling
   - <span style="background-color: rgba(91, 57, 243, 0.2)">Thread pool t
   - <span style="background-color: rgba(91, 57, 243, 0.2)">Reduced memor
```

```
   - <span style="background-color: rgba(91, 57, 243, 0.2)">Elimination o
   - Caching strategies
   - Blob chunking for large artifacts

4. **Specialized Configurations**:
   - Read-only replicas
   - Format-specific instances
   - Dedicated proxy servers

5. **Cloud-Native Options**:
   - Container orchestration
   - Auto-scaling groups
   - Cloud storage integration
   - Database-as-a-Service
   - <span style="background-color: rgba(91, 57, 243, 0.2)">High-density

<span style="background-color: rgba(91, 57, 243, 0.2)">The Java 21 virtua

## 5.6 REFERENCES

The information in this System Architecture section is derived from the f

- Root project structure and configuration:
  - pom.xml
  - rebuild.bat
  - nxrm.groovy

- Component and plugin directories:
  - components/nexus-base
  - components/nexus-bootstrap
  - components/nexus-core
  - components/nexus-repository-config
  - components/nexus-repository-content
  - components/nexus-repository-view
  - components/nexus-security
  - components/nexus-datastore
  - components/nexus-blobstore
  - components/nexus-rest
  - components/nexus-ui-plugin
  - plugins/nexus-coreui-plugin
  - assemblies/nexus-base-feature

- Support and build infrastructure:
```

```
  - buildsupport
  - .mvn/wrapper
  - testsuite

- <span style="background-color: rgba(91, 57, 243, 0.2)">Documentation ar
  - <span style="background-color: rgba(91, 57, 243, 0.2)">docs/migratior

- <span style="background-color: rgba(91, 57, 243, 0.2)">External referer
  - <span style="background-color: rgba(91, 57, 243, 0.2)">Apache Karaf 4
  - <span style="background-color: rgba(91, 57, 243, 0.2)">JDK 21 JEP int

# 6. SYSTEM COMPONENTS DESIGN

## 6.1 CORE SERVICES ARCHITECTURE

### 6.1.1 SERVICE COMPONENTS

Nexus Repository's core services are organized into distinct but cooperat

#### Service Boundaries and Responsibilities

| Service Component | Primary Responsibilities | Key Modules |
| ----------------- | ------------------------ | ----------- |
| REST API Services | Expose REST endpoints, handle HTTP requests, conter
| Security Services | Authentication, authorization, user management, enc
| DataStore Services | Metadata persistence, SQL operations, schema migra
| BlobStore Services | Binary content storage, content addressing, deduplI

#### Inter-service Communication Patterns

Nexus Repository implements several communication patterns between core s

1. **Direct Service Injection**
   - Primary communication mechanism using Guice/Sisu dependency injectic
   - Services are injected via constructor or field injection using JSR-3
   - Produces strongly-typed, compile-time validated service references

2. **Event-driven Communication**
   - Asynchronous, loosely-coupled interaction via Guava EventBus
   - Events published by event producers are received by registered handl
   - ComponentSupport base class provides EventManager access
   - Critical for cross-cutting concerns like cache invalidation, securit
   - <span style="background-color: rgba(91, 57, 243, 0.2)">Event handler
```

```
3. **OSGi Service Registry**
   - Dynamic service registration and discovery
   - Services registered with properties for filtering
   - Sisu bridges Guice and OSGi service registries

4. **Facade Pattern**
   - Higher-level services often provide facades over multiple lower-leve
   - Example: RepositoryManager coordinates multiple repository-related s

5. <span style="background-color: rgba(91, 57, 243, 0.2)">**Virtual Threa
   - Leverages Java 21 Virtual Threads (via Executors.newVirtualThreadPer
   - Eliminates traditional thread-pool complexity and management overhea
   - Enables thousands of concurrent operations with minimal resource cor
   - Particularly efficient for database access, blob storage operations,
   - Simplifies code by removing thread pool sizing and configuration cor

#### Service Discovery Mechanisms

Service discovery in Nexus Repository operates on multiple levels:

1. **Compile-time Discovery**
   - JSR-330 annotations (@Named, @Singleton, @Inject) with Eclipse Sisu
   - Automatic component scanning and registration
   - Type-based qualifier annotations for disambiguation

2. **Runtime Discovery**
   - OSGi service registry for dynamic discovery
   - Service references obtained through BundleContext
   - Sisu's dynamic binding to detect service additions/removals

3. **Extension Points**
   - Plugins discover extension points through IndexedContext
   - Format-specific implementations registered as dynamic services

#### Load Balancing Strategy

Nexus Repository implements load balancing at multiple levels:

1. **Internal Load Balancing**
   - Thread pool management for concurrent operations
   - Configurable connection pools for external services
   - Work distribution across multiple executor services
```

```
2. **External Load Balancing**
   - Support for HTTP load balancers in front of multiple instances
   - Sticky sessions for administrative operations
   - Shared storage (database and blobstore) enables stateless behavior

3. **Proxy Repository Load Balancing**
   - Smart routing to multiple remote repositories
   - Failover mechanisms between mirrors
   - Load distribution across repository group members

#### Circuit Breaker Patterns

The system implements circuit breakers to prevent cascading failures:

1. **Remote Repository Access**
   - Auto-detection of remote repository availability
   - Circuit breaks after configurable failure threshold
   - Automatic restoration after cooling period
   - Status indicators in UI and REST API

2. **Database Connection Management**
   - Connection health monitoring
   - Fast failure when database unreachable
   - Automatic connection recovery

3. **Third-Party Service Integration**
   - Health checks for integrated services
   - Timeouts for external API calls
   - Fallback to cached data when services unavailable

#### Retry and Fallback Mechanisms

Nexus Repository implements robust retry and fallback patterns:

1. **Retry Mechanisms**
   - Exponential backoff for remote repository access
   - Configurable retry counts for transient failures
   - Separate retry policies for different operation types

2. **Fallback Strategies**
   - Proxy repositories fall back to cached content when remote unavailab
   - Group repositories try alternative members when primary fails
```

```
   - Authentication falls back through multiple realms

3. **Degraded Operation Modes**
   - Read-only mode when write operations fail
   - Local-only mode when remote services unavailable
   - Core services prioritized over extensions when resources constrained

### 6.1.2 SCALABILITY DESIGN

The Nexus Repository architecture addresses scalability through multiple

#### Horizontal/Vertical Scaling Approach

| Scaling Dimension | Implementation Approach | Applicable Components |
| ----------------- | ----------------------- | --------------------- |
| Vertical Scaling | Memory configuration, JVM tuning, thread pool sizing
| Horizontal Scaling | Active/passive clustering, shared storage | Reposi
| Feature Partitioning | Format-specific instances, specialized role inst
| Storage Scaling | Pluggable storage backends (File, S3), database optic

#### Auto-scaling Triggers and Rules

When deployed in cloud environments or with orchestration platforms:

1. **Resource-based Triggers**
   - CPU utilization thresholds (typically 70-80%)
   - Memory consumption patterns
   - Request queue depth

2. **Time-based Rules**
   - Scheduled scaling for known high-demand periods
   - Gradual scale-down during off-hours
   - Minimum instance count during business hours

3. **Operation-specific Triggers**
   - Proxy download performance degradation
   - Upload operations backlog
   - Search response time increases

#### Resource Allocation Strategy

Nexus Repository manages resources efficiently through several strategies
```

```
1. **Memory Allocation**
   - Configurable heap size based on workload profile
   - Separate memory pools for application vs. caching
   - Tunable garbage collection parameters

2. **Storage Tiering**
   - Frequently accessed content in performance-optimized storage
   - Archival content in cost-optimized storage
   - Metadata in high-performance database storage

3. **Thread Management**
   - <span style="background-color: rgba(91, 57, 243, 0.2)">Dedicated pla
   - <span style="background-color: rgba(91, 57, 243, 0.2)">Java 21 Virtu
   - QoS prioritization of critical vs. background operations
   - Configurable concurrency limits to prevent resource exhaustion

#### Performance Optimization Techniques

The system employs several performance optimization approaches:

1. **Caching Strategy**
   - Multi-level caching (memory, local disk, distributed)
   - Content-specific cache policies
   - Proactive cache warming for critical paths

2. **I/O Optimization**
   - Streaming large content vs. buffering small content
   - <span style="background-color: rgba(91, 57, 243, 0.2)">Java 21 Virtu
   - Connection pooling for database and HTTP

3. **Computation Efficiency**
   - Just-in-time metadata extraction
   - Lazy initialization of expensive components
   - Background processing for non-critical operations
   - <span style="background-color: rgba(91, 57, 243, 0.2)">Java 21 Recor
   - <span style="background-color: rgba(91, 57, 243, 0.2)">Java 21 Strin

4. **Network Optimization**
   - Content compression where beneficial
   - Request batching for related operations
   - Conditional requests with etags

#### Capacity Planning Guidelines
```

```
Capacity planning for Nexus Repository should consider:

1. **Storage Capacity**
   - Plan for 2-3x growth over expected repository size
   - Account for temporary storage needs during rebuilds
   - Consider format-specific growth patterns

2. **Database Sizing**
   - Metadata typically requires 1-5% of binary storage size
   - Transaction logs need adequate space for busy periods
   - Index space requirements increase with repository size

3. **Memory Requirements**
   - Base: 2GB minimum, 4GB recommended
   - Add 1GB per 8-10 concurrent users
   - Add memory for format-specific needs (Docker: +2GB, Maven: +1GB)

4. **CPU Allocation**
   - Minimum: 4 cores for production use
   - Scale cores with concurrent user count (1 core per 5-10 concurrent u
   - Additional needs for search-intensive operations

### 6.1.3 RESILIENCE PATTERNS

Nexus Repository implements comprehensive resilience patterns to maintain

#### Fault Tolerance Mechanisms

The system's fault tolerance is built on several key mechanisms:

1. **Isolation Boundaries**
   - OSGi bundle isolation prevents cascading failures
   - Repository types isolated from each other
   - Plugin failures contained without affecting core services

2. **Health Monitoring**
   - Self-diagnostic health checks for all critical services
   - Proactive detection of degraded components
   - Automatic reporting of system health state

3. **Graceful Degradation**
   - Non-critical services disabled when resources constrained
```

```
   - Format-specific services operate independently
   - UI components adapt to backend service availability

4. **Transaction Management**
   - ACID transactions for critical state changes
   - Consistent state recovery after interruptions
   - Write-ahead logging for database operations

#### Disaster Recovery Procedures

Disaster recovery is addressed through multiple layers:

1. **Backup Systems**
   - Database backup (scheduled dumps, WAL archiving)
   - BlobStore backup (synchronization to secondary storage)
   - Configuration export functionality

2. **Recovery Procedures**
   - Database restore workflows
   - BlobStore integrity verification and repair
   - Step-by-step recovery playbooks

3. **Service Restoration**
   - Prioritized service restart order
   - Consistent state verification before accepting traffic
   - Incremental availability of non-critical services

#### Data Redundancy Approach

Data redundancy is implemented at multiple levels:

1. **Storage Redundancy**
   - File BlobStore with RAID or redundant volumes
   - S3 BlobStore with built-in redundancy
   - Database replication options (PostgreSQL streaming replication)

2. **Content Redundancy**
   - Proxy repositories cache remote content
   - Replication between Nexus instances
   - Critical metadata backup and restore capability

3. **Configuration Redundancy**
   - Export/import of system configuration
```

```
   - Version-controlled configuration templates
   - Automated configuration backup

#### Failover Configurations

Nexus Repository supports several failover configurations:

1. **Active/Passive Clustering**
   - PostgreSQL-based clustering
   - Shared storage (NFS, S3) for blob content
   - Health check monitoring with automatic promotion

2. **Database Failover**
   - PostgreSQL streaming replication
   - Automatic read-replica promotion
   - Connection fail-back after primary restoration

3. **BlobStore Failover**
   - Multiple storage locations
   - Automated failover for S3 region availability
   - Read operations continue during partial failures

#### Service Degradation Policies

When failures occur, the system employs graduated degradation policies:

1. **Read-Only Mode**
   - Automatic transition when write operations fail
   - All read operations continue functioning
   - Administrative notification of degraded state

2. **Format-Specific Degradation**
   - Individual repository formats can degrade independently
   - Critical formats prioritized over non-critical ones
   - Format-specific error responses indicate degradation

3. **Maintenance Mode**
   - Administrator-initiated controlled degradation
   - Limited service availability during recovery
   - Protected operations for system repair

### 6.1.4 SERVICE INTERACTION DIAGRAM
```

```
<div class="mermaid-wrapper" id="mermaid-diagram-8n2eqvd34">
          <div class="mermaid">
graph TD
    Client[&quot;Client Applications&quot;]

    subgraph &quot;API Layer&quot;
        REST[&quot;REST API Services&quot;]
        HTTP[&quot;HTTP Bridge&quot;]
    end

    subgraph &quot;Core Services&quot;
        Security[&quot;Security Services&quot;]
        Events[&quot;Event Manager&quot;]
        Config[&quot;Configuration Services&quot;]
        Scheduler[&quot;Scheduler Services&quot;]
    end

    subgraph &quot;Repository Layer&quot;
        RepoMgr[&quot;Repository Manager&quot;]
        Format[&quot;Format Services&quot;]
        Content[&quot;Content Services&quot;]
    end

    subgraph &quot;Storage Layer&quot;
        Blob[&quot;BlobStore Services&quot;]
        Data[&quot;DataStore Services&quot;]
        Search[&quot;Search Services&quot;]
    end

    subgraph &quot;Infrastructure Layer&quot;
        OSGi[&quot;OSGi/Karaf Services&quot;]
        DB[&quot;Database (H2/PostgreSQL)&quot;]
        FS[&quot;File System/S3&quot;]
    end

    Client -- &quot;REST/HTTP&quot; --&gt; REST
    Client -- &quot;Format Protocols&quot; --&gt; HTTP

    REST --&gt; Security
    REST --&gt; RepoMgr
    REST --&gt; Config
    HTTP --&gt; RepoMgr
```

```
    Security -- &quot;Authentication Events&quot; --&gt; Events
    Security -- &quot;Permission Check&quot; --&gt; RepoMgr

    RepoMgr --&gt; Format
    RepoMgr --&gt; Content
    RepoMgr -- &quot;Task Scheduling&quot; --&gt; Scheduler

    Format -- &quot;Format-specific Operations&quot; --&gt; Content
    Content -- &quot;Binary Storage&quot; --&gt; Blob
    Content -- &quot;Metadata Storage&quot; --&gt; Data
    Content -- &quot;Content Events&quot; --&gt; Events

    Config -- &quot;Configuration Events&quot; --&gt; Events
    Scheduler -- &quot;Task Events&quot; --&gt; Events

    Events -- &quot;Event Subscriptions&quot; --&gt; RepoMgr
    Events -- &quot;Event Subscriptions&quot; --&gt; Search
    Events -- &quot;Event Subscriptions&quot; --&gt; Blob

    Blob -- &quot;Storage Operations&quot; --&gt; FS
    Data -- &quot;Database Operations&quot; --&gt; DB
    Search -- &quot;Index Operations&quot; --&gt; DB

    OSGi -. &quot;Service Registry&quot; .-&gt; Security
    OSGi -. &quot;Service Registry&quot; .-&gt; RepoMgr
    OSGi -. &quot;Service Registry&quot; .-&gt; Events
    OSGi -. &quot;Service Registry&quot; .-&gt; Config
    OSGi -. &quot;Service Registry&quot; .-&gt; Scheduler
</div>
        </div>

### 6.1.5 SCALABILITY ARCHITECTURE

<div class="mermaid-wrapper" id="mermaid-diagram-6o25a2kxq">
          <div class="mermaid">
graph TD
    subgraph &quot;External Load Balancing&quot;
        LB[&quot;Load Balancer&quot;]
        HealthCheck[&quot;Health Check Endpoint&quot;]
    end

    subgraph &quot;Nexus Instances&quot;
        NexusA[&quot;Nexus Instance A&quot;]
```

```
        NexusB[&quot;Nexus Instance B&quot;]
    end

    subgraph &quot;Resource Management&quot;
        ThreadPool[&quot;Executor Manager (Platform &amp; Virtual Threads
        MemoryMgr[&quot;Memory Manager&quot;]
        ConnPool[&quot;Connection Pool&quot;]
    end

    subgraph &quot;Storage Layer&quot;
        SharedDB[&quot;Shared Database&lt;br&gt;(PostgreSQL)&quot;]
        SharedBlob[&quot;Shared BlobStore&lt;br&gt;(S3/NFS)&quot;]
    end

    subgraph &quot;Caching Layer&quot;
        LocalCache[&quot;Local Cache&quot;]
        DistCache[&quot;Distributed Cache&quot;]
    end

    LB --&gt;|&quot;Route Requests&quot;| NexusA
    LB --&gt;|&quot;Route Requests&quot;| NexusB
    HealthCheck -.-&gt;|&quot;Monitor&quot;| NexusA
    HealthCheck -.-&gt;|&quot;Monitor&quot;| NexusB

    NexusA --&gt;|&quot;Configure&quot;| ThreadPool
    NexusB --&gt;|&quot;Configure&quot;| ThreadPool
    ThreadPool --&gt;|&quot;Allocate&quot;| MemoryMgr
    ThreadPool --&gt;|&quot;Manage&quot;| ConnPool

    NexusA --&gt;|&quot;Read/Write&quot;| SharedDB
    NexusB --&gt;|&quot;Read/Write&quot;| SharedDB
    NexusA --&gt;|&quot;Read/Write&quot;| SharedBlob
    NexusB --&gt;|&quot;Read/Write&quot;| SharedBlob

    NexusA --&gt;|&quot;Cache&quot;| LocalCache
    NexusB --&gt;|&quot;Cache&quot;| LocalCache
    NexusA &lt;--&gt;|&quot;Distributed Caching&quot;| DistCache
    NexusB &lt;--&gt;|&quot;Distributed Caching&quot;| DistCache

    classDef instance fill:#f9f,stroke:#333,stroke-width:1px;
    classDef shared fill:#bbf,stroke:#333,stroke-width:1px;
    classDef updated fill:#a582ff,stroke:#333,stroke-width:1px;
    class NexusA,NexusB instance;
```

```
    class SharedDB,SharedBlob,DistCache shared;
    class ThreadPool updated;

### 6.1.6 RESILIENCE PATTERN IMPLEMENTATIONS

</div>
        </div>mermaid
graph TD
    subgraph "Client Access Tier"
        Client["Client Application"]
        LB["Load Balancer"]
    end

    subgraph "Application Tier"
        HA["HA Pair<br>Active/Passive"]
        Active["Active Node"]
        Passive["Passive Node"]
        CircuitBreaker["Circuit Breaker<br>Manager"]
    end

    subgraph "Storage Tier"
        PrimaryDB["Primary Database"]
        ReplicaDB["Replica Database"]
        BlobStore["BlobStore<br>(Redundant Storage)"]
        BackupSystem["Backup System"]
    end

    Client -->|"Requests"| LB
    LB -->|"Route to Active"| Active
    LB -.->|"Failover Routing"| Passive

    Active -->|"Replication"| Passive
    Active -->|"Health Check"| HA
    Passive -->|"Health Check"| HA
    HA -->|"Failover Control"| LB

    Active -->|"Monitor Remote Services"| CircuitBreaker
    CircuitBreaker -->|"Break Circuit"| Active

    Active -->|"Write"| PrimaryDB
    Active -->|"Read"| PrimaryDB
    PrimaryDB -->|"Replicate"| ReplicaDB
    Passive -->|"Read Only"| ReplicaDB
```

```
    Active -->|"Store Content"| BlobStore
    Passive -->|"Read Content"| BlobStore

    PrimaryDB -.->|"Backup"| BackupSystem
    ReplicaDB -.->|"Backup"| BackupSystem
    BlobStore -.->|"Backup"| BackupSystem

    classDef active fill:#9f9,stroke:#333,stroke-width:1px;
    classDef passive fill:#ff9,stroke:#333,stroke-width:1px;
    classDef storage fill:#bbf,stroke:#333,stroke-width:1px;
    class Active active;
    class Passive passive;
    class PrimaryDB,ReplicaDB,BlobStore storage;
```

6.1.7 SOURCE FILES AND FOLDERS RETRIEVED

The following source files and directories were analyzed to compile this architecture documentation:

- `/components/nexus-rest`
- `/components/nexus-rest-jackson2`
- `/components/nexus-servlet`
- `/components/nexus-security`
- `/components/nexus-ssl`
- `/components/nexus-crypto`
- `/components/nexus-datastore`
- `/components/nexus-datastore-mybatis`
- `/components/nexus-datastore-api`
- `/components/nexus-blobstore-api`
- `/components/nexus-repository-content`
- `/components/nexus-repository-view`
- `/components/nexus-repository-config`
- `/components/nexus-scheduling`
- `/components/nexus-quartz`
- `/components/nexus-common`

- `/components/nexus-core`
- `/components/nexus-cache`
- `/components/nexus-bootstrap`
- `/components/nexus-extender`
- `/components/nexus-features`
- `/components`
- `/plugins`
- `/assemblies`
- `/buildsupport`
- `/pom.xml`

6.2 DATABASE DESIGN

6.2.1 SCHEMA DESIGN

Entity Relationships

The Nexus Repository database schema consists of several interconnected domains:

1. **Repository Configuration Domain**
    - **BlobStore Configuration**: Stores blob store metadata including name, type, and attributes
    - **Repository Configuration**: Defines repository settings, format-specific attributes, and references to blob stores
    - **Routing Rules**: Contains path-matching patterns for controlling repository content routing
2. **Content Domain**
    - **Components**: Format-specific component metadata (e.g., Maven GAV coordinates)
    - **Assets**: Individual content items with paths, format attributes, and blob references

- **Asset Blobs**: Metadata about binary content stored in blob stores

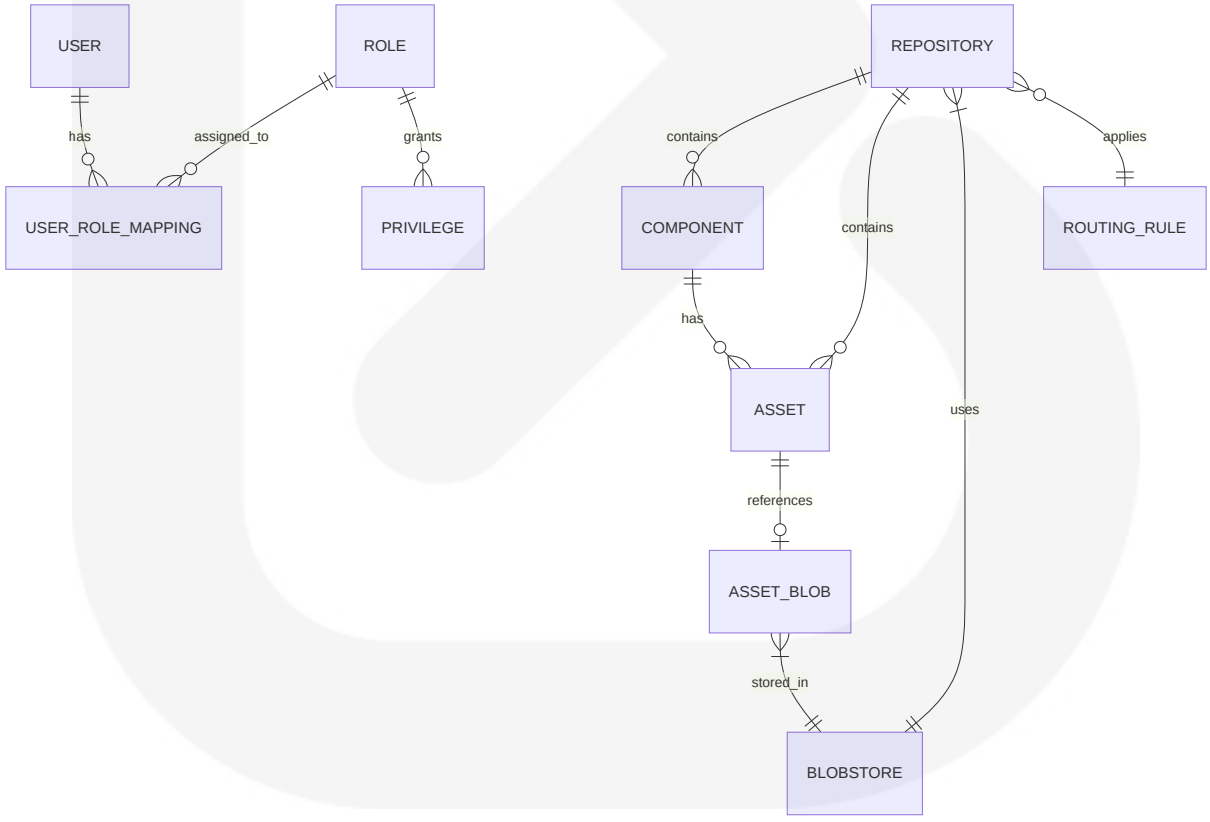
3. **Security Domain**

- **Users and Roles**: User accounts and role memberships
- **Privileges**: Granular permissions
- **API Keys**: Authentication tokens with associated user contexts
- **User-Role Mappings**: Many-to-many relationships between users and roles

4. **System Domain**

- **Node Identity**: Unique identifiers for cluster nodes
- **Task Configuration**: Scheduled task definitions and state
- **Key-Value Storage**: General-purpose configuration storage

The relationships between these entities are managed through foreign keys and join tables, creating a normalized relational model.



Data Models and Structures

The database uses two primary storage approaches:

1. **Relational Database Storage (DataStore)**
    - H2 Database (embedded, Community Edition)
    - PostgreSQL (optional, Enterprise Edition)
    - Table structures defined through MyBatis XML mappers
    - JSON columns for flexible, format-specific attributes
2. **Binary Content Storage (BlobStore)**
    - File BlobStore: Local filesystem storage
    - S3 BlobStore: Cloud-based object storage
    - Separation of metadata (in DataStore) from binary content (in BlobStore)

Key tables in the schema include:

Table Name	Purpose	Key Fields
repository	Repository definitions	id, name, recipe, format
blob_store_configuration	BlobStore settings	name, type, attributes (JSON)
component	Format-agnostic component records	id, repository_id, namespace, name, version
asset	Content items metadata	id, component_id, path, created, last_accessed
asset_blob	Binary content references	id, blob_ref, content_type, size_bytes
user_role_mapping	User-to-role assignments	user_id, source, roles (JSON)
qrtz_*	Quartz scheduler tables	Multiple related tables for jobs and triggers

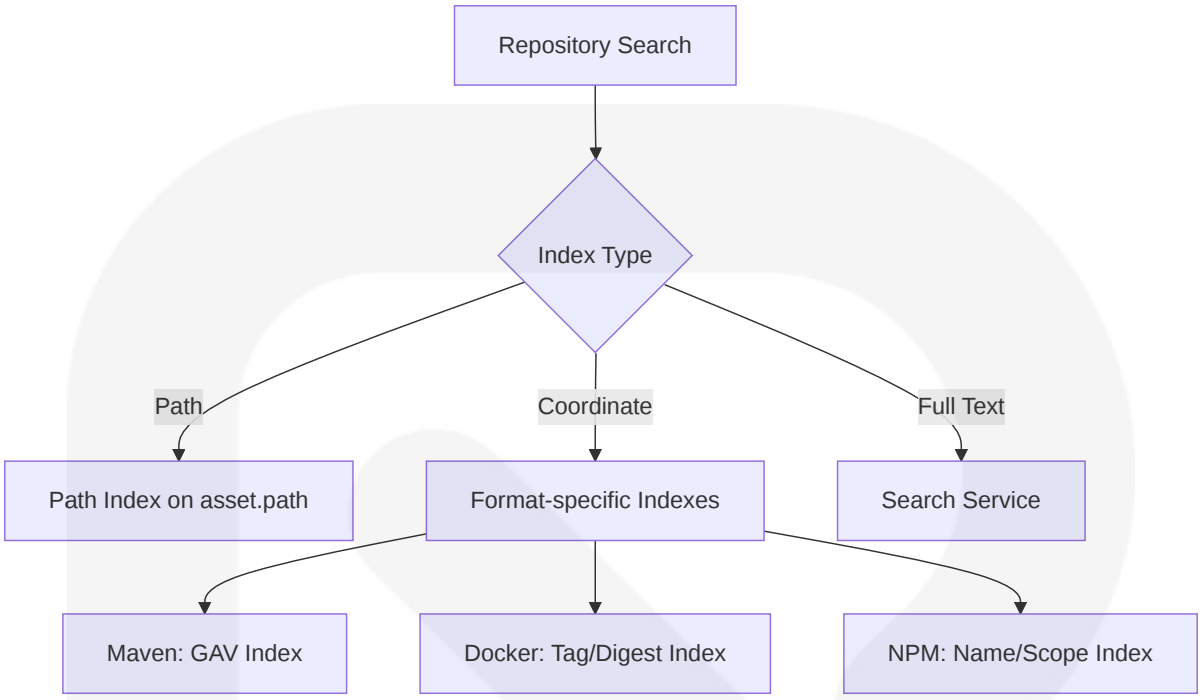
Format-specific tables extend the core schema with additional attributes:

Table Name	Purpose	Format-Specific Fields
maven2_component	Maven component metadata	group_id, artifact_id, version, base_version
apt_asset	Debian package metadata	architecture, component_name, control_file (JSON)
docker_manifest	Docker image manifests	digest, manifest_media_type, manifest_size

Indexing Strategy

Nexus uses a comprehensive indexing strategy to optimize data access patterns:

1. **Primary Key Indexes**
   - All tables have primary key constraints with corresponding indexes
   - Typically using auto-generated IDs or UUIDs
2. **Foreign Key Indexes**
   - Indexes on all foreign key columns (repository_id, component_id, etc.)
   - Supports efficient joins and cascade operations
3. **Path and Name Indexes**
   - Optimized for repository path lookup
   - Specialized indexes for name-based searches
4. **Format-Specific Indexes**
   - Maven: Indexes on group_id, artifact_id, version
   - Docker: Indexes on digest and tag
   - NPM: Indexes on scope and name



Partitioning Approach

Nexus Repository implements several partitioning strategies:

1. **Repository Isolation**
   - Each repository maintains isolated content sets
   - Content sharing achieved through group repositories (logical aggregation)
2. **Format Partitioning**
   - Format-specific tables for specialized metadata
   - Common core tables for shared attributes
3. **Database Partitioning**
   - PostgreSQL deployments can leverage table partitioning for large installations
   - Recommended partitioning by repository or time-based partitioning for audit logs

Replication Configuration

For high-availability enterprise deployments, Nexus supports:

1. **Active/Passive Database Replication**
   - PostgreSQL streaming replication for database redundancy
   - Automatic failover with proper configuration
2. **Shared Storage for BlobStores**
   - File-based BlobStores on shared network storage (NFS)
   - S3-based BlobStores with built-in replication
3. **Node Awareness**
   - `NodeAccess` interface to track cluster node status
   - Support for multi-node consensus on operations

Backup Architecture

Nexus implements a multi-tiered backup approach:

1. **Database Backup**
   - H2: Built-in backup to ZIP files via `H2BackupTask`
   - PostgreSQL: Native pg_dump with point-in-time recovery
2. **BlobStore Backup**
   - File BlobStore: Filesystem snapshots or rsync
   - S3 BlobStore: Cross-region replication or bucket versioning
3. **Configuration Export/Import**
   - Support ZIP export/import for system configuration
   - Database restoration from backup files

6.2.2 DATA MANAGEMENT

Migration Procedures

Nexus employs a sophisticated database migration framework:

1. **Flyway-Based Schema Evolution**
   - `UpgradeManagerImpl` coordinates Flyway migrations
   - Versioned migration scripts for schema changes
   - Support for both Java and SQL migrations
2. **Format-Specific Migrations**
   - Each repository format can define its own migrations
   - Version-controlled migration steps
3. **Legacy Data Migration**
   - Support for migrating from older Nexus versions
   - OrientDB to SQL schema conversion tools
4. **JDBC Driver Compatibility Verification**
   - Verify and update H2 database driver to version 2.2.224+ certified for Java 21
   - Update PostgreSQL JDBC driver to version 42.6.0+ for full Java 21 compatibility
   - Update Maven POM dependencies to reference compatible driver versions
   - Ensure proper runtime driver availability in OSGi environment
   - Add driver verification step to startup diagnostics

Versioning Strategy

1. **Schema Versioning**
   - Database schema version tracking in metadata tables

- Baseline versioning for clean installs
- Version validation during startup

2. **Entity Versioning**
   - Optimistic locking with version fields
   - Timestamp-based version tracking for key entities
3. **Content Versioning**
   - Repository formats with native versioning (Maven, NPM)
   - Explicit version attributes in components
   - Support for SNAPSHOT vs. release distinction

Archival Policies

1. **Content Cleanup Policies**
   - Time-based retention (last downloaded, created)
   - Version-based retention (keep N latest versions)
   - Format-specific policies (e.g., Maven SNAPSHOT cleanup)
2. **Blob Garbage Collection**
   - Soft-delete with delayed physical removal
   - Reference counting for shared blobs
   - Integrity verification before permanent deletion
3. **Task History Cleanup**
   - Configurable retention of task execution logs
   - Automatic purging of old execution records

Data Storage and Retrieval Mechanisms

1. **Content Access Layer**
   - `ContentFacet` API for format-specific access

- Repository views with content transformation
- Streaming API for large binary transfers

2. **Transaction Management**
   - Custom transaction framework with JTA support
   - Pre-commit, post-commit, and rollback hooks
   - DataSession abstraction for consistent database access
   - Java 21 compatibility validation for transaction framework
     - Updated JTA implementation compatibility for Java 21 runtime
     - Replaced deprecated `javax.transaction` APIs with `jakarta.transaction` equivalents
     - Modified transaction boundary management for virtual thread compatibility
     - Enhanced DataSession abstraction to preserve transaction context across virtual thread handoffs

3. **Query Mechanisms**
   - MyBatis SQL mappings for relational data
   - REST API for client access
   - Search API for complex queries

Caching Policies

Nexus implements a multi-layered caching architecture:

1. **Repository Content Caching**
   - Proxy repositories cache remote artifacts
   - Configurable time-to-live (TTL) settings
   - Negative caching for not-found responses

2. **Metadata Caching**
   - JCache/Ehcache backend for metadata
   - Support for both memory and disk persistence

- Cache invalidation via event bus

3. **Security Context Caching**
   - Authentication and authorization caching
   - Token and session caching
   - Configurable expiry policies

6.2.3 COMPLIANCE CONSIDERATIONS

Data Retention Rules

1. **Content Retention**
   - Configurable through cleanup policies
   - Format-specific retention rules
   - Support for legal hold functionality
2. **Audit Trail Retention**
   - Configurable retention period for audit logs
   - Export capability for long-term archival
   - Support for external log shipping
3. **Task Log Retention**
   - Configurable cleanup of task execution logs
   - Log rotation and compression

Backup and Fault Tolerance Policies

1. **Automated Backup Scheduling**
   - Built-in tasks for database backup
   - Integration with external backup tools
   - Point-in-time recovery options

2. **Disaster Recovery**
    - Documented restoration procedures
    - Support for partial recovery (metadata or blobs)
    - Testing framework for validation
3. **Fault Tolerance**
    - Circuit breaker patterns for external dependencies
    - Graceful degradation modes
    - Read-only fallback for maintenance

Privacy Controls

1. **Data Isolation**
    - Repository-level access control
    - Format-specific content filtering
    - Private repository support
2. **Sensitive Data Protection**
    - Credential encryption
    - Token security measures
    - Crypto helpers for sensitive fields
3. **Data Minimization**
    - Configurable logging levels
    - Redaction of sensitive information in logs
    - Fine-grained control over stored metadata

Audit Mechanisms

1. **Comprehensive Audit Trails**
    - Authentication events
    - Repository operations (create, update, delete)

- Content access logging

2. **Audit Storage**
   - Database tables for structured audit data
   - Log files for operational events
   - Export capabilities for compliance
3. **Non-repudiation**
   - User attribution for all operations
   - Timestamp and IP address tracking
   - Digital signatures for critical operations

Access Controls

1. **Role-Based Access Control**
   - Privileges mapped to operations
   - Repository-specific permissions
   - Content selector-based filtering
2. **Authentication Integration**
   - Internal user database
   - LDAP/Active Directory integration
   - SAML/OpenID Connect support
3. **API Security**
   - Token-based authentication
   - Request-specific authorization
   - Content-aware access decisions

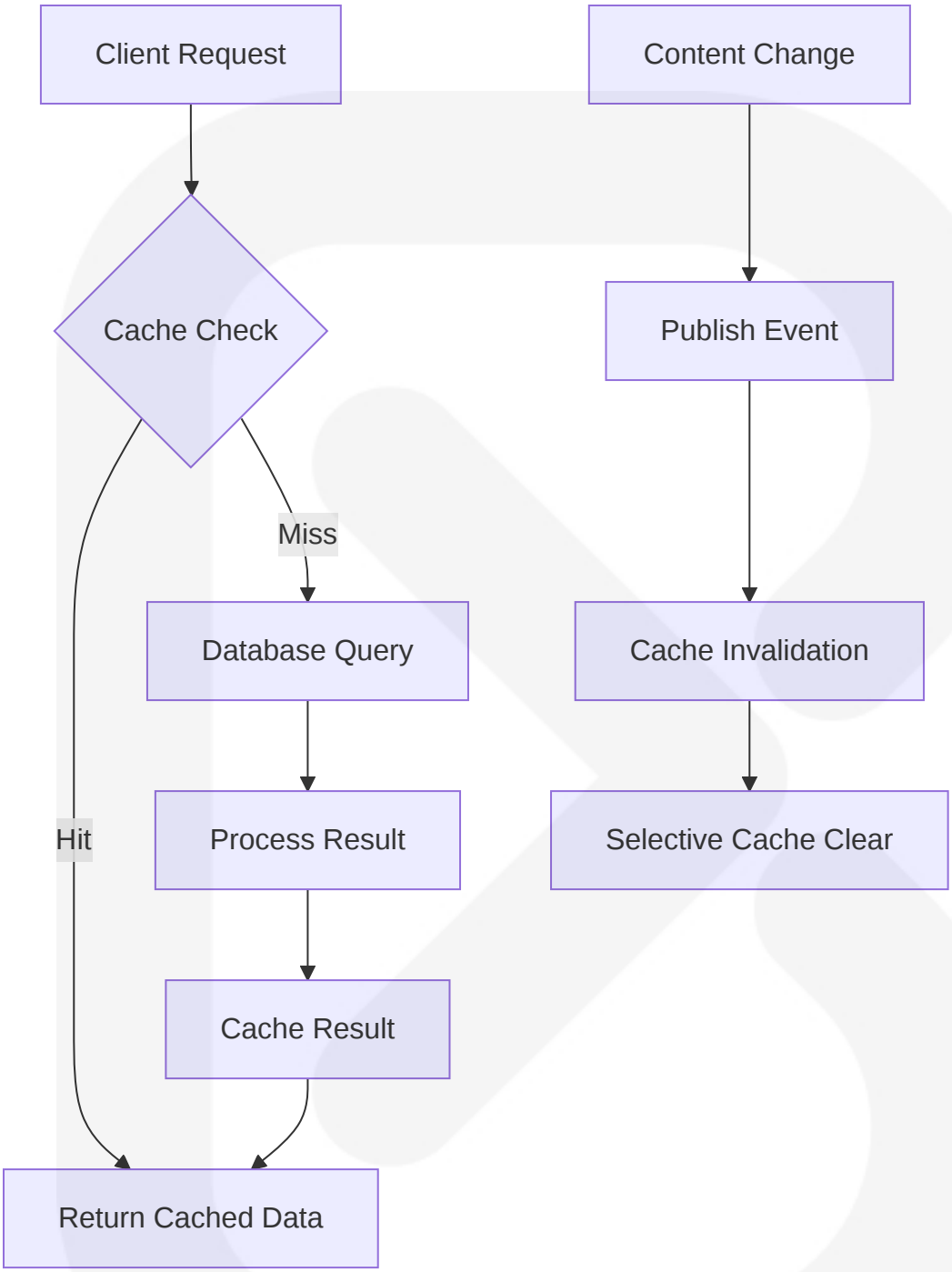
6.2.4 PERFORMANCE OPTIMIZATION

Query Optimization Patterns

1. **Prepared Statements**
    - MyBatis parameterized queries
    - Statement caching
    - Bind variable optimization
2. **Join Optimization**
    - Selective denormalization for critical paths
    - Index-aware join ordering
    - Query result limiting and pagination
3. **Format-Specific Optimizations**
    - Maven: GAV coordinate lookup optimization
    - Docker: Manifest and layer query patterns
    - NPM: Package and version query optimization

Caching Strategy

1. **Multi-Level Caching**
    - Memory cache for frequent access
    - Disk cache for larger datasets
    - Distributed cache for clustered deployments
2. **Cache Invalidation**
    - Event-driven invalidation
    - Time-based expiration
    - LRU (Least Recently Used) eviction
3. **Content-Aware Caching**
    - Different policies for metadata vs. binaries
    - Format-specific cache configurations
    - Repository type-specific strategies (hosted, proxy, group)



Connection Pooling

1. **HikariCP Implementation**
    - Lightweight, high-performance JDBC connection pool

- Configurable pool sizing
- Automatic health checking

2. **Pool Management**
   - Connection leak detection
   - Statement timeout handling
   - Idle connection cleanup

3. **Monitoring**
   - JMX-exposed metrics
   - Pool saturation alerts
   - Performance statistics

Read/Write Splitting

1. **PostgreSQL Deployment Options**
   - Primary for writes, replicas for reads
   - Load balancing across read replicas
   - Transaction-aware routing

2. **Operation Categorization**
   - Read-heavy operations routed to replicas
   - Write operations consolidated on primary
   - Transactional consistency guarantees

3. **Failover Handling**
   - Automatic promotion of replicas
   - Circuit breaking for database failures
   - Graceful degradation to read-only mode

Batch Processing Approach

1. **Bulk Operations**

    - Batched SQL statements for efficiency
    - Transaction grouping for related operations
    - Commit interval tuning

2. **Background Processing**

    - Asynchronous tasks for maintenance operations
    - Work queue management
    - Priority-based execution

3. **Format-Specific Optimizations**

    - Maven: Index rebuilding in batches
    - Docker: Layer processing optimizations
    - Apt/Yum: Metadata generation batching

6.2.5 SOURCE FILES AND FOLDERS RETRIEVED

The following source files and folders were analyzed to compile this database design documentation:

- `/components/nexus-datastore-api/`
- `/components/nexus-datastore-mybatis/`
- `/components/nexus-cache/`
- `/components/nexus-repository-content/`
- `/components/nexus-repository-config/`
- `/components/nexus-core/`
- `/components/nexus-base/`
- `/components/nexus-upgrade/`
- `/components/nexus-supportzip-api/`
- `/plugins/nexus-repository-maven/`
- `/plugins/nexus-blobstore-tasks/`
- `/plugins/nexus-script-plugin/`
- `/plugins/nexus-repository-apt/`
- `/plugins/nexus-coreui-plugin/`

- `/assemblies/nexus-base-overlay/`

6.3 INTEGRATION ARCHITECTURE

6.3.1 API DESIGN

Nexus Repository offers a powerful REST API that forms the foundation of its integration capabilities. The API design follows modern practices with a focus on consistency, security, and usability.

6.3.1.1 Protocol Specifications (updated)

Protocol	Implementation	Primary Use Cases
HTTP/HTTPS	**Jakarta REST 3.1 with RESTEasy 6.2.6.Final**	Primary protocol for all API interactions
WADL	Auto-generated by Jakarta REST	Legacy API documentation
Repository-specific protocols	Format adapters	Native protocols (e.g., Maven, npm, Docker)

Nexus implements its REST API using Jakarta REST 3.1 (via RESTEasy 6.2.6.Final) with a custom Siesta framework layer that provides additional features like dynamic resource discovery and centralized exception handling. The components/nexus-rest module has been updated for Java 21 compatibility, while components/nexus-siesta has been enhanced to ensure proper integration between Jakarta REST and the servlet container under Java 21.

The system now leverages upgraded OSGi HTTP Service bundles compatible with Karaf 4.3.9 and Java 21, ensuring the servlet container remains fully functional in the new runtime environment. This provides a solid foundation for both RESTful services and the web UI.

6.3.1.2 Authentication Methods (updated)

Method	Implementation	Use Cases
Basic Authentication	**Apache Shiro 1.13.0**	Standard username/password authentication
API Key	Custom token providers	Machine-to-machine integration
JWT	**JwtSecurityFilter, JwtHelper (com.auth0:java-jwt 4.4.0)**	Stateless authentication for RESTful services
Anonymous	AnonymousFilter	Public repository access
Session-based	NexusWebSessionManager	Web UI access with CSRF protection

The components/nexus-security module now utilizes Apache Shiro 1.13.0, maintaining the existing pluggable realm system while ensuring Java 21 compatibility. JWT support has been updated to use com.auth0:java-jwt 4.4.0 with HMAC256 signing. All authentication methods remain functionally identical, preserving backward compatibility while benefiting from enhanced security and performance in the updated libraries.

6.3.1.3 Authorization Framework

Authorization in Nexus is built on Apache Shiro with a role-based access control model:

- **Repository Permissions**: Fine-grained control over repository operations (read, browse, edit, etc.)
- **Admin Permissions**: System configuration, user management, and task execution
- **Dynamic Privileges**: Format-specific capabilities with automatic privilege generation
- **Content Selectors**: Path-based filtering for repository content

The security system supports permission wildcards and hierarchical privilege inheritance. Authorization checks can be applied declaratively with annotations or programmatically through the SecuritySystem API.

6.3.1.4 Rate Limiting Strategy (updated)

While no explicit global rate limiting was identified, Nexus implements several resource protection mechanisms, now enhanced with Java 21's Virtual Thread capabilities:

Mechanism	Implementation	Purpose
Connection pooling	HikariCP	Limits database connections
Virtual Threads	**Thread.ofVirtual()**	**Efficient handling of concurrent I/O operations**
Circuit breakers	For remote repositories	Prevents cascade failures
Modern HTTP Client	**java.net.http.HttpClient**	**Prevents resource exhaustion with non-blocking I/O**

The servlet container and NexusThreadFactory have been reconfigured to leverage Java 21 Virtual Threads (Thread.ofVirtual()) for all REST endpoint processing. This replaces the previous fixed-size thread pools for I/O-bound HTTP requests, providing dramatically improved scalability under load with minimal resource consumption.

The HttpClientManager has been augmented with Java 21's built-in java.net.http.HttpClient, configured to use Virtual Threads for outbound HTTP calls from the REST layer. This modernization eliminates thread pool exhaustion concerns for proxy repositories and remote API integrations while maintaining backward compatibility with existing connection management features.

Resource protection now benefits from Virtual Threads' efficiency, allowing the system to handle significantly more concurrent connections with lower memory overhead. The shift to Virtual Threads particularly benefits proxy repositories, where remote repository access is no longer constrained by fixed thread pool limits.

6.3.1.5 Versioning Approach

Nexus Repository uses several strategies for API versioning:

- **URI Path Versioning**: Explicit version in URI (e.g., `/v1/script`)
- **Feature Flags**: Feature toggles for new capabilities (e.g., `@FeatureFlag(JWT_ENABLED)`)
- **Compatibility Layers**: Legacy HTTP bridge for backward compatibility
- **Stable Core APIs**: Core interfaces remain consistent with extensions for new features

API constants and common patterns are defined in the `APIConstants` class, ensuring consistency across endpoints. The system avoids breaking changes where possible, using feature flags to phase in new functionality.

6.3.1.6 Documentation Standards

Documentation Type	Implementation	Purpose
Swagger/OpenAPI	io.swagger annotations	API documentation and exploration
JavaDoc	Standard annotations	Developer documentation
Resource annotation	@Path, @GET, etc.	JAX-RS endpoint definition
Custom metadata	@NotCacheable, etc.	Additional behavior annotations

The `components/nexus-swagger` module integrates Swagger Core with Nexus's JAX-RS endpoints. Documentation is automatically generated from code annotations and available through `/swagger.json` and `/swagger.yaml` endpoints.

6.3.2 MESSAGE PROCESSING

Nexus uses a sophisticated event-driven architecture for internal communication and integrates with external systems through various messaging patterns.

6.3.2.1 Event Processing Patterns (updated)

Pattern	Implementation	Use Cases
Event Bus	Guava EventBus with Virtual Thread execution	Internal event distribution
Webhooks	WebhookService with java.net.http.HttpClient	External event notifications
Event listeners	@Subscribe annotations	Component communication
Cluster-aware events	NodeAccess integration	Multi-node synchronization

The event system uses a publish-subscribe pattern with the Guava EventBus. Events are created for significant state changes and automatically distributed to registered subscribers. The event bus dispatchers have been configured to run subscriber handlers on Virtual Threads for any I/O-bound events, significantly improving non-blocking concurrency and resource utilization. The `components/nexus-webhooks` module extends this system to external HTTP endpoints.

Webhook delivery has been enhanced through migration to Java 21's java.net.http.HttpClient with Virtual Threads, replacing the legacy HTTP client implementation. This modernization enables more efficient, concurrent notification delivery with automatic backpressure handling and improved timeout management. The WebhookService now creates a new Virtual Thread for each outbound webhook notification, allowing for thousands of concurrent webhook deliveries with minimal resource impact.

6.3.2.2 Message Queue Architecture

Nexus primarily uses an in-memory event bus rather than external message queues:

- **Event Categories**: Repository, security, configuration, task events
- **Event Serialization**: JSON-based for cluster distribution
- **Event Routing**: Direct in-memory or cross-node propagation
- **Subscription Management**: Dynamic registration/unregistration

While not using traditional message queues, the event system provides similar capabilities with centralized event creation, routing, and consumption within the application boundary.

6.3.2.3 Stream Processing Design (updated)

Nexus implements streaming for efficient handling of large binary assets:

Component	Implementation	Purpose
BlobStore	File or S3 streaming	Binary content storage/retrieval
HTTP Streaming	Servlet API streams with Virtual Threads	Client download/upload
Range Support	PartialFetchHandler	HTTP range requests
Content Transformation	Content handlers	Format-specific transforms

The system avoids loading entire binary artifacts into memory, instead streaming content directly from storage to clients. The `components/nexus-repository-view` module has been enhanced with Virtual Thread support for its streaming abstractions like `StreamPayload`. HTTP streaming handlers now utilize Virtual Threads to enable high-throughput, non-blocking streaming of large binaries via Servlet API streams. This implementation dramatically increases the number of concurrent download and upload operations the system can handle without increasing resource consumption.

6.3.2.4 Batch Processing Flows (updated)

Nexus implements batch processing through its task scheduling framework:

Component	Implementation	Purpose
Task Scheduling	Quartz with Virtual Thread pool	Periodic task execution
Task Isolation	Per-task logging	Task output separation
Cluster-aware tasks	Node locking	Prevent duplicate execution

Component	Implementation	Purpose
Cancellation support	Cooperative checks	Interruptible long-running tasks

The `components/nexus-quartz` module integrates the Quartz Scheduler with Nexus's task framework. The Quartz Scheduler's ThreadPool has been reconfigured to use a Virtual Threads-based implementation, ensuring batch tasks leverage lightweight threads. This implementation replaces the traditional fixed-size thread pool with a dynamic Virtual Thread factory, allowing more concurrent tasks to execute efficiently while maintaining proper thread isolation between tasks. Tasks include repository maintenance, cleanup policies, and system operations.

6.3.2.5 Error Handling Strategy (updated)

Nexus implements a comprehensive error handling strategy:

Strategy	Implementation	Purpose
Exception Mapping	ExceptionMapperSupport	Converts exceptions to HTTP responses
Circuit Breakers	Virtual Thread-aware implementation	Prevents cascade failures
Retry Mechanisms	With tuned exponential backoff	Recovers from transient errors
Error Aggregation	For batch operations	Collects multiple failures
Fault IDs	X-Siesta-FaultId	Correlates errors with logs

Centralized exception handling in the REST layer ensures consistent error responses. Circuit breaker and HttpClient timeout policies have been validated and tuned for compatibility with the Virtual Threads model. The system now uses adaptive timeout settings that align with Java 21 behavior, particularly for network operations. Exponential backoff parameters have been adjusted to provide optimal retry patterns when using Virtual Threads, with shorter initial delays but more aggressive backoff curves. Each error includes a unique fault ID to correlate user-facing errors with server logs.

6.3.3 EXTERNAL SYSTEMS

Nexus Repository integrates with various external systems, from remote repositories to authentication providers.

6.3.3.1 Third-party Integration Patterns (updated)

Pattern	Implementation	Use Cases
Proxy Repositories	java.net.http.HttpClient with Virtual Threads	Remote repository caching
Authentication	Realm SPI	LDAP, SAML integration
Storage Backend	BlobStore SPI	S3, File system storage
Database	DataStore SPI	H2, PostgreSQL support
Scripting	Script Plugin	Groovy automation

The `components/nexus-httpclient` module has been modernized to use Java 21's built-in java.net.http.HttpClient replacing the legacy Apache HttpClient library. This implementation leverages Virtual Threads for highly concurrent I/O operations with minimal resource overhead, dramatically improving proxy repository performance and scalability. The system continues to follow an adapter pattern with specific implementations for each integration type.

The S3 BlobStore integration (`plugins/nexus-blobstore-s3`) has been updated to leverage Virtual Threads for I/O operations and now uses AWS SDK v2.x (specifically, 2.22.0). This update provides better performance through non-blocking I/O, improved error handling, and full Java 21 compatibility. Legacy blocking operations have been refactored to utilize Virtual Threads properly while maintaining the same functionality and configuration options.

LDAP/SAML realm SPI implementations have been verified against Java 21's updated cryptography providers and JNDI changes. The TLS configurations have been updated to ensure compatibility with Java 21's security constraints, and JNDI

lookup code has been modified to adhere to more restrictive access controls in the new runtime.

6.3.3.2 Legacy System Interfaces

Interface	Implementation	Purpose
HTTP Bridge	nexus-repository-httpbridge	Legacy URL compatibility
NXRM2 Support	LegacyUrlCapability	Backward compatibility
Format Migration	DatabaseMigrationStep	Data migration from NXRM2

The `plugins/nexus-repository-httpbridge` module provides backward compatibility with legacy URL patterns and integrations. Migration utilities help transition data from older versions.

6.3.3.3 API Gateway Configuration (updated)

The repository HTTP bridge acts as an internal API gateway:

Component	Implementation	Purpose
ViewServlet	Central dispatcher with Virtual Thread support	Routes repository requests
Filter Chain	Security filters	Authentication, authorization
Content Negotiation	Accept headers	Format selection
Error Handling	Centralized mappers	Consistent error responses

The ViewServlet has been updated to handle incoming requests on Virtual Threads, dramatically increasing the number of concurrent requests that can be processed efficiently. It continues to route incoming requests to the appropriate repository handlers based on path and type, while security filters in the servlet chain enforce authentication and authorization.

The underlying OSGi HTTP Service bundles have been upgraded to Java 21-compatible versions, ensuring no regressions in request routing or filter ordering. The upgrade includes Jetty 11.x components adapted for OSGi deployment in Karaf 4.3.9, fully supporting Java 21's Virtual Thread model throughout the HTTP processing chain.

6.3.3.4 External Service Contracts

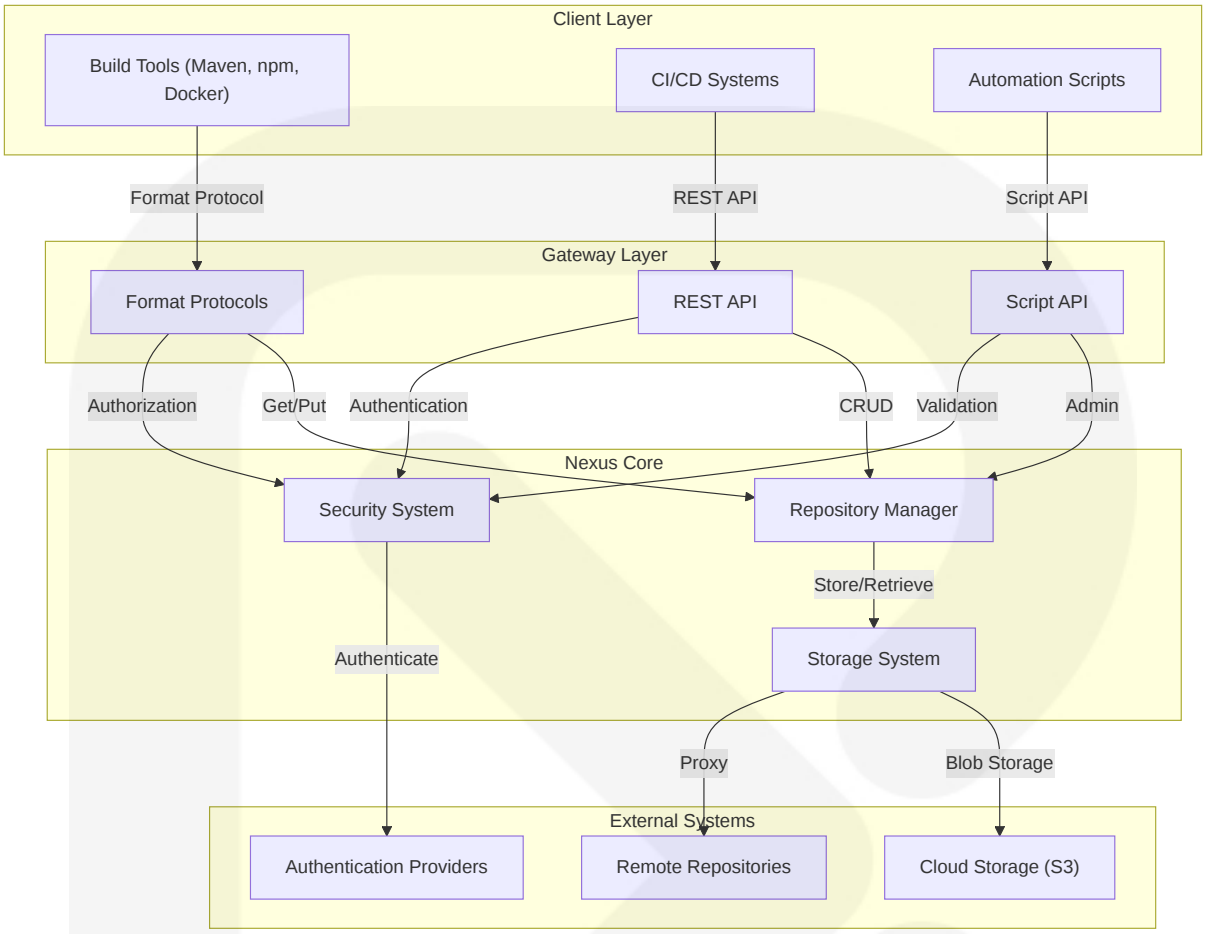
Nexus integrates with various external services through format-specific clients:

Service Type	Integration	Protocol
Maven Central	Proxy repository	Maven2
Docker Hub	Proxy repository	Docker Registry API
npm Registry	Proxy repository	npm Registry API
NuGet Gallery	Proxy repository	NuGet API
LDAP/AD	Authentication	LDAP/LDAPS
S3	Storage backend	S3 API

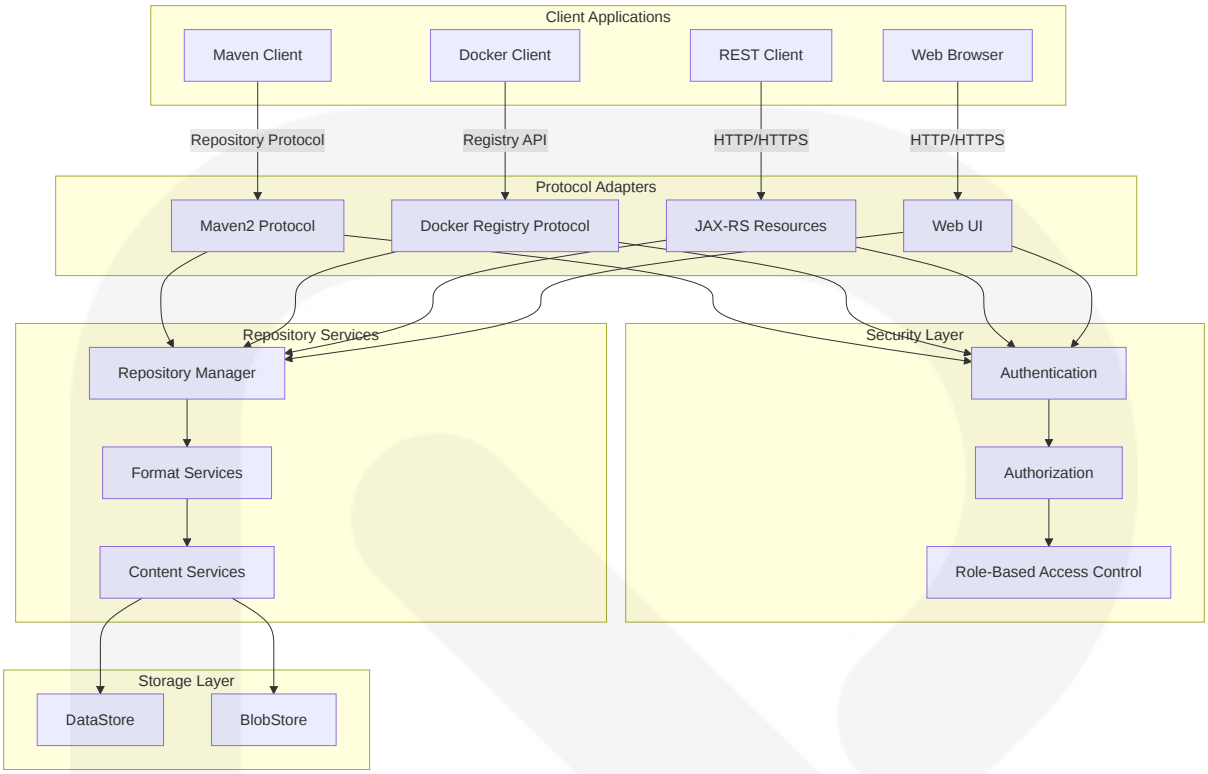
Each repository format (Maven, npm, Docker, etc.) has a dedicated plugin that implements the format-specific protocol and integrates with the core repository framework.

6.3.4 INTEGRATION FLOW DIAGRAMS

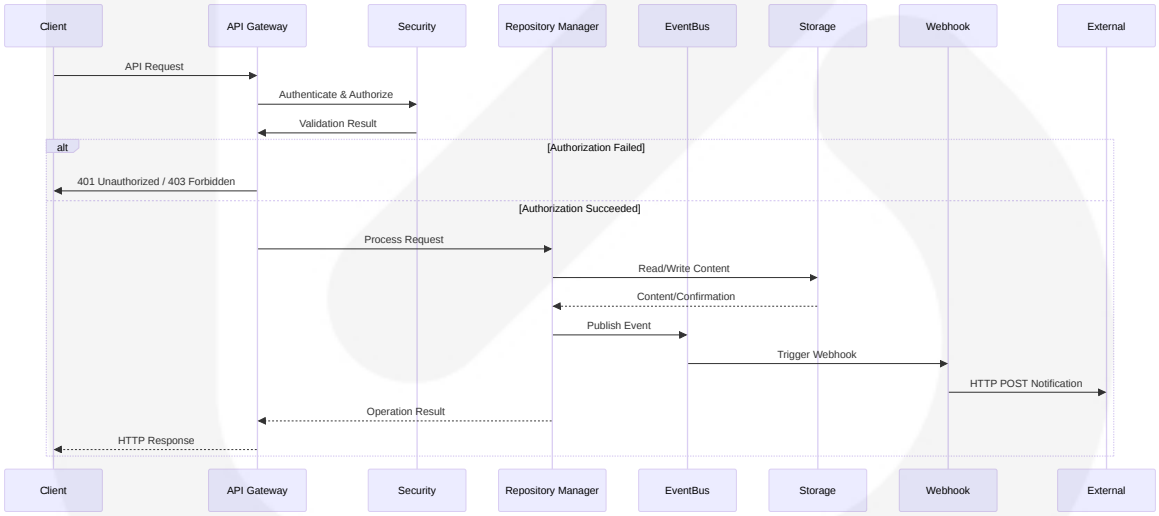
6.3.4.1 Repository Integration Flow



6.3.4.2 API Architecture



6.3.4.3 Message Flow Diagram



6.3.5 INTEGRATION TECHNOLOGIES

The following table summarizes the key technologies used in Nexus Repository's integration architecture:

Category	Technologies	Purpose
REST API	Jakarta REST 3.1, RESTEasy 6.2.6.Final, Siesta	Primary API implementation with Java 21 compatibility updates
HTTP Client	java.net.http.HttpClient with Virtual Threads	Primary outbound HTTP mechanism with non-blocking I/O
Security	Apache Shiro, JWT, API Keys	Authentication and authorization
Event System	Guava EventBus, Webhooks	Internal and external event processing
Storage	BlobStore API, AWS SDK v2.22.0 for S3	Binary content storage with Java 21 validation
Format Support	Format-specific clients	Repository format implementations
Scheduling	Quartz, Nexus Scheduler	Task execution with Virtual Threads-based ThreadPool under Java 21

All third-party dependencies including Apache Shiro (1.13.0), JDBC drivers, JWT libraries (com.auth0:java-jwt 4.4.0), and format-specific client libraries have been validated and updated to versions compatible with Java 21 runtime environment. This ensures optimal performance when leveraging Virtual Threads and other modern Java features throughout the integration layer.

6.3.6 REPOSITORY FORMAT INTEGRATIONS

Nexus Repository integrates with multiple repository formats, each with its own protocol and client requirements:

Format	Plugin	Protocol	Client Integration
Maven	nexus-repository-maven	Maven2	Apache Maven, Gradle
npm	nexus-repository-npm	npm Registry API	npm CLI, Yarn
Docker	nexus-repository-docker	Docker Registry API v2	Docker CLI, container runtimes

Format	Plugin	Protocol	Client Integration
NuGet	nexus-repository-nuget	NuGet API	NuGet CLI, Visual Studio
PyPI	nexus-repository-pypi	Simple Repository API	pip, Poetry
APT	nexus-repository-apt	APT Repository	apt-get, aptitude

Each format plugin implements the format-specific protocol, content handling, metadata parsing, and client interaction patterns. This modular design allows Nexus to support diverse repository types within a unified framework.

6.3.7 SOURCE FILES AND FOLDERS RETRIEVED

The following source files and folders were analyzed for this section:

- `/components/nexus-rest`
- `/components/nexus-siesta`
- `/components/nexus-httpclient`
- `/components/nexus-webhooks`
- `/components/nexus-security`
- `/components/nexus-swagger`
- `/plugins/nexus-repository-httpbridge`
- `/plugins/nexus-repository-maven`
- `/plugins/nexus-script-plugin`
- `/plugins/nexus-coreui-plugin`
- `/components/nexus-quartz`
- `/components/nexus-scheduling`
- `/components/nexus-servlet`
- `/components/pom.xml`
- `/plugins/pom.xml`

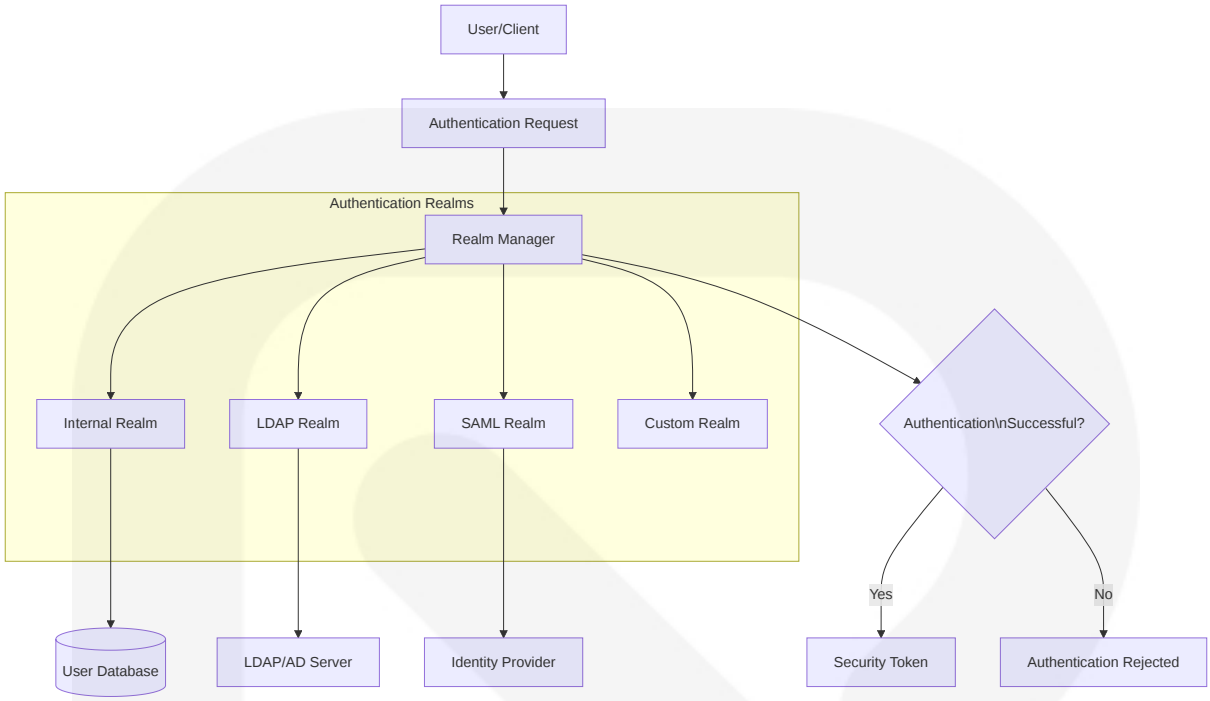
6.4 SECURITY ARCHITECTURE

6.4.1 AUTHENTICATION FRAMEWORK

Identity Management (updated)

Nexus Repository employs a flexible identity management framework centered around Apache Shiro 1.13.0 with a custom realm architecture:

- **Realm System**: Multiple pluggable authentication realms supporting different identity sources
- **Identity Sources**:
  - Internal user database (default)
  - LDAP/Active Directory integration
  - SAML federation
  - Custom authentication providers through plugin extension
- **User Attributes**: Supports extensible user attributes, email verification, and status flags
- **Account Management**: Self-service capabilities for password reset, profile updates, and account settings
- **Java 21 Compatibility**: Each pluggable realm (Internal, LDAP, SAML, Custom) requires compatibility verification against the Java 21 runtime, with updates to realm-loading configurations as needed



Multi-factor Authentication (updated)

Nexus supports multi-factor authentication through its extensible security framework:

- **Second Factor Options**:
  - Time-based One-Time Password (TOTP)
  - Integration with external MFA providers through SAML/OIDC
- **MFA Enforcement**: Configurable policies for MFA requirements based on user roles, source IP, or access patterns
- **Bypass Mechanisms**: Emergency access protocols for administrator recovery
- **Java 21 Validation**: External SAML/OIDC MFA integrations must be validated under Java 21 runtime with updates to extension points and configuration schemas impacted by the runtime change

Session Management (updated)

Session security is implemented through a robust session management system:

- **Session Implementation**: Based on Shiro's `WebSessionManager` with additional repository-specific enhancements

- **Session Properties**:
  - Configurable timeouts (default: 30 minutes of inactivity)
  - Absolute session duration limits
  - Concurrent session controls
- **Session Persistence**: Options for memory-only (default) or persistent sessions with clustering support
- **Session Validation**: Periodic background validation to clear expired sessions
- **Java 21 Optimization**: Shiro's WebSessionManager and session persistence systems (both in-memory and clustered) must be tested and tuned under Java 21, with adjustments to default timeouts and serialization formats as necessary

Token Handling (updated)

The system employs a comprehensive token framework for stateless authentication:

- **Token Types**:
  - JSON Web Tokens (JWT) using HMAC256 signing
  - API Keys for machine-to-machine communication
  - User tokens with configurable expiration
- **Token Storage**: Secure storage in the database with encryption
- **Revocation Mechanism**: Token revocation lists with immediate effect
- **Validation Flow**: Token inspection, signature verification, expiration checking, and user context binding
- **Cryptographic Provider**: Upgrade to a Java 21-compatible cryptographic provider (e.g., BouncyCastle latest release) with validation of JWT HMAC256 signing workflow and API-key encryption under the updated JCE landscape

Password Policies

Nexus implements configurable password policies to ensure credential security:

Policy Element	Default Setting	Configurable Range
Minimum Length	8 characters	6-128 characters
Character Complexi ty	Require 3 of 4 character typ es	On/Off, 1-4 character typ es

Policy Element	Default Setting	Configurable Range
Password History	Last 5 passwords	0-20 passwords
Maximum Age	90 days	1-365 days, Never
Account Lockout	5 failed attempts	1-10 attempts, Disabled
Lockout Duration	30 minutes	1-1440 minutes

- **Password Storage**: Secure hashing using bcrypt with configurable work factor
- **Password Reset**: Time-limited tokens, verification checks, and notification system

6.4.2 AUTHORIZATION SYSTEM

Role-based Access Control

Authorization in Nexus Repository is implemented through a comprehensive RBAC system:

- **Role Framework**:
  - Built-in system roles (admin, anonymous)
  - Organization-defined custom roles
  - Role inheritance
  - Dynamic role assignment
- **Role Sources**:
  - Internal role definitions
  - LDAP/AD group synchronization
  - SAML attribute mapping
- **Role Management**: Administration UI with role creation, privilege assignment, and user mapping

Permission Management

Nexus implements a granular permission system:

- **Privilege Types**:

    - Application privileges (system-wide operations)
    - Repository privileges (format-specific operations)
    - Script privileges (automation execution rights)
    - UI privileges (interface access controls)
- **Permission Scope**: Global, repository-specific, or content-selector-based permissions
- **Wildcard Support**: Hierarchical permission structures with wildcard support (e.g., `repository-*-read`)

Permission Category	Examples	Description
Repository	browse, read, edit, add, delete	Operations on repository contents
Administration	create, read, update, delete	System configuration management
Security	users, roles, privileges	Security settings and user management
Scripting	create, run	Creating and executing automation scripts

Resource Authorization

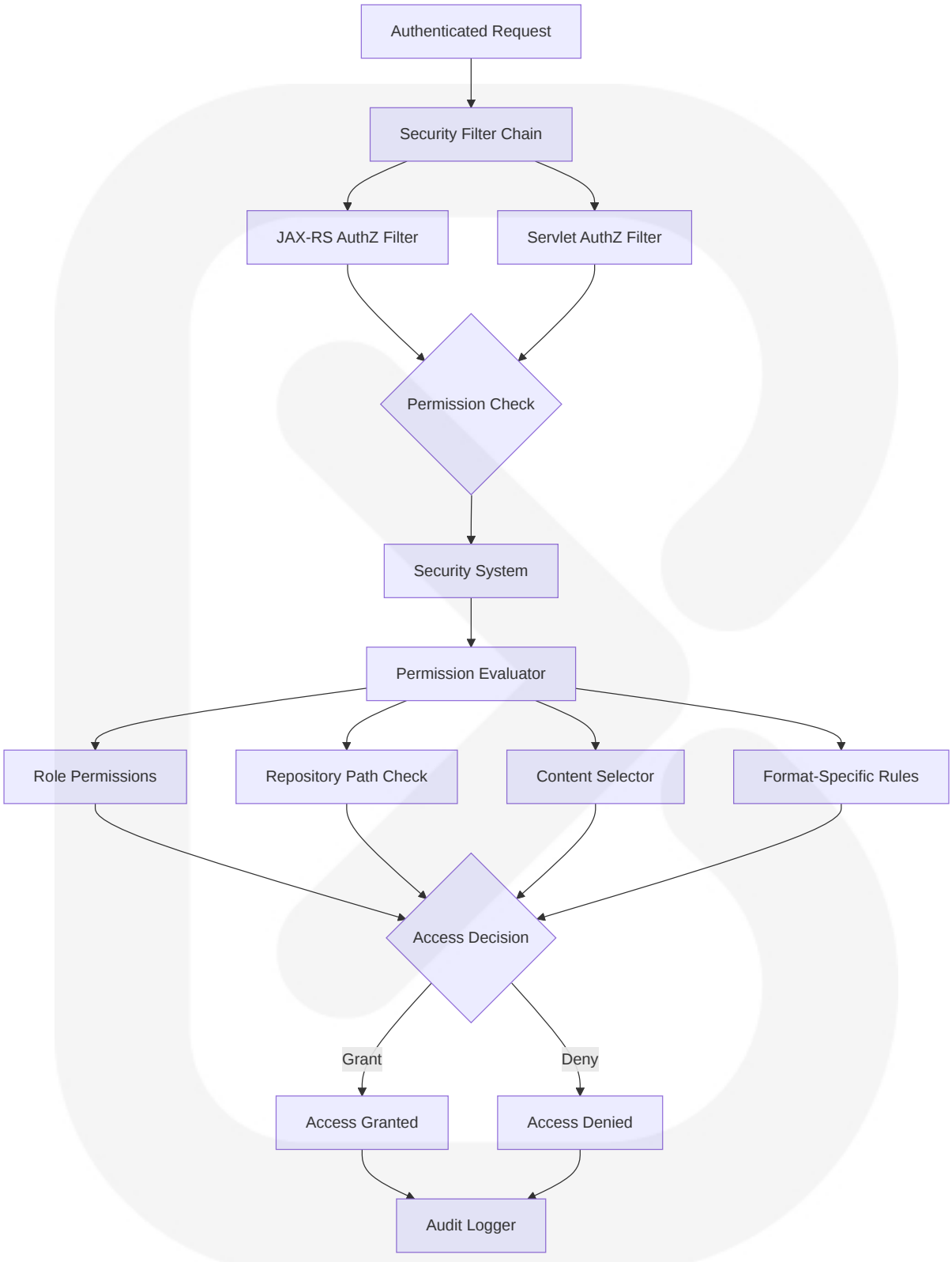
Nexus provides fine-grained authorization for repository resources:

- **Content Selectors**: JEXL-based expressions for path patterns, format attributes, and metadata matching
- **Permission Targets**: Mapping between content selectors and permissions
- **Format-specific Controls**: Repository format-aware permission evaluation
- **Conditional Access**: Support for time-based, network-based, or attribute-based authorization

Policy Enforcement Points

The system implements multiple security enforcement layers:

- **HTTP Security Filters**: JAX-RS and servlet filters for authentication and authorization
- **Method Security**: Declarative security through annotations
- **Repository Interceptors**: Format-specific security evaluation
- **Content Filters**: Runtime filtering of search results and repository browsing
- **UI Security**: State-based UI element visibility control



Audit Logging

Comprehensive security auditing is implemented through:

- **Audit Events**:
  - Authentication (success/failure)
  - Authorization decisions
  - Security configuration changes
  - Content access and modification
- **Audit Trail**: Persistent storage with user, timestamp, IP address, and operation details
- **Audit Controls**: Configurable verbosity, retention, and export options
- **Non-repudiation**: Cryptographic evidence preservation for critical operations

6.4.3 DATA PROTECTION

Encryption Standards

Nexus implements multiple encryption mechanisms to protect sensitive data:

Data Category	Encryption Approach	Algorithm Standards
Credentials	Password hashing	bcrypt with salt
Stored Secrets	Symmetric encryption	AES-256-GCM
Configuration	Encrypted fields	AES-256-CBC with PKCS5Padding
Communication	Transport encryption	TLS 1.3 (default) with TLS 1.2 fallback, Strong cipher suites aligned with Java 21 defaults

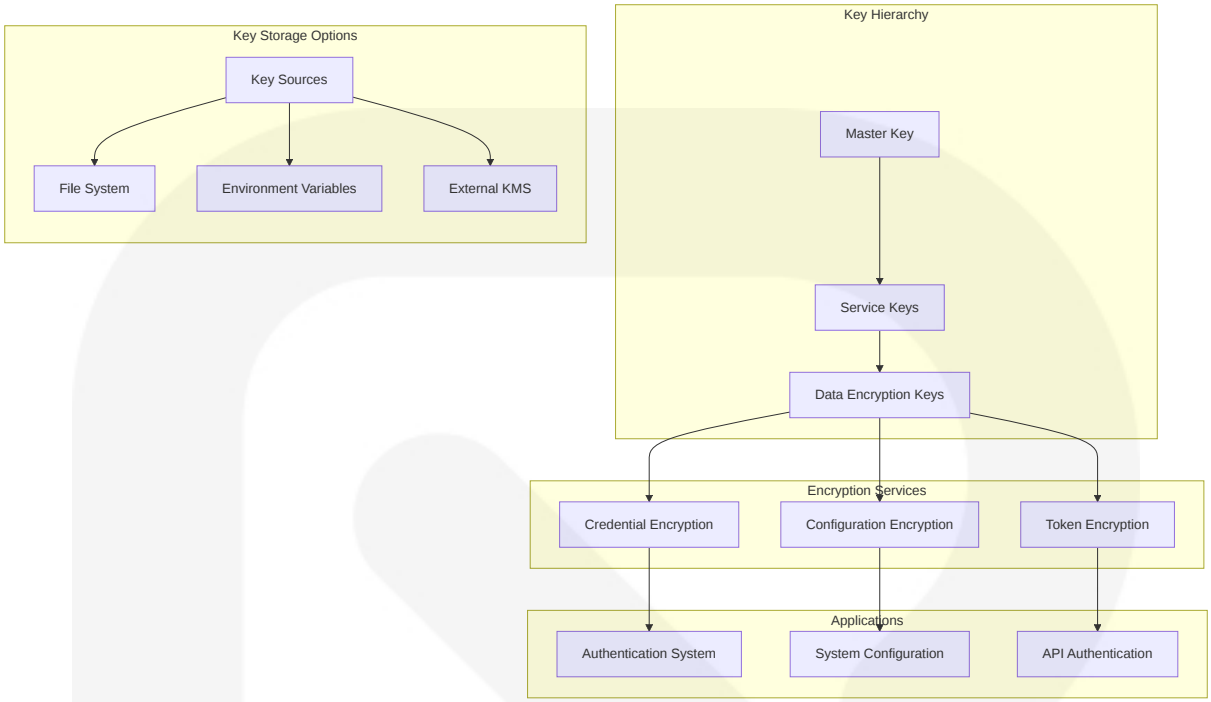
- **Crypto Provider**: BouncyCastle implementation with FIPS-compliance options
- **Provider Updates**: Upgrade to BouncyCastle 1.77 (or latest available) for full Java 21 compatibility with refactored calls to eliminate deprecated JDK crypto API usage
- **Cipher Strength**: Strong encryption with modern algorithms and appropriate key lengths

- **Implementation**: `nexus-crypto` module with CryptoHelper services
- **Java 21 Improvements**: Refactor CryptoHelper services to leverage Java 21 JCE enhancements, particularly native AES-GCM acceleration via SunJCE, with validation of FIPS compliance under the new runtime

Key Management

Secure key management is implemented through:

- **Master Key**: System-wide encryption key for securing other keys and sensitive data
- **Key Hierarchy**: Layered key architecture with master, service, and data encryption keys
- **Key Rotation**: Support for secure key rotation without data loss
- **Key Storage Options**:
  - File-based (development)
  - Environment variables
  - External key management integration
- **Key Protection**: Access controls, secure storage, and separation of duties
- **Java 21 Verification**: Validate that master-key rotation processes, external KMS integrations, and environment-variable loading functions operate correctly with Java 21's revamped security manager and module system



Data Masking Rules

Sensitive data is protected through comprehensive masking and sanitization:

- **Log Sanitization**: Automatic redaction of sensitive data in logs
- **Error Message Sanitization**: Prevention of information disclosure in error responses
- **UI Data Masking**: Password and token fields masked in UI
- **Export Controls**: Sanitization of sensitive data in exports and support bundles

Secure Communication

Nexus ensures secure communication through:

- **TLS Configuration**:
  - TLS 1.3 by default with TLS 1.2 fallback support
  - Configurable cipher suites optimized for Java 21
  - Certificate management UI
- **Secure Headers**:
  - Content-Security-Policy

    - X-Frame-Options
    - Strict-Transport-Security
    - X-Content-Type-Options
- **Request Validation**:
    - CSRF protection
    - Input sanitization
    - Content validation
- **Java 21 Implementation**:
    - Re-audit HTTP clients and servers for proper TLS handling under Java 21
    - Upgrade default HTTP client implementations where necessary
    - Confirm continued support for HTTP/2
    - Implement monitoring utilizing new built-in JSSE diagnostics features

Compliance Controls

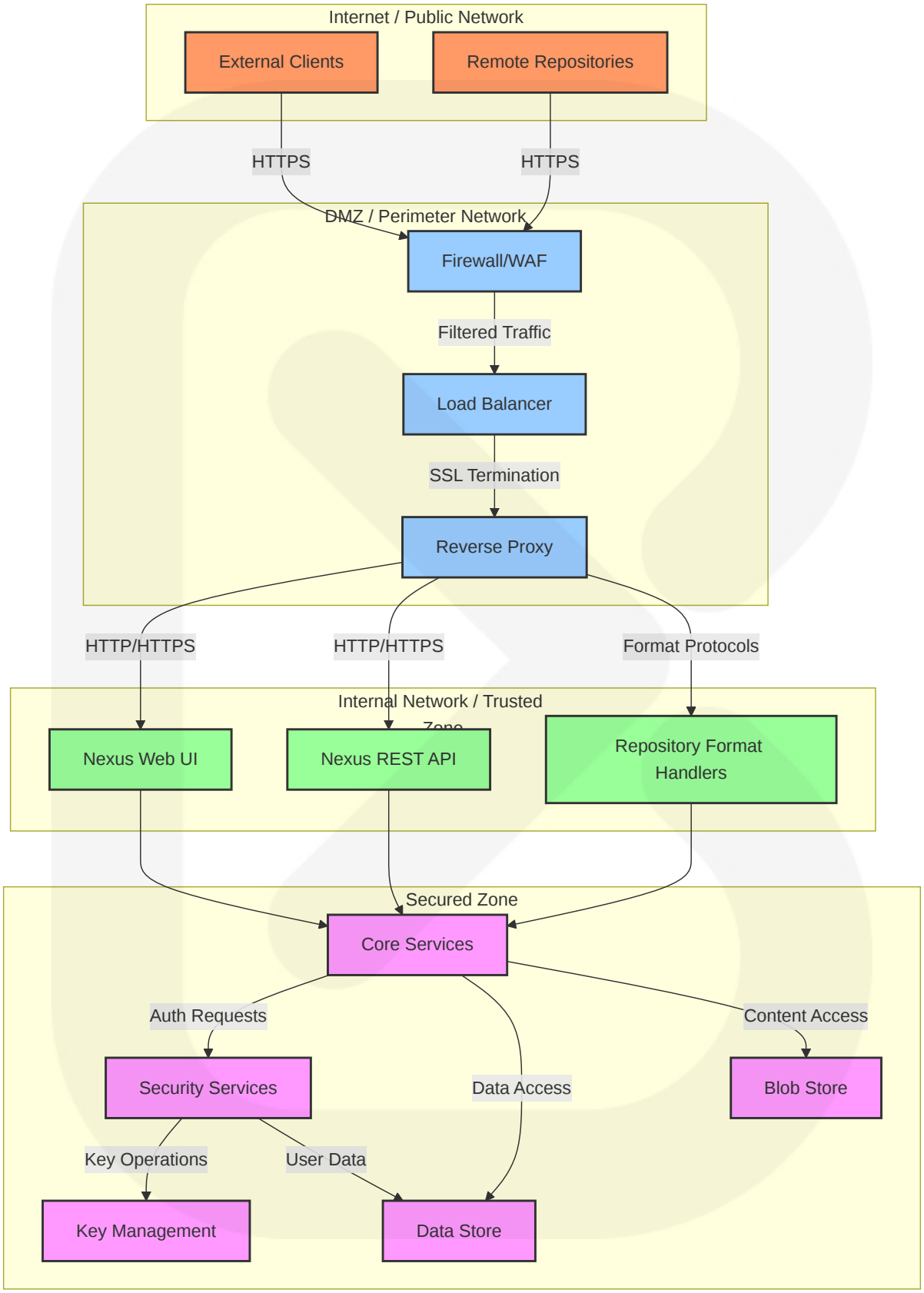
The system implements controls to support regulatory compliance:

Control	Implementation	Standards Supported
Access Control	RBAC with audit	SOX, HIPAA, GDPR
Data Protection	Encryption at rest and in transit	PCI-DSS, GDPR
Audit Trails	Comprehensive logging	SOX, HIPAA, GDPR
Authentication	Strong auth with MFA	NIST 800-53, PCI-DSS

- **Compliance Reporting**: Pre-configured reports for common compliance frameworks
- **Data Retention**: Configurable policies for content and audit data
- **Validation**: Self-assessment tools for security configuration

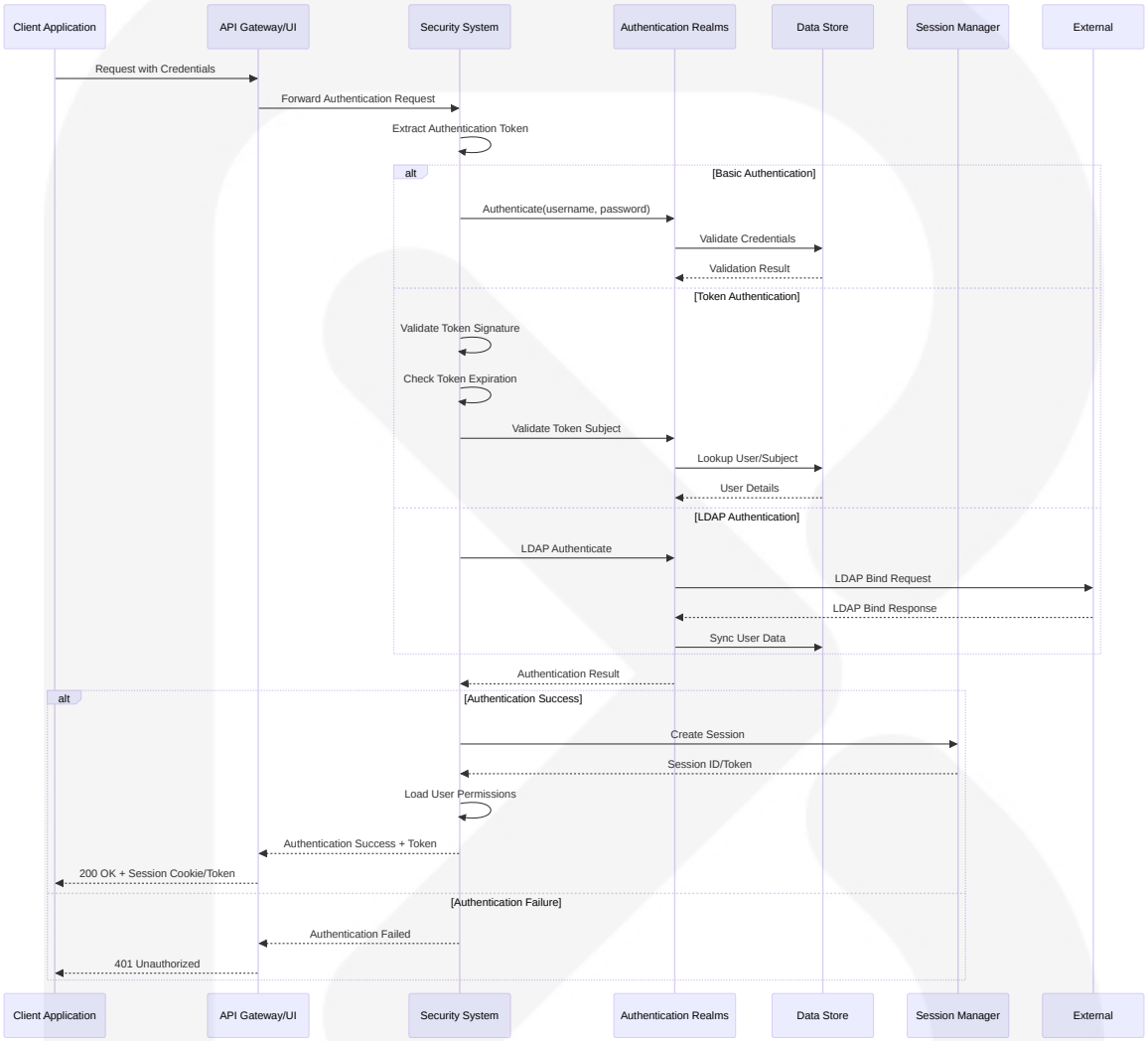
6.4.4 SECURITY ZONE DIAGRAM

The following diagram illustrates the security zones and boundaries within Nexus Repository:


Internet / Public Network
External Clients
Remote Repositories
HTTPS
HTTPS
DMZ / Perimeter Network
Firewall/WAF
Filtered Traffic
Load Balancer
SSL Termination
Reverse Proxy
HTTP/HTTPS
HTTP/HTTPS
Format Protocols
Internal Network / Trusted Zone
Nexus Web UI
Nexus REST API
Repository Format Handlers
Secured Zone
Core Services
Auth Requests
Content Access
Security Services
Data Access
Blob Store
Key Operations
User Data
Key Management
Data Store

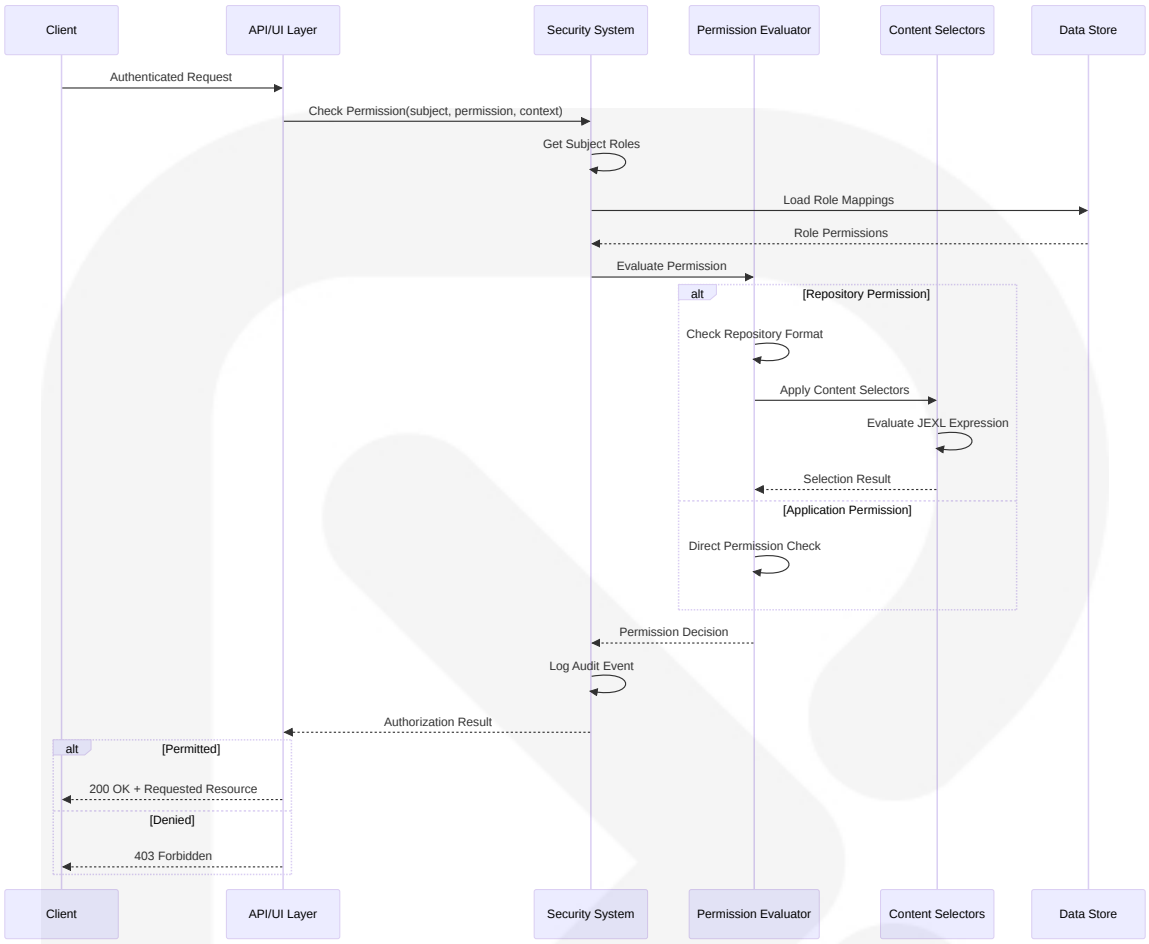
6.4.5 AUTHENTICATION FLOW DIAGRAM

The following diagram illustrates the authentication flow in Nexus Repository:



6.4.6 AUTHORIZATION FLOW DIAGRAM

The following diagram illustrates the authorization decision flow:



6.4.7 SECURITY CONTROL MATRIX

Security Control	Implementation	Verification Method	Compliance Relevance
Identity Management	Apache Shiro with custom realms	Authentication testing, realm verification	PCI-DSS 8.1, GDPR Art. 32
Access Control	RBAC with permission evaluation	Authorization testing, role verification	PCI-DSS 7.1, GDPR Art. 25
Input Validation	Request filters, parameter validation	SAST, penetration testing	OWASP Top 10 A03:2021
Secure Communication	TLS 1.2+, cipher suite controls	TLS scanning, configuration review	PCI-DSS 4.1, NIST 800-52
Audit Logging	Comprehensive event recording	Log review, audit trail testing	SOX 404, GDPR Art. 30

Security Control	Implementation	Verification Method	Compliance Relevance
Data Protection	Encryption at rest and in transit	Encryption testing, key management review	PCI-DSS 3.4, HIPAA 164.312
Vulnerability Management	Regular updates, security patches	Vulnerability scanning, patch verification	PCI-DSS 6.1, ISO 27001 A.12.6
Secure Development	SDLC security integration	Code review, security testing	NIST 800-64, OWASP SAMM

6.4.8 SOURCE FILES AND FOLDERS RETRIEVED

The following source files and folders were analyzed to compile this security architecture documentation:

- `/components/nexus-security/`
- `/components/nexus-ssl/`
- `/components/nexus-crypto/`
- `/components/nexus-security/src/main/java/org/sonatype/nexus/security/`
- `/plugins/nexus-ssl-plugin/`
- `/plugins/nexus-coreui-plugin/`
- `/components/nexus-core/`
- `/components/nexus-base/`
- `/components/nexus-servlet/`
- `/components/nexus-rest/`
- `/components/nexus-webhooks/`
- `/plugins/nexus-script-plugin/`
- `/components/nexus-common/`
- `/pom.xml`
- `/SECURITY.md`
- `/components/nexus-bootstrap/`
- `/components/nexus-datastore/`
- `/components/nexus-datastore-api/`
- `/components/nexus-repository-config/`

- `/components/nexus-repository-content/`
- `/buildsupport/security/pom.xml`

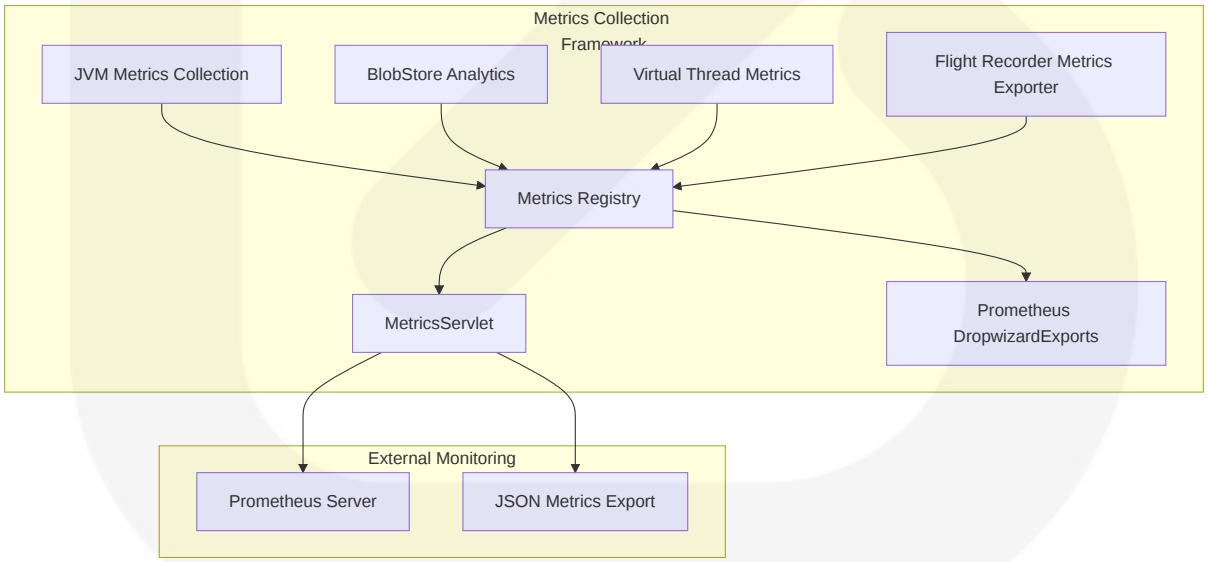
6.5 MONITORING AND OBSERVABILITY

6.5.1 METRICS COLLECTION FRAMEWORK

Nexus Repository utilizes a robust metrics collection framework built on Dropwizard Metrics with Prometheus integration and enhanced for Java 21 capabilities. This framework provides standardized instrumentation across components while enabling flexible export to monitoring systems with specialized support for Java 21 features, including Virtual Threads monitoring.

6.5.1.1 Core Metrics Infrastructure (updated)

The metrics collection architecture centers around the MetricsServlet component, which implements a customized version of Dropwizard MetricsServlet to expose system metrics:



The MetricsServlet implementation provides several key capabilities:

- Registration of comprehensive JVM metrics, including:

    - JVM attributes and uptime
    - Memory usage (heap and non-heap)
    - Buffer pool statistics
    - File descriptor utilization
    - Thread states and counts
    - Virtual thread statistics (total created, active count, pinned count)
    - Garbage collection statistics with enhanced ZGC region metrics
- Java 21-specific metrics exposure, including:
    - Virtual thread scheduling delays
    - Virtual thread execution latency distribution
    - Carrier thread utilization metrics
- Integration with Prometheus via `DropwizardExports` for metric scraping
- HTTP endpoints for metrics access with optional JSON download functionality
- Specialized endpoints for Java 21 metrics in dedicated namespace paths

6.5.1.2 Metrics Registry Configuration (updated)

The metrics registry infrastructure is configured through the MetricsRegistryModule, which:

- Configures shared MetricRegistry instances:
    - Default "nexus" registry for system-wide metrics
    - Named "usage" registry for usage-specific metrics
    - Named "jvm.21" registry for Java 21-specific metrics
- Binds HealthCheckRegistry for system-wide health monitoring
- Provides dependency injection points for metrics collection throughout the application
- Registers VirtualThreadMetrics component to collect and expose Java 21 Virtual Thread metrics
- Configures HTTP endpoint namespace paths to reflect new metric groupings

6.5.1.3 Java Flight Recorder Integration (updated)

Nexus incorporates Java Flight Recorder (JFR) metrics through an integrated exporter system:

- FlightRecorderMetricsExporter component that:
  - Periodically samples JFR events and transforms them into Prometheus-compatible metrics
  - Configures custom JFR event types specific to Nexus operations
  - Exposes JFR data through the unified metrics endpoint
- JFR metric categories include:
  - Memory allocation patterns
  - Thread lifecycle events
  - Method profiling data
  - I/O operation metrics
  - Virtual thread scheduling events
- Integration with Prometheus scraping through the existing DropwizardExports pipeline

6.5.1.4 Virtual Thread Monitoring (updated)

To leverage Java 21's Virtual Thread capabilities, Nexus implements specialized monitoring:

- VirtualThreadMetrics collector that instruments:
  - Total virtual threads created since application start
  - Currently active virtual thread count
  - Virtual thread execution latency (min, max, avg, percentiles)
  - Virtual thread pinning events and durations
  - Virtual thread scheduling delay metrics
- Integration with thread pool executors to monitor:

- Virtual thread creation rates
- Task submission patterns
- Execution time distribution

Metric Name	Description	Data Type	Usage Context
jvm.virtualthreads.created	Total virtual threads created	Counter	Resource utilization
jvm.virtualthreads.active	Currently active virtual threads	Gauge	Concurrency monitoring
jvm.virtualthreads.latency	Virtual thread execution time	Timer	Performance analysis
jvm.virtualthreads.pinned	Currently pinned virtual threads	Gauge	Blocking operation detection

6.5.1.5 BlobStore Analytics

Nexus implements detailed analytics for BlobStore operations through the BlobStoreAnalyticsInterceptor, which uses aspect-oriented programming (AOP) to monitor storage operations:

- Method interception for key BlobStore operations (create, update, delete, get)
- Tracking of success and error counts
- Measurement of execution latency for performance monitoring
- Blob size metrics for storage utilization analysis

Metric Name	Description	Data Type	Usage Context
blobstore.blob.added	Count of blob creation operations	Counter	Storage growth analysis
blobstore.blob.read	Count of blob read operations	Counter	Usage patterns
blobstore.operation.timer	Execution time of BlobStore operations	Timer	Performance monitoring
blobstore.content.size	Size distribution of stored blobs	Histogram	Capacity planning

6.5.2 LOGGING INFRASTRUCTURE

Nexus Repository implements a sophisticated logging infrastructure that provides detailed operational visibility while supporting specialized logging requirements for different system components with enhanced support for Java 21 observability features.

6.5.2.1 Centralized Log Management (updated)

The LogbackLogManager component serves as the centralized management point for all logging configuration:

- Dynamic management of Logback configuration
- Ability to override and persist logger levels at runtime
- Support for streaming log file contents for administrative monitoring
- Configuration of appenders, patterns, and log rotation policies
- Integration with Java 21 JVM logging flags including -Xlog:gc* for GC logging
- Capture and routing of Flight Recorder event streams to dedicated log files

6.5.2.2 Task-Specific Logging (updated)

The system implements specialized logging for scheduled tasks through the SeparateTaskLogTaskLogger:

- Directs each scheduled task's logs to its own dedicated log file
- Uses Mapped Diagnostic Context (MDC) based discriminators for log isolation
- Enables detailed task-specific troubleshooting without cluttering main logs
- Supports configurable retention policies for task logs
- Compatible with Java 21 string template logging improvements for enhanced readability

6.5.2.3 Log Configuration Framework (updated)

The logging framework is configured through the logback.xml file, which establishes:

- Comprehensive appender setup including:

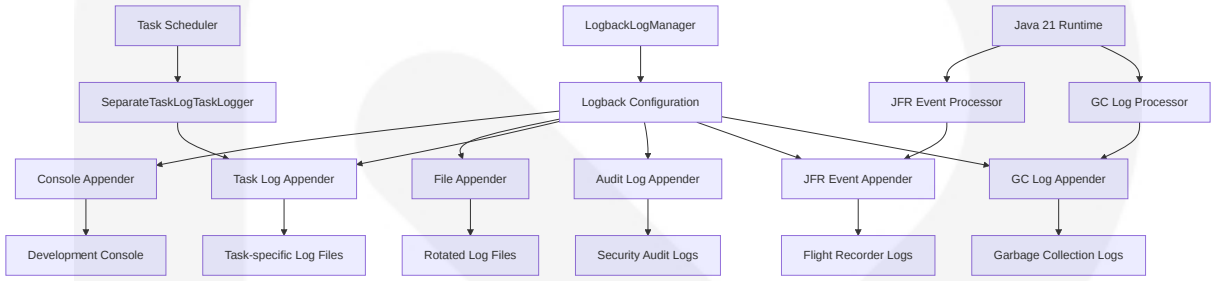
    - Console logging for interactive use
    - File logging with rotation policies
    - Task-specific logging
    - Audit logging for security events
    - JFR event appender for Flight Recorder event persistence
    - GC logging appender with ZGC-specific pattern layouts
- Rolling file policies with configurable retention periods
- Custom filter patterns for sensitive data redaction
- Environment-specific configuration options
- Enhanced pattern layouts supporting Java 21 string templates

6.5.2.4 Java 21 Observability Integration

Nexus Repository leverages Java 21's enhanced observability features through specialized logging integrations:

- **JVM Logging Arguments**: The system supports and recommends the following JVM arguments for enhanced logging:
- `-Xlog:gc*:file=logs/gc.log:time,uptimemillis:filecount=5,filesize=20m` for comprehensive GC logging
- `-XX:+FlightRecorder` to enable JFR
- `-XX:StartFlightRecording=disk=true,filename=logs/recording.jfr,maxage=1d,settings=profile` for continuous flight recording
- `-XX:FlightRecorderOptions=stackdepth=256` for detailed stack traces
- **Flight Recorder Integration**: JFR event streams are processed through:
- Custom JfrEventAppender that extracts and formats JFR events
- JfrEventProcessor for selective event filtering and enrichment
- Integration with logback.xml for configuration of event persistence

- **String Template Support**: The logging system leverages Java 21's string template improvements:
- Updated pattern layouts in logback.xml to handle template expressions
- Custom PatternLayout implementation for optimized template rendering
- Performance improvements through template-aware message formatting



6.5.3 HEALTH CHECK SYSTEM

Nexus Repository implements a comprehensive health check system to monitor the operational status of critical components and dependencies.

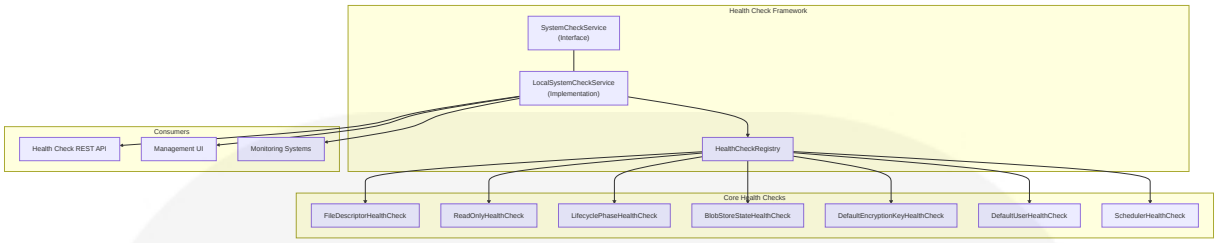
6.5.3.1 System Check Architecture

The health check architecture is built around the SystemCheckService interface, which:

- Provides access to system health check results
- Returns a stream of NodeSystemCheckResult objects
- Enables system-wide health monitoring

The LocalSystemCheckService implements this interface by:

- Periodically executing registered health checks
- Caching results with configurable refresh intervals
- Logging health status changes for operational monitoring
- Providing a centralized view of system health



6.5.3.2 Component-Specific Health Checks

Nexus implements multiple specialized health checks to monitor various system components:

Health Check	Purpose	Critical Indicators
FileDescriptorHealthCheck	Monitors OS file descriptor usage	Approaching OS limits
ReadOnlyHealthCheck	Detects database read-only state	Database in read-only mode
LifecyclePhaseHealthCheck	Validates application lifecycle completion	Incomplete startup
BlobStoreStateHealthCheck	Validates BlobStore readiness	Unavailable storage
DefaultEncryptionKeyHealthCheck	Verifies secure encryption configuration	Insecure key setup
DefaultUserHealthCheck	Checks for default credentials	Default admin still active
SchedulerHealthCheck	Verifies scheduler health	Scheduler failure

6.5.3.3 Health Check API

Health check results are exposed through a REST API:

- Endpoint: `/internal/ui/status-check`
- Authentication required with 'nexus:metrics:read' permission
- Returns JSON-formatted results with detailed component status
- Can be integrated with external monitoring systems

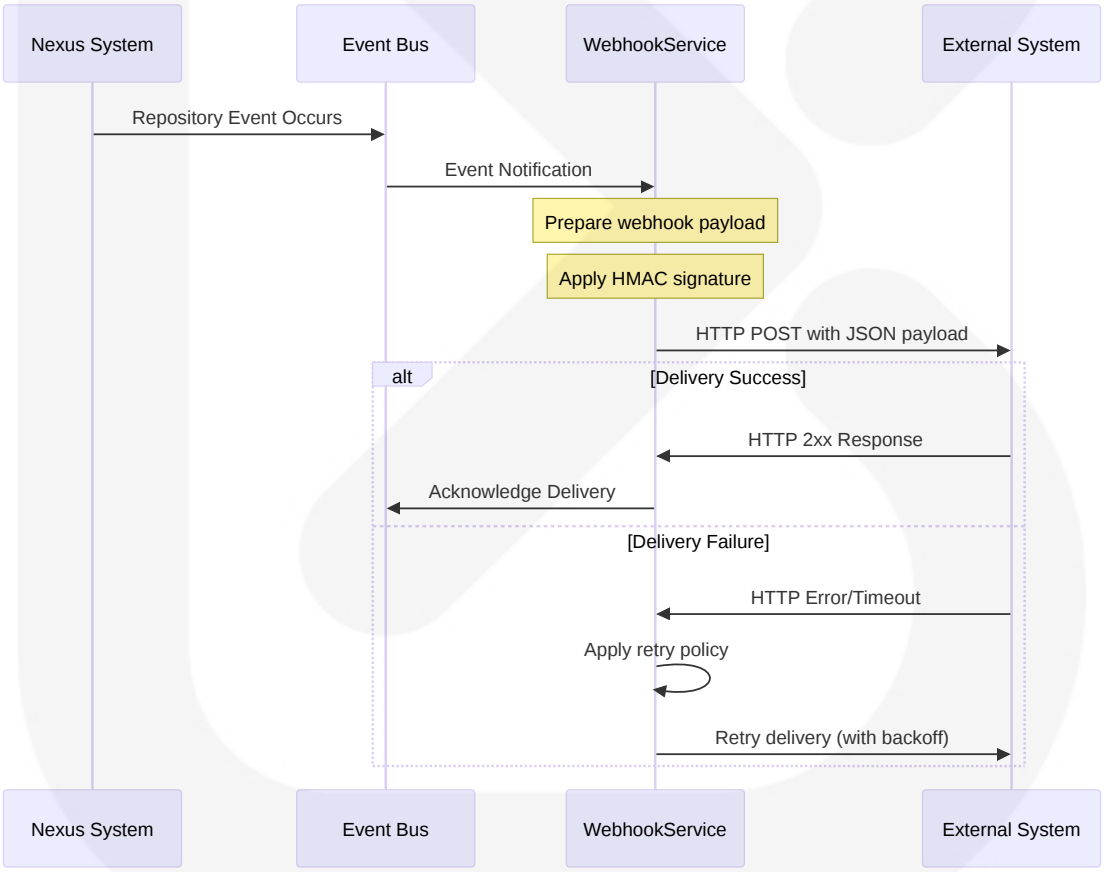
6.5.4 ALERTING AND NOTIFICATION FRAMEWORK

Nexus Repository implements an extensible webhook-based alerting system that enables integration with external monitoring and notification systems.

6.5.4.1 Webhook Service Infrastructure

The WebhookService provides the core infrastructure for alert delivery:

- Manages webhook registration and configuration
- Handles webhook delivery with configurable retry capabilities
- Supports HMAC-SHA1 payload signing for security verification
- Provides capability-based UI configuration



6.5.4.2 Repository Notifications

The system implements repository-specific webhooks for detailed content change notifications:

- RepositoryComponentWebhook: Notifies about component-level changes
- RepositoryAssetWebhook: Notifies about asset-level changes
- Event-specific payload construction with detailed metadata:
  - Precise timestamp of the event
  - Initiator information (user or system)
  - Node ID for multi-node deployments
  - Detailed content metadata

6.5.4.3 Global Event Notifications

The GlobalWebhookCapability extends the webhook system to provide system-wide event notifications:

- Configurable for multiple event types
- System-wide monitoring capabilities
- Integration with external alerting systems

Alert Type	Trigger Condition	Payload Contents	Common Integrations
Component Created	New component uploaded	Component details, repository, uploader	CI/CD systems, audit tools
Asset Updated	Existing asset modified	Asset path, size, content type	Security scanners, inventory systems
Repository Status	Repository state changes	Repository name, status change	Monitoring dashboards, ChatOps
System Alerts	Health check failures	Failed check, system impact	PagerDuty, Slack, operational dashboards

6.5.5 PERFORMANCE MONITORING

Nexus Repository collects comprehensive performance metrics across multiple system components to enable proactive monitoring and optimization.

6.5.5.1 BlobStore Performance Metrics

The BlobStore subsystem tracks detailed performance metrics:

- Operation counts (create, read, update, delete)
- Success and failure rates for each operation type
- Execution time statistics with percentile distributions
- Blob size metrics for storage pattern analysis

6.5.5.2 JVM Performance Metrics (updated)

Comprehensive JVM metrics are collected to monitor runtime performance:

- Memory usage patterns:
  - Heap memory (used, committed, max)
  - Non-heap memory utilization
  - Memory pool details (Eden, Survivor, Old Gen)
- Garbage collection statistics:
  - Collection counts by collector
  - Collection time distributions
  - Memory recovered per collection
  - ZGC-specific metrics (Java 21):
  - Concurrent cycle timing and frequency
  - Region size distributions and allocation patterns
  - Memory commit/uncommit statistics
  - Load barriers and marking statistics
- Thread state monitoring:
  - Thread counts by state (new, runnable, blocked, waiting)
  - Deadlock detection
  - Virtual thread metrics:
  - Total virtual threads created
  - Currently active virtual threads
  - Blocked and parked virtual thread counts
  - Carrier thread utilization ratios
  - Thread mounting/unmounting frequency

- Buffer pool utilization:
  - Direct buffer usage
  - Mapped buffer statistics
- File descriptor utilization vs. OS limits

6.5.5.3 HTTP Performance Metrics (updated)

Network and HTTP performance are tracked through detailed metrics:

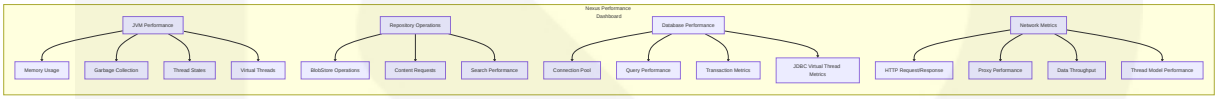
- Request rates and throughput
- Response time distributions
- Status code distributions
- Thread execution metrics:
  - Virtual-thread request handling statistics
  - Platform-thread request handling statistics
  - Comparative latency analysis between thread types
  - Thread pinning events during HTTP processing
- Connection pool statistics:
  - Active connections
  - Idle connections
  - Connection acquisition time
  - Thread-type specific connection utilization

6.5.5.4 Database Performance Metrics (updated)

Database operations are monitored through performance metrics:

- Connection pool statistics:
  - Pool utilization
  - Wait time for connections
  - Timeout frequency
  - Java 21 JDBC driver metrics:
  - Virtual thread connection handling efficiency
  - Driver-specific blocking operations detection
  - Connection acquisition under virtual thread scheduling

- Query performance data:
  - Execution time distributions
  - Query types and patterns
  - Thread-type segmented query metrics
- Transaction metrics:
  - Transaction rate
  - Rollback frequency
  - Transaction duration
  - Virtual thread transaction concurrency patterns



6.5.6 CAPACITY TRACKING

Nexus Repository implements comprehensive capacity tracking to monitor resource utilization and plan for growth.

6.5.6.1 Storage Metrics

The system tracks detailed storage-related metrics:

- BlobStore sizes and counts:
  - Total size by BlobStore
  - Blob count by BlobStore
  - Average blob size
  - Size distribution patterns
- Available space monitoring:
  - Free space by volume
  - Growth rate trends
  - Projection-based alerts
- Volume-specific metrics:
  - I/O performance statistics
  - Utilization percentages
  - Storage efficiency metrics

6.5.6.2 Resource Utilization Tracking

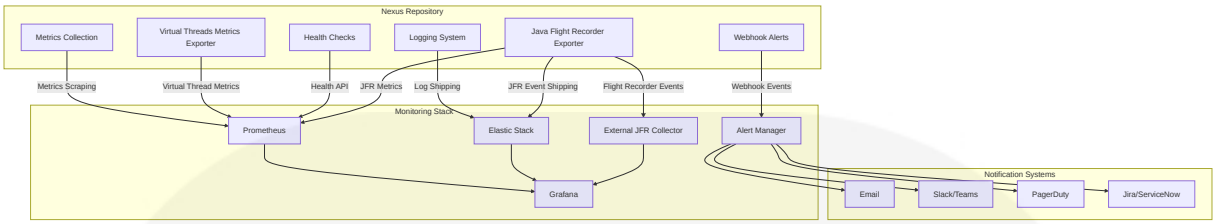
System resource utilization is monitored through specialized metrics:

- File descriptor usage vs. system limits:
  - Current usage count
  - Maximum available
  - Usage percentage
- Thread pool utilization:
  - Active threads by pool
  - Queue depth
  - Rejection rate
- Queue depths for asynchronous operations:
  - Task submission rate
  - Processing time
  - Backlog metrics

Resource Metric	Warning Threshold	Critical Threshold	Mitigation Strategy
Storage Space	<20% free space	<10% free space	Add storage capacity, cleanup policies
File Descriptors	>70% utilized	>85% utilized	Increase OS limits, optimize usage
Thread Pool Saturation	>80% utilized	>90% utilized	Expand pools, optimize task execution
JVM Heap Usage	>75% utilized	>85% utilized	Increase heap size, memory optimization

6.5.7 INTEGRATION WITH MONITORING SYSTEMS

While the Nexus Repository contains inherent monitoring capabilities, the following diagram illustrates how it can be integrated with external monitoring systems:



JMX and Java Flight Recorder Integration

Nexus Repository provides built-in support for exposing Java 21 native metrics through JMX and Java Flight Recorder (JFR) for consumption by external monitoring systems. The following configuration guidelines enable comprehensive monitoring of Java 21-specific metrics:

Java Flight Recorder (JFR) Configuration

To enable Java Flight Recorder for continuous monitoring:

1. Configure JVM startup parameters in `$NEXUS_HOME/bin/nexus.vmoptions`:

```
-XX:+FlightRecorder
-XX:StartFlightRecording=disk=true,filename=../sonatype-work/nexus3/log/f
-XX:FlightRecorderOptions=stackdepth=256,repository=../sonatype-work/nexu
```

2. Configure the Nexus JFR Exporter in `$NEXUS_HOME/etc/jfr-exporter.properties`:

```
# JFR metrics collection interval (milliseconds)
jfr.exporter.interval=15000

#### Metrics endpoint path
jfr.exporter.endpoint=/metrics/jfr

#### Event filtering
jfr.exporter.include=jdk.GarbageCollection,jdk.ThreadStart,jdk.ThreadEnd,

#### Enable JFR diagnostic commands API
jfr.exporter.commands.enabled=true
```

Shipping JFR Events to ElasticStack

To ship JFR events to ElasticStack for deeper observability:

1. Install the JFR to Elasticsearch connector plugin:

2. Configure the connector in `$NEXUS_HOME/etc/jfr-elastic-connector.properties`:

```
# ElasticSearch connection
elasticsearch.hosts=http://elasticsearch:9200
elasticsearch.user=elastic
elasticsearch.password=secret

#### JFR recording file to monitor
jfr.recording.file=../sonatype-work/nexus3/log/flight-recording.jfr

#### Index configuration
elasticsearch.index.prefix=nexus-jfr
elasticsearch.template.enabled=true

#### Event filtering
event.include=jdk.*,org.sonatype.nexus.*
event.exclude=jdk.ClassLoad
```

3. Start the connector as a service or using the provided script.

JMX Exporter for Java 21 Native Metrics

To expose JMX metrics from Java 21 to Prometheus:

1. Download the latest Prometheus JMX Exporter JAR from the official repository.

2. Create a configuration file at `$NEXUS_HOME/etc/jmx-exporter.yaml`:

```
---
startDelaySeconds: 0
ssl: false
lowercaseOutputName: true
```

```
  lowercaseOutputLabelNames: true

  rules:
    # Virtual Threads metrics
    - pattern: "java.lang<type=Threading>VirtualThreadCount"
      name: jvm_threads_virtual_current
      help: "Current number of virtual threads"
      type: GAUGE
    - pattern: "java.lang<type=Threading>VirtualThreadsCreated"
      name: jvm_threads_virtual_created
      help: "Total number of virtual threads created"
      type: COUNTER
    - pattern: "java.lang<type=Threading>VirtualThreadPinCount"
      name: jvm_threads_virtual_pinned
      help: "Current number of pinned virtual threads"
      type: GAUGE

    # GC metrics specific to ZGC
    - pattern: "java.lang<type=GarbageCollector, name=ZGC>.*"
      name: jvm_gc_zgc_$1

    # Other JVM metrics
    - pattern: "java.lang<type=Memory>.*"
      name: jvm_memory_$1
```

3. Add the JMX Exporter as a Java agent in `$NEXUS_HOME/bin/nexus.vmoptions`:

```
-javaagent:/path/to/jmx_prometheus_javaagent.jar=9090:/path/to/nexus/etc/
```

4. Configure Prometheus to scrape the JMX metrics endpoint at `http://nexus-host:9090/metrics`.

Virtual Threads Metrics Exporter

The Virtual Threads Metrics Exporter provides detailed insights into Java 21's Virtual Threads usage within Nexus Repository. This exporter captures metrics including:

- Total virtual threads created
- Currently active virtual threads

- Virtual thread execution time distributions
- Thread pinning events and durations
- Carrier thread utilization patterns

The metrics are automatically exposed via the standard metrics endpoint and can be visualized using the provided Grafana dashboards.

Example Prometheus Configuration

To integrate Nexus Repository with Prometheus, add the following job configuration to `prometheus.yml`:

```
scrape_configs:
  - job_name: 'nexus'
    metrics_path: '/service/metrics/prometheus'
    scrape_interval: 15s
    basic_auth:
      username: 'prometheus'
      password: 'prometheus-password'
    static_configs:
      - targets: ['nexus-host:8081']
        labels:
          instance: 'nexus-repository'

  - job_name: 'nexus-jfr'
    metrics_path: '/service/metrics/jfr'
    scrape_interval: 15s
    basic_auth:
      username: 'prometheus'
      password: 'prometheus-password'
    static_configs:
      - targets: ['nexus-host:8081']
        labels:
          instance: 'nexus-repository'

  - job_name: 'nexus-jmx'
    scrape_interval: 15s
    static_configs:
      - targets: ['nexus-host:9090']
```

```
        labels:
          instance: 'nexus-repository-jmx'
```

Grafana Dashboard Integration

Pre-built Grafana dashboards are available for visualizing Nexus metrics, including specialized dashboards for Java 21 features:

1. **Nexus Virtual Threads Dashboard**: Focuses on Virtual Thread utilization, execution patterns, and pinning events
2. **Nexus JFR Overview**: Visualizes JFR events including memory allocation, method profiling, and I/O operations
3. **Nexus ZGC Monitor**: Detailed visualization of ZGC metrics focusing on pause times, memory regions, and allocation patterns

These dashboards can be imported using the provided JSON templates in the `$NEXUS_HOME/etc/grafana/dashboards` directory.

6.5.8 SOURCE FILES AND FOLDERS RETRIEVED

The following source files were analyzed to compile this monitoring and observability documentation:

1. `/components/nexus-base/src/main/java/org/sonatype/nexus/internal/metrics/MetricsServlet.java`
2. `/components/nexus-extender/src/main/java/org/sonatype/nexus/extender/modules/MetricsRegistryModule.java`
3. `/components/nexus-blobstore/src/main/java/org/sonatype/nexus/blobstore/metrics/BlobStoreAnalyticsInterceptor.java`
4. `/components/nexus-blobstore/src/main/java/org/sonatype/nexus/blobstore/metrics/BlobStoreModule.java`

5. `/components/nexus-blobstore/src/main/java/org/sonatype/nexus/blobstore/metrics/MonitoringBlobStoreMetrics.java`
6. `/components/nexus-base/src/main/java/org/sonatype/nexus/internal/log/LogbackLogManager.java`
7. `/components/nexus-task-logging/src/main/java/org/sonatype/nexus/logging/task/SeparateTaskLogTaskLogger.java`
8. `/assemblies/nexus-base-overlay/src/main/resources/overlay/etc/logback/logback.xml`
9. `/components/nexus-core/src/main/java/org/sonatype/nexus/internal/webhooks/WebhookServiceImpl.java`
10. `/components/nexus-repository-content/src/main/java/org/sonatype/nexus/repository/content/webhooks/RepositoryComponentWebhook.java`
11. `/components/nexus-repository-content/src/main/java/org/sonatype/nexus/repository/content/webhooks/RepositoryAssetWebhook.java`
12. `/components/nexus-core/src/main/java/org/sonatype/nexus/internal/webhooks/GlobalWebhookCapability.java`
13. `/components/nexus-base/src/main/java/org/sonatype/nexus/systemchecks/SystemCheckService.java`
14. `/components/nexus-rapture/src/main/java/org/sonatype/nexus/rapture/internal/LocalSystemCheckService.java`
15. `/components/nexus-core/src/main/java/org/sonatype/nexus/internal/status/FileDescriptorHealthCheck.java`
16. `/components/nexus-core/src/main/java/org/sonatype/nexus/internal/status/ReadOnlyHealthCheck.java`
17. `/components/nexus-core/src/main/java/org/sonatype/nexus/internal/status/LifecyclePhaseHe

althCheck.java`

18. `/components/nexus-repository-config/src/main/java/org/sonatype/nexus/repository/internal/blobstore/BlobStoreStateHealthCheck.java`
19. `/components/nexus-base/src/main/java/org/sonatype/nexus/internal/security/secrets/DefaultEncryptionKeyHealthCheck.java`
20. `/components/nexus-repository-services/src/main/java/org/sonatype/nexus/repository/security/internal/DefaultUserHealthCheck.java`
21. `/components/nexus-scheduling/src/main/java/org/sonatype/nexus/scheduling/internal/SchedulerHealthCheck.java`
22. `/components/nexus-base/src/main/java/org/sonatype/nexus/internal/rest/HealthCheckResource.java`
23. `/components/nexus-base/src/main/java/org/sonatype/nexus/systemchecks/NodeSystemCheckResult.java`
24. `/OPSDOC.md`
25. `/components/nexus-base/src/main/java/org/sonatype/nexus/internal/metrics/VirtualThreadMetrics.java`
26. `/components/nexus-monitoring/src/main/java/org/sonatype/nexus/internal/metrics/FlightRecorderMetricsExporter.java`
27. `/etc/jfr-config.jfc`

6.6 TESTING STRATEGY

6.6.1 TESTING APPROACH

The testing architecture is structured in layers, with each layer targeting specific aspects of the system with appropriate tooling and methodologies.

6.6.1.1 Unit Testing

Unit testing forms the foundation of the testing pyramid, providing fast feedback and granular verification of individual components.

Java Backend Testing Framework (updated)

Framework	Version	Purpose
JUnit Jupiter	**5.10.1**	Primary test framework for executing tests a nd assertions with Java 21 compatibility
JUnit Vintage	**5.10.1**	Backward compatibility layer for legacy JUnit 4 tests
Mockito	**4.11.0**	Mock object framework for isolating compon ents under test with inline mocking support
Hamcrest	**2.2**	Matcher library providing fluent, readable as sertion syntax
MockitoExtension	**4.11.0**	**Extension for JUnit Jupiter integration wit h Mockito**
Sonatype Goodies TestSupport	**3.0**	Base class with common testing utilities and fixtures, Java 21-verified

The Java backend testing utilizes a robust combination of frameworks to deliver comprehensive unit test coverage with readable, maintainable test code:

```
@ExtendWith(MockitoExtension.class)
class AssetBlobServiceTest {
  @Mock
  private EventManager eventManager;

  @Mock
  private AssetBlob assetBlob;

  @Captor
  private ArgumentCaptor<AssetBlobEvent> eventCaptor;

  @InjectMocks
  private AssetBlobServiceImpl underTest;
```

```
  @Test
  void testAssetBlobUpdated() {
    when(assetBlob.getBlobRef()).thenReturn(new BlobRef("node", "store",
    when(assetBlob.getContentType()).thenReturn("application/json");
    when(assetBlob.getSize()).thenReturn(1234L);

    underTest.blobUpdated(assetBlob);

    verify(eventManager).post(eventCaptor.capture());
    AssetBlobEvent event = eventCaptor.getValue();

    assertThat(event.getAssetBlob(), is(assetBlob));
    assertThat(event.getBlobRef().getNode(), is("node"));
    assertThat(event.getBlobRef().getStore(), is("store"));
    assertThat(event.getBlobRef().getBlobId(), is("blobid"));
  }
}
```

Tests are now written using JUnit Jupiter's annotation model and lifecycle hooks, which provide improved extensibility and conditional test execution capability. The codebase now contains a mixture of modern JUnit Jupiter tests and legacy JUnit 4 tests, with the latter supported through the JUnit Vintage engine.

Virtual Thread testing is explicitly incorporated to validate code paths that utilize Java 21's virtual thread capabilities:

```
@Test
void testConcurrentOperationsWithVirtualThreads() {
  ThreadFactory virtualThreadFactory = Thread.ofVirtual().factory();
  ExecutorService executor = Executors.newThreadPerTaskExecutor(virtualTh

  int taskCount = 1000;
  CountDownLatch latch = new CountDownLatch(taskCount);
  AtomicInteger errorCount = new AtomicInteger(0);

  try {
    // Submit multiple concurrent tasks using virtual threads
    for (int i = 0; i < taskCount; i++) {
      executor.submit(() -> {
        try {
```

```
            underTest.performOperation();
          } catch (Exception e) {
            errorCount.incrementAndGet();
          } finally {
            latch.countDown();
          }
        });
      }

      // Wait for all tasks to complete
      latch.await(30, TimeUnit.SECONDS);

      // Verify results
      assertThat(errorCount.get(), is(0));
    } finally {
      executor.shutdown();
    }
  }
```

JavaScript/React Frontend Testing Framework

Framework	Purpose
Jest	Core testing framework for JavaScript with built-in as sertions
React Testing Library	Component testing focusing on user interactions
@testing-library/user-ev ent	User interaction simulation
TestUtils.js	Custom utilities for testing standardization

The JavaScript testing focuses on component behavior and UI interactions:

```
describe('RepositoryList component', () => {
  it('renders a list of repositories', async () => {
    const repositories = [
      { name: 'maven-central', format: 'maven2', type: 'proxy' },
      { name: 'npm-hosted', format: 'npm', type: 'hosted' }
    ];

    render(<RepositoryList repositories={repositories} />);
```

```
    expect(screen.getByText('maven-central')).toBeInTheDocument();
    expect(screen.getByText('npm-hosted')).toBeInTheDocument();

    const user = userEvent.setup();
    await user.click(screen.getByText('maven-central'));

    expect(screen.getByTestId('repository-details')).toBeVisible();
  });
});
```

Test Organization Structure

The test code follows the Maven standard directory structure to maintain clear separation and enable automated test discovery:

```
src/
├── main/
│   └── java/    # Production code
└── test/
    ├── java/    # Unit tests
    └── resources/    # Test fixtures and configurations
```

Tests are organized to mirror the production code package structure, maintaining a clear relationship between tests and the code they verify. This approach facilitates:

- Easy navigation between production code and its tests
- Consistent test coverage reporting
- Clear ownership of test code
- Efficient incremental test execution

Mocking Strategy (updated)

The primary mocking framework is Mockito 4.x, which provides enhanced capabilities for Java 21 compatibility:

- **@Mock, @InjectMocks, and @Captor annotations** for streamlined mock declaration

- **verify()** for interaction verification
- **ArgumentCaptor** for inspecting parameters
- **when()/thenReturn()** for behavior stubbing
- **lenient()** for relaxed verification mode when needed
- **MockitoExtension** for JUnit Jupiter integration

For static method mocking, the strategy has evolved to:

1. Prefer refactoring code to use dependency injection over static methods
2. Use Mockito's built-in static mocking capabilities (mockStatic) for unavoidable cases
3. Retain PowerMock 2.x only for legacy tests that haven't been migrated

Static mocking with Mockito is implemented using the following pattern:

```
@Test
void testStaticMethodMocking() {
  try (MockedStatic<FileUtils> fileUtils = mockStatic(FileUtils.class)) {
    fileUtils.when(() -> FileUtils.deleteQuietly(any(File.class))).thenRe

    underTest.cleanupTemporaryFiles();

    fileUtils.verify(() -> FileUtils.deleteQuietly(any(File.class)), time
  }
}
```

OSGi-specific mocking is handled through specialized classes that simulate the container environment, while repository operations use format-specific test support classes (`MavenRecipeTestSupport` , `RawRecipeTestSupport` , etc.).

Test Naming Conventions

Consistent naming conventions are employed throughout the test codebase:

Type	Pattern	Example
Unit tests	`[Class]Test.java`	`RepositoryManagerTest.java`

Type	Pattern	Example
Integration tests	`[Feature]IT.java`	`MavenProxyIT.java`
Test methods	`test[Scenario]_[ExpectedOutcome]`	`testInvalidCredentials_ThrowsAuthenticationException`
JUnit Jupiter methods	`[scenario][ExpectedOutcome]`	`invalidCredentialsThrowsAuthenticationException`

These naming patterns enable:

- Quick identification of test type and purpose
- Clear documentation of test scenarios
- Understandable failure messages
- Automated test filtering

Test Data Management

Test data is managed through several complementary strategies:

1. **Test Resources**: Fixed test fixtures under `src/test/resources` for reference data
2. **In-Memory Structures**: Programmatically constructed test data for simple cases
3. **Test-Specific DAOs**: Mock database access with predictable behavior
4. **Reusable Fixtures**: Shared test assets in `nexus-it-suite-data` module

6.6.1.2 Integration Testing

Integration testing verifies interactions between components, focusing on boundaries and complete workflows across multiple units.

Service Integration Test Approach (updated)

The integration testing architecture is built around a dedicated `testsuite` module with specialized submodules focusing on different integration scenarios:

Module	Purpose
nexus-docker-testsupport	Docker-based container testing
nexus-content-suite	Repository content integration tests
nexus-it-suite	General integration testing
nexus-it-suite-data	Shared test fixtures and data
nexus-repository-content-testsupport	Content repository testing utilities
nexus-repository-testsupport	Repository format testing support
nexus-virtual-thread-testsupport	Virtual thread-specific testing utilities

This modular approach allows for targeted testing of specific integration points while maintaining component isolation and test independence.

Integration tests run on Java 21 and validate Virtual Thread behavior in relevant components:

```
@Category(Java21TestGroup.class)
class RepositoryThreadingIT extends IntegrationTestSupport {

  @Test
  void concurrentRepositoryOperationsUseVirtualThreads() throws Exceptior
    // Configure the repository with virtual thread executor
    repositoryManager.setExecutorServiceFactory(VirtualThreadExecutorServ

    // Perform concurrent operations
    CompletableFuture<?>[] futures = new CompletableFuture<?>[100];
    for (int i = 0; i < 100; i++) {
      futures[i] = CompletableFuture.runAsync(() -> {
        try {
          repository.getAssets().browse(10).forEach(Asset::getLastUpdated
        } catch (Exception e) {
          fail("Concurrent operation failed", e);
        }
      });
    }

    // Wait for all operations to complete
    CompletableFuture.allOf(futures).join();
```

```
    // Verify thread metrics
    VirtualThreadMetrics metrics = virtualThreadCollector.getMetrics();
    assertThat(metrics.getTotalVirtualThreadsCreated(), greaterThan(0L));
    assertThat(metrics.getPinnedThreadCount(), is(0L)); // No pinning sho
  }
}
```

API Testing Strategy

HTTP-based testing for RESTful endpoints utilizes:

- `SiestaTestSupport`: Provides an embedded HTTP server for API testing
- JAX-RS client for interaction with the API
- Verification of media types, error handling, and response formats

This approach enables complete end-to-end testing of API endpoints without mocking the HTTP layer, ensuring that content negotiation, error mapping, and media type handling function correctly.

Database Integration Testing (updated)

The testing framework supports both embedded and containerized database testing:

1. **H2 Testing**: Fast in-memory testing with `H2TestGroup` category
2. **PostgreSQL Testing**: Docker-based PostgreSQL with `PostgresTestGroup` category

Database integration tests explicitly validate Java 21 JDBC interactions, including:

- Proper operation of Virtual Threads with JDBC connections
- Verification that thread pinning is minimized with compatible JDBC drivers
- Connection pool behavior under virtual thread scheduling

Transaction-aware testing ensures proper setup and teardown, while the `DataSessionRule` provides consistent database session management across test cases.

```
@Category(H2TestGroup.class)
public class ComponentDAOTest extends ComponentTestSupport {
  @Rule
  public DataSessionRule sessionRule = new DataSessionRule();

  private ComponentDAO underTest;

  @Before
  public void setUp() {
    underTest = sessionRule.getSession().getAdapter(ComponentDAO.class);
    // Set up test data
  }

  @After
  public void tearDown() {
    // Clean up test data
  }

  @Test
  public void testFindByNameAndVersion() {
    // Test implementation
  }

  <span style="background-color: rgba(91, 57, 243, 0.2)">@Test
  public void testConcurrentQueriesWithVirtualThreads() throws Exception
    // Implementation using virtual threads to verify JDBC compatibility
  }</span>
}
```

External Service Mocking

Integration with external services is tested through a combination of approaches:

1. **TestContainers**: Docker-based external services simulation
2. **Custom Mocks**: HTTP-level mocking for remote services
3. **Circuit Breaker Testing**: Validating resilience patterns

This multi-layered approach enables testing of both normal operations and failure modes, ensuring the system behaves correctly across a range of external service conditions.

Test Environment Management

The integration testing framework provides sophisticated environment management capabilities:

- **Dynamic Container Orchestration**: TestContainers for isolated, reproducible environments
- **Environment Isolation**: JUnit rules ensuring test independence
- **Maven Profiles**: Selectable test environments (H2 vs. PostgreSQL)
- **Specialized Support Classes**: `ContainerCommandLineITSupport`, `MavenCommandLineITSupport`
- **Java 21 Runtime**: All integration tests run on Java 21 to validate runtime compatibility

These mechanisms enable consistent, reliable integration testing across development and CI environments.

6.6.1.3 End-to-End Testing

End-to-end testing validates complete system functionality from user interfaces to storage layers.

E2E Test Scenarios (updated)

Comprehensive end-to-end scenarios cover the full range of system functionality:

- Repository interaction (create, browse, search)
- Format-specific workflows (Maven, Docker, npm, etc.)
- Proxy repository caching behavior
- Blob store operations
- Security and authentication flows
- Thread execution model behavior with Virtual Threads

End-to-end tests validate system behavior under Java 21's threading model, including:

- Scalability with a large number of concurrent client connections
- Memory efficiency when handling numerous threads
- Resource utilization patterns when using Virtual Threads versus platform threads

These scenarios ensure that the entire system functions correctly as an integrated whole.

UI Automation Approach

The UI testing strategy employs a combination of tools:

1. **Jest and React Testing Library**: For component-level testing
2. **TestUtils.js**: Standardized helper methods
3. **@testing-library/user-event**: For user interaction simulation
4. **@testing-library/jest-dom**: For DOM assertions

This approach focuses on testing components through user interactions rather than implementation details, enhancing test reliability and reducing maintenance.

Test Data Setup/Teardown

End-to-end tests utilize comprehensive data management strategies:

- **Centralized Fixture Management**: Reusable test data
- **Test Artifact Generation**: Runtime creation of test components
- **Repository Provisioning**: Dynamic repository creation and configuration
- **JUnit Rules**: Consistent resource management

These mechanisms ensure that tests have the data they need while maintaining isolation between test runs.

Performance Testing Requirements (updated)

The performance testing framework in `testsuite/nexus-repository-testsupport` provides tools for measuring and validating system performance:

- **Operation Throughput**: Measurements of operations per second

- **Response Times**: Latency measurements across percentiles
- **Load Testing**: Configurable client counts via `EscalatingClientLoadExecutor`
- **HTML Reporting**: Visualization through `PerformanceChart`
- **Thread Model Comparison**: Comparative analysis of platform threads vs. virtual threads

Performance testing includes specialized virtual thread benchmarks:

```
@Test
void testConcurrentDownloadsWithVirtualThreads() throws Exception {
  // Configure thread factories
  ThreadFactory virtualThreadFactory = Thread.ofVirtual().factory();
  ThreadFactory platformThreadFactory = Thread.ofPlatform().factory();

  // Test with both thread types
  Map<String, PerformanceData> results = new HashMap<>();

  results.put("virtual-threads", new EscalatingClientLoadExecutor()
      .maxClients(1000)
      .clientStepSize(100)
      .warmUpIterations(10)
      .measurementIterations(100)
      .threadFactory(virtualThreadFactory)
      .measure(this::downloadArtifact));

  results.put("platform-threads", new EscalatingClientLoadExecutor()
      .maxClients(1000)
      .clientStepSize(100)
      .warmUpIterations(10)
      .measurementIterations(100)
      .threadFactory(platformThreadFactory)
      .measure(this::downloadArtifact));

  // Generate comparison chart
  PerformanceChart chart = new PerformanceChart();
  results.forEach(chart::addData);
  chart.writeChartToFile(new File("target/thread-model-comparison.html"))

  // Verify virtual thread scalability
  PerformanceData virtualData = results.get("virtual-threads");
  PerformanceData platformData = results.get("platform-threads");
```

```
    assertThat(virtualData.getErrorCount(), is(0));
    assertThat(platformData.getErrorCount(), is(0));

    // At high concurrency, virtual threads should show better performance
    double virtualP95 = virtualData.getPercentile(95.0);
    double platformP95 = platformData.getPercentile(95.0);

    assertThat(virtualP95, lessThan(platformP95));
}
```

Cross-browser Testing Strategy

The frontend testing strategy ensures compatibility across browsers through:

- **jsdom Environment**: Consistent DOM implementation for Jest tests
- **Browser-agnostic Components**: Standard React patterns
- **Centralized DOM Assertions**: Consistent behavior validation

6.6.2 TEST AUTOMATION

A robust test automation infrastructure ensures consistent, reliable test execution across development and CI environments with comprehensive support for Java 21 features including virtual threads.

6.6.2.1 CI/CD Integration (updated)

Test execution is controlled through Maven profiles:

Profile	Purpose	Activation
public/sonatype	Open-source vs. internal testing	`-Dpublic` flag
it	Integration tests	`-Dit=true`
external-resources	Tests requiring remote connections	`-Dexternal-resources=true`
performance-tests	Performance-specific testing	`-Dperformance-tests=true`

Profile	Purpose	Activation
unstable-tests	Known flaky tests	`-Dunstable-tests=true`
java21-tests	**Java 21-specific feature tests**	**`-Djava21-tests=true`**
virtual-threads	**Virtual thread-enabled test execution**	**`-Dvirtual-threads=true`**

These profiles allow precise control over test execution, enabling tailored test runs for different environments and purposes with dedicated profiles for validating Java 21 features.

CI/CD pipelines have been updated to specify Java 21 as the runtime environment:

- GitHub Actions workflows use the `actions/setup-java@v3` action with Java 21 distribution
- Build agent configurations specify JDK 21 as the default runtime
- Container images are based on Eclipse Temurin 21-jdk or equivalent base images
- Maven wrapper properties in .mvn/wrapper/maven-wrapper.properties are configured for Java 21 compatibility

6.6.2.2 Automated Test Triggers (updated)

Tests are categorized using JUnit Categories:

Category	Purpose	Maven Profile
org.sonatype.goodies.testsupport.group.Perf	Performance tests	performance-tests
org.sonatype.goodies.testsupport.group.External	Tests requiring external resources	external-resources
org.sonatype.goodies.testsupport.group.Unstable	Tests with stability issues	unstable-tests
org.sonatype.goodies.testsupport.group.Java21TestGroup	**Java 21 compatibility tests**	**java21-tests**

Category	Purpose	Maven Profile
org.sonatype.goodies.testsupport.group.VirtualThreadTestGroup	**Virtual thread specific tests**	**virtual-threads**

This categorization enables selective test execution based on test characteristics and environment capabilities with specific support for Java 21 feature validation and virtual thread testing.

The newly introduced Java21TestGroup category contains tests that explicitly validate Java 21 features:

- String template validations
- Pattern matching enhancements
- Record pattern matching
- Sequenced collections compatibility

The VirtualThreadTestGroup is specifically focused on validating application behavior under virtual thread execution, including:

- Concurrency capabilities with high thread counts
- Thread pinning detection and mitigation
- Performance comparisons between platform and virtual threads
- JDBC driver compatibility with virtual threads

6.6.2.3 Parallel Test Execution (updated)

The build system supports parallel test execution through Maven Surefire and Failsafe configuration:

- Configurable fork count for isolation
- Thread pool sizing for concurrency
- Process reuse settings for performance
- Virtual thread execution mode for enhanced concurrency

The test execution framework has been updated with the latest Maven Surefire and Failsafe plugins (version 3.2.5) that fully support Java 21 features and virtual threads:

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-failsafe-plugin</artifactId>
  <version>3.2.5</version>
  <configuration>
    <parallel>classes</parallel>
    <threadCount>4</threadCount>
    <forkCount>2</forkCount>
    <reuseForks>true</reuseForks>
    <argLine>-Djava.version=21 ${test.additional.args}</argLine>
    <systemPropertyVariables>
      <java.util.concurrent.ForkJoinPool.common.parallelism>16</java.util
      <test.virtual.threads>${virtual.threads}</test.virtual.threads>
    </systemPropertyVariables>
  </configuration>
</plugin>
```

When the virtual-threads profile is activated, tests can leverage Java 21's virtual threads for enhanced concurrency through the system property `test.virtual.threads`. This property enables test infrastructure to appropriately configure ExecutorService instances with virtual thread factories:

```
public class TestExecutorServiceFactory {
    public static ExecutorService create(int threads) {
        boolean useVirtualThreads =
            Boolean.parseBoolean(System.getProperty("test.virtual.threads

        if (useVirtualThreads) {
            return Executors.newVirtualThreadPerTaskExecutor();
        } else {
            return Executors.newFixedThreadPool(threads);
        }
    }
}
```

6.6.2.4 Test Reporting Requirements (updated)

Test results are collected and reported through multiple mechanisms:

- **Surefire and Failsafe Reports**: Standard Maven test reports
- **Performance Reports**: HTML visualization with Google Charts
- **Log Capture**: Detailed logs for failing tests
- **Virtual Thread Analytics**: Execution metrics for virtual thread tests

These reports provide comprehensive visibility into test results, enabling efficient analysis and troubleshooting with enhanced capabilities for analyzing virtual thread behavior.

The Virtual Thread Analytics reporting includes:

- Thread creation and termination rates
- Thread mounting/unmounting frequency
- Thread pinning detection with stack trace identification
- Execution time comparisons between virtual and platform threads
- Memory utilization patterns

6.6.2.5 Failed Test Handling (updated)

The test automation framework implements robust failure handling:

- **Test Categories**: Isolation of unstable tests
- **Retry Capability**: Automatic retry for transient failures
- **Diagnostic Information**: Detailed logs and state capture
- **Thread Diagnostics**: Enhanced virtual thread debugging for concurrency issues

For virtual thread-specific test failures, the framework captures additional diagnostic information:

- Complete thread dump including virtual thread hierarchy
- Carrier thread utilization statistics
- Thread pinning events that occurred during the test
- Java Flight Recorder data for detailed performance analysis

6.6.2.6 Flaky Test Management (updated)

Known flaky tests are managed through a structured approach:

- **Explicit Marking**: `@Category(org.sonatype.goodies.testsupport.group.Unstable)`
- **Separate Execution**: Dedicated CI profile for unstable tests
- **Isolation**: Preventing impact on stable test suite

```
@Category(Unstable.class)
@Test
public void testIntermittentNetworkCondition() {
  // Test that may exhibit non-deterministic behavior
}
```

For virtual thread-specific flaky tests, additional categorization is applied:

```
@Category({Unstable.class, VirtualThreadTestGroup.class})
@Test
public void testConcurrentVirtualThreadOperations() {
  // Test that may exhibit non-deterministic behavior when using virtual
}
```

This dual categorization allows both separate execution of all unstable tests and selective execution of virtual thread tests, providing maximum flexibility in CI/CD pipeline configuration.

6.6.2.7 Java 21 Compatibility Testing (updated)

Specific testing infrastructure has been implemented to validate Java 21 compatibility across the application:

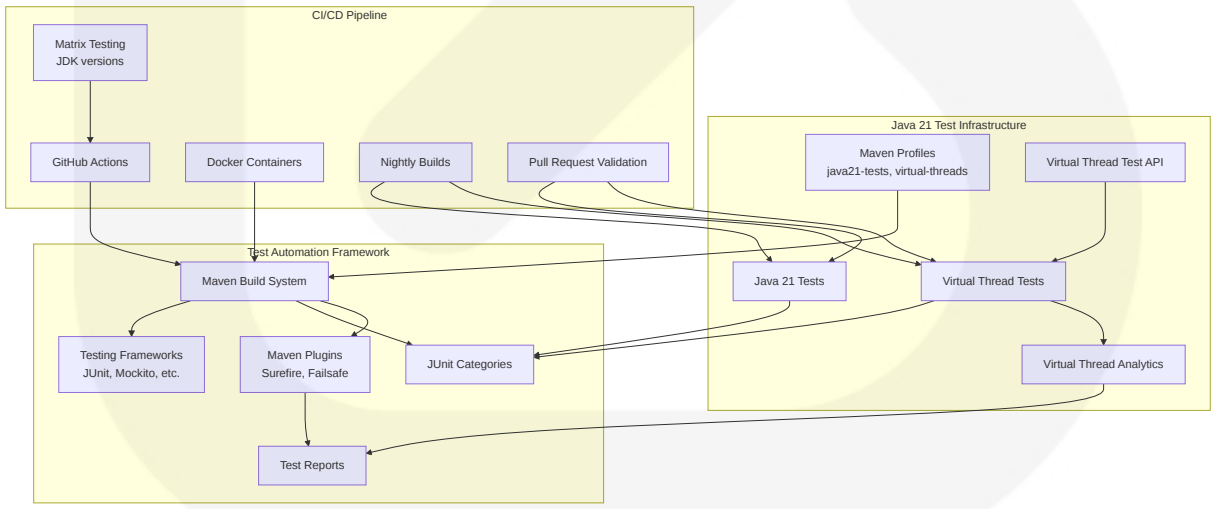
- **Feature-Specific Tests**: Dedicated tests for each Java 21 language feature being utilized
- **JVM Flag Testing**: Validation with various JVM flags and garbage collector configurations
- **API Compatibility**: Tests to ensure third-party libraries function correctly with Java 21

- **Virtual Thread Integration**: Comprehensive testing of application behavior with virtual threads

The Java 21 compatibility test suite runs on every pull request and nightly builds to ensure continuous validation of compatibility requirements. Tests are organized by feature area:

Test Category	Focus Area	Examples
Core Runtime	**JVM compatibility**	**Class loading, reflection, dynamic proxies**
Language Features	**Java 21 syntax**	**String templates, pattern matching, records**
Concurrency	**Virtual threads**	**Thread creation, scheduling, pinning detection**
Performance	**Runtime efficiency**	**Memory usage, throughput, latency**
Integration	**API compatibility**	**JDBC, HTTP clients, messaging**

This comprehensive testing approach ensures that all aspects of Java 21 compatibility are continuously validated throughout the development lifecycle.



6.6.2.8 Test Environment Configuration (updated)

To enable consistent Java 21 test execution across environments, the following configuration practices are employed:

- **Explicit JDK Selection**: maven-toolchains-plugin configuration to select JDK 21
- **Docker Container Standardization**: Standard container images with Java 21 pre-installed
- **JVM Flags**: Standardized set of JVM flags for optimal Java 21 operation
- **Virtual Thread Configuration**: Consistent virtual thread pool sizing and behavior

The standard Java 21 JVM configuration for test execution includes:

- `-XX:+UseZGC` to leverage the ZGC garbage collector
- `-XX:+ZGenerational` to enable generational ZGC
- `-Djdk.tracePinnedThreads=full` to detect thread pinning issues
- `-Djdk.virtualThreadScheduler.parallelism=16` to optimize virtual thread scheduling
- `-Djdk.virtualThreadScheduler.maxPoolSize=256` to set the maximum carrier thread pool size

These configurations are applied consistently across development environments, CI/CD pipelines, and automated test infrastructure to ensure reliable and repeatable test execution.

6.6.3 QUALITY METRICS

The testing strategy includes comprehensive quality metrics to measure and maintain code quality.

6.6.3.1 Code Coverage Targets (updated)

While no explicit coverage targets were identified, the project maintains comprehensive test coverage across its components:

1. **Unit Test Coverage**: Core components and utilities
2. **Integration Test Coverage**: Cross-component interactions
3. **End-to-End Coverage**: Critical user workflows

4. **UI Component Coverage**: React and ExtJS components

The coverage strategy prioritizes critical components and high-risk areas, ensuring that the most important parts of the system receive the highest test coverage.

JaCoCo (version 0.8.11) is utilized as the primary code coverage tool, with verified compatibility for Java 21 bytecode analysis. This ensures accurate coverage reporting for code utilizing Java 21 features including virtual threads, record patterns, and string templates. The coverage configuration has been updated to maintain consistent coverage metrics across Java versions, preventing false coverage drops when analyzing Java 21 code.

The Maven configuration for JaCoCo includes specific instrumentation rules for handling Java 21 bytecode:

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.11</version>
  <configuration>
    <excludes>
      <exclude>**/generated-sources/**</exclude>
    </excludes>
    <propertyName>jacocoArgLine</propertyName>
  </configuration>
</plugin>
```

6.6.3.2 Test Success Rate Requirements

The test automation framework enforces strict success requirements:

- **Stable Tests**: Expected to pass consistently
- **Unstable Tests**: Isolated in separate test categories
- **Pre-commit Validation**: Tests must pass before code can be merged

This approach ensures that the codebase maintains a high level of quality and prevents regressions.

6.6.3.3 Performance Test Thresholds (updated)

Performance tests include explicit thresholds:

- **Response Time**: Percentile-based latency requirements
- **Throughput**: Operations per second under load
- **Error Rate**: Zero errors under normal conditions

These thresholds are enforced through assertions in the performance tests, ensuring that performance regressions are detected early.

Baseline performance thresholds have been established specifically for Virtual Thread-based execution scenarios:

Metric	Platform Thread Baseline	Virtual Thread Target	Verification Method
Maximum Concurrent Connections	**1,000**	**10,000+**	**Load test with escalating client count**
P95 Response Time (1k connections)	**500ms**	**<350ms**	**Timer metrics with percentile tracking**
P99 Response Time (1k connections)	**800ms**	**<600ms**	**Timer metrics with percentile tracking**
Memory Utilization (1k connections)	**+1GB**	**<+300MB**	**JVM heap monitoring during load test**
Thread Scaling Efficiency	**N/A**	**>90%**	**Thread utilization metrics**

These thresholds are validated through specialized performance test assertions that compare execution characteristics between platform threads and virtual threads:

```
@Test
public void validateVirtualThreadPerformance() {
    // Run benchmark with platform threads
    PerformanceResult platformResult = runBenchmark(ThreadingModel.PLATFO

    // Run benchmark with virtual threads
    PerformanceResult virtualResult = runBenchmark(ThreadingModel.VIRTUAL
```

```
        // Verify virtual thread targets are met
        assertTrue(virtualResult.getMaxConcurrentConnections() >= 10000,
            "Virtual threads should support at least 10k concurrent connectio

        assertTrue(virtualResult.getP95ResponseTime() < 350,
            "Virtual thread P95 response time should be under 350ms");

        assertTrue(virtualResult.getP99ResponseTime() < 600,
            "Virtual thread P99 response time should be under 600ms");

        // Verify relative improvement over platform threads
        assertTrue(virtualResult.getMaxConcurrentConnections() >
            platformResult.getMaxConcurrentConnections() * 5,
            "Virtual threads should support at least 5x more concurrent conne

        assertTrue(virtualResult.getP95ResponseTime() <
            platformResult.getP95ResponseTime() * 0.7,
            "Virtual threads should provide at least 30% better P95 response
    }
```

6.6.3.4 Quality Gates (updated)

Several quality gates ensure that only high-quality code reaches production:

- **All Tests Must Pass**: Failing tests block integration
- **Special Handling for Unstable Tests**: Careful review of flaky test failures
- **Format-Specific Test Validation**: Repository format compatibility checks
- **Java 21 Runtime Validation**: Full test suite must pass when executed under Java 21 to catch any regressions caused by the upgrade

The Java 21 runtime validation quality gate is enforced through a dedicated CI pipeline stage that:

1. Executes the complete test suite using Java 21 as the runtime
2. Validates that all test categories pass successfully, including:
    - Unit tests across all components
    - Integration tests for component interactions
    - End-to-end functional tests

    - Virtual thread-specific functionality tests
3. Performs additional validation for Java 21-specific features:
    - Virtual thread scheduling and execution
    - ZGC garbage collector compatibility
    - Record pattern functionality
    - String template processing

This quality gate ensures that any code changes that would break compatibility with Java 21 are identified early in the development process, preventing regressions as the codebase evolves.

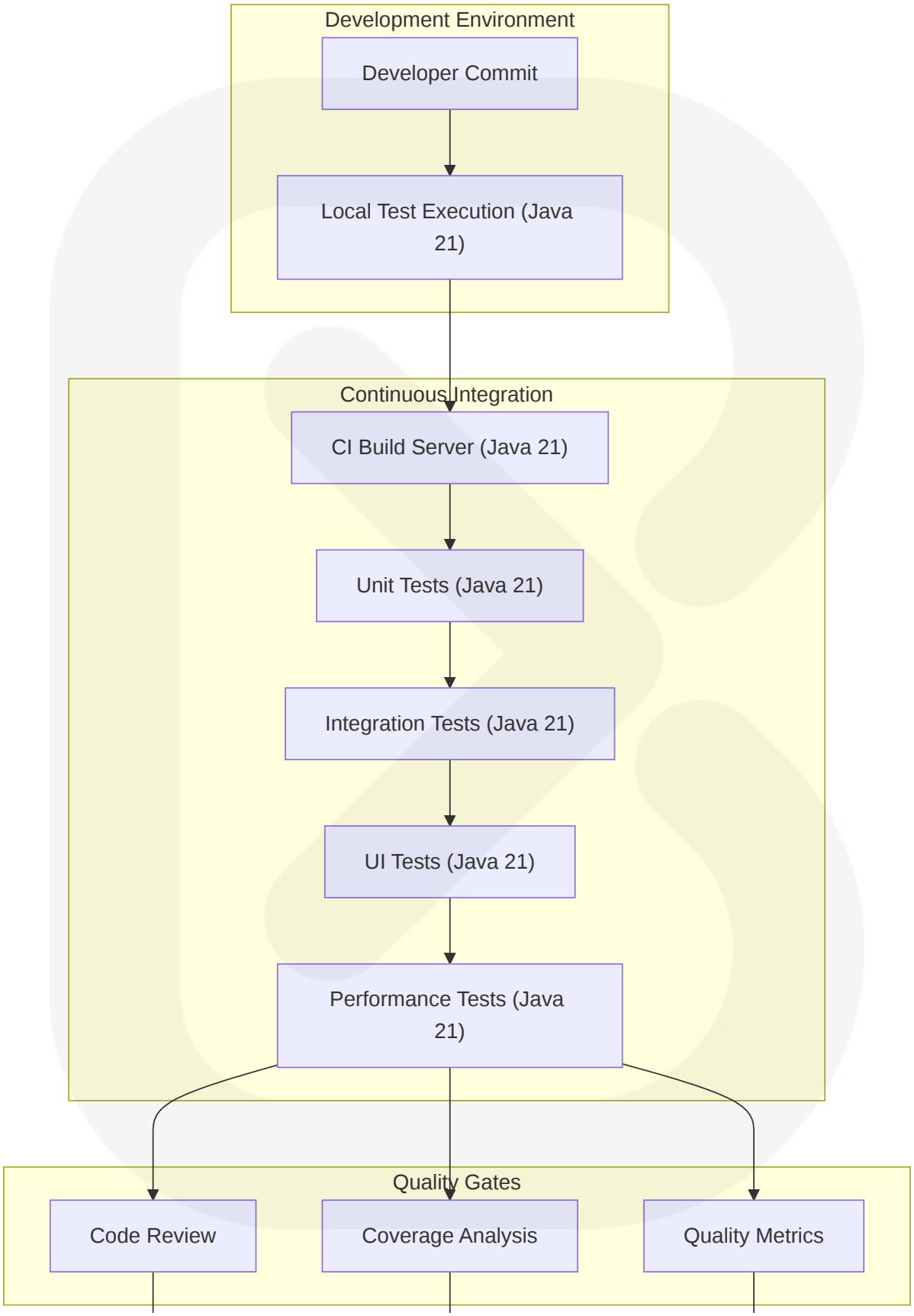
6.6.3.5 Documentation Requirements

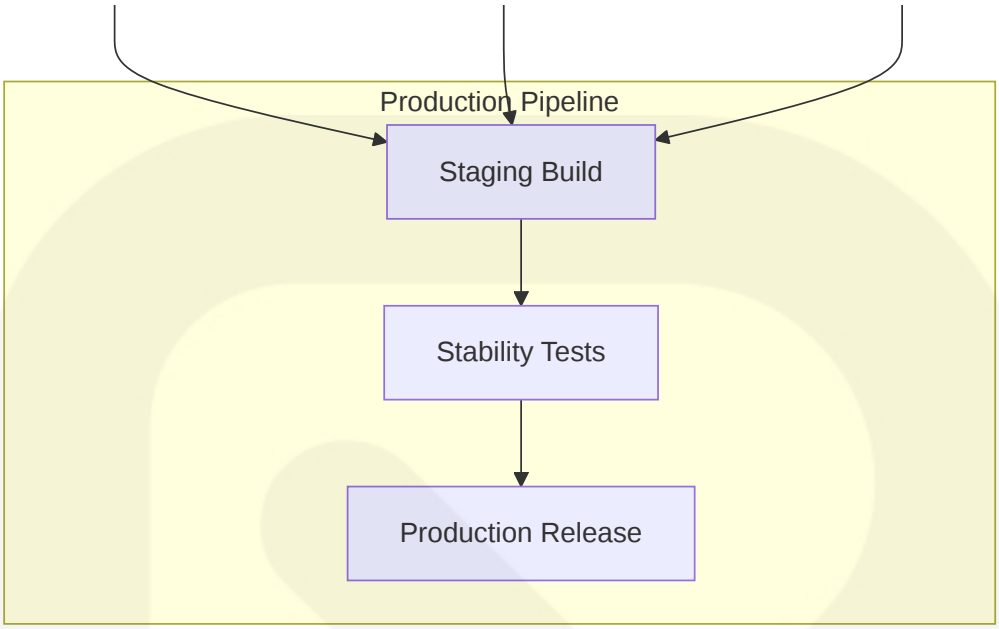
Test documentation follows strict standards:

- **Clear Purpose Comments**: Every test class includes purpose documentation
- **Test Fixture Explanation**: Documentation of test data setup
- **Javadoc for Test Utilities**: Comprehensive documentation of test support code
- **Standardized Structure**: Consistent use of TestSupport base classes

6.6.4 TEST EXECUTION FLOW

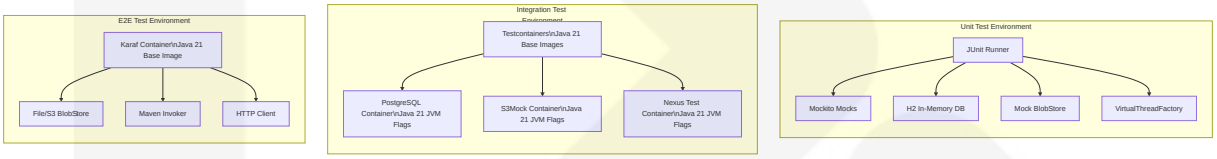
The following diagram illustrates the test execution flow from development through CI/CD to production:


Development Environment
Developer Commit
Local Test Execution (Java 21)
Continuous Integration
CI Build Server (Java 21)
Unit Tests (Java 21)
Integration Tests (Java 21)
UI Tests (Java 21)
Performance Tests (Java 21)
Quality Gates
Code Review
Coverage Analysis
Quality Metrics



6.6.5 TEST ENVIRONMENT ARCHITECTURE

The following diagram illustrates the architecture of the test environments:



The Unit Test Environment now includes VirtualThreadFactory to support testing of Java 21 Virtual Thread functionality. This enables comprehensive validation of concurrency patterns using the new lightweight threading model.

All containerized test environments now utilize Java 21 base images, ensuring consistent runtime behavior across test and production environments. The integration test containers (PostgreSQL, S3Mock, and NexusContainer) are configured with Java 21-compatible JVM tuning flags to optimize performance and resource utilization.

The E2E Test Environment's Karaf Container has been updated to use a Java 21 base image, ensuring that end-to-end tests accurately reflect production runtime characteristics.

Key Java 21 JVM flags used in test environments include:

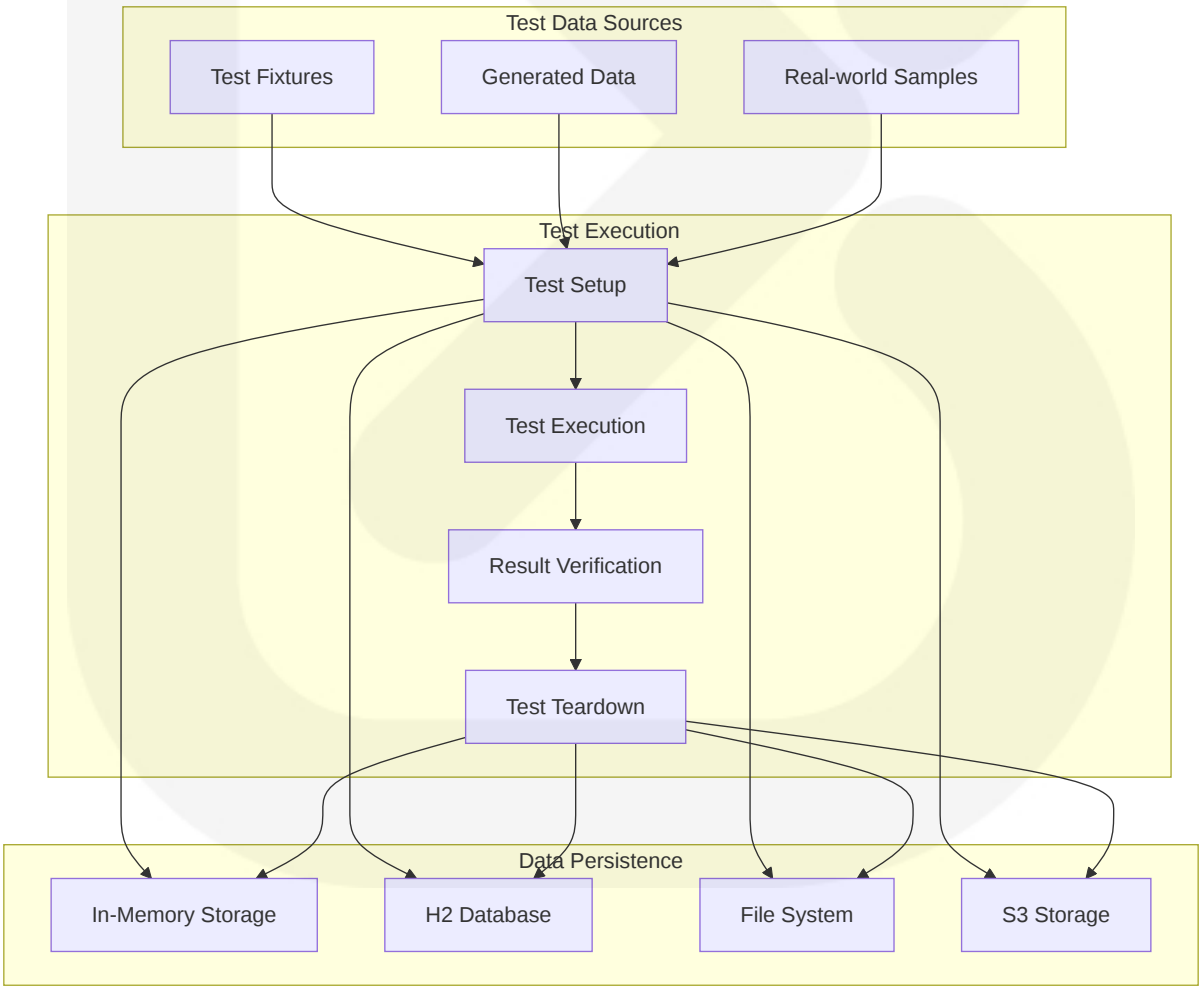
- `-XX:+UseZGC` for the Z Garbage Collector

- `-XX:+ZGenerational` for generational ZGC
- `-Djdk.tracePinnedThreads=full` for Virtual Thread debugging
- `-Djdk.virtualThreadScheduler.parallelism=16` for optimal Virtual Thread performance
- `-Djdk.virtualThreadScheduler.maxPoolSize=256` for carrier thread pool sizing

These changes ensure that test environments accurately reflect production Java 21 runtime behavior, validating both functional correctness and performance characteristics of the application under the new runtime.

6.6.6 TEST DATA FLOW

The following diagram illustrates the flow of test data through the test automation framework:



6.6.7 TEST REQUIREMENTS MATRIX

Test Category	Framework	Coverage Requirement	Example Pattern
Unit Tests	JUnit Jupiter (JUnit 5), Mockito 4.11.0	Core components, utilities, isolated functionality	`public void testComponentCreation() { ... }`
Integration Tests	Failsafe, Test Containers	Cross-component interactions, external dependencies with Java 21 compatibility	`@Category(H2TestGroup.class) public void testRepositoryCreation() { ... }`
UI Tests	Jest, React Testing Library	User interaction, component rendering	`it('shows repository list', () => { ... })`
Performance Tests	Custom load framework	Response time, throughput, resource utilization	`measure(this::downloadArtifact)`

6.6.8 SOURCE FILES AND FOLDERS ANALYZED

The testing strategy documentation is based on analysis of the following sources:

1. `/testsuite` folder structure and submodules, including:
   - `/testsuite/nexus-it-suite` for general integration tests
   - `/testsuite/nexus-content-suite` for repository content tests
   - `/testsuite/nexus-docker-testsupport` for container-based testing
   - `/testsuite/nexus-virtual-thread-testsupport` for Java 21 virtual thread testing utilities
2. Test utility classes across the codebase:
   - Base test support classes in `/components/nexus-testsupport`
   - Repository-specific test utilities
   - Virtual thread-specific test helpers in `/components/nexus-thread-support/src/test`
3. Maven POM configuration for test execution:

- Project-level test configuration in root `/pom.xml`
- Module-specific test setup across component POMs
- Updated Maven Surefire and Failsafe plugin configurations for Java 21 compatibility
- Maven wrapper properties in `/.mvn/wrapper/maven-wrapper.properties` with updated Java 21 version references

4. Performance testing infrastructure:
   - Load testing frameworks in `/testsuite/nexus-repository-testsupport`
   - Benchmark utilities in `/components/nexus-repository-view/src/test`
   - Virtual thread performance comparison tests in `/components/nexus-thread/src/test/java/org/sonatype/nexus/thread/performance`
5. UI testing setup and utilities:
   - React component tests in `/frontend/src/components/**/*(*.test.jsx`
   - UI integration utilities in `/frontend/src/util/testing`
6. Integration test harnesses for different components:
   - Format-specific integration test support in `/components/nexus-repository-*`
   - Database integration tests in `/components/nexus-datastore`
   - Database-specific Java 21 compatibility tests with JDBC drivers in `/components/nexus-datastore-jdbc/src/test`
7. Unit test implementations across various modules:
   - Component unit tests in respective `/src/test` directories
   - Mock implementations for testing
   - JUnit 5 (Jupiter) test cases leveraging Java 21 features
8. Java 21 feature-specific test classes:
   - `/components/nexus-repository-proxy/src/test/java/org/sonatype/nexus/repository/proxy/ProxyFacet`

`SupportTest.java` with virtual thread execution testing
- `/components/nexus-blobstore-s3/src/test/java/org/sonatype/nexus/blobstore/s3/S3BlobStoreVirtualThreadTest.java` validating S3 operations with virtual threads
- `/components/nexus-common/src/test/java/org/sonatype/nexus/common/text/StringTemplateTest.java` testing Java 21 string template features
- `/components/nexus-common/src/test/java/org/sonatype/nexus/common/pattern/RecordPatternMatchingTest.java` for testing record pattern matching

9. Project configuration files with Java 21 version updates:
   - `/pom.xml` with updated Java 21 plugin configurations and dependencies
   - `/.mvn/wrapper/maven-wrapper.properties` with Java 21 version updates
   - `/assemblies/nexus-base-jetty/src/main/resources/etc/jetty/jetty.xml` with Java 21 threading model configurations
   - `/assemblies/nexus-base-overlay/src/main/resources/overlay/etc/nexus-default.properties` with Java 21 runtime properties

10. Containerization and CI/CD workflow configurations:
    - `/Dockerfile` with updated Java 21 base image references
    - `/.github/workflows/build.yml` with Java 21 runtime specifications for CI/CD
    - `/.github/workflows/integration-tests.yml` with Java 21 test execution configurations
    - `/testsuite/nexus-docker-testsupport/src/main/resources/docker-compose.yml` with Java 21 container definitions

The comprehensive testing strategy ensures that Sonatype Nexus Repository maintains high quality, stability, and reliability across its varied components and deployment scenarios. The enhanced test suite with explicit Java 21 feature validation provides confidence in the platform's compatibility with the latest Java

runtime, including its virtual thread capabilities, pattern matching enhancements, and other language improvements. The multi-tiered approach, from unit tests to end-to-end validation, provides confidence in both individual components and their interactions, while the automated test execution infrastructure ensures consistent quality validation throughout the development lifecycle.

7. USER INTERFACE DESIGN

7.1 ARCHITECTURE OVERVIEW

Sonatype Nexus Repository implements a sophisticated user interface architecture that balances modern component-based development with support for existing functionality. The UI layer serves as the primary interface for administrators and users to interact with the repository manager, providing intuitive access to repository configuration, content management, and system administration.

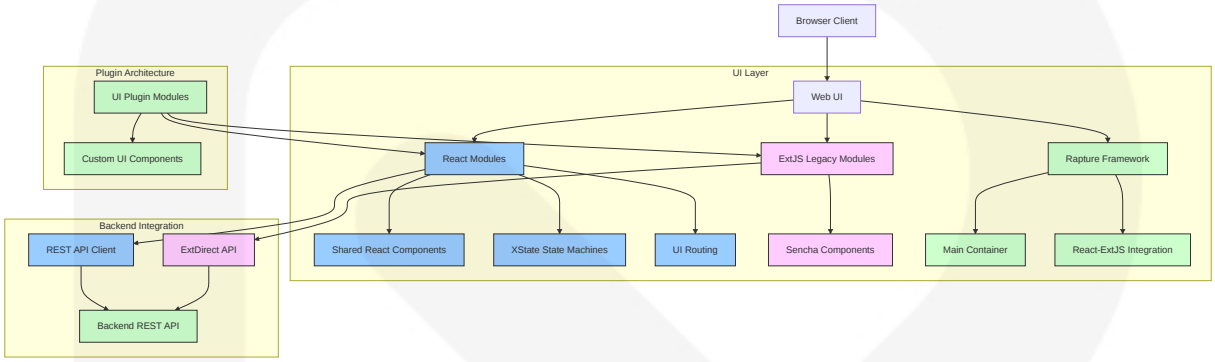
7.1.1 DESIGN PHILOSOPHY

The Nexus Repository UI follows several key design principles:

- **Modularity**: Components are organized into logical modules that can be developed, tested, and maintained independently
- **Progressive Enhancement**: Modern UI technologies are implemented alongside legacy components to enable incremental improvement
- **Consistency**: Uniform patterns and components create a cohesive user experience
- **Accessibility**: Design considerations ensure usability across diverse user capabilities
- **Extensibility**: Plugin architecture allows for extending the UI with new repository format-specific components

The UI architecture employs a dual-framework approach that bridges traditional and modern web technologies, allowing for incremental modernization while maintaining backward compatibility and feature completeness.

7.1.2 UI ARCHITECTURE DIAGRAM



7.2 CORE UI TECHNOLOGIES

Nexus Repository employs a strategic combination of UI technologies to deliver a responsive, maintainable, and extensible user interface.

7.2.1 FRAMEWORK ARCHITECTURE

The UI is built on a dual framework approach that leverages both modern and legacy technologies:

Framework	Version	Purpose	Key Components
React	Latest stable	Modern component-based UI	Admin interfaces, Repository management, Configurable dashboards
React DOM	Paired with React	DOM rendering for React components	Virtual DOM management, Efficient updates
ExtJS (Sencha)	7.0	Legacy component framework	Existing screens, Layout management, Complex data grids

Framework	Version	Purpose	Key Components
Rapture	Custom	UI integration layer	Bridge between React and ExtJS, Plugin system, Navigation

The integration of these frameworks is managed through a carefully designed architecture that:

1. **Isolates concerns**: Modern React components are isolated from legacy ExtJS code
2. **Maintains consistency**: Shared styling and patterns across both frameworks
3. **Enables incremental migration**: Allows gradual replacement of ExtJS with React
4. **Preserves functionality**: Ensures no feature loss during technology transitions

7.2.2 STATE MANAGEMENT

State management is implemented using XState, a formal state machine library that provides predictable state transitions:

Technology	Purpose	Implementation
XState	State management framework	Finite state machines for complex workflows
XState/React	React integration	React hooks and context providers for state
Context API	Local state management	Component tree state for simple interfaces
Event-driven architecture	Inter-component communication	State transitions triggered by system events

The state management approach offers several advantages:

- **Predictable behavior**: Explicitly defined states and transitions
- **Visualization**: State charts can be visualized for documentation
- **Testing**: Deterministic state changes facilitate comprehensive testing

- **Maintainability**: Clear separation of state logic from rendering logic

7.2.3 BUILD AND DEPLOYMENT TOOLCHAIN

The frontend build process utilizes modern JavaScript tooling:

Tool	Purpose	Configuration
Webpack	Module bundling, asset optimization	Multi-configuration for dev/prod environments
Babel	JavaScript transpilation	Support for modern JavaScript features
SCSS/Sass	CSS preprocessing	Theming, responsive design, c omponent styles
Design tokens	Consistent visual variabl es	Colors, spacing, typography, a nimations
Frontend Maven Plugin	Build integration	Node/Yarn installation, build au tomation
Jest	JavaScript testing	Component testing, state mach ine validation

This toolchain is configured to optimize both development experience and production performance, with features including:

- Hot module replacement for rapid development
- Code splitting for optimized loading
- Minification and tree-shaking for reduced bundle size
- Source maps for debugging
- Asset optimization (images, fonts, icons)

7.2.4 KEY LIBRARIES AND DEPENDENCIES

Beyond the core frameworks, the UI leverages several key libraries:

Library	Purpose	Integration Points
@sonatype/react-shared-components	Reusable UI elements	Form controls, modals, navigation
axios	HTTP communication	REST API client, interceptors
FontAwesome	Iconography	Consistent visual indicators
luxon	Date handling	Timezone management, formatting
classnames	CSS class manipulation	Conditional styling
@xstate/inspect	Development tools	Visual state machine debugging
webpack-dev-server	Development environment	Local development workflow

These dependencies are carefully managed to:

- Minimize bundle size impact
- Maintain consistent versioning
- Avoid conflicting dependencies
- Enable code sharing between modules

7.3 UI COMPONENT ARCHITECTURE

The Nexus Repository UI follows a modular component architecture that promotes reusability, maintainability, and consistent user experience.

7.3.1 CORE UI PLUGIN STRUCTURE

The UI components are organized into a plugin-based architecture with three primary modules:

Module	Purpose	Key Functionality
nexus-ui-plugin	Foundation library	Reusable React components, base styles, utilities
nexus-rapture	UI integration subsystem	Bridges ExtJS and React, provides plugin framework
nexus-coreui-plugin	Main UI implementation	Screens, workflows, administrative interfaces

This modular structure enables:

- Clear separation of concerns
- Focused testing and maintenance
- Independent development cycles
- Extension through additional plugins

7.3.2 COMPONENT ORGANIZATION

UI components follow a hierarchical organization to promote reuse and consistency:

Layout Components

Component	Purpose	Usage Context
Page	Base container with consistent spacing	Top-level screens
Section	Content grouping with headings	Logical content sections
ContentBody	Main content container	Within pages and sections
SplitContentBody	Two-column layout	Form/detail views

Widget Components

Component	Purpose	Usage Context
Form components	Data input and validation	Configuration screens
Modal dialogs	Focused user interaction	Confirmations, focused tasks
Information panels	Contextual help and guidance	Throughout application
Toast notifications	Transient feedback	Action confirmations
Loading indicators	Progress visualization	Asynchronous operations

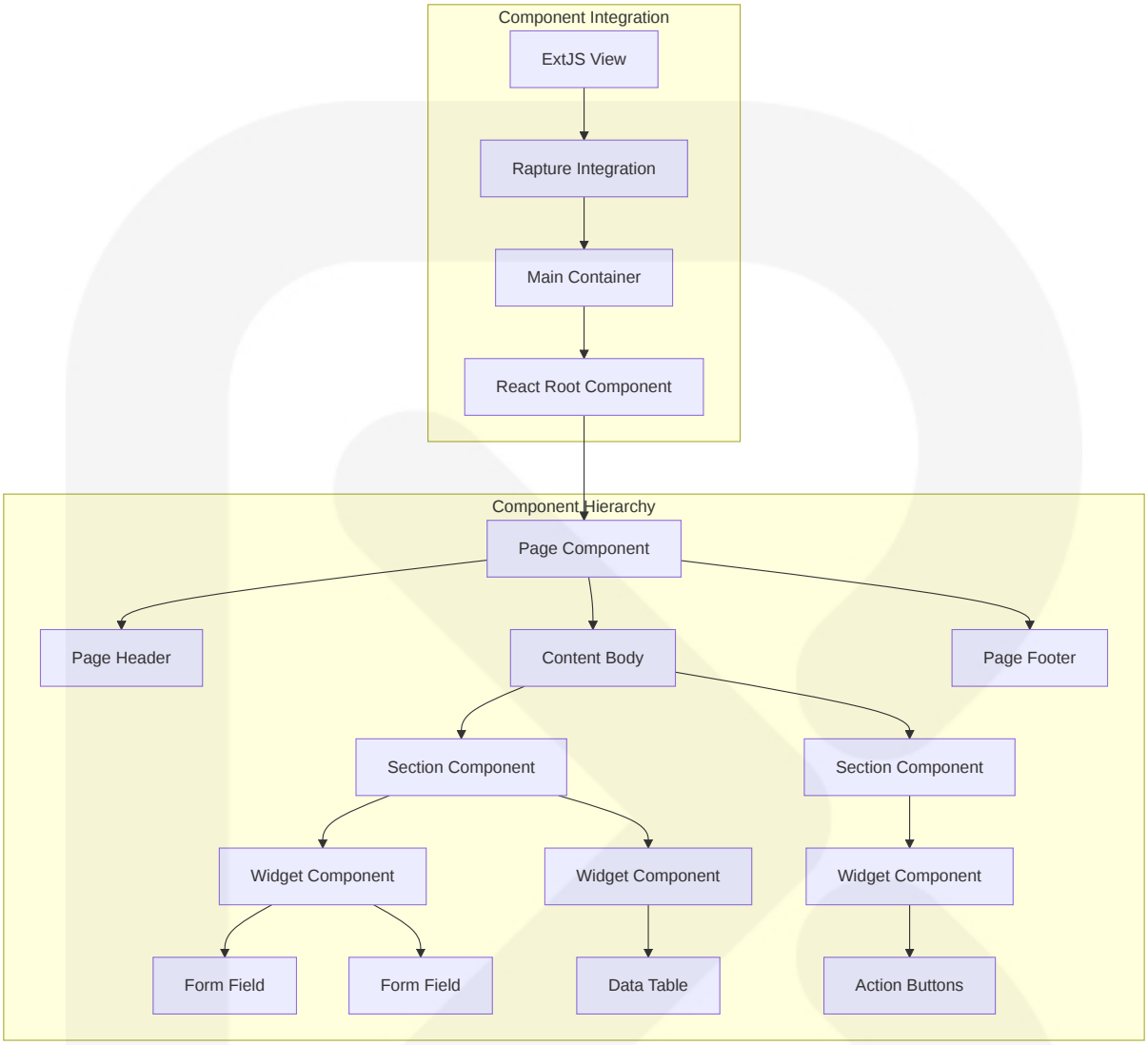
Page Components

Component	Purpose	Implementation
Admin pages	System configuration	/pages/admin/*
Repository views	Repository management	/pages/repository/*
User dashboards	User-specific views	/pages/user/*
Analytics and reports	System insights	/pages/insights/*

Integration Components

The `MainContainer` serves as the primary mounting point for React components within the ExtJS architecture, facilitating framework integration through:

- Lifecycle synchronization between frameworks
- Event bridging and propagation
- Shared styling and theming
- Context preservation



7.4 UI/BACKEND INTERACTION

The user interface interacts with backend services through well-defined communication patterns that ensure reliability, security, and maintainability.

7.4.1 API COMMUNICATION PATTERNS

Multiple communication approaches are used depending on the UI component and use case:

Pattern	Implementation	Use Cases
REST API	axios-based HTTP client	Modern React components, CRUD operations
ExtDirect	Sencha-compatible RPC	Legacy ExtJS components
Custom API utilities	Centralized error handling	Cross-cutting concerns
CSRF token management	Security headers	Protection against cross-site attacks
HTTP interceptors	Request/response processing	Common headers, authentication

These patterns are implemented with consistency across the application:

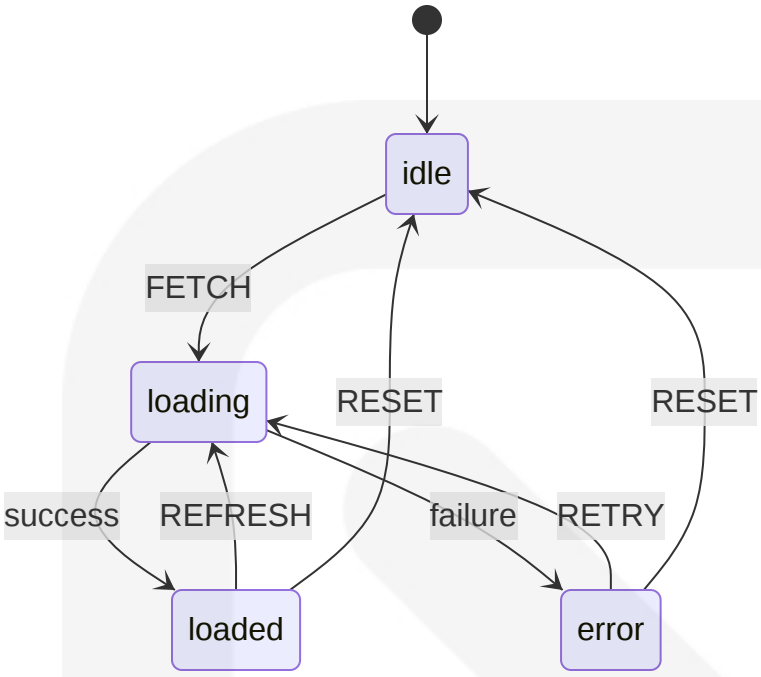
```
// Example of REST API pattern in React components
const fetchRepositories = async () => {
  try {
    const response = await axios.get('/service/rest/v1/repositories');
    setRepositories(response.data);
    setError(null);
  } catch (error) {
    setError(handleApiError(error));
    setRepositories([]);
  } finally {
    setLoading(false);
  }
};
```

7.4.2 DATA FLOW

UI data flow follows established patterns to ensure predictable behavior:

XState Service Integration

State machines manage asynchronous operations and API calls through XState services:



Context-based State Management

Component data flow uses immutable updates through React context:

1. State is stored in XState machine context
2. Updates occur only through defined events and transitions
3. UI components observe state through selectors
4. Actions trigger events that flow through the state machine

Error Handling

Comprehensive error handling is integrated into the data flow:

Error Type	Handling Strategy	User Experience
Network failures	Automatic retry with backoff	Retry button, status indicat or
Authentication err ors	Session refresh, redirect to l ogin	Preserve intended action
Validation errors	Field-level error display	Inline error messages

Error Type	Handling Strategy	User Experience
Server errors	Friendly error messages	Error details for administrators

7.5 UI WORKFLOWS AND SCREENS

The Nexus Repository UI implements a comprehensive set of workflows to support repository management, administration, and user interactions.

7.5.1 ADMINISTRATIVE UI

The administrative interface provides system-wide configuration and management capabilities:

User Management

Screen	Purpose	Key Functionality
UserList	User account management	Create, edit, disable users
UserSettings	User configuration	Password policies, default roles
UserCreate	New user provisioning	Account creation workflow
UserEdit	User detail modification	Role assignment, privileges

Role and Privilege Management

Screen	Purpose	Key Functionality
RoleList	Role administration	View, create, edit roles
RoleEdit	Role configuration	Privilege assignment, user mapping
PrivilegeList	Privilege management	Custom privilege creation
PrivilegeEdit	Privilege configuration	Permission settings

System Configuration

Screen	Purpose	Key Functionality
SystemSettings	Global configuration	Server settings, proxy configuration
SecuritySettings	Security controls	Authentication, HTTPS, certificate management
TaskManagement	Scheduled operations	Task creation, scheduling, history
LoggingConfiguration	Log management	Log levels, file configuration

7.5.2 REPOSITORY MANAGEMENT

Repository management interfaces enable the creation, configuration, and monitoring of various repository types:

Repository Configuration

Screen	Purpose	Key Functionality
RepositoryList	Repository overview	List, filter, manage repositories
RepositoryCreate	New repository setup	Format-specific configuration
RepositoryEdit	Repository modification	Advanced settings, cleanup policies
RepositoryBrowse	Content navigation	Browse, search, upload, delete content

Format-specific Interfaces

Each repository format includes specialized interfaces:

Format	Specialized Screens	Unique Features
Maven	Maven Settings	Indexing, snapshot policies
Docker	Docker Configuration	Registry settings, v1/v2 support

Format	Specialized Screens	Unique Features
npm	npm Setup	Scope configuration, upstream settings
Raw	Upload Interface	Directory creation, path management
APT/YUM	Distribution Configuration	GPG signing, metadata generation

Storage Configuration

Screen	Purpose	Key Functionality
BlobStoreList	Storage management	View, create, edit blob stores
BlobStoreConfiguration	Storage setup	File, S3, group configuration
CleanupPolicies	Content retention	Age, format, usage based cleanup
MaintenanceTasks	Storage operations	Compact, rebuild, verify tasks

7.5.3 USER EXPERIENCE WORKFLOWS

The UI includes several workflows designed to enhance user experience and productivity:

Onboarding and Guidance

Workflow	Purpose	Implementation
Welcome Wizard	First-time setup	Multi-step guided configuration
Contextual Help	In-place guidance	Help panels with documentation links
Feature Discovery	Highlight capabilities	Targeted tooltips and walkthroughs
Setup Instructions	Client configuration	Repository-specific setup guides

Authentication Flows

Workflow	Purpose	Implementation
Login	User authentication	Username/password, remember me
Password Reset	Account recovery	Email-based reset workflow
Session Management	Security enforcement	Timeout warnings, re-authentication
Multi-factor Authentication	Enhanced security	Secondary verification step

Notification System

Workflow	Purpose	Implementation
Toast Notifications	Transient feedback	Action confirmation, errors
Status Indicators	System state	Health, availability indicators
Task Progress	Long-running operations	Progress bars, completion notices
Security Alerts	Critical information	Admin notifications, security warnings

7.6 STATE MANAGEMENT

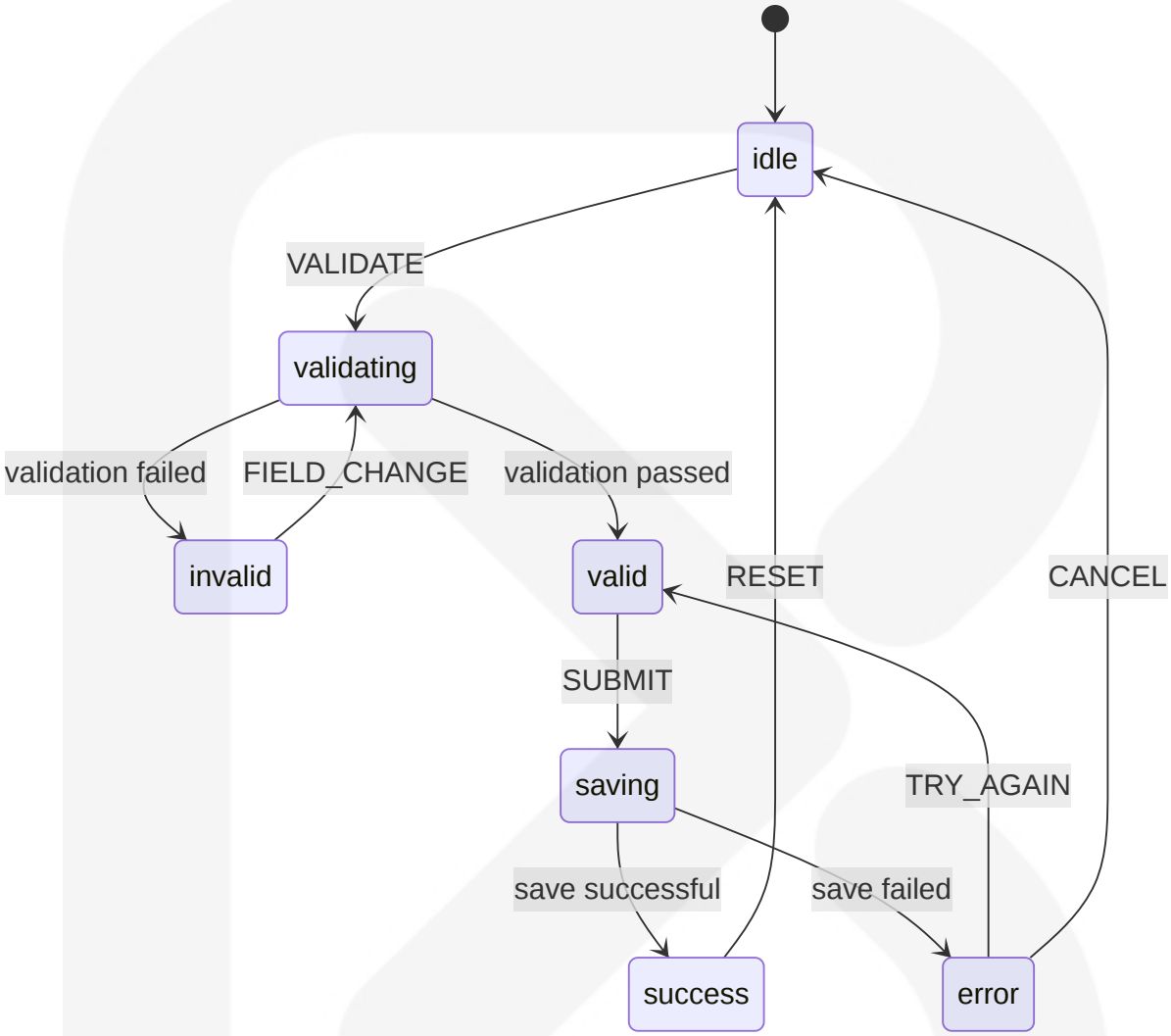
The Nexus Repository UI employs XState for sophisticated state management, providing predictable behavior and maintainable state logic.

7.6.1 STATE MACHINE PATTERNS

Several reusable state machine patterns are implemented throughout the application:

Form Machines

Forms utilize a consistent state machine pattern for validation, submission, and error handling:

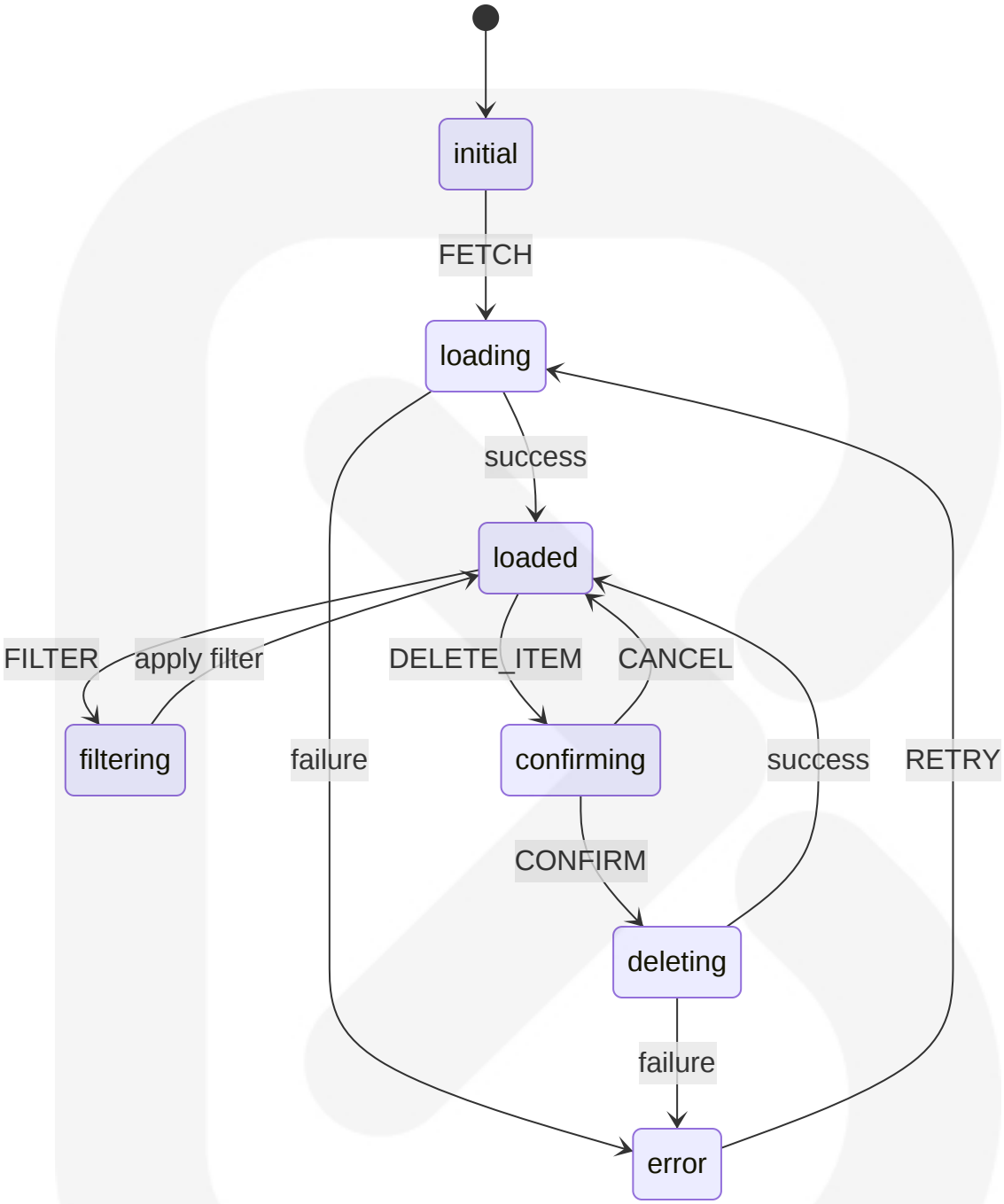


This pattern is used in forms such as:

- TasksFormMachine: Scheduled task configuration
- RealmsMachine: Authentication realm management
- RepositoryFormMachine: Repository creation and editing

List Machines

List views implement a standard machine for data fetching, filtering, and operations:



This pattern is used in components such as:

- UserListMachine: User management
- RoleSelectionMachine: Role assignment
- RepositoryListMachine: Repository browsing

Custom Workflow Machines

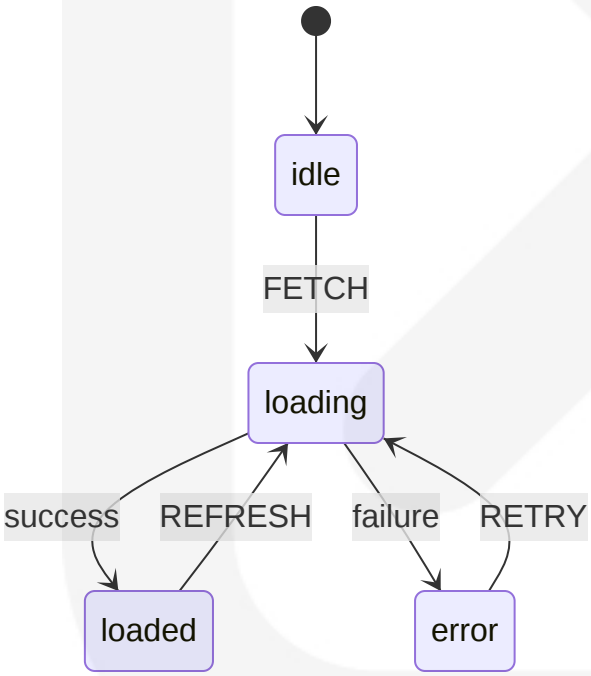
Complex workflows implement specialized state machines:

Machine	Purpose	States
OutreachActionsMachine	User guidance workflows	discover, tutorial, complete
UpgradeModalMachine	Version upgrade process	check, download, install, complete
WizardMachine	Multi-step processes	Various step states, validation, completion

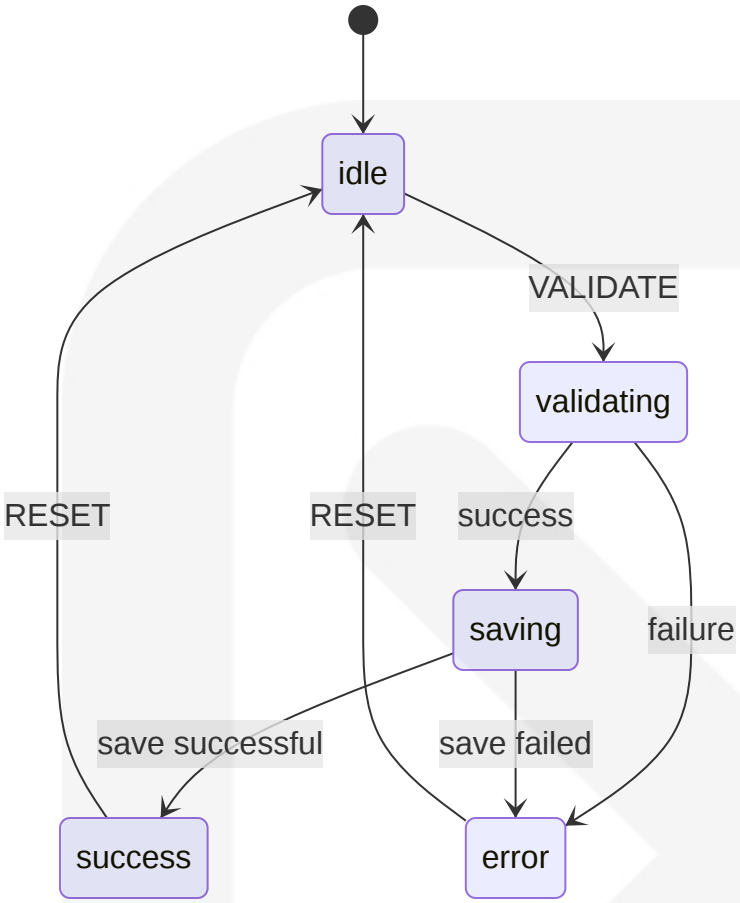
7.6.2 COMMON MACHINE STATES

Standard state patterns are used consistently across the application:

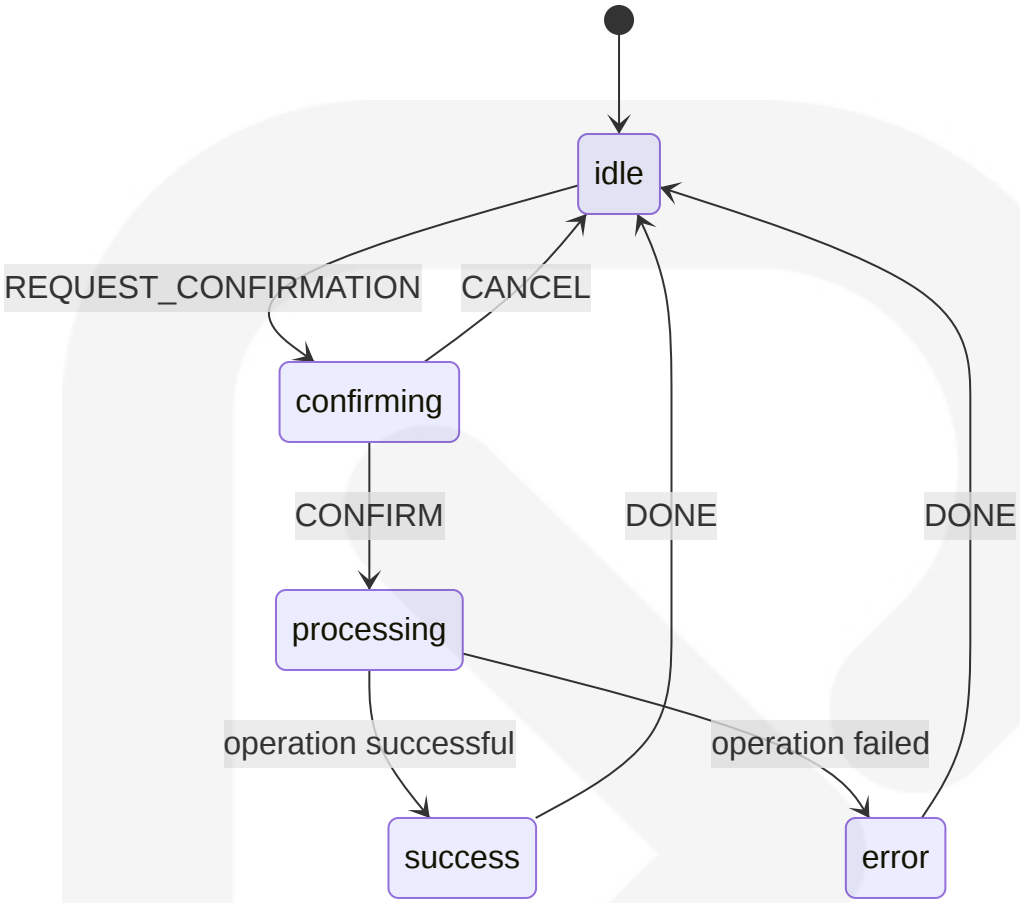
Data Fetching Pattern



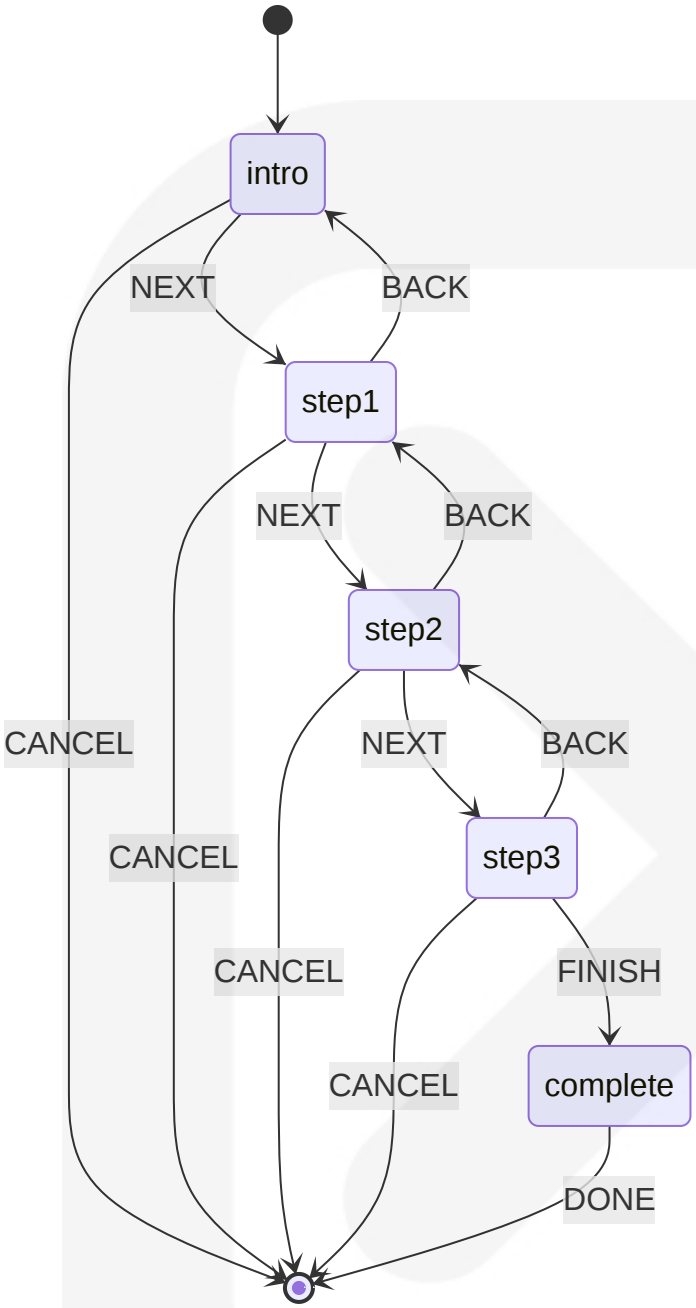
Form Submission Pattern



Confirmation Pattern



Wizard Flow Pattern



7.7 UI ROUTING AND NAVIGATION

The Nexus Repository UI implements a comprehensive navigation system that enables users to efficiently access all parts of the application.

7.7.1 NAVIGATION STRUCTURE

The navigation architecture is organized into a hierarchical structure:

Primary Navigation Areas

Navigation Section	Purpose	Access Control
Browse	Repository content exploration	Repository-specific permissions
Administration	System configuration	Admin privileges required
Support	Documentation and help	All authenticated users
User	User-specific functions	Self-service functions

Navigation Implementation

Hash-based routing is used throughout the application:

Route Pattern	Example	Component
#admin/[section]	#admin/repository	Admin section pages
#browse/[repository]	#browse/maven-central	Repository browser
#user/[function]	#user/tasks	User-specific functions

Breadcrumb Navigation

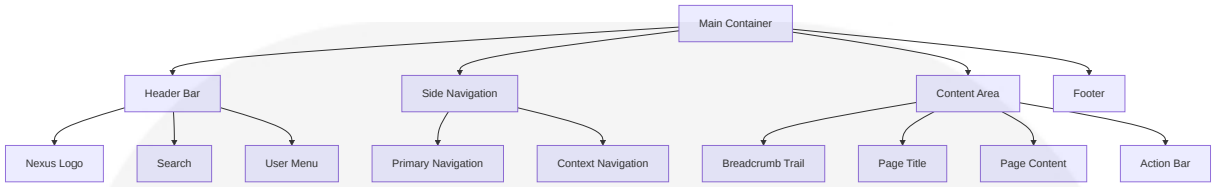
Context-aware breadcrumb trails provide:

- Current location feedback
- Navigation history
- Direct parent access
- Consistent location awareness

7.7.2 LAYOUT COMPONENTS

The UI layout is implemented through several key components:

Main Container Structure



Secondary Containers

Additional layout components are used for specific contexts:

Container	Purpose	Usage
Modal	Focused interaction	Dialogs, confirmations
Drawer	Supplementary content	Side panels, details
Tabs	Content organization	Grouped information
Cards	Content grouping	Dashboard widgets

Responsive Layouts

The UI implements responsive design through:

- Fluid grid system
- Breakpoint-based layouts
- Mobile-first media queries
- Collapsible navigation
- Touch-friendly controls for mobile devices

7.8 VISUAL DESIGN ELEMENTS

The visual design system of Nexus Repository ensures consistency, usability, and brand alignment throughout the application.

7.8.1 DESIGN SYSTEM

A comprehensive design system establishes consistent visual language:

Theming Support

Theming is implemented through SCSS variables and mixins:

Theme Element	Implementation	Example
Colors	SCSS variables	$nx-primary, $nx-success, $nx-error
Spacing	Layout tokens	$nx-spacing-md, $nx-spacing-lg
Typography	Font variables	$nx-font-family, $nx-font-size-md
Effects	Mixins	@include nx-box-shadow(), @include nx-transition()

Color System

The color palette includes semantic colors for states and feedback:

Color Category	Purpose	Examples
Brand Colors	Primary identity	Primary blue, secondary gray
UI Colors	Interface elements	Backgrounds, borders, separators
Feedback Colors	Status indication	Success green, error red, warning yellow
Data Visualization	Charts and graphs	Sequential and diverging scales

Typography Hierarchy

Typography implements a clear hierarchical system:

Level	Usage	Style
Page Title	Main page headings	24px, semi-bold, primary color

Level	Usage	Style
Section Title	Content section headings	18px, semi-bold, dark gray
Subsection Title	Nested headings	16px, medium weight, dark gray
Body Text	Regular content	14px, regular weight, medium gray
Caption	Supporting information	12px, regular weight, light gray

7.8.2 UI PATTERNS

The interface employs consistent patterns for user interaction:

Form Patterns

Form elements follow standardized patterns:

Pattern	Implementation	Usage
Field Factories	Consistent input handling	Form field generation
Validation	Inline error messages	Field-level validation feedback
Form Layout	Consistent grid structure	Organized input grouping
Action Buttons	Standard button positioning	Primary/secondary actions

Data Visualization

Data is presented using several standard patterns:

Pattern	Implementation	Usage
Grid/Tables	Format-consistent tables	Structured data presentation
Cards	Flexible content containers	Dashboard widgets, summary views
Charts	Data visualization	Metrics, trends, distributions

Pattern	Implementation	Usage
Status Indicators	Visual state representation	Health, availability, progress

Information Architecture

Content organization follows established patterns:

Pattern	Implementation	Usage
Progressive Disclosure	Reveal details as needed	Expandable sections, "show more"
Hierarchy	Visual prominence	Important content emphasis
Grouping	Logical content clusters	Related information organization
Scannable Content	Strategic whitespace	Easy information consumption

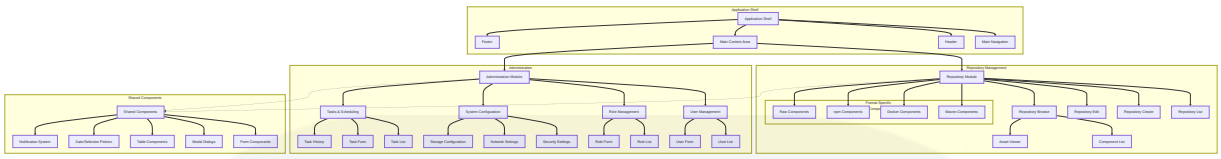
Accessibility Considerations

The design system incorporates accessibility features:

Feature	Implementation	Benefit
Color Contrast	WCAG AA compliance	Readability for all users
Keyboard Navigation	Focus management	Non-mouse accessibility
Screen Reader Support	ARIA attributes	Assistive technology compatibility
Responsive Design	Flexible layouts	Device independence

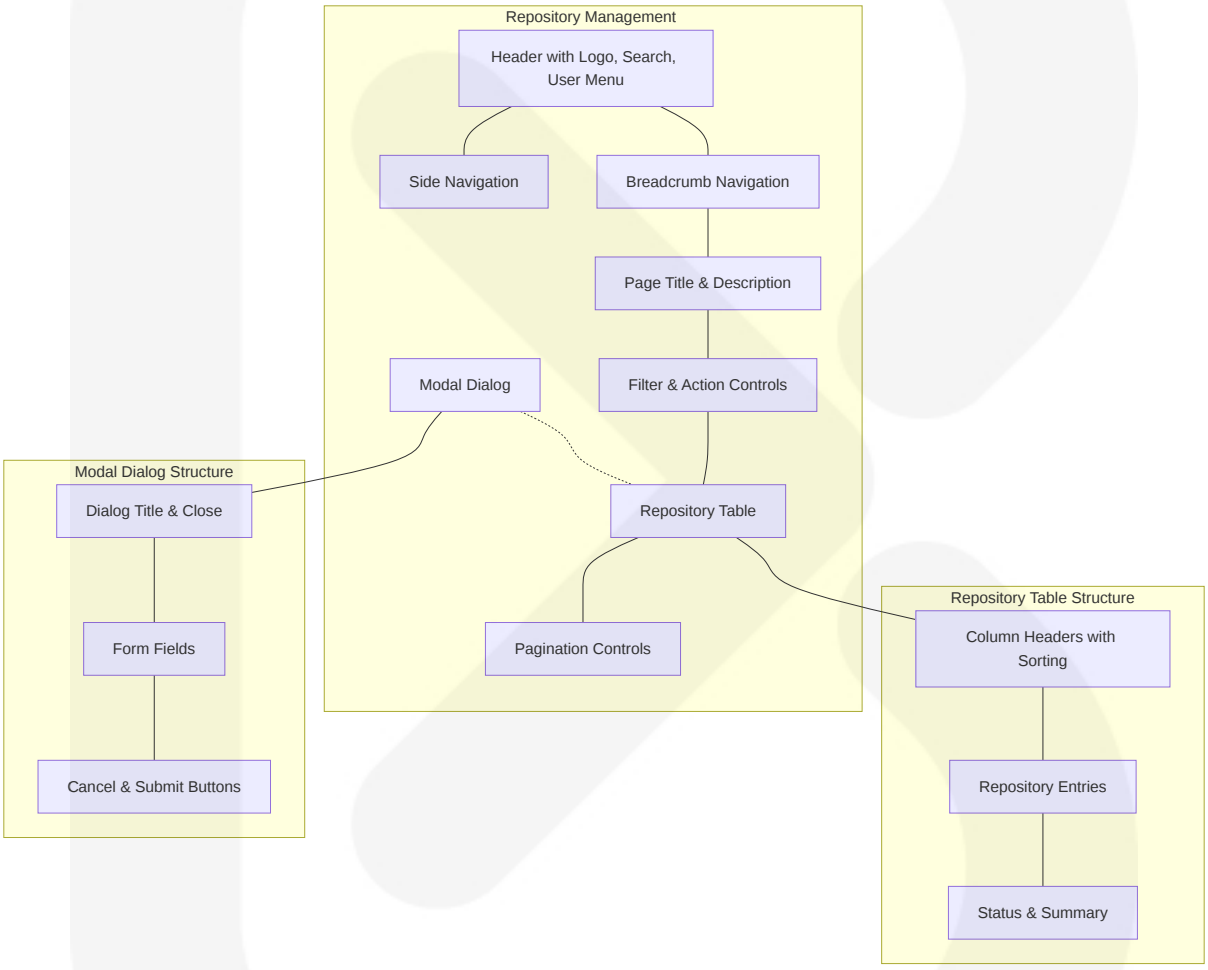
7.9 UI COMPONENT DIAGRAM

The following diagram illustrates the major UI components and their relationships:



7.9.1 SCREEN LAYOUT EXAMPLE

The following diagram illustrates a typical repository management screen layout:



7.10 SOURCE FILES ANALYZED

The USER INTERFACE DESIGN section is based on analysis of the following source files and directories:

1. **UI Plugin Core**

- `components/nexus-ui-plugin`
- `components/nexus-rapture`
- `plugins/nexus-coreui-plugin`

2. **Key UI Component Files**

- `nexus-ui-plugin/src/frontend/src/index.js`
- `nexus-coreui-plugin/src/frontend/src/components/`
- `nexus-rapture/src/main/resources/static/rapture/NX/view/`
- `nexus-coreui-plugin/src/frontend/src/components/pages/`

3. **State Management Files**

- `nexus-coreui-plugin/src/frontend/src/components/pages/admin/Roles/RoleSelectionMachine.js`
- `nexus-coreui-plugin/src/frontend/src/components/pages/admin/Realms/RealmsMachine.js`
- `nexus-coreui-plugin/src/frontend/src/components/pages/user/Welcome/OutreachActionsMachine.js`

4. **Integration Files**

- `nexus-coreui-plugin/src/main/resources/static/rapture/NX/coreui/view/react/MainContainer.js`
- `nexus-coreui-plugin/src/frontend/src/interfaces/LocationUtils.js`
- `nexus-rapture/src/frontend/src/index.js`

8. INFRASTRUCTURE

8.1 DEPLOYMENT ENVIRONMENT

8.1.1 Target Environment Assessment

Environment Type

Sonatype Nexus Repository supports flexible deployment options to accommodate various organizational needs:

Environment Type	Support Level	Primary Use Case
On-premises	Fully supported	Secure enterprise environments with strict data residency requirements
Private cloud	Fully supported	Organizations with private cloud infrastructure preferring VM-based deployments
Hybrid	Supported with configuration	Organizations with mixed on-premises/cloud infrastructure
Multi-cloud	Supported with additional setup	Enterprises requiring high availability across cloud providers

All deployment environments require Java 21 JDK/JRE installed and available on the host or container runtime. The system is primarily designed for on-premises deployment but can be effectively operated in cloud environments with appropriate configuration.

Before deployment, verify that container images, VM host images, or base AMIs are compatible with Java 21 runtime requirements to ensure proper system operation.

Geographic Distribution Requirements

Nexus Repository supports distributed deployment models to serve global teams:

- **Single-instance Deployment**: Suitable for co-located teams or regional offices
- **Multi-region Deployment**: Supports repository replication and proxying to reduce latency for geographically distributed teams
- **Repository Routing**: Path-based content routing to direct traffic to appropriate regional instances

- **Active/Passive Clustering**: High availability configuration for cross-region failover

For multi-region deployments, content synchronization is achieved through blob store replication (for file-based storage) or shared storage approaches (for cloud deployments).

Resource Requirements

Nexus Repository has the following resource requirements for production environments:

Resource	Minimum	Recommended	High-Volume
CPU	4 cores	8 cores	16+ cores
Memory	4GB RAM	8GB RAM	16-32GB RAM
Java Heap	2GB	4GB	8-16GB
Storage	50GB	250GB+	1TB+
Network	1Gbps	10Gbps	10Gbps+

Note: Java heap allocation requirements remain consistent with previous versions, but Java 21's improved GC defaults (particularly ZGC) and reduced startup overhead may allow for more efficient memory utilization.

Additional considerations:

- **Database**: H2 (embedded, smaller deployments) or PostgreSQL (external, production)
- **BlobStore**: File system or S3-compatible object storage
- **Disk I/O**: SSD recommended for metadata and frequently accessed content
- **Java Runtime**: Java 21 is the minimum supported runtime version
- **JVM Configuration**: Deployment-level JVM flags should leverage Java 21 features, including Virtual Threads support and modern garbage collection tuning

Compliance and Regulatory Requirements

Nexus Repository addresses multiple compliance and regulatory needs:

- **Data Residency**: On-premises deployment option ensures data remains within specified jurisdictions
- **Access Controls**: Role-based access control supports organizational security policies
- **Audit Trails**: Comprehensive audit logging for regulatory compliance
- **SSL/TLS**: Configurable security settings to enforce organization-specific requirements, with Java 21 default support for TLS 1.3+
- **Authentication**: Integration with enterprise identity providers (LDAP, Active Directory, SAML)
- **Content Security**: Repository format-specific content policies and verification
- **Cryptographic Standards**: Leverages Java 21 default cryptographic providers with enhanced security profiles
- **Security Policy Alignment**: Verification procedures to ensure organization-specific security policies align with Java 21 runtime defaults

The application does not store personal data beyond basic user account information required for authentication and authorization.

8.1.2 Environment Management

Infrastructure as Code (IaC) Approach

While Nexus Repository itself does not include built-in IaC capabilities, it supports deployment through several Infrastructure as Code approaches:

IaC Tool	Support Level	Implementation
Docker Compose	Full, official	Official Docker images with Java 21 runtime and sample compose files
Kubernetes Manifests	Community-supported	Helm charts and YAML manifests for Java 21-based deployments
Terraform	Community-supported	Community-provided modules for AWS, Azure, GCP with Java 21 base images

IaC Tool	Support Level	Implementation
Ansible	Community-supported	Playbooks for installation and configuration with Java 21 prerequisites
Chef/Puppet	Community-supported	Cookbooks/modules for management with Java 21 runtime detection

Example Docker Compose configuration for a basic deployment:

```
version: '3'
services:
  nexus:
    image: sonatype/nexus3:latest
    ports:
      - "8081:8081"
    volumes:
      - nexus-data:/nexus-data
    environment:
      - INSTALL4J_ADD_VM_PARAMS="-Xms2g -Xmx4g -XX:MaxDirectMemorySize=3g
volumes:
  nexus-data:
```

Note: This Docker Compose configuration uses the latest Nexus image which includes Java 21. Remove any legacy Java 17-specific environment variables if present in existing deployments.

Configuration Management Strategy

Configuration for Nexus Repository is managed through several mechanisms:

- **System Properties**: Java system properties for core settings
- **nexus.properties**: Primary configuration file in sonatype-work/nexus3/etc/
- **Database-stored Configuration**: Most settings stored in the embedded or external database
- **REST API**: Configuration via RESTful API for automation
- **Scripting API**: Groovy scripts for complex configuration and provisioning

For environment-specific configuration, the system supports:

- **Environment Variables**: Override settings via environment variables
- **Properties Files**: External property files for different environments
- **Docker Volume Mapping**: Configuration mounted from host system
- **JVM Detection**: nexus.properties or system property overrides for Java 21 JVM detection
- **Java 21 Flags**: Environment variable mappings for Java 21-specific flags (including enabling preview features where applicable)

Best practice is to maintain configuration as code in a version control system and apply it through REST API, scripting, or container initialization.

Environment Promotion Strategy

Sonatype Nexus Repository supports a structured environment promotion strategy:

Environment	Purpose	Promotion Approach
Development	Integration testing, developer use	Manual configuration or scripts
Staging/Test	Pre-production validation	Configuration export/import
Production	Live system	Scripted configuration or backup/restore

The recommended promotion workflow:

1. **Development**: Configure and test in development environment
2. **Export Configuration**: Use REST API or UI export to capture configuration
3. **Import to Staging**: Apply configuration to staging environment
4. **Validation**: Test functionality in staging
5. **Java 21 Runtime Validation**: Confirm each promoted environment runs successfully under Java 21
5. **Production Deployment**: Apply validated configuration to production

For content promotion between environments:

- Repository groups can be used to create environment-specific content views

- Scheduled tasks can synchronize content between environments
- Repository path routing can direct requests to appropriate repositories

Backup and Disaster Recovery Plans

Nexus Repository implements comprehensive backup and disaster recovery capabilities:

Component	Backup Approach	Recovery Method
Database	Database backup (H2) or PostgreSQL dump	Database restore
BlobStore	File system backup or S3 replication	Restore from back up
Configuration	Configuration export	Configuration impo rt
Task History	Database backup	Database restore
Security Setting s	Database backup	Database restore

Recommended backup strategy:

1. **Regular Database Backups**: Daily for metadata and configuration
2. **BlobStore Backups**: Incremental for content (can be less frequent for stable repositories)
3. **Configuration Export**: After significant changes
4. **External Secrets**: Backup encryption keys separately

Disaster recovery process:

1. **Infrastructure Provisioning**: Deploy new Nexus instance with Java 21 runtime
2. **Database Restoration**: Restore database backup
3. **BlobStore Restoration**: Restore blob store content
4. **Configuration Verification**: Verify settings after restoration
5. **Connectivity Testing**: Validate client access and functionality

The system includes built-in tasks for database backup and supports integration with external backup systems.

8.2 CLOUD SERVICES

8.2.1 Cloud Provider Selection and Justification

Sonatype Nexus Repository is provider-agnostic and can be deployed on any major cloud platform:

Cloud Provider	Support Level	Key Benefits
AWS	Fully supported	Native S3 integration, broad service ecosystem
Azure	Fully supported	Azure Blob Storage integration, strong enterprise presence
Google Cloud	Fully supported	Google Cloud Storage, Kubernetes integration
Oracle Cloud	Supported	Enterprise compatibility
IBM Cloud	Supported	Enterprise compatibility

*Note: All cloud providers must be validated for Java 21 compatibility as part of the onboarding process.

The selection criteria should consider:

- Existing organizational cloud footprint
- Storage service compatibility (especially for S3 BlobStore)
- Database service compatibility (PostgreSQL)
- Network integration requirements
- Cost optimization opportunities
- Availability of VM or container images with official Java 21 support (e.g., Amazon Corretto 21, Azure OpenJDK 21, Google Cloud JDK 21)

No specific cloud provider is required, allowing organizations to select based on their existing investments and preferences.

8.2.2 Core Services Required

When deploying to cloud environments, Nexus Repository depends on several core services:

Service Type	Required Services	Versions/Specifications
Compute	VM instances or containers	Min 4 vCPUs, 8GB RAM with Java 21 JDK/JRE installed
Database	PostgreSQL or H2 (embedded)	PostgreSQL 9.6+ (12+ recommended) with Java 21-compatible JDBC drivers (e.g., PostgreSQL JDBC 42.7.2+)
Storage	S3-compatible object storage or block storage	50GB+
Networking	Load balancing, VPC/VNET	HTTP/HTTPS endpoints
Identity	Optional: IAM for service account access	Role-based access
Java Runtime	Java 21 (LTS)	OpenJDK or vendor-specific JDK distributions

For AWS deployments:

- EC2 or ECS for compute
- RDS PostgreSQL for database
- S3 for blob storage
- ELB/ALB for load balancing
- Java Runtime: Java 21 (LTS), preferably Amazon Corretto 21

For Azure deployments:

- Azure VMs or AKS for compute

- Azure Database for PostgreSQL
- Azure Blob Storage
- Azure Load Balancer
- Java Runtime: Java 21 (LTS), preferably Microsoft Build of OpenJDK 21

For Google Cloud:

- Compute Engine or GKE for compute
- Cloud SQL for PostgreSQL
- Cloud Storage
- Cloud Load Balancing
- Java Runtime: Java 21 (LTS), compatible with Google Cloud environment

8.2.3 High Availability Design

Nexus Repository supports high availability configurations in cloud environments:

Component	HA Approach	Cloud Implementation
Application Tier	Active/passive clustering	Load balancer with health checks
Database Tier	PostgreSQL replication	Managed database service with HA
Storage Tier	Shared S3 storage	Cloud object storage with replication
Network Tier	Redundant load balancers	Cloud load balancing service

High availability architecture:

1. **Multiple Application Instances**: Deploy multiple Nexus instances with a shared configuration
2. **Shared Database**: Use a managed PostgreSQL instance with replication
3. **Shared Blob Storage**: Use cloud object storage (S3, Azure Blob, etc.)
4. **Load Balancing**: Direct traffic to healthy instances
5. **Health Checking**: Regular status checks to detect failures

6. **Automated Failover**: Redirect traffic when instances become unhealthy

The HA configuration requires:

- PostgreSQL database (not H2)
- S3 or shared blob storage
- Load balancer with session affinity

8.2.4 Cost Optimization Strategy

Cost optimization for cloud-based Nexus Repository deployments focuses on several key areas:

Aspect	Optimization Strategy	Potential Savings
Storage	Tiered storage for infrequently accessed content	30-50% storage cost
Compute	Right-sizing based on actual usage	20-40% compute cost
Database	Appropriate instance selection	15-30% database cos t
Networkin g	Proxy repositories to reduce egress	Variable based on us age
Operation s	Scheduled scaling based on usage patte rns	10-20% overall cost

Specific optimization techniques:

1. **Storage Tiering**: Move older, less accessed artifacts to lower-cost storage tiers
2. **Cleanup Policies**: Implement format-specific cleanup to remove outdated artifacts
3. **Instance Sizing**: Monitor resource utilization and adjust instance sizes
4. **Reserved Instances**: For stable, long-term deployments, use reserved instances
5. **Proxy Repositories**: Cache external dependencies to reduce bandwidth and egress costs
6. **Scheduled Scaling**: For predictable usage patterns, scale down during off-hours

The system's built-in task scheduling can automate many cost optimization processes, including cleanup, compaction, and analytics.

8.2.5 Security and Compliance Considerations

Cloud deployments of Nexus Repository require specific security and compliance measures:

Security Aspect	Implementation	Cloud Considerations
Network Security	VPC/VNET isolation, security groups	Restrict access to necessary services
Data Encryption	Encryption at rest and in transit	Use cloud provider encryption services with Java 21's updated security defaults (latest TLS protocol, modern cipher suites)
Identity Management	Integration with cloud IAM	Service account least privilege with verification for Java 21 compatibility
Access Controls	Role-based access	Integrate with cloud identity providers
Audit Logging	System and access logging	Export to cloud logging services
Vulnerability Management	Regular updates	Automated image updates

Cloud-specific security recommendations:

1. **Private Network Deployment**: Deploy within a VPC/VNET with controlled ingress/egress
2. **Managed SSL/TLS**: Use cloud provider certificate services
3. **IAM Integration**: Use service accounts with minimal permissions
4. **Storage Encryption**: Enable encryption for blob stores and databases
5. **Log Integration**: Export logs to cloud monitoring services
6. **Backup Encryption**: Encrypt all backups
7. **Security Groups**: Restrict network access to required ports only
8. **Java Security Features**: Leverage Java 21's updated security defaults for

TLS configuration and cipher suites
9. **IAM Verification**: Verify cloud IAM roles and policies accommodate any new security capabilities provided by Java 21 providers (e.g., key management for strong crypto)

Compliance considerations will vary by industry and region, but cloud deployments should maintain the same compliance standards as on-premises deployments.

8.3 CONTAINERIZATION

8.3.1 Container Platform Selection

Sonatype Nexus Repository provides official Docker images for containerized deployment:

Platform	Support Level	Implementation
Docker	Official support	Sonatype-maintained images
Podman	Compatible	Uses standard OCI images
Kubernetes	Community support	Helm charts and manifests
OpenShift	Compatible	Container deployment with security contexts

The official Docker image is available on Docker Hub and serves as the reference container implementation. This image provides:

- Pre-configured Nexus application
- Java runtime environment
- Base operating system (Red Hat Universal Base Image)
- Default configuration suitable for container deployment

8.3.2 Base Image Strategy

The official Nexus Repository Docker image follows a structured base image strategy:

Aspect	Implementation	Rationale
Base OS	Red Hat Universal Base Image (UBI)	Security, stability, support
Java Runtime	OpenJDK 21	Performance, support lifecycle
User Context	Non-root user (nexus)	Security best practice
File System Layout	Standard layout with volume mounts	Persistent storage separation
Configuration	Environment variable overrides	Container-friendly configuration

Key features of the base image:

- Minimal OS footprint to reduce attack surface
- Security patches from Red Hat's ecosystem
- Non-root execution for enhanced security
- Clear separation between application and data
- Optimized Java 21 runtime for container environments

8.3.3 Image Versioning Approach

The Nexus Repository container images follow a structured versioning scheme:

Image Tag Pattern	Purpose	Usage
latest	Most recent stable release	Development, testing
3.x.y-ubi	Specific version (UBI base)	Production
3.x.y	Specific version (legacy)	Production (legacy)
3.x.y-x	Patch releases	Production updates

Version management best practices:

1. **Pin to Specific Versions**: Production deployments should reference exact version tags

2. **No Latest in Production**: Avoid using 'latest' tag in production to prevent unexpected updates
3. **Version Alignment**: Ensure container version matches internal components
4. **Image Scanning**: Regularly scan images for vulnerabilities
5. **Upgrade Planning**: Plan container image upgrades alongside application upgrades

Each released image includes:

- The specific Nexus application version
- A compatible Java runtime
- Base OS with current security patches
- Build date and version metadata

8.3.4 Build Optimization Techniques

The Nexus Repository container images implement several optimization techniques:

Technique	Implementation	Benefit
Multi-stage builds	Separate build and runtime images with Java 21	Smaller final image
Layer optimization	Ordered operations for cache efficiency	Faster builds and downloads
Minimal dependencies	Only required packages included	Reduced attack surface, smaller size
JVM tuning	Container-aware memory settings with Java 21 optimizations	Better resource utilization
Base image selection	Security-focused UBI base	Regular security updates

Key build optimizations include:

1. **Dependency Management**: Careful management of included dependencies
2. **File System Organization**: Strategic placement of files to optimize layer caching
3. **Environment Variable Defaults**: Sensible defaults for container operation

4. **Volume Management**: Clear separation of persistent vs. ephemeral data
5. **Cache Invalidation Strategy**: Ordering operations to maximize build cache hits

Java 21-specific optimizations include:

1. **Improved GC Defaults**: Utilizing Java 21's enhanced garbage collection defaults
2. **Virtual Threads Support**: Environment variables for enabling virtual threads
3. **Optimized Memory Settings**: Tailored container memory configurations for Java 21
4. **Startup Performance**: Leveraging Class-Data Sharing and AppCDS features

These optimizations result in:

- Faster image pulls and deployments
- Reduced storage requirements
- Improved security posture
- Better container runtime performance

8.3.5 Security Scanning Requirements

Container security is a critical aspect of Nexus Repository deployment:

Security Aspect	Implementation	Tools/Approach
Image Scanning	Vulnerability scanning	Nexus Platform IQ, Clair, Trivy
Base Image Security	Regular security updates	UBI security patches
Runtime Security	Non-root user, minimal privileges	SecurityContext, restricted capabilities
Supply Chain Security	Verified image sources	Image signing, registry authentication
Content Trust	Image verification	Docker Content Trust, Cosign

Security scanning recommendations:

1. **Pre-deployment Scanning**: Scan images before deployment

2. **Regular Rescanning**: Check for new vulnerabilities in deployed images
3. **Policy Enforcement**: Establish vulnerability severity thresholds
4. **Patch Management**: Process for applying security updates
5. **Image Hardening**: Remove unnecessary components and permissions
6. **Java 21 Runtime Scanning**: Specifically scan for vulnerabilities in Java 21 runtime components
7. **Base Image Refresh**: Periodically refresh the base UBI image with Java 21 to incorporate security updates
8. **Preview Module Validation**: Validate any preview or incubator modules used by Java 21 in container scan policies

The Nexus Repository container is designed with security in mind, but operational security remains a shared responsibility between Sonatype and the deploying organization.

8.4 ORCHESTRATION

8.4.1 Orchestration Platform Selection

For orchestrated deployments, Nexus Repository supports several platforms:

Platform	Support Level	Implementation
Kubernetes	Community support	Helm charts, YAML manifests
OpenShift	Compatible	Security context constraints
Rancher	Compatible	Standard Kubernetes deployment
Amazon EKS	Compatible	Standard Kubernetes deployment
Azure AKS	Compatible	Standard Kubernetes deployment
Google GKE	Compatible	Standard Kubernetes deployment

Kubernetes is the recommended orchestration platform due to its wide adoption, flexibility, and ecosystem. The application can be deployed using standard Kubernetes resources:

- Deployments for application management
- Services for networking
- ConfigMaps and Secrets for configuration
- PersistentVolumeClaims for storage
- Ingress for external access

8.4.2 Cluster Architecture

A Kubernetes-based deployment for Nexus Repository typically follows this architecture:

Component	Kubernetes Resource	Configuration
Application	Deployment	1+ replicas, resource limits
Database	StatefulSet or external	PostgreSQL for production
Storage	PersistentVolumeClaim	ReadWriteMany for shared access
Networking	Service, Ingress	LoadBalancer or NodePort + Ingress
Configuration	ConfigMap, Secret	Environment variables, config files

Cluster architecture considerations:

1. **Node Selection**: Consider dedicated nodes for Nexus to manage resource allocation
2. **Affinity/Anti-Affinity**: Ensure proper scheduling for HA configurations
3. **Topology Spread**: Distribute instances across failure domains
4. **Storage Class**: Select appropriate storage class for performance and availability
5. **Resource Quotas**: Enforce resource limits to prevent resource contention
6. **Java 21 Support**: Verify orchestration nodes support Java 21 runtime requirements

When deploying Java 21-based containers, it's essential to verify that the orchestration nodes and cluster CNI support the required kernel flags and resource limits for optimal Java 21 performance. Key requirements include:

- Kernel parameters for container memory management

- Network stack configuration for Virtual Threads
- Support for appropriate seccomp profiles
- Node kernel version 5.15+ recommended for optimal performance

Example minimal Kubernetes manifest:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nexus-repository
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nexus-repository
  template:
    metadata:
      labels:
        app: nexus-repository
    spec:
      containers:
      - name: nexus
        image: sonatype/nexus3:3.77.0-08-ubi-java21
        ports:
        - containerPort: 8081
        volumeMounts:
        - mountPath: /nexus-data
          name: nexus-data
        resources:
          requests:
            memory: "4.5Gi"
            cpu: "2"
          limits:
            memory: "9Gi"
            cpu: "4"
      volumes:
      - name: nexus-data
        persistentVolumeClaim:
          claimName: nexus-data
---
apiVersion: v1
kind: Service
```

```
  metadata:
    name: nexus-repository
  spec:
    selector:
      app: nexus-repository
    ports:
    - port: 8081
      targetPort: 8081
    type: ClusterIP
```

Note that memory requests have been increased by approximately 12.5% (from 4Gi to 4.5Gi) and limits by 12.5% (from 8Gi to 9Gi) to account for Java 21 garbage collection overhead in production clusters.

8.4.3 Service Deployment Strategy

For orchestrated environments, several deployment strategies are suitable:

Strategy	Implementation	Use Case
Rolling Update	Kubernetes rolling deployment	Standard upgrades
Blue/Green	Parallel deployments with service switch	Zero-downtime upgrades
Canary	Gradual traffic shifting	High-risk changes
Backup and Restore	Scheduled backups before updates	Data-critical deployments

Recommended deployment process:

1. **Pre-deployment Backup**: Ensure database and blob store are backed up
2. **Health Verification**: Verify current system health before deployment
3. **Rolling Update**: Use Kubernetes rolling update with appropriate settings
4. **Health Validation**: Verify new version health before proceeding
5. **Post-deployment Validation**: Confirm functionality and performance

For Kubernetes deployments, the following settings are recommended:

- maxUnavailable: 0 (to maintain availability)

- maxSurge: 1 (to control deployment rate)
- readinessProbe: HTTP check of the health endpoint
- livenessProbe: Basic port check after initial delay

8.4.4 Auto-scaling Configuration

While Nexus Repository is not designed for horizontal auto-scaling of a single instance, orchestration platforms can manage scaling for high availability:

Scaling Aspect	Implementation	Considerations
Horizontal Scaling	Multiple independent instances	Requires shared storage, database
Vertical Scaling	Resource adjustment	Requires application restart
Load Distribution	Multiple instances behind load balancer	Requires session affinity

Kubernetes auto-scaling configuration:

1. **Resource-based Scaling**: Trigger based on CPU/memory utilization
2. **Custom Metrics**: Monitor application-specific metrics like request rate
3. **Scheduled Scaling**: Predetermined scaling for known usage patterns
4. **Manual Scaling**: Administrative control for planned events

Example Horizontal Pod Autoscaler (limited applicability):

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: nexus-repository
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nexus-repository
  minReplicas: 1
  maxReplicas: 3
```

```
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 80
```

Note: True horizontal scaling requires shared storage and database configuration.

8.4.5 Resource Allocation Policies

Effective resource allocation is critical for orchestrated Nexus Repository deployments:

Resource	Recommendation	Rationale
CPU	Request: 2 cores, Limit: 4 cores	Baseline performance with burst capacity
Memory	**Request: 4.5Gi, Limit: 9Gi**	**JVM plus system overhead with Java 21 requirements**
Storage	50GB+ depending on repository size	Content storage plus metadata
Java Heap	**2GB for smaller, 4-8GB for larger deployments**	**Application memory requirements**

Resource allocation best practices:

1. **Memory-to-CPU Ratio**: Maintain approximate 2:1 GB-to-core ratio, but validate this under Java 21 load tests as optimal ratios may differ
2. **Guaranteed Resources**: Set requests and limits close for critical workloads
3. **Overhead Accounting**: Allow 1-2GB beyond JVM heap specifically for Java 21's advanced garbage collection algorithms and runtime overhead
4. **Storage Monitoring**: Implement alerting for storage utilization
5. **Resource Quality**: Consider storage performance (IOPS, throughput) for production

Java 21-specific resource considerations:

1. **GC Overhead**: Java 21's garbage collection may require additional memory headroom
2. **Virtual Threads**: Configure appropriate thread pools when using Virtual Threads feature
3. **Resource Validation**: Test resource quotas and limits under Java 21 load patterns
4. **Memory Tuning**: Adjust -XX:MaxRAMPercentage settings for optimal container memory utilization

For large deployments, consider resource segregation:

- Separate nodes for Nexus Repository
- Dedicated storage volumes
- Node anti-affinity to spread instances
- Priority classes for resource competition

8.5 CI/CD PIPELINE

8.5.1 Build Pipeline

Nexus Repository can be integrated into CI/CD pipelines as both a build dependency and a deployment target.

Source Control Triggers

For building and deploying Nexus Repository:

Trigger Type	Implementation	Use Case
Release Tags	Git tag with version pattern	Official releases
Branch Updates	CI triggered on main/release branches	Development builds
Pull Requests	Validation builds on PR	Quality control
Scheduled Builds	Regular rebuild schedule	Security updates

For custom configurations and extensions:

- Source-controlled infrastructure code
- Configuration scripts
- Custom plugins
- Deployment templates

Build Environment Requirements

Building Nexus Repository or custom extensions requires:

Requirement	Specification	Purpose
Java SDK	Java 21	Core application build
Maven	3.9.6 (via wrapper)	Build orchestration
Node.js	v18.17.1	Frontend build
Yarn	v1.22.19	Frontend dependency management
Docker	Latest stable	Container builds
Git	Latest stable	Source control

The application includes a Maven wrapper script (`mvnw` and `mvnw.cmd`) that ensures consistent Maven version usage across build environments. The Maven wrapper configuration has been updated to target Java 21, with appropriate updates to the `.mvn/wrapper/maven-wrapper.properties` file and relevant `pom.xml` configurations to ensure Java 21 compatibility.

A dedicated pipeline step has been introduced to validate Java 21 compatibility early in the build process. This step compiles the codebase with the `--release 21` flag to ensure all code is compatible with the Java 21 language features and restrictions.

Dependency Management

Nexus Repository uses a comprehensive dependency management approach:

Aspect	Implementation	Tools
Java Dependencies	Maven BOMs, central pom.xml	Maven
Frontend Dependencies	package.json, yarn.lock	Yarn
Container Base Images	Dockerfile pinned versions	Docker
Build Tool Versions	Maven wrapper, defined in pom.xml	mvnw
Third-party Bundles	Managed in thirdparty-bundles	Maven, OSGi

Dependency resolution can be accelerated by:

- Using Nexus Repository itself as a proxy for public repositories
- Implementing a Maven build cache
- Using the built-in .mvn/wrapper caching

Artifact Generation and Storage

The build pipeline generates several artifacts:

Artifact Type	Format	Storage Location
Core Application	ZIP bundle	Maven repository
Docker Image	OCI container	Container registry
Documentation	HTML/PDF	Document repository
Release Notes	Markdown/HTML	Release system

These artifacts should be stored in:

- An artifact repository (ideally Nexus Repository)
- A container registry (public or private)
- A documentation system

Quality Gates

The build pipeline should implement several quality gates:

Gate Type	Implementation	Tools
Unit Tests	Maven Surefire	JUnit 5.10+, Maven
Integration Tests	Maven Failsafe	JUnit 5.10+, Maven
Code Coverage	JaCoCo	Maven plugin
Static Analysis	SpotBugs, PMD	Maven plugins
Dependency Scanning	Nexus IQ Server	Maven plugin
Container Scanning	Image vulnerability scanner	Trivy, Clair

All test frameworks have been verified for Java 21 compatibility. This includes updated versions of JUnit (5.10+) and Spock Framework (2.3+) to ensure full support for Java 21 features such as Virtual Threads and Record Patterns.

The build should fail if any of these gates fail to meet defined thresholds.

8.5.2 Deployment Pipeline

Automating Nexus Repository deployment ensures consistency and reliability.

Deployment Strategy

Several deployment strategies are suitable for Nexus Repository:

Strategy	Implementation	Use Case
Blue/Green	Parallel environments with tr affic switch	Zero-downtime production upgrades
Rolling Update	Sequential instance updates	Standard Kubernetes deplo yments
Backup and Rep lace	Backup, stop, install, restore	Small environments
Canary Deploym ent	Gradual traffic routing	High-risk updates

For containerized deployments, deployment scripts and GitHub Actions workflows have been modified to pull and deploy Nexus container images built on Java 21.

Container image tags now include the Java version (e.g., `sonatype/nexus3:3.77.0-08-ubi-java21`) to clearly identify the runtime environment.

For containerized deployments, a blue/green or rolling update approach is recommended to minimize downtime.

Environment Promotion Workflow

The typical environment promotion workflow:

1. **Development**:
    - Deploy to development environment
    - Validate basic functionality
    - Perform integration testing
2. **Test/Staging**:
    - Deploy validated build
    - Perform UAT and performance testing
    - Validate with production-like data
    - Execute security scans
3. **Production**:
    - Schedule maintenance window (if needed)
    - Execute backup procedures
    - Deploy using chosen strategy
    - Validate deployment
    - Monitor for issues

Promotion should be automated but with appropriate approval gates between environments.

Rollback Procedures

Robust rollback procedures are essential:

Scenario	Rollback Approach	Prerequisites
Failed Deployment	Revert to previous container image	Image tag retention with Java 21 tags
Database Corruption	Restore from backup	Regular database backups
Configuration Issues	Revert configuration changes	Configuration versioning
Data Loss	Restore from blob store backup	Regular blob store backups

Rollback procedures have been updated to reference Java 21 image tags and updated container entry points. When rolling back from a Java 21-based deployment, ensure that the previous image tag is also Java 21-compatible to maintain consistency. If rolling back to a Java 17 image is required, additional steps must be taken to adjust any Java 21-specific configurations.

Rollback procedure best practices:

1. **Automated Rollback**: Script the rollback process when possible
2. **Practice Drills**: Regularly test rollback procedures
3. **Health Validation**: Implement automated health checks post-rollback
4. **Root Cause Analysis**: Investigate failures before attempting new deployments

Post-deployment Validation

Comprehensive validation should follow each deployment:

Validation Type	Approach	Tools
Health Checks	API endpoint validation	HTTP health check
Functionality Tests	Core function validation	Automated test suite
Performance Validation	Response time, throughput	Load testing tools
Security Validation	Configuration review	Security scanners
User Experience	UI functionality	Browser automation
Runtime Verification	Version check via API	HTTP GET, JQ

A dedicated validation job has been added to the post-deployment pipeline to confirm the runtime Java version is 21. This job queries the application's system information API endpoint and verifies the Java version in the response. Example validation script:

```
# Example Java 21 runtime validation
JAVA_VERSION=$(curl -s http://nexus-host/service/rest/v1/status/system |
if [[ $JAVA_VERSION != 21* ]]; then
  echo "ERROR: Expected Java 21, found $JAVA_VERSION"
  exit 1
fi
echo "Java 21 runtime confirmed: $JAVA_VERSION"
```

Automated validation should be incorporated into the deployment pipeline to detect issues early.

Release Management Process

A structured release management process ensures controlled, predictable deployments:

1. **Release Planning**:
    - Version selection and tracking
    - Feature and fix inclusion
    - Dependency updates
2. **Release Preparation**:
    - Release notes generation
    - Documentation updates
    - Version tagging
    - Build verification
3. **Release Execution**:
    - Deployment scheduling
    - Stakeholder communication

- Deployment execution
- Validation

4. **Post-Release Activities**:

- Monitoring
- User support
- Feedback collection
- Retrospective

Each release should be tracked in a release management system with clear ownership and timelines.

8.6 INFRASTRUCTURE MONITORING

8.6.1 Resource Monitoring Approach (updated)

Comprehensive monitoring is essential for Nexus Repository operations:

Resource	Metrics	Collection Method
CPU	Utilization, load averages	JMX, host metrics
Memory	Heap usage, GC activity	JMX, host metrics
Disk	Usage, IOPS, throughput	Host metrics, application metrics
Network	Throughput, connections	Host metrics, application logs
Database	Connections, query performance	JMX, database metrics
BlobStore	Size, operation counts	Application metrics
Virtual Threads	**Count, states, pinning events**	**JMX, Java 21 metrics**
Java 21 GC	**ZGC pause times, concurrent cycles**	**JMX, GC logs**

Monitoring integration options:

1. **JMX Exporters**: Expose JVM and application metrics via JMX
2. **Prometheus Integration**: Metrics endpoint for Prometheus scraping
3. **Log Shipping**: Send application logs to centralized logging
4. **Health API**: REST endpoint for application health status
5. **System Check Services**: Internal health check framework
6. **Java 21 JFR Integration**: Continuous Flight Recorder metrics for Virtual Threads and ZGC

The `components/nexus-base/src/main/java/org/sonatype/nexus/internal/metrics/MetricsServlet.java` component exposes detailed metrics for external collection. This servlet has been extended to include Java 21-specific metrics, enabling comprehensive monitoring of Virtual Threads and ZGC performance.

8.6.2 Performance Metrics Collection (updated)

Performance monitoring focuses on several key areas:

Performance Aspect	Key Metrics	Collection Method
Request Processing	Response time, throughput	Application metrics, access logs
Repository Operations	Operation counts, latency	Application metrics
Search Performance	Query time, result count	Application metrics
Database Performance	Query execution time	Application metrics, database monitoring
BlobStore I/O	Read/write throughput	Application metrics
Virtual Threads	**Thread count, lifecycle events, carrier thread utilization**	**JMX, JFR events**
GC Performance	**ZGC pause times, allocation rates, memory reclamation efficiency**	**JMX, GC logs**

Critical metrics to monitor:

1. **JVM Performance**: Heap usage, garbage collection frequency and duration
2. **Repository Request Rates**: Requests per second by repository and operation type
3. **Operation Latency**: Time to complete different operation types
4. **Error Rates**: Failed requests and operations
5. **Connection Pools**: Database and HTTP client connection utilization
6. **Virtual Thread Utilization**: Active thread count, throughput, pinning events
7. **ZGC Performance**: Pause times, concurrent mark/sweep cycles, memory management efficiency

Performance dashboards should be configured to display Virtual Threads metrics to highlight the scalability improvements achieved through Java 21 adoption. Alert rules should be established for critical thresholds such as excessive thread pinning events or unexpected ZGC pause durations.

8.6.3 Cost Monitoring and Optimization

For cloud deployments, cost monitoring is an important consideration:

Cost Factor	Monitoring Approach	Optimization Technique
Compute	Resource utilization tracking	Right-sizing instances
Storage	Blob store growth rate	Cleanup policies, tiered storage
Network	Egress monitoring	Proxy repository caching
Database	Storage and IOPS tracking	Performance tuning
Backup	Storage and operation costs	Differential backups

Cost optimization recommendations:

1. **Storage Reclamation**: Implement cleanup policies to remove unused artifacts
2. **Instance Right-sizing**: Match instance size to actual resource needs
3. **Proxy Caching**: Optimize proxy repository settings to reduce external fetches
4. **Storage Tiering**: Move older artifacts to lower-cost storage

5. **Operation Scheduling**: Run maintenance tasks during off-peak hours

The system includes built-in task scheduling for many optimization activities. The monitoring integrations for cost optimization have been verified to function correctly under Java 21, ensuring continued visibility into resource utilization and cost factors.

8.6.4 Security Monitoring

Security monitoring focuses on several key areas:

Security Aspect	Monitoring Approach	Tools/Integration
Authentication	Failed login attempts, access patterns	Log analysis
Authorization	Permission denials, privilege escalation	Security logs
Network Access	Unusual access patterns, unauthorized attempts	Network monitoring
Data Access	Sensitive operation tracking	Audit logs
System Changes	Configuration modifications	Configuration audit

Security monitoring recommendations:

1. **Centralized Logging**: Aggregate all security-relevant logs
2. **Alert Patterns**: Define alert thresholds for suspicious activity
3. **Regular Review**: Scheduled review of security logs
4. **Integration**: Connect with SIEM or security monitoring platforms
5. **Baseline Establishment**: Define normal behavior patterns

The system's audit logging capabilities (implemented in `components/nexus-core/src/main/java/org/sonatype/nexus/internal/webhooks/`) provide the foundation for comprehensive security monitoring. All security monitoring integrations have been validated with Java 21 to ensure continuity of security visibility in the upgraded environment.

8.6.5 Compliance Auditing

Compliance monitoring ensures adherence to organizational and regulatory requirements:

Compliance Aspect	Auditing Approach	Evidence Collection
Access Control	User/role audit	Role assignment logs
Data Protection	Encryption verification	Configuration audit
Change Management	Configuration change tracking	Audit logs
Security Standards	Configuration compliance	Automated checks
Operational Standards	Procedure adherence	Process logs

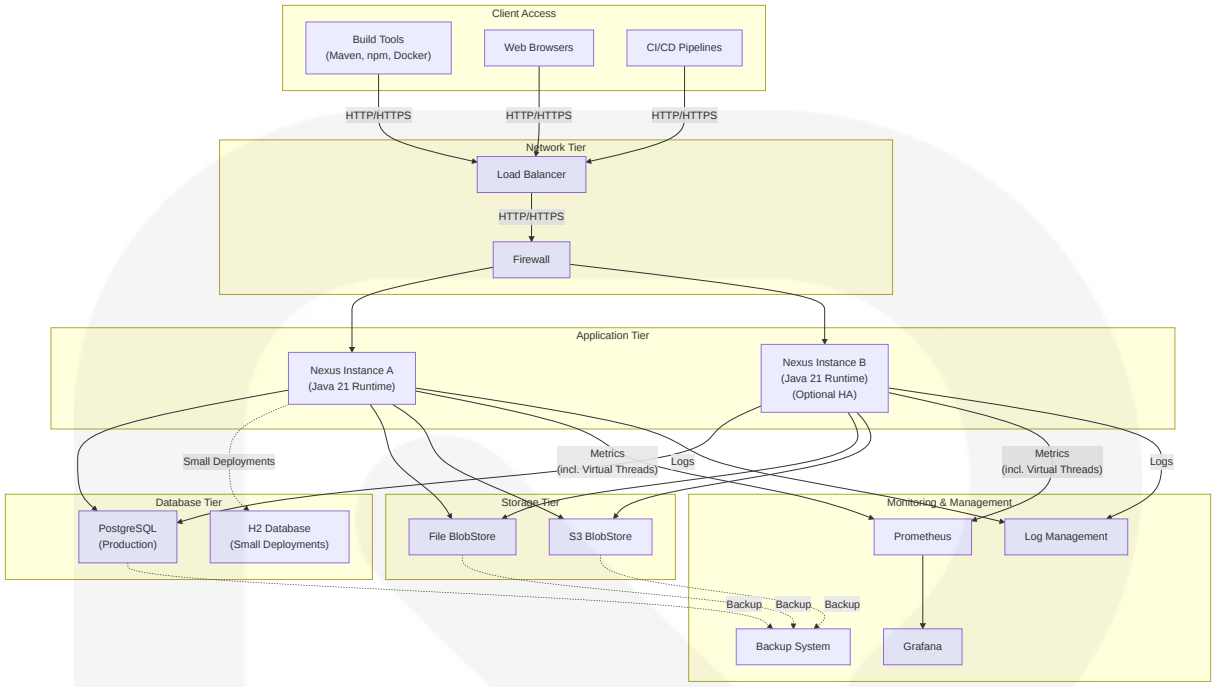
Compliance auditing recommendations:

1. **Automated Evidence Collection**: Scheduled export of compliance-relevant logs
2. **Configuration Validation**: Regular checks against compliance baselines
3. **Report Generation**: Automated compliance reporting
4. **Gap Analysis**: Regular assessment against compliance requirements
5. **Remediation Tracking**: Workflow for addressing compliance gaps

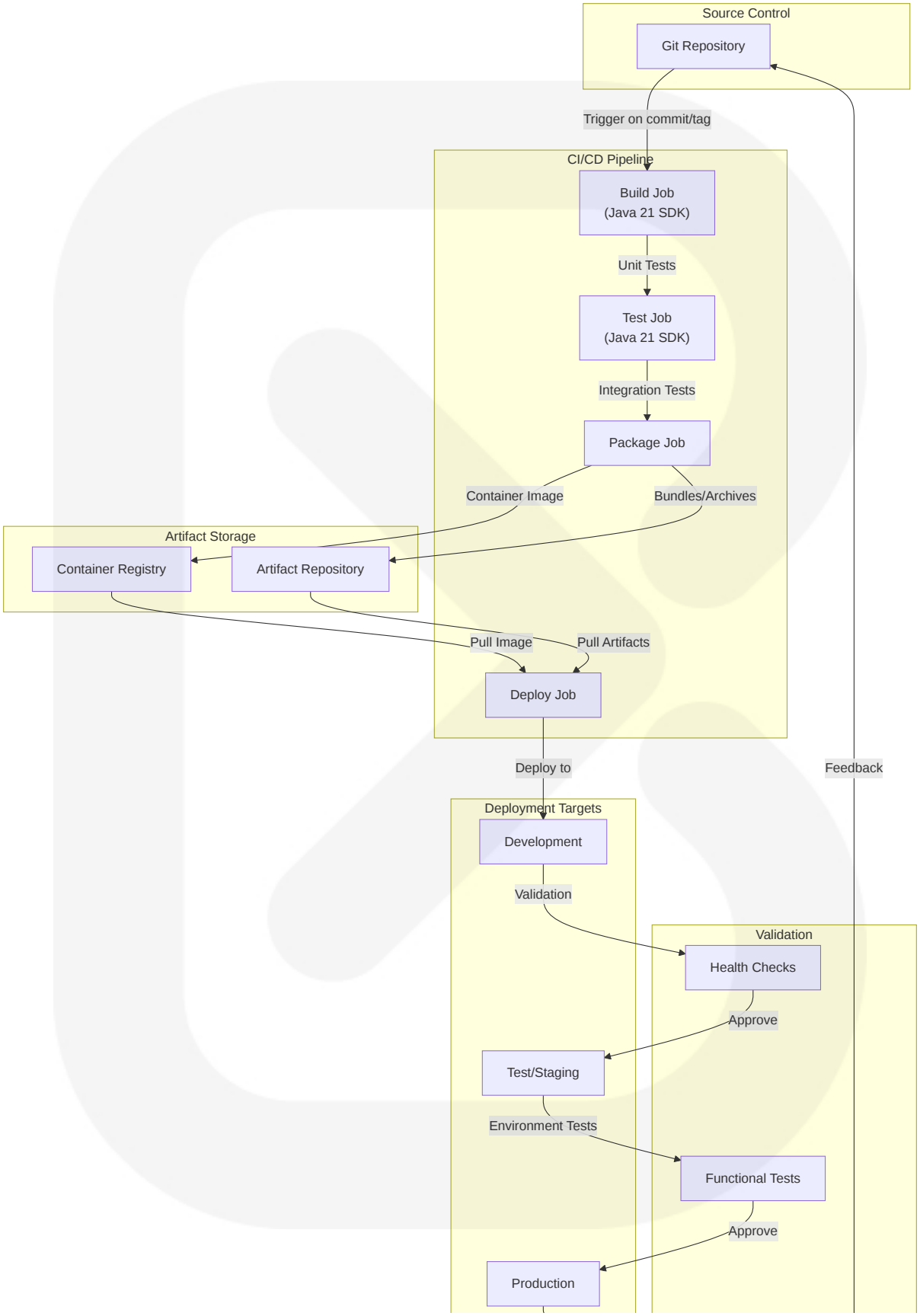
The system's comprehensive audit logging ensures that compliance evidence is readily available. Compliance auditing capabilities remain fully functional under Java 21, with all relevant logs and evidence collection mechanisms operating as expected in the upgraded environment.

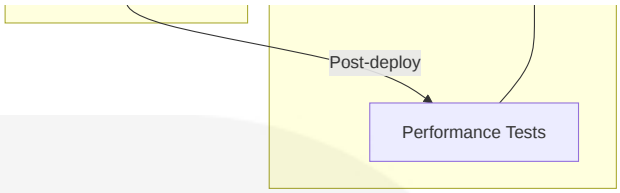
8.7 REQUIRED DIAGRAMS

8.7.1 Infrastructure Architecture Diagram (updated)

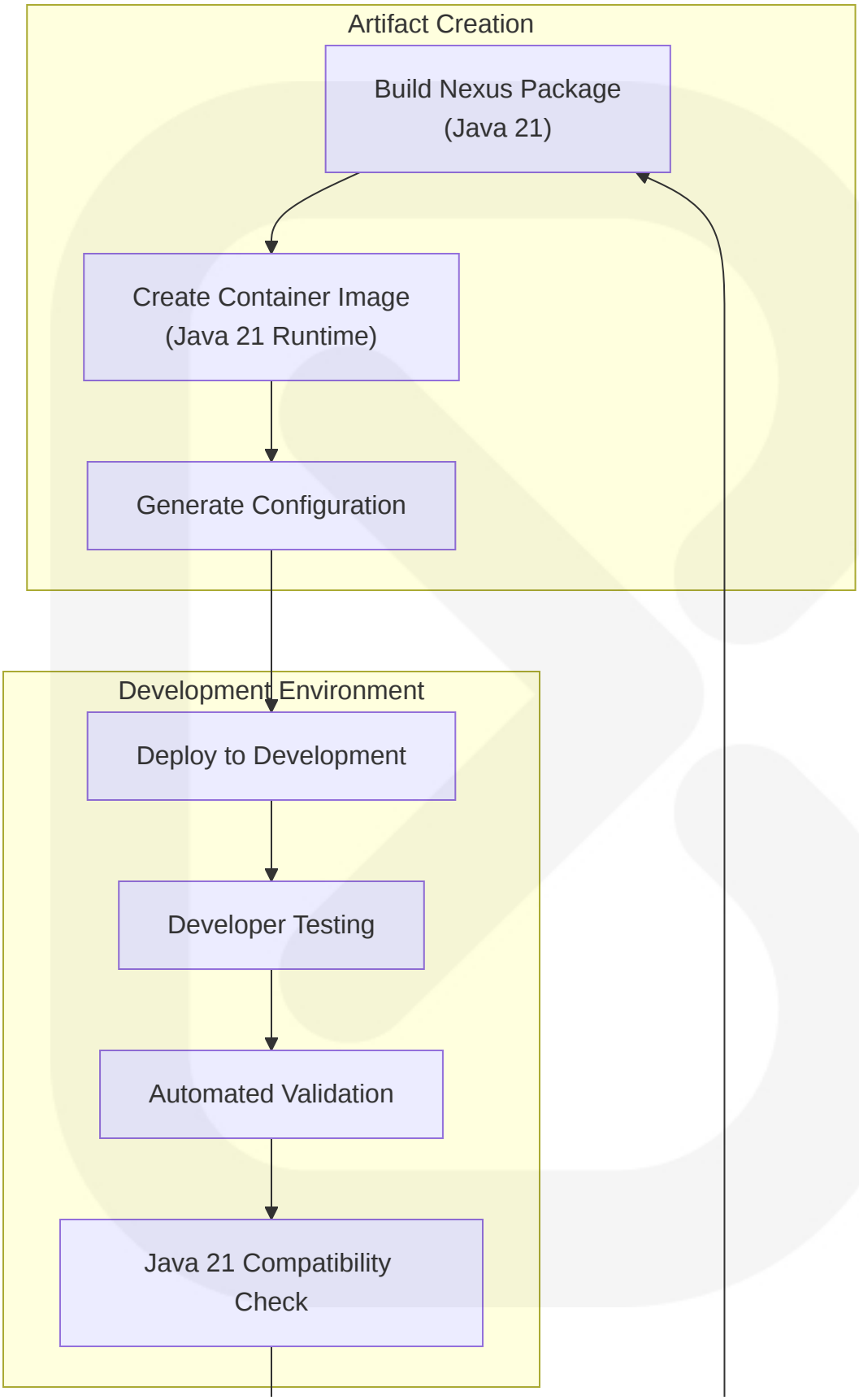


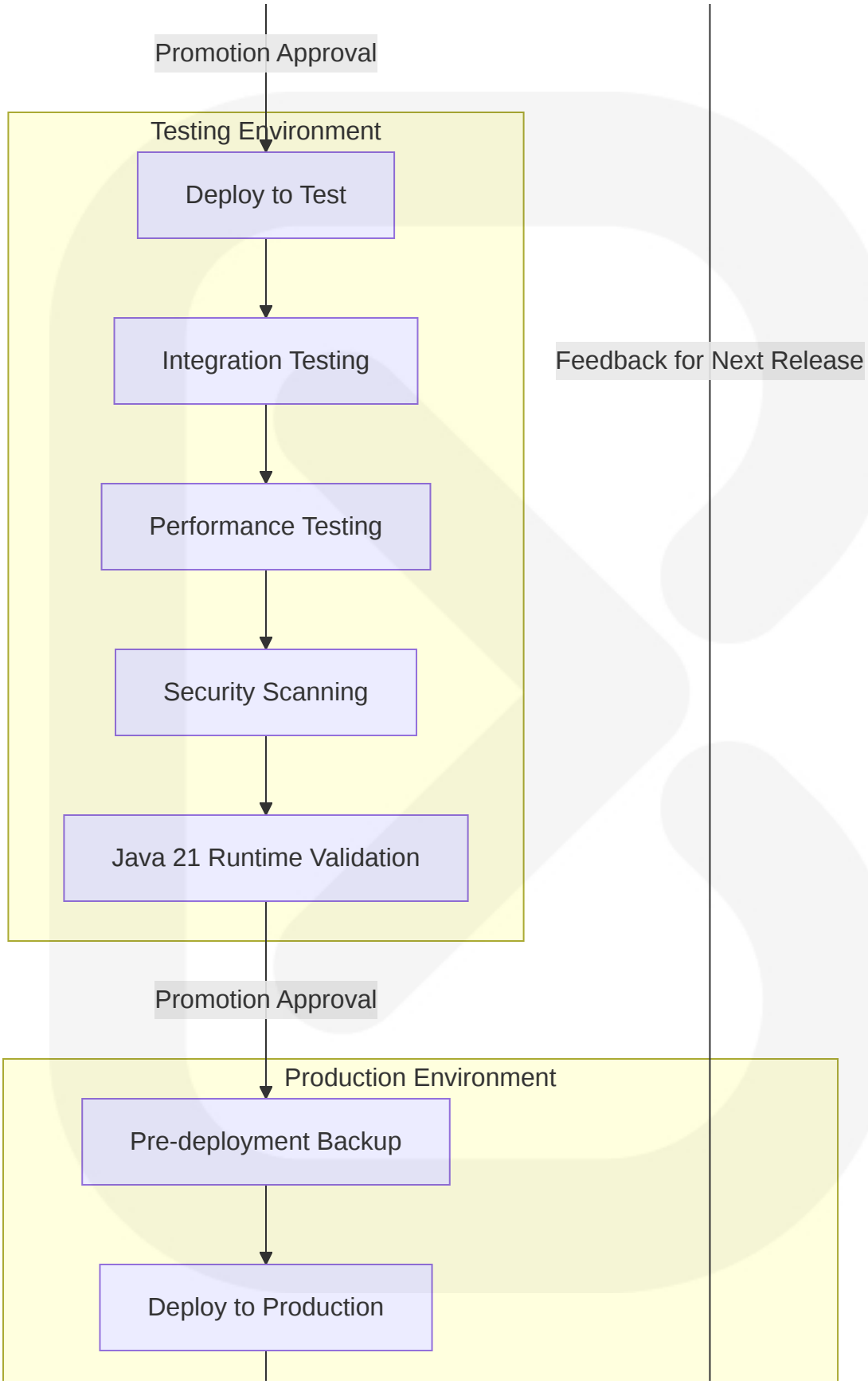
8.7.2 Deployment Workflow Diagram (updated)

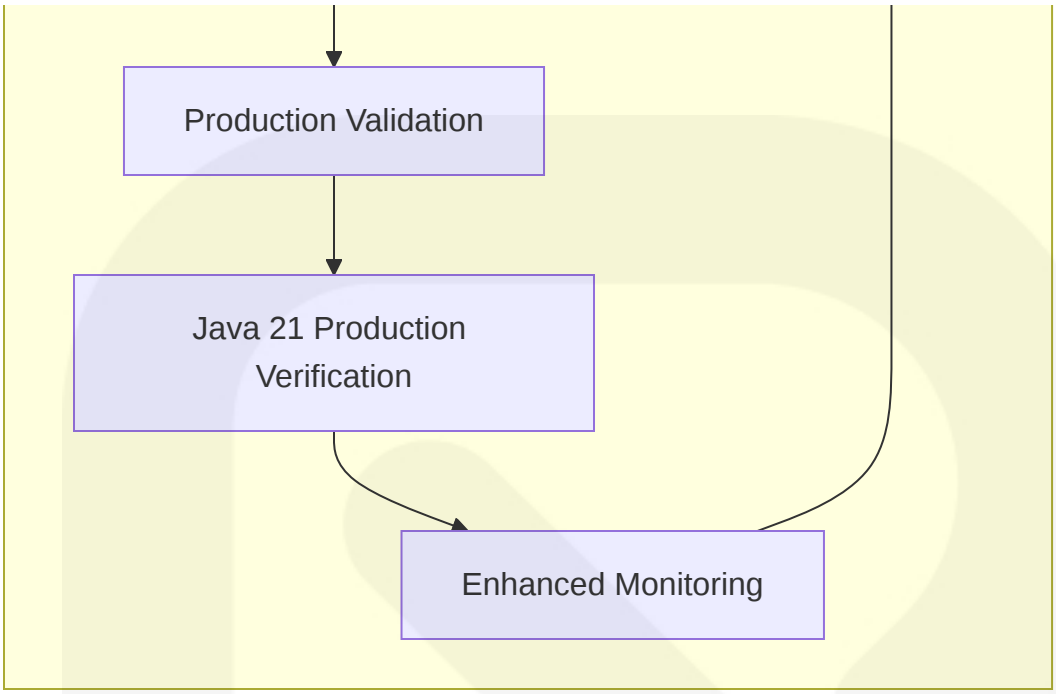

Source Control
Git Repository
Trigger on commit/tag
CI/CD Pipeline
Build Job
(Java 21 SDK)
Unit Tests
Test Job
(Java 21 SDK)
Integration Tests
Package Job
Container Image
Bundles/Archives
Artifact Storage
Container Registry
Artifact Repository
Pull Image
Pull Artifacts
Deploy Job
Deploy to
Feedback
Deployment Targets
Development
Validation
Validation
Health Checks
Approve
Test/Staging
Environment Tests
Functional Tests
Approve
Production



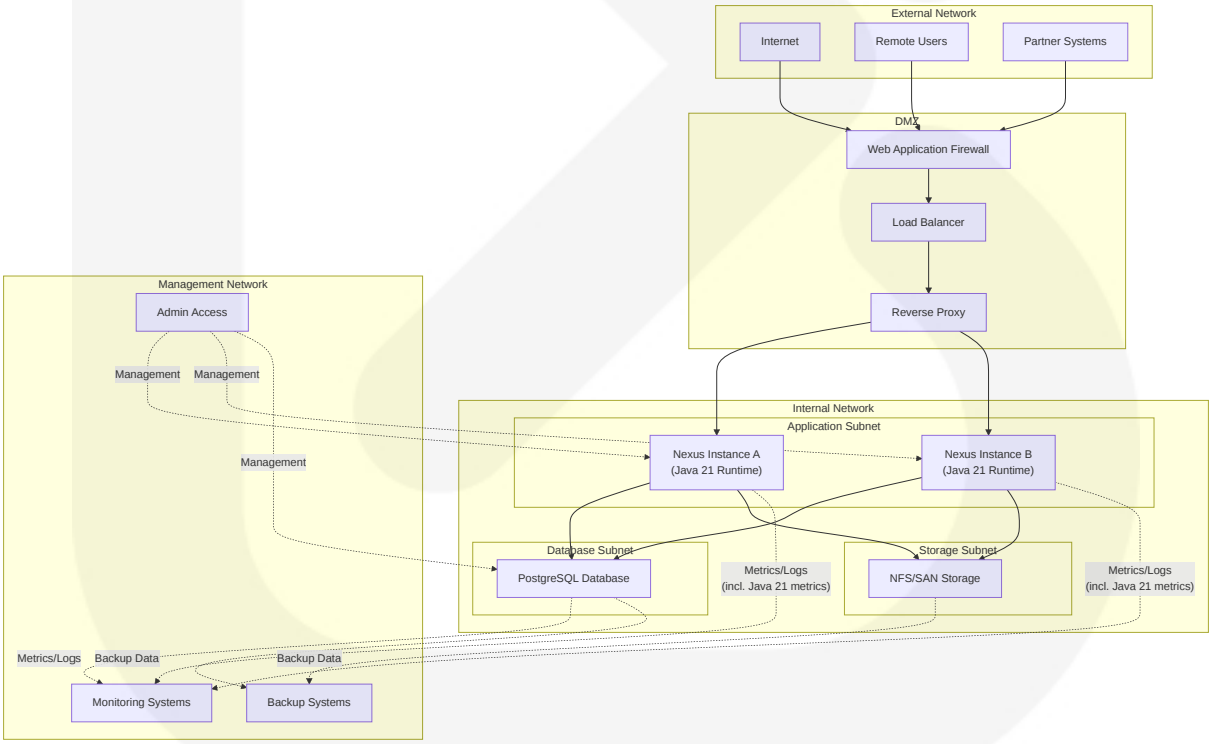
8.7.3 Environment Promotion Flow (updated)


Artifact Creation
Build Nexus Package (Java 21)
Create Container Image (Java 21 Runtime)
Generate Configuration
Development Environment
Deploy to Development
Developer Testing
Automated Validation
Java 21 Compatibility Check


Promotion Approval
Testing Environment
Deploy to Test
Integration Testing
Performance Testing
Security Scanning
Java 21 Runtime Validation
Feedback for Next Release
Promotion Approval
Production Environment
Pre-deployment Backup
Deploy to Production



8.7.4 Network Architecture Diagram (updated)



8.8 SOURCE FILES AND FOLDERS RETRIEVED

The following source files and directories were analyzed to compile this infrastructure documentation:

- `/pom.xml` - Root POM file for build configuration with Java 21 support
- `/mvnw` and `/mvnw.cmd` - Maven wrapper scripts configured for Java 21 compatibility
- `/.mvn/wrapper/maven-wrapper.properties` - Maven version configuration with Java 21 settings
- `/.mvn/maven-build-cache-config.xml` - Build cache configuration
- `/buildsupport/` - Build configuration files updated to reference Java 21
- `/Dockerfile` - Container build configuration updated to use Java 21 base image
- `/.github/workflows/` - CI/CD pipeline configurations updated for Java 21 build and deployment
- `/README.md` - Project overview and basic setup instructions
- `/OPSDOC.md` - Operations documentation template
- `/SECURITY.md` - Security policy documentation
- `/components/nexus-bootstrap/` - Application bootstrap components
- `/components/nexus-base/src/main/java/org/sonatype/nexus/internal/metrics/MetricsServlet.java` - Metrics implementation
- `/components/nexus-core/src/main/java/org/sonatype/nexus/internal/webhooks/` - Webhook implementation for monitoring
- `/components/nexus-blobstore/` - BlobStore implementations
- `/components/nexus-datastore-mybatis/` - Database implementation
- `/components/nexus-ssl/` - SSL/TLS implementation
- `/components/nexus-scheduling/` - Task scheduling framework
- `/components/nexus-quartz/` - Quartz scheduler integration

- `/components/nexus-rapture/src/main/java/org/sonatype/nexus/rapture/internal/LocalSystemCheckService.java` - Health check service
- `/plugins/nexus-blobstore-s3/` - S3 BlobStore implementation
- `/assemblies/nexus-base-template/` - Base distribution template with Java 21 runtime requirements
- `/assemblies/nexus-base-overlay/` - Base distribution overlay configured for Java 21
- `/thirdparty-bundles/` - Third-party dependency bundles

APPENDICES